CLOUD DATA PLATFORMS • SQL • PYTHON

SNOWFLAKE DATA ENGINEERING

From Fundamentals to Production-Ready Data Pipelines

180+

REAL-WORLD PROJECTS

GitHub references and practical learning resources



ARMAN GOLBIDI

From experience to an engineering handbook

A data pipeline is easy to describe when every component is working. The deeper lessons begin when an unexpected record changes a metric, a service becomes unavailable, a report is delivered late, or two valid-looking results disagree. In those situations, a data engineer cannot rely on attractive diagrams or memorised commands. The engineer needs a clear model of the system, evidence about its behaviour, and a practical way to recover.

This book grew out of that conviction. Its ideas, notes, practical experiments, and engineering questions have developed over a four-year period of professional learning and reflection, informed by a longer career in software, data, and continuously operating technical systems. I wanted to create a book that moves beyond a catalogue of Snowflake features and instead helps readers ask why one design is more dependable than another. The result is an applied guide to building, testing, explaining, and operating modern data pipelines.

From Iran to Finland

My professional foundations were shaped in Iran, where I studied computer software engineering and worked in the technical environment of the Islamic Republic of Iran Broadcasting. Beginning in 2017, my responsibilities developed from data preparation and database-oriented software work into data engineering, analytics, and the support of real-time digital systems. Those systems taught me that reliability is not an optional quality to add at the end. Clear interfaces, predictable recovery, monitoring, and careful communication are requirements from the beginning.

A later chapter of my work took me to Finland. At Valueqon Oy, I worked on data integration, transformation, data quality, and business intelligence using Azure, SQL, Python, Power BI, and related analytics tools. At LUT University, I contributed to research on uncertainty-aware, 168-hour building-energy forecasting. In May 2026, I completed a Master of Science (Technology) in Data-Centric Engineering at LUT University. These experiences reinforced the connection between trustworthy data preparation and meaningful decisions.

Why Snowflake?

Snowflake offers a focused setting in which to study principles that extend beyond any one platform: separation of storage and compute, explicit data contracts, reproducible transformations, incremental processing, observability, governance, and cost-aware architecture. The objective is not to present one vendor as the answer to every workload. It is to develop enough practical judgement to recognise both the strengths and the boundaries of a design.

Learn by building, checking, and questioning

The chapters move from Snowflake architecture, SQL and modelling to batch and streaming ingestion, incremental processing, Snowpark Python, dbt and Power BI. Later chapters address query performance, security, governance, cost and carefully evaluated AI. The final capstone joins these concerns into an energy-data-platform design.

Practical projects, not just demonstrations

The book includes worked examples, code, exercises and companion materials. The continuing *Finland Energy Analytics* scenario connects time grains, energy units, corrections, validation, auditability and reporting. The companion catalogue indexes 181 external GitHub repositories as educational references, not as claims of independent execution, certification or permission to republish their code.

Many exercises are tested offline with synthetic data. Live Snowflake integrations, permissions, regional feature availability, Power BI deployments and cloud charges require separate verification. An illustrative result is not proof of production readiness.

How to read this book

New Snowflake users can work sequentially and reproduce controlled exercises before modifying a real environment. Experienced SQL and Python users can focus on ingestion and transformation, consulting earlier explanations as needed. Every engineering choice deserves an example, a failure case and a check.

This is an independent educational book, not endorsed by Snowflake or Microsoft. Product names remain their owners' trademarks. Third-party reuse requires a licence check; production deployment requires accountable review and testing.

An editorial note and a word of thanks

Manuscript preparation included AI-based drafting and editorial assistance. The author's professional experience shaped the educational direction, and independent technical and publishing review remains necessary. I thank the colleagues, teachers and engineering teams in Iran and Finland whose questions have shaped my thinking.

My hope is that this book encourages readers not merely to make pipelines run, but to make their assumptions inspectable, their failures recoverable, and their results worthy of trust.

Arman Golbidi

Lappeenranta, Finland | October 2026

Contents

Folio numbers refer to the original printed chapter pages, retained unchanged.

Chapter 01 Introduction to Snowflake and Modern Data Engineering 9

1.1 The evolution of modern data platforms 10
1.2 Data warehouses, lakes, lakehouses, and data clouds 11
1.3 OLTP versus OLAP: workloads and design choices 12
1.4 Snowflake ecosystem and core use cases 13
1.5 The end-to-end data engineering lifecycle 14
1.6 Batch, streaming, and analytics architectures 15
1.7 When Snowflake is (and is not) the right choice 16
1.8 Introducing the Finland Energy Analytics capstone 17
1.L Guided lab: design the first data product 18

Chapter 02 Snowflake Architecture: Storage, Compute, and Services 21

2.1 The three architectural layers 22
2.2 Storage organization and micro-partitions 23
2.3 Virtual warehouses and compute isolation 24
2.4 Cloud services, metadata, and query processing 25
2.5 Query pruning, caching, and data access 28
2.6 Concurrency and multi-cluster warehouses 31
2.7 Time Travel, Fail-safe, and data retention 32
2.8 Cloud regions, deployment models, and limitations 34
2.9 Reference architectures and design trade-offs 36
2.L Guided lab: follow an analytical query 37

Chapter 03 Setting Up Snowflake and Development Environments 39

3.1 Accounts, organizations, editions, and regions 40
3.2 Getting started with Snowsight 41
3.3 Databases, schemas, tables, and warehouses 42
3.4 Users, roles, and basic grants 45
3.5 SQL worksheets, Snowflake CLI, and Git 46
3.6 Connecting Python and SQL tools 49
3.7 Naming and DEV/TEST/PROD boundaries 51
3.8 Budgets, resource monitors, and cost guardrails 52
3.9 Reusable environment bootstrap scripts 54
3.L Guided lab: a governed first workspace 55

Chapter 04 Snowflake SQL and Data Modeling 57

4.1 Snowflake SQL syntax and data types 58
4.2 Filtering, joins, aggregations, and subqueries 61
4.3 Common table expressions and window functions 63
4.4 Working with dates, time zones, and time series 65
4.5 Tables, views, and materialized views 66
4.6 Fact tables, dimensions, and star schemas 67
4.7 Keys, surrogate identifiers, and slowly changing dimensions 68
4.8 Data validation and SQL testing patterns 69
4.9 Naming, documentation, and maintainable SQL 72
4.L Guided lab: a tested energy star model 75

Chapter 05 Batch Data Ingestion and Loading 77

5.1 Data source assessment and ingestion requirements 78
5.2 File formats: CSV, JSON, Parquet, and others 81
5.3 Internal and external stages 83
5.4 Loading data with COPY INTO 85
5.5 File validation and rejected-record handling 88
5.6 Schema evolution, metadata, and provenance 89
5.7 Idempotency, deduplication, and load auditing 92
5.8 Python-assisted batch ingestion 93
5.9 Recovery procedures and ingestion monitoring 94
5.L Guided lab: a trustworthy bronze ingest 95

Chapter 06 JSON and Semi-Structured Data 97

6.1 Semi-structured data and the VARIANT type 98
6.2 OBJECT and ARRAY structures 101
6.3 Parsing, inspecting, and validating JSON 103
6.4 Querying nested keys and paths 104
6.5 Unnesting arrays with FLATTEN and LATERAL 105
6.6 Handling schema drift and missing values 106
6.7 Parquet, mixed-format sources, and normalization 107
6.8 Performance and modeling considerations 108
6.L Guided lab: energy API payloads to silver facts 111

Chapter 07 Snowpipe, Streaming, and Real-Time Ingestion 113

7.1 Batch versus near-real-time versus streaming 114
7.2 Snowpipe architecture and auto-ingest concepts 117
7.3 Cloud event notifications and staged files 118
7.4 Snowpipe Streaming fundamentals 119
7.5 SDK-based and API-based streaming workflows 122
7.6 Event ordering, duplicates, and late-arriving data 125
7.7 Schema changes and failure recovery 127
7.8 Openflow and connector-based ingestion overview 128
7.9 Latency, monitoring, throughput, and cost 129
7.L Guided lab: a failing energy telemetry feed 131

Chapter 08 Streams, Tasks, and Dynamic Tables 133

8.1 Change data capture and incremental updates 134
8.2 Change tracking with Streams 137
8.3 Scheduling jobs and task dependency graphs 139
8.4 Incremental MERGE and upsert design 140
8.5 Defining transformations with Dynamic Tables 141
8.6 Refresh modes, target lag, and dependencies 143
8.7 Dynamic Tables versus Streams/Tasks versus views 144
8.8 Recovery, backfills, and pipeline observability 146
8.9 Incremental transformation design exercise 148
8.L Guided lab: two ways to maintain energy state 149

Chapter 09 Data Engineering with Snowpark Python 151

9.1 Where Snowpark fits in the platform 152
9.2 Python sessions and connection management 155
9.3 DataFrames, expressions, and lazy evaluation 156
9.4 Transformations, joins, and aggregations 159
9.5 User-defined functions and stored procedures 160
9.6 Dependencies, packaging, and code organisation 161
9.7 Local development and automated testing 162
9.8 Snowpark performance and execution trade-offs 163
9.9 Production-oriented Python pipeline patterns 166
9.L Guided lab: clean and enrich energy observations 169

Chapter 10 ETL/ELT Pipelines, dbt, and Delivery Workflows 171

10.1 ETL versus ELT: architecture and trade-offs 172
10.2 Bronze, silver, and gold transformation layers 175
10.3 Structuring a dbt project for Snowflake 177
10.4 Models, sources, tests, and documentation 179
10.5 Incremental models and snapshot strategies 180
10.6 Native and external orchestration options 181
10.7 Git branching, CI/CD, and environments 182
10.8 Data contracts, quality gates, and observability 183
10.9 Deployment, rollback, and release checklists 186
10.L Guided lab, chapter review, and exercises 189

Chapter 11 Analytics Engineering and Power BI Integration 191

11.1 Analytics engineering and serving-layer design 192
11.2 Defining trusted metrics and KPI contracts 194
11.3 Snowflake connectors and BI integration 196
11.4 Power BI Import and DirectQuery considerations 198
11.5 Semantic modelling and refresh strategies 199
11.6 Building energy analytics dashboards 200
11.7 Dashboard performance and query behaviour 201
11.8 Data-sharing patterns and operational handoff 204
11.L Guided lab, chapter review, and exercises 207

Chapter 12 Query Performance and Warehouse Optimization 209

12.1 A structured performance troubleshooting workflow 210
12.2 Query History and Query Profile 213
12.3 Micro-partition pruning and clustering 215
12.4 Warehouse sizing and concurrency management 217
12.5 Caching behavior and materialized results 219
12.6 Query acceleration and search optimization concepts 220
12.7 Improving joins, filters, and aggregations 221
12.8 Benchmarking performance, credits, and latency 222
12.L Guided lab, chapter review, and exercises 225

Chapter 13 Security, Governance, Reliability, and Cost 227

13.1 Security architecture and least privilege 228
13.2 Role-based access control and role hierarchies 231
13.3 Authentication, secrets, and network controls 233
13.4 Masking policies, row access, and classification 234
13.5 Tags, lineage, auditing, and data sharing 235
13.6 Privacy, retention, and compliance basics 238
13.7 Compute, storage, serverless, and AI cost drivers 239
13.8 Budgets, monitoring, and workload guardrails 242
13.9 Operational reliability, recovery, and incident response 243
13.10 Governance and platform runbooks 244
13.L Guided lab, chapter review, and exercises 247

Chapter 14 Snowflake Cortex AI and Machine Learning 249

14.1 AI/ML use cases in a modern data platform 250
14.2 Cortex AI capabilities and availability caveats 252
14.3 AI SQL functions and evaluation principles 253
14.4 Search, retrieval, and natural-language analytics 255
14.5 Data preparation and feature engineering with Snowpark 256
14.6 Forecasting and anomaly detection use cases 258
14.7 Model evaluation, security, and responsible AI 259
14.8 Token usage, compute charges, and AI cost controls 261
14.L Guided lab, chapter review, and exercises 263

Chapter 15 Capstone: A Production-Ready Energy Data Platform 265

15.1 Business scenario, assumptions, and acceptance criteria 266
15.2 Source selection, architecture, and data contracts 268
15.3 Reproducible environment and repository structure 271
15.4 Batch and continuous ingestion design 273
15.5 Bronze-to-silver data quality and transformations 274
15.6 Gold marts, KPIs, and incremental refreshes 277
15.7 Python components, tests, and CI/CD 278
15.8 Power BI dashboards and optional AI extension 279
15.9 Access controls, performance, and cost budget 280
15.10 Monitoring, failure drills, and operational handover 283
15.11 Documentation, architecture decision records, and demo 284
15.12 Project review and future improvements 285
15.L Guided lab, chapter review, and exercises 286

CHAPTER 01 / FOUNDATIONS

Introduction to Snowflake and Modern Data Engineering

From raw measurements to a trustworthy decision

At 08:00, an energy analyst opens a dashboard that shows electricity generation in Finland. The chart is current, its labels look convincing, and the query returned successfully. But the source has retried yesterday's data, two timestamps have been interpreted in local time rather than UTC, and a missing quarter-hour has quietly disappeared from the total. The platform did what it was asked to do; the information is still wrong.

This is why modern data engineering is not primarily the art of moving bytes. It is the discipline of turning uncertain, changing inputs into *reliable data products* that people can interpret and use. Snowflake can provide a powerful environment for that work, but architecture, semantics, quality rules, and operational judgment remain the engineer's responsibility.

The central question of this chapter
How do we choose a data architecture that delivers correct, timely, maintainable answers without paying for complexity that our users do not need?

By the end of Chapter 1, you will be able to:

- Explain the evolution from traditional warehouses to cloud-based data platforms.
- Distinguish warehouses, lakes, lakehouses, and transactional stores by workload.
- Explain the role of Snowflake's storage, compute, and cloud-services layers.
- Compare batch, micro-batch, and streaming with measurable freshness requirements.
- Draft the first requirements, metrics, and context diagram for the Finland Energy Analytics capstone.

Prerequisites: no Snowflake experience; elementary SQL is helpful. The guided lab can be completed offline without a paid cloud account. The architecture discussion draws on Snowflake documentation [1]; all version-dependent features require verification before publication.

Reader's route Sections 1.1–1.3 build a historical and architectural mental model; 1.4–1.7 test engineering decisions; 1.8 introduces the capstone; 1.L turns the ideas into a reviewable specification. We will implement the platform incrementally in later chapters.

1.1 The evolution of modern data platforms

1.1.1 On-premises systems and traditional data warehouses

Imagine a manufacturer with order data inside an enterprise resource planning database, meter readings in CSV files, and finance reports in spreadsheets. Ten years ago, a common response was to operate a dedicated analytical database: extract records overnight, clean and reshape them, and load the results into a central warehouse. Analysts queried that carefully managed copy rather than overloading the system that processed orders.

That architecture solved a real problem: an *operational* system is optimized for timely, correct transactions, while analytical questions often scan many records, join histories, and compute aggregates. Separating them protects the daily business from an expensive reporting query. Dimensional models further help readers understand what a number means: "electricity delivered" requires a defined measure, grain, unit, time basis, and source.

Traditional warehouses were not a mistake. They were designed around understandable constraints: hardware procurement, fixed storage capacity, scheduled maintenance, limited processor headroom, and teams that had to decide in advance how much infrastructure to buy. Their weaknesses appeared when workload shapes changed faster than procurement cycles. A monthly reporting peak might require hardware that remained idle for the rest of the month.

The architectural distinction is not old versus new. It is *which responsibilities the system centralizes.* Even on cloud infrastructure, a poorly designed pipeline can reproduce every problem of its on-premises predecessor: stale extracts, undocumented transformations, and no recovery plan.



Figure 1.1. A familiar scheduled warehouse pattern. The issue is not nightly loading itself, but whether its delay and failure behavior meet the reporting contract.

Three questions turn the historical model into engineering judgment: *How fresh must the result be? Can the operational workload safely serve analysis? What happens if the extract runs twice or fails halfway through?* Those questions matter regardless of the vendor.

Explore the source project: P001: Modern Data Engineering with Snowflake [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.1 From fixed capacity to elastic analytics

1.1.2 Elastic cloud analytics and separated workloads

Cloud platforms changed the procurement problem. Rather than purchase a machine sized for the year's busiest afternoon, a team can provision compute for a job and release it after the work ends. Snowflake applies this model through *virtual warehouses*: compute clusters can be assigned to different analytical activities while accessing a common data platform. Snowflake documents three architectural layers—database storage, compute, and cloud services—which we examine in detail in Chapter 2 [1].

The separation matters because analytical workloads compete differently. A data transformation job may scan and update large tables; a dashboard may issue many smaller queries during working hours; an experimental Python notebook may be unpredictable. Assigning distinct virtual warehouses can reduce contention between these compute workloads. It does not eliminate every shared dependency, and it does not make credits free: warehouses consume credits while running, subject to billing rules and minimums [2].

1.1.3 Evolving responsibilities of the data engineer

Infrastructure automation also shifts work rather than removing it. Modern data engineers must design data contracts, select ingestion modes, reason about schema changes, organize testing, secure access, monitor freshness, and explain costs. They ask whether a reported number can be traced back to its source and whether a corrected upstream record will update a downstream metric safely.

Worked decision: two teams, one dataset
The energy-analysis team refreshes a daily report once each morning; an operations team investigates current power measurements throughout the day. Both can use a shared governed dataset, but the same refresh target and compute schedule need not fit both. The engineering decision is to separate *service objectives* before selecting services or warehouse sizes.

In Snowflake, "elastic" is an engineering affordance, not a performance guarantee. Starting more compute might reduce queueing, but it will not repair an incorrect join or a duplicated source file. Likewise, a high-throughput platform will not resolve the meaning of an energy statistic. Correctness begins with agreement on identifiers, units, boundaries, and acceptable delays.

Decision checkpoint. Suppose reporting needs only yesterday's validated totals. A scheduled batch is a defensible default. If a control room requires updates within a short interval, revisit that design with an explicit freshness target and failure budget. You should justify the extra moving parts in operational terms, not because "streaming" sounds more modern.

Common failure. Teams often equate successful pipeline execution with accurate data. A green job status proves a process completed according to its controls; it does not prove the business meaning of the result. We will add independent invariants and reconciliation checks later.

1.2 Data warehouses, lakes, lakehouses, and data clouds

1.2.1 Data warehouses and data lakes

A *data warehouse* organizes information for repeatable analytical queries. Curated tables, stable dimensions, explicit metric definitions, and well-understood access patterns make it appropriate for business intelligence. A *data lake* is typically an object-storage-based collection of data in diverse formats and stages of refinement; it can accommodate raw files cheaply, but a folder of files is not by itself a governed, reliable analytics system.

The distinction is functional, not a claim that warehouses can store only tables or that

lakes cannot support analytics. Snowflake accepts structured and semi-structured data, while modern engines can query files in external storage. The critical question is who manages transaction semantics, metadata, security, transformation, and performance.

1.2.2 Lakehouse and data-cloud concepts

A *lakehouse* aims to combine open or lake-style storage with table metadata, reliable reads, and analytical processing. Apache Iceberg is one table format used in this ecosystem. Snowflake documents both Snowflake-managed storage and external-volume arrangements for Iceberg tables; those choices have different governance and operational implications [1, 3]. "Data cloud" is broader platform language describing governed access, processing, and collaboration around data; it is not a precise replacement for a physical architecture diagram.

1.2.3 Selecting architecture by workload rather than marketing labels

Approach	Useful when	Question to test
Warehouse	Curated BI, governed SQL metrics	Are model and query contracts stable?
Lake	Diverse raw files and replay history	Who enforces usable table semantics?
Lakehouse	Shared table formats and multiple engines	How are catalogs and writes coordinated?

For our energy case, retain an auditable source snapshot and build trustworthy analytical tables. Whether the source snapshot stays in an external object store, in Snowflake-managed storage, or in both is a cost, governance, and recovery decision. It is not dictated by the word "lakehouse."

Failure to avoid: assuming an open table format means every engine supports identical semantics, security, and performance. Verify catalog behavior, write ownership, compatibility, and cross-tool tests before committing to a multi-engine architecture.

Explore the source project: P002: Getting Started with Iceberg Tables [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.3 OLTP versus OLAP: workloads and design choices

1.3.1 Transactional versus analytical access patterns

An electricity billing application may need to update one customer's payment atomically. An analyst may ask for two years of hourly electricity generation by source, region, and season. The first resembles *online transaction processing* (OLTP): short operations, selective reads, concurrent updates, and carefully enforced invariants. The second resembles *online analytical processing* (OLAP): scans, aggregates, joins, and historical comparisons. Both are databases; their principal workloads differ.

1.3.2 Read/write characteristics and latency

A workload specification should record operation mix, concurrency, data volume, read and write patterns, and acceptable latency. "Real time" is not a measurable target. "An event

must appear in the operational display within 60 seconds for 95% of accepted events" is closer, although it still needs a definition of event time and acceptance time.

Snowflake standard tables are primarily oriented toward analytics. Snowflake also documents *hybrid tables*, designed for lower-latency indexed reads and writes with transactional capabilities, but their support and limitations depend on cloud region and account features [1, 4]. The existence of a transactional-oriented option does not imply that Snowflake should replace every operational database.

1.3.3 Selecting a store for energy-meter use cases

For the running project, imagine two independent applications. A device-registration system changes individual meter records throughout the day, while an analytical platform calculates generation trends and power-to-energy aggregates across many readings. We would ordinarily evaluate a transactional system for registration state and a governed analytical model for historical analysis, with an explicit ingestion boundary between them.

Why granularity matters
A row recording 480 MW of average power for a 15-minute interval contributes $480 \times \frac{15}{60} = 120$ MWh of energy for that interval under the stated sampling assumption. Summing MW readings alone does *not* yield energy. An architecture that returns queries quickly but confuses those units is not successful.

Decision checkpoint. Describe a requirement for a single-record correction and another for a two-year trend query. What write behavior, consistency guarantees, latency, and history will each need? Write your answers before naming a database.

Common failure. Running a broad reporting query directly against the transactional application during peak business hours can harm its primary duty. Isolate and govern analytical access unless a measured workload justifies otherwise.

Explore the source project: P003: Hybrid Tables Performance Testing [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.4 Snowflake ecosystem and core use cases

1.4.1 Core platform components and SQL/Python roles

Snowflake is a managed cloud data platform rather than software that you install on a local server. Its database storage layer manages persistent analytical data; virtual warehouses supply compute for supported SQL and Snowpark workloads; cloud services coordinate authentication, metadata, parsing, optimization, and other control functions [1]. Cloud-managed does not mean responsibility-free. The customer still defines permissions, trustworthy models, retention choices, and spending controls.

SQL is a first-class language for transformations and analysis. Python enters through connectors, Snowpark, and external tooling, among other workflows. Choosing one does not exclude the other: Python may retrieve a licensed API extract and validate it, while SQL defines the repeatable analytical model. Later chapters will distinguish where code executes and which component pays for it.

1.4.2 Structured versus semi-structured processing

An energy record may have a timestamp, a source category, and power measurement:

```
timestamp_utc / source_type / power_mw
```

An API may instead return nested JSON: a time stamp, several measurements, metadata, and quality flags inside one object. Snowflake can process structured and semi-structured data [1], but accepting flexible JSON is not the same as validating its semantics. Explicit checks are needed for missing values, units, time zones, and schema drift.

1.4.3 Business intelligence, data sharing, and AI use cases

Three broad patterns are relevant here: prepare stable data for BI, share governed data products with authorized consumers, and apply forecasting or other models to curated features. Snowflake supports sharing selected database objects with other Snowflake accounts, using provider-controlled shares rather than naïvely exporting a separate copy in every intra-platform case [5]. Neither this feature nor AI capability eliminates privacy reviews, contract requirements, or the need to measure correctness.

A small, defensible initial platform
`Public energy source` → `raw observations` → `validated energy intervals` → `daily metrics` → `BI report`. Start with this traceable path. Add continuous loading, data sharing, or AI only when they serve an explicit requirement.

Failure to avoid. Treating "supported by the platform" as "available in every edition and region." Product surfaces and pricing change; use official documentation, account-specific verification, and a recorded test date.

Explore the source project: P004: Zero to Snowflake [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

Explore the source project: P005: Data Apps Demo [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.5 The end-to-end data engineering lifecycle

1.5.1 Source discovery and data contracts

A useful data product begins with people and decisions rather than tables. Who owns the source? What can a "missing" value mean? How often is the feed updated? Which conditions make a record acceptable? A *data contract* captures such promises: schema, grain, units, identity, timeliness, provenance, and expected quality. A public API that returns JSON is only an entry point. Its schema and licensing terms must be recorded before the team builds dashboards around it.

1.5.2 Ingestion, transformation, and validation

The lifecycle then moves through ingestion, preservation of raw evidence, validation, transformation, and publication. Treat each boundary as a place where meaning can change. Parsing a timestamp can erase its offset. Filtering nulls can alter totals. Joining two tables at incompatible grains can multiply records. A reproducible pipeline makes such decisions

inspectable.

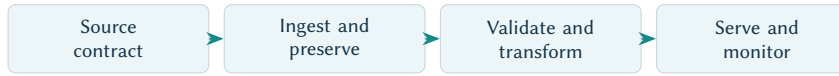


Figure 1.2. Data engineering is a lifecycle of contracts, transformations, tests, and responsibility—not a one-way file copy.

1.5.3 Monitoring, quality, and maintenance

Operational ownership begins after deployment. Useful indicators include latest source timestamp, delay to publication, rejected-record count, test failure rate, and cost by workload. A successful query is only one observation. If the source stops sending readings, yesterday's results may still look plausible; a freshness alert detects the silent failure.

Six questions for every stage
What went in? What came out? Which invariants hold? Which records were rejected? Can the step be repeated safely? Who responds when it fails? A team that can answer these questions is closer to operating a data product than a team that merely has a collection of scripts.

For a daily energy report, the first useful contract might be: every accepted interval is uniquely identified by source and UTC timestamp; power is expressed in MW; energy calculations respect interval length; no dashboard is marked "complete" until required quality checks pass. These are proposed *project requirements*, not claims about a public dataset.

Common failure. Monitoring only pipeline status ignores source freshness and business correctness. A row count can look normal even when all new rows are duplicates.

Explore the source project: P006: Snowflake Northstar Data Engineering [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.6 Batch, streaming, and analytics architectures

1.6.1 Batch, micro-batch, and streaming models

A batch loads a bounded set of data at scheduled or deliberate times. A micro-batch collects smaller intervals, often frequently. Streaming receives records or events continuously and typically involves state, delivery, ordering, and recovery concerns. These describe different processing patterns; they are not interchangeable promises about end-to-end latency.

Snowflake supports bulk file loading with `COPY INTO`, file-oriented continuous loading with Snowpipe, and row-oriented ingestion using Snowpipe Streaming. Their sources, compute models, latency characteristics, and costs differ [6]. A data engineer selects an ingestion pattern only after agreeing on the event-to-reporting requirement.

1.6.2 Freshness and end-to-end latency metrics

Define *freshness* as how recently published data reflects the source, and *latency* as the elapsed time across identified pipeline boundaries. If a sensor reports at 09:00 and the curated table becomes queryable at 09:08, the observed source-to-table delay is eight minutes.

If the BI dashboard does not refresh until 09:30, the user's effective delay is thirty minutes. Measuring only ingestion hides the true result.

A proposed service-level objective (SLO) for a daily dashboard might be "the previous UTC day's validated metrics are available by 08:00 Europe/Helsinki, on 99% of scheduled days." For a near-live view, an SLO might instead target the 95th percentile of event-to-serving delay. These are *illustrative targets*; source limitations and stakeholder needs must determine the real ones.

1.6.3 Cost, complexity, and latency trade-offs

Pattern	Likely benefit	Operational burden
Scheduled batch	Simple replay and reconciliation	Delayed availability between runs
Frequent micro-batch	More frequent publication	More scheduling and file overhead
Continuous rows	Lower ingestion delay potential	Delivery state, monitoring, paid service footprint

Decision checkpoint. Would a 15-minute refresh meet the user's actual need? If yes, start there and test it before paying for a more elaborate architecture. If not, quantify what faster results change and design for backpressure, duplicates, and recovery.

Failure to avoid. Advertising "five-second real time" because a vendor documents a best-case ingestion figure. Benchmark the full path including external API, validation, transformations, BI refresh, and users' region [6].

Explore the source project: P007: Time-Series and Streaming Guide [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

Explore the source project: P008: Recommender Pipeline [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.7 When Snowflake is (and is not) the right choice

1.7.1 Strengths in BI, ELT, and governed sharing

Snowflake is worth considering when multiple teams need governed analytical data, a SQL-centric transformation workflow, independently configurable compute, and managed infrastructure rather than self-operated database servers. It can reduce infrastructure work and support collaboration around shared analytical objects. These are capabilities, not proof of cost or speed superiority for a specific workload [1, 5].

For a team already using SQL and BI tools, an attractive design may be to ingest raw observations, transform them in Snowflake, and publish reusable metrics. Such a design still depends on good dimensional modeling, tested definitions, and clearly assigned ownership.

1.7.2 Limitations, pricing, and vendor lock-in

Costs require a workload model. Warehouse compute, serverless services, storage, transfers, and specialized features can be billed differently. A virtual warehouse billed while active is not equivalent to a free always-on process; storage and region choices also matter [2, 7]. Avoid publishing a fixed price per run without date, account assumptions, and measurement.

Operational dependencies matter too. Proprietary SQL features, security policies, managed orchestration, and some platform-specific AI functionality can increase migration cost. A customer may reasonably choose another platform when transactional latency, open multi-engine access, existing infrastructure, data sovereignty, or specific budget constraints dominate.

1.7.3 Fair comparison criteria for alternatives

Compare products using *the same* data volume, source conditions, transformation semantics, concurrency, security constraints, recovery objectives, and cost window. Specify which systems handle metadata and storage, which execute computation, and who must operate each component. Never compare a fully warmed cached query with an uncached competitor's query and call the difference a platform result.

Architecture decision record — a fair beginning
Context: daily Finnish energy reporting with a small team. **Options:** SQL warehouse, open-table lakehouse, or managed analytics platform. **Criteria:** reliable source integration, reproducibility, governance, BI compatibility, staffing, monthly credits, and exit strategy. **Decision:** not yet final. Prototype the smallest adequate design, measure it, and record why alternatives were rejected.

Failure to avoid. Buying capacity or complex streaming infrastructure before documenting a freshness requirement. In engineering, a simple solution that demonstrably meets a contract is often superior to an impressive architecture with unbounded maintenance obligations.

Explore the source project: P009: Advanced Data Engineering with Snowflake [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.8 Introducing the Finland Energy Analytics capstone

1.8.1 Stakeholders and business problem

We will build a *Finland Energy Analytics Platform*: a documented, testable data product that transforms licensed public energy observations into interpretable metrics. This is a teaching design, not an official Finnish energy statistic or a live production service. A system operator may care about timeliness; an analyst may care about comparability across years; a researcher may need provenance and reproducibility. Each needs a different kind of confidence in the same pipeline.

Before collecting data, define a narrow decision: *How did the composition and amount of reported electricity generation change over a chosen period?* We must select sources according to available variables and permitted reuse, rather than assume every public API exposes the desired measurements.

1.8.2 Energy generation/consumption indicators and time grain

A metric catalogue prevents arithmetic from becoming disguised business logic. At minimum, specify metric name, source, unit, grain, aggregation rule, missing-value behavior, and timezone. Power (MW) is not energy (MWh); power averaged over a time interval must be multiplied by that interval's duration in hours to obtain an energy contribution.

For time-series comparison, preserve original UTC timestamps and convert to local presentation time only with explicit daylight-saving handling.

Indicator	Unit / grain	Definition / caution
Mean power	MW / 15-min	Average reported power for each valid interval
Interval energy	MWh / 15-min	Mean MW multiplied by 0.25 h, *if* sample is an interval mean
Source freshness	Minutes / source	Publication time minus latest accepted event time
Completeness	Percent / day	Expected intervals received and accepted

1.8.3 Project roadmap across the book

The first milestone is a specification, not a database. Chapters 3–6 will establish an environment and load/normalize sources; Chapters 7–10 will compare and implement transformations; Chapters 11–13 will deliver metrics, governance, and operations; Chapter 14 will explore carefully evaluated AI extensions; Chapter 15 will integrate and assess the end-to-end result.

Scope boundary: In Chapter 1 all figures and sample readings are synthetic. Real Finnish data will be selected after verifying each provider's available variables, API terms, citation requirements, and download limits. The case study does not claim live grid coverage or forecasts.

Explore the source project: P010: Predictive Grid Maintenance [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

Explore the source project: P011: Green-Energy IoT Engineering [GitHub]. *External reference; not executed or licensed for reproduction as part of this chapter.*

1.L Guided lab: design the first data product

1.L.1 Prerequisites, inputs, permissions, and cost guardrails

Goal: turn the running scenario into three reviewable artifacts before creating a paid Snowflake account. Produce three files: `problem_statement.md`, `requirements.md`, and `architecture_context.svg`. Use the synthetic input below; no account, credential, warehouse, or live API is required.

Synthetic input assumptions
Each row represents *mean power* in MW for one 15-minute UTC interval. Rows are uniquely identified by the pair (timestamp, source). The values are invented solely to check a dimensional and arithmetic decision; they are not Fingrid or Statistics Finland measurements.

Listing 1: Synthetic sample for the offline lab (energy_sample.csv).

```
timestamp_utc,source,power_mw,minutes
2026-01-15T00:00:00Z,WIND,480,15
2026-01-15T00:15:00Z,WIND,500,15
2026-01-15T00:00:00Z,SOLAR,0,15
2026-01-15T00:15:00Z,SOLAR,0,15
```

1.L.2 Step-by-step specification

1. **Problem statement:** write one paragraph identifying the reader's question, decision, source categories, out-of-scope uses, and success criterion. Do not promise a live national statistic before identifying a source.
2. **Stakeholders:** specify an analyst, platform operator, and reviewer. Give each an explicit question and desired freshness.
3. **Metric catalogue:** define `interval_energy_mwh` as `power_mw` $\times$ `minutes`/60, under the mean-power assumption. Define completeness on an expected-interval denominator.
4. **Data contract:** require UTC timestamps, known units, positive duration, source IDs, uniqueness, and a decision about rejected records.
5. **Architecture:** draw source $\rightarrow$ ingest $\rightarrow$ raw $\rightarrow$ validated $\rightarrow$ curated metric $\rightarrow$ dashboard, adding trust boundaries and ownership.

Suggested first acceptance criteria: the metric formula is traceable; source and grain are unambiguous; duplicate (timestamp, source) keys are rejected; no private credentials appear in deliverables; and a reviewer can reproduce the synthetic calculation offline.

Operational guardrail: the chapter requires zero Snowflake credits. For later labs, establish a separate learning account, a least-privilege role, warehouse auto-suspension, budget alerts, and cleanup steps before running cloud services [7]. Budgets are notifications and monitoring tools, not a guaranteed hard ceiling for every cost category.

1.L Verify, review, and extend

1.L.3 A runnable offline check and expected result

Save the CSV from the previous page as `energy_sample.csv`. The following standard-library Python code checks uniqueness, positive durations, and the energy calculation. It has been designed to run without Snowflake; the included companion file is exercised separately from any cloud integration.

Listing 2: Synthetic energy validation (chapter01_lab.py).

```
import csv
from collections import defaultdict

energy = defaultdict(float)
seen = set()
with open("energy_sample.csv", newline="", encoding="utf-8") as f:
    for row in csv.DictReader(f):
        key = (row["timestamp_utc"], row["source"])
        assert key not in seen, f"Duplicate: {key}"
        seen.add(key)
        minutes = int(row["minutes"])
        assert minutes > 0, "Invalid interval"
        energy[row["source"]] += float(row["power_mw"]) * minutes / 60
print({k: round(v, 2) for k, v in sorted(energy.items())})
```

Expected output for the *synthetic* file: `{'SOLAR': 0.0, 'WIND': 245.0}`. A duplicate `WIND` timestamp should fail the uniqueness assertion. The code does not check licensing, true signal validity, or whether values are interval averages; those are separate checks. Production validation should raise explicit errors rather than rely on assertions that Python can disable.

1.L.4 Exercises and scoring rubric

Foundation: explain why 480 MW + 500 MW is not itself 980 MWh. **Practitioner:** add a rejected row with duration zero and describe the failure. **Production-design question:** specify how you would replay late corrections without double counting. **Scoring:** requirements 25%, metric semantics 25%, test evidence 25%, security/cost/limitations 25%.

Reviewer's acceptance checklist

- A business decision and primary reader are stated clearly.
- The metric catalogue distinguishes MW from MWh and names its time grain.
- UTC timestamps, duplicate identity, missing data, and source rights are addressed.
- Synthetic results are reproducible and plainly labelled synthetic.
- The architecture records alternatives, a freshness objective, and a cost boundary.

Chapter summary. Platform selection begins with the workload and its definition of correctness. Snowflake provides managed storage, compute, and cloud services, but the engineer must design the contracts, quality checks, operating model, and evidence that make results trustworthy. Chapter 2 follows a query through Snowflake's architecture.

SELECTED TECHNICAL REFERENCES

[1] Snowflake, Key Concepts and Architecture.

[2] Snowflake, Understanding Overall Cost. [3] Snowflake, Iceberg External Volumes.

[4] Snowflake, Hybrid Tables. [5] Snowflake, Secure Data Sharing.

[6] Snowflake, Snowpipe and Ingestion Choices.

[7] Snowflake, Controlling Cost.

Documentation consulted 21 October 2026; recheck editions, regions, features, licenses, and pricing before publication.

CHAPTER 02 / ARCHITECTURE

Snowflake Architecture: Storage, Compute, and Services

Follow one query, then decide what to scale.

At 09:01, a Finnish energy analyst opens an hourly-generation report. The query returns in 1.7 seconds. At 09:02, twelve colleagues refresh the same dashboard and several requests wait in a queue. The team could buy a larger warehouse, add compute clusters, change the table's organization, or improve the query. Each response has a different cost and attacks a different bottleneck.

The right choice depends on where the time was spent. Did Snowflake read too many micro-partitions? Was the data cached? Did the request wait for compute? Did an authorization check or compilation dominate? The words *storage*, *compute*, and *cloud services* are useful only when they help answer these questions. This chapter builds a precise mental model of those responsibilities and shows how to inspect evidence before making a change.

The central question
When a Snowflake workload becomes slow, expensive, or difficult to govern, which architectural layer is responsible—and which change is justified by the evidence?

Learning outcomes. By the end of the chapter you can:

- Draw the three-layer architecture and trace an authorized query from request to result.
- Explain columnar micro-partitions, metadata pruning, and why loading order can matter.
- Distinguish warehouse resizing, compute isolation, and concurrency scaling.
- Read a Query Profile without confusing a query-result cache hit with actual execution speed.
- Describe Time Travel, Fail-safe, retention limits, region dependencies, and table-type decisions.
- Produce an architecture decision record and a safe, reviewable performance investigation.

Prerequisites. Chapter 1 and basic familiarity with `SELECT`, `WHERE`, and `GROUP BY`. The architecture exercise has an offline route. Optional SQL investigation requires an appropriately configured Snowflake account and a low-cost, least-privilege test warehouse.

2.1 The three architectural layers

2.1.1 Storage, virtual warehouses, and cloud services

Snowflake's core model separates *persistent data*, *computing resources*, and *coordination services*. It is a managed cloud service, not a package installed on a local server. A table's persistent bytes are managed independently from a virtual warehouse that scans and transforms them. The cloud-services layer maintains much of the coordination required to authenticate a user, resolve metadata, compile a query, and dispatch its plan [1].

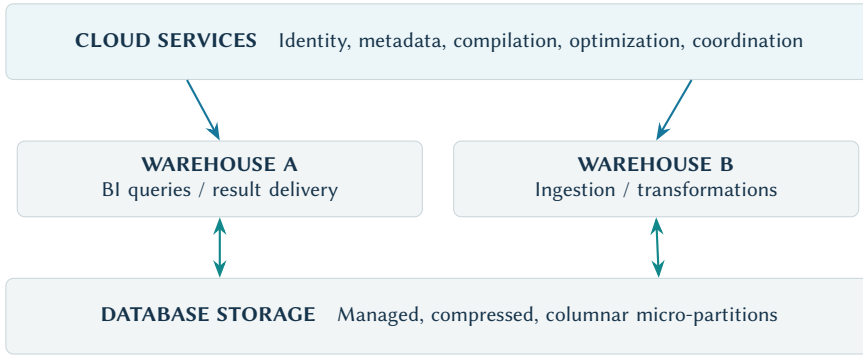


Figure 2.1. A conceptual view of Snowflake. The arrows explain responsibility, not private implementation internals or a guaranteed physical data path.

2.1.2 Responsibilities and relative isolation

Separate warehouses can use the same logical tables while having distinct compute resources and billing. This is valuable when a heavy transformation should not contend for the same execution slots as interactive reporting. It does *not* mean that all platform activity is isolated: warehouses still depend on common storage, account governance, metadata, and cloud services. Nor does it mean an idle warehouse can read data without any compute being used by a query; cached-result retrieval is a separate behavior.

2.1.3 Tracing a data and command flow

A simplified SELECT passes through authentication and authorization, object resolution, compilation/optimization, scheduling on a warehouse if execution is needed, columnar scanning and operators, and result delivery. Some requests may reuse an eligible persisted result without rerunning the original work. The important distinction is between *logical steps* and claims about proprietary internal execution order; Figure 2.1 depicts the former [1,5].

Decision check. If dashboard queries queue only when ELT begins, first test separate warehouses; if one filtered query reads most of a large table, inspect pruning before adding compute. **Failure mode:** concluding that storage and warehouse separation removes every shared dependency.

Project reference: P012: Data Engineering with Notebooks. *Not independently executed; rights review pending.*

2.2 Storage organization and micro-partitions

2.2.1 Micro-partitions and metadata

A standard Snowflake table is not a single row-oriented file. Snowflake stores its data in compressed, columnar micro-partitions and maintains metadata that helps avoid unnecessary reads. The documented typical size is roughly 50–500 MB of *uncompressed* data per micro-partition, not a fixed number of disk bytes. Metadata includes value ranges and distinct-value information that can inform pruning [2]. You do not allocate micro-partitions manually as you might create directories for date partitions in object storage.

Mental model. A micro-partition is a unit of organized columnar storage with useful statistics. It is not the same as a warehouse, a database schema, an external file, or the partition clause of an Apache Iceberg table.

2.2.2 Pruning and load-order effects

Imagine energy readings loaded in increasing event time. If the timestamp ranges of micro-partitions are relatively narrow, a query for one day may discard many other partitions using metadata. If data for that day is scattered across a large portion of the table, many micro-partitions may overlap the predicate. The load pattern and subsequent changes influence clustering; *there is no guarantee* that chronological ingestion always results in perfect pruning.

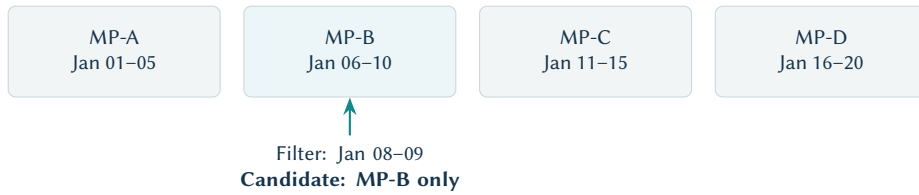


Figure 2.2. Illustrative non-overlapping date ranges. Real micro-partition ranges may overlap; the figure is not a measured Query Profile.

2.2.3 Compared with manually managed partitions

Unlike explicitly partitioned files in a data lake, the engineer does not usually write a `PARTITION BY` scheme for standard Snowflake tables. Instead, concentrate first on data types, selective predicates, load patterns, and observed scans. Clustering keys and automated reclustering are further tools, not automatic prerequisites for every table; maintenance has costs [2].

Failure mode. A filter such as `DATE(event_ts) = ...` may be easier to read but should not automatically be assumed to prune like a direct range predicate; compare query plans with your actual data. **Evidence to seek:** TableScan *Partitions scanned* against *Partitions total* and logical bytes read [6].

Project reference: P013: Getting Started with pandas on Snowflake. *Not independently executed; rights review pending.*

2.3 Virtual warehouses and compute isolation

2.3.1 Warehouse roles, start/stop, and auto-suspend

A virtual warehouse supplies compute for many SQL queries and certain processing jobs; it is distinct from the account that owns tables and roles. Assign different virtual warehouses to different workloads where isolation matters. A warehouse can be started, resized, and suspended. AUTO_SUSPEND and AUTO_RESUME help avoid leaving test compute active, but do not remove storage, cloud-service, serverless, or transfer costs [3].

Listing 1: Illustrative warehouse definition; execute only with approved privileges.

```
-- Run in a dedicated training account if authorized.
CREATE WAREHOUSE IF NOT EXISTS BOOK_CH02_XS
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE;
```

The command is a *proposed configuration*, not an executed lab result. In an existing account, use an already-provided test warehouse instead of creating resources. Verify current size names, billing minima, required grants, and organization policies before execution.

2.3.2 Isolation for BI and ELT

Suppose a nightly transformation and an interactive BI dashboard share a warehouse. A transformation with expensive joins may compete for memory and execution slots with dozens of short BI queries. Moving ELT to one warehouse and BI to another can protect the BI workload from direct compute contention. However, the second warehouse adds a second billable compute resource while running. The data is not duplicated merely by creating another warehouse [1,3].

2.3.3 Compute resizing versus expense

Scale up means using a larger warehouse cluster; it may speed CPU- and memory-bound queries. *Scale out* adds clusters to a multi-cluster warehouse, primarily to address concurrency and queueing, subject to edition and configuration. A single slow query may not become faster just because an extra cluster exists [3,4]. Always differentiate *query execution time*, *queue wait*, *startup time*, and *total user latency*.

First response to a slow dashboard
If the Query Profile shows large partition scans, test predicate and organization changes first. If it shows extensive spilling, a larger warehouse may be justified. If queueing rises only during bursts, consider workload isolation and then concurrency scaling. Never infer the cause from the dashboard's spinner alone.

Failure mode. Calling an X-Small warehouse "free" because it is configured to suspend. Active compute is billable; low idle cost is not zero total cost. More detail and cost safeguards follow in Chapters 3, 12, and 13.

Project reference: P014: Official Snowflake CLI. *Not independently executed; rights review pending.*

2.4 Cloud services, metadata, and query processing

2.4.1 Identity and authorization control

Before any meaningful query can return energy observations, Snowflake must recognize the requester and decide whether the active role may access the referenced objects. Authentication answers *who is making this request?*; authorization answers *what may the requester do?*. In the Snowflake model, privileges are typically assigned to roles and apply to named objects. Database access also depends on the relevant container privileges; a role with no table access should not obtain the table's rows merely because another warehouse is available.

Think of the architecture as two cooperating paths. The *control path* establishes session context, resolves objects, and authorizes operations. The *data path* executes an approved plan against the necessary storage using provisioned compute. The distinction helps explain why an authentication failure is not a data-scanning problem, and why adding compute cannot repair a missing grant.

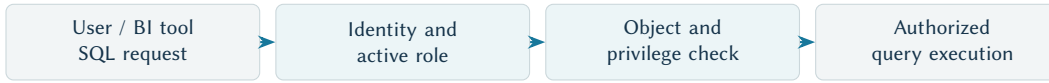


Figure 2.3. A simplified authorization boundary. Policies and specific object privileges can add checks not shown.

An architectural consequence. Shared data does not imply shared authority. The same storage may serve authorized analysts and transformation engineers while role hierarchies and governance policies constrain access. A secure architecture therefore documents *which identity runs each task*, which role it activates, and the precise privileges needed.

Failure investigation: permission versus performance
An "insufficient privileges" error is a configuration or policy issue, not evidence of a slow warehouse. Record the role, intended object, expected access, and observed error. Never solve a tutorial's access problem by casually switching every step to `ACCOUNTADMIN`.

Decision checkpoint. For a public energy dashboard, a narrow read-only role should see curated aggregates rather than necessarily all raw records. That boundary is more durable than relying on the BI tool to hide restricted columns. Section 13.2 provides role-hierarchy implementation details.

2.4 From SQL to a physical plan

2.4.2 Query compilation, optimization, and metadata

The SQL text is a request for a result, not a program that dictates each storage access. A query such as "return the average reported wind power for yesterday" can be implemented through different physical strategies. Snowflake resolves names and types, validates the expression, considers object metadata and statistics, and constructs an execution plan. Optimization concerns the amount of data to scan, the way operators are arranged, and how work is distributed; the private implementation may change across releases [1,6].

Listing 2: A deliberately small, semantically explicit analytical query.

```
SELECT source,
       AVG(power_mw) AS mean_power_mw
FROM BOOK_DEV.CH02_LAB.ENERGY_READING
WHERE event_utc >= '2026-01-15'::TIMESTAMP_NTZ
  AND event_utc <  '2026-01-16'::TIMESTAMP_NTZ
GROUP BY source;
```

This is *not* meaningful energy in MWh. An arithmetic average of MW readings becomes a valid measure of mean power only when sampling semantics support the calculation. Unequal intervals or missing samples require explicit weighting and completeness rules. The distinction carries forward from Chapter 1.

Conceptual execution path:

1. Resolve the database, schema, table, and the active session context.
2. Confirm privileges and determine whether an eligible persisted result can be reused.
3. If the query requires execution, compile and optimize operators using metadata and constraints.
4. Execute the plan, scan required columns and partitions, aggregate, and return results.
5. Record execution activity that authorized users can inspect through Query History and Query Profile.

> **Why selectivity is architectural**
> A precise date range may let metadata eliminate irrelevant micro-partitions before their bytes reach the query operators. This is a storage-organizational advantage expressed through the query plan. A faster result is therefore not necessarily evidence that the warehouse has become more powerful.

Common misconception. "The table has an index on the date." Standard Snowflake tables use automated micro-partition organization, not a user-managed B-tree index on each column. Hybrid tables have a different indexed storage engine and serve different workloads [1,11]. **Common diagnostic mistake.** Calling an optimized query slow based on one run that included resume/compile overhead, while comparing it with a result-cache hit on another run.

Project reference: P015: A Simple API Powered by Snowflake. *Not independently executed; rights review pending.*

2.4 Managed coordination has costs too

2.4.3 Managed services versus user-managed compute

Snowflake operates the cloud-services layer; the customer does not provision its underlying service machines. That does not mean coordination work is economically irrelevant. Metadata queries, compilation, access-control checks, and other services still consume platform resources and may contribute to cloud-services billing under the account's current terms. Compute charges for virtual warehouses, serverless features, data storage, and transfers are distinct cost categories [1,10].

A useful distinction is between *what the engineer configures* and *what the engineer operates*. You select warehouse sizes and choose SQL/data models; Snowflake maintains much of the underlying infrastructure and query service. Your team remains accountable for query semantics, role design, observability, recoverability, and the bill resulting from the architecture.

Concern	Managed by platform	Owner's decision
Persistent layout	Micro-partition encoding and metadata	Data model, table type, load pattern
Query planning	Parsing and optimization services	SQL semantics, filters, workload tests
Execution capacity	Warehouse infrastructure	Warehouse selection, size, suspend policy
Authorization	Role/privilege enforcement	Least-privilege design and assignments
Recovery	Supported retention mechanisms	Retention choice, backup and incident plan

A practical governance scenario. The Finland Energy Analytics project is run by one data engineer but read by many BI users. It is tempting to use a single high-privilege role for every script and dashboard. A better model assigns a deployment role to create approved objects, a transformation role to write curated tables, and a reporting role that reads only the required outputs. The specific grants must be tested later; what matters here is the boundary between responsibilities.

Cost reasoning. For a new job, record the number of calls, query durations, warehouse uptime, bytes transferred, and any use of managed serverless functionality. A *credit-per-query* estimate is incomplete when other billable services also contribute. No static USD-per-credit value should be treated as universal across regions and contracts.

Small architecture exercise

An analyst receives a permissions error, another user sees a ten-second warehouse resume delay, and a third user sees a three-second table scan. Assign each event to its primary layer and propose one observation that would confirm your diagnosis. The three cases require different remedies, even though every user describes the dashboard as "not working."

Project reference: P016: Getting Started with the Snowflake Python API. *Not independently executed; rights review pending.*

2.5 Query pruning, caching, and data access

2.5.1 Micro-partition pruning

The cheapest byte to process is often the byte never read. When a filter narrows the requested time range, Snowflake can use metadata to exclude micro-partitions that could not contain qualifying values. This is *pruning*. It differs from filtering rows *after* scanning: both can produce the correct answer, but the number of partitions read may differ substantially. Query Profile exposes TableScan statistics, including *Partitions scanned* and *Partitions total*, that help diagnose the difference [2,6].

Illustrative metric	Interpreting the observation
Scanned 12 of 1,200 partitions	Strong candidate for effective pruning; confirm query and bytes
Scanned 1,175 of 1,200	Filter eliminated little stored data; investigate necessity and layout
Scanned 0; returned instantly	May represent result reuse; verify rather than infer a fast scan

Those counts are made-up *examples*, not measured Snowflake outputs. Define a pruning ratio as $r = \text{scanned}/\text{total}$ only when total is nonzero. A lower ratio may be desirable for selective queries; it is not an overall performance score. One query may scan nearly every partition because the business genuinely asked for all history.

A predicate experiment. In an isolated test table containing many days of readings, compare a bounded UTC interval with a query for the entire table. Keep projection, warehouse, and session conditions as stable as practicable. Make sure the result is not served from a previously persisted result; then inspect the TableScan metrics. Do not extrapolate a large-data performance claim from a ten-row classroom table.

Listing 3: An explicit half-open UTC interval predicate.

```
WHERE event_utc >= '2026-01-01'::TIMESTAMP_NTZ
  AND event_utc <  '2026-02-01'::TIMESTAMP_NTZ
```

A half-open interval has a clear boundary, avoids constructing a last millisecond, and works naturally across adjacent periods. Its semantics still depend on storing and interpreting timestamps consistently: `TIMESTAMP_NTZ` does not itself carry a time-zone offset. Preserve the chosen UTC convention in the data contract.

Failure mode. Declare clustering keys without validating selectivity, write patterns, or maintenance cost. Begin with the workload: identify common filters and joins, then decide whether physical reorganization is justified.

Project reference: P017: Query Performance Deep-Dive. *Not independently executed; rights review pending.*

2.5 Three different meanings of cache

2.5.2 Result reuse and data/cache behavior

Snowflake users often say, "The second query is faster because it is cached," but several distinct effects can produce shorter response time. *Persisted query-result reuse* can return an eligible prior result without repeating execution. Separately, *warehouse data caching* can retain frequently used data on an active warehouse's local resources, reducing future remote reads. Finally, query compilation, initialization, and system load can vary between runs even without a cache hit [3,5].

Mechanism	What is reused?	Investigation question
Persisted result	Final earlier query result, when eligible	Did the query actually execute?
Warehouse cache	Accessed data on local warehouse resources	Was remote I/O reduced?
Plan and runtime state	Compilation/runtime conditions can vary	Were startup and workloads comparable?

Snowflake documents a normal 24-hour lifetime for persisted query results, subject to eligibility and reuse behavior; this is not a promise that every repeated query will hit the cache. Data changes, mismatched SQL text, permissions, non-reusable functions, and other conditions can defeat reuse [5]. A suspended warehouse may lose some or all useful local cache; consequently aggressive suspension can save credits but also alter performance. Neither rule justifies a universal recommendation without observing the workload.

Listing 4: Session-level control for a diagnostic, not a speed guarantee.

```
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
-- Run the target SELECT and inspect Query Profile.
-- Then restore the usual behavior:
ALTER SESSION SET USE_CACHED_RESULT = TRUE;
```

Why this matters. If the first execution takes 8 seconds and the next returns a previously persisted result in 0.2 seconds, the engineer has not demonstrated a 40-fold improvement in the query plan. The second execution may have bypassed the physical scan altogether. A fair benchmark labels which observations include cached results and which are actual executions.

Rule of evidence
A successful cache hit demonstrates effective result reuse for that request. It does not prove the underlying table organization, join strategy, or warehouse size is efficient for *new* queries.

Failure mode. Citing the existence of a cached result as an explanation for correctness. Caches must respect Snowflake's reuse and authorization conditions, but metric correctness still depends on the pipeline and data contract.

2.5 Designing a fair query experiment

2.5.3 Cache misses and fair comparisons

A performance experiment starts with one hypothesis, not a collection of buttons to press. For the Finland Energy Analytics workload, consider this testable statement: *a selective half-open timestamp predicate scans fewer partitions than the unbounded monthly aggregate on the same fixed dataset.* The observation is a partition count and scanned data volume; elapsed time is secondary until overhead and warehouse conditions have been controlled.

Experimental protocol:

1. Freeze or document dataset version, row count, timestamp grain, and table type.
2. Record account cloud/region, edition, warehouse size, query text, role, and query tag.
3. Use a dedicated test workload or note concurrent queries; disable persisted-result reuse for the specific diagnostic.
4. Repeat each query enough times to identify obvious outliers; keep warm and cold conditions separately labelled.
5. Inspect scanned/total partitions, bytes read, spill behavior, elapsed time, and queueing.
6. Confirm both queries deliver the correct *intended* answers before interpreting speed.

Correctness comes first. If one query computes average MW for 24 equal intervals and another computes sum of MW, their execution time is not a fair benchmark because their answers mean different things. Likewise, using UTC in one query and Finnish local time in another may change which intervals belong to the reporting day.

Log field	Why keep it?
`query_id`, UTC run time	Locate the exact history and profile
SQL hash / saved query	Establish that comparison text is controlled
Warehouse, size, cache state	Separate execution effects from environment effects
Partitions scanned / total	Quantify pruning for scans with relevant operators
Runtime and queue wait	Distinguish CPU/I/O work from capacity contention
Result checksum or invariant	Verify correctness independently of latency

A table with 800 rows may not have enough micro-partitions to show a useful pruning contrast. *A small laboratory is valuable for validating SQL behavior, not for establishing production performance.* A larger representative dataset and an isolated account are needed for meaningful optimization conclusions.

Diagnostic outcome. If both runs have low queue time but scan almost all partitions, add compute only after investigating data organization and filter usefulness. If pruning is efficient but one join spills to remote disk, examine warehouse memory and join strategy. If execution is short but elapsed time includes long queueing, consider workload isolation or concurrency capacity.

2.6 Concurrency and multi-cluster warehouses

2.6.1 Queueing and throughput under concurrency

A slow report can mean either *the query itself executes slowly* or *the query waits to begin*. Consider a classroom example: one warehouse can complete a particular analytical request in two seconds, but thirty users submit requests in a short burst. Finite execution slots mean some requests may queue, and end-to-end latency rises although an individual scan remains efficient. Capacity and scheduling must be measured instead of inferred from concurrent user count alone.

2.6.2 Multi-cluster use cases and edition limits

Multi-cluster warehouses can add compute clusters dynamically or use a fixed number of clusters to handle many concurrent requests. The feature requires Enterprise Edition or higher under the documented availability model [4]. Auto-scale policies balance faster response under load against extra credits. A sensible approach is to monitor queueing, reduce expensive or unnecessary requests, isolate workloads if appropriate, and then validate whether additional clusters improve the service-level objective.

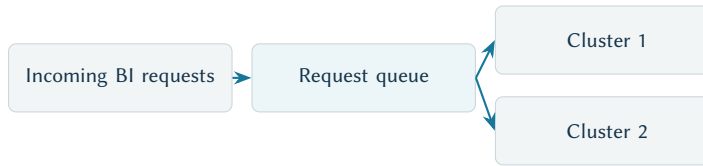


Figure 2.4. Conceptual distribution under concurrent load. A multi-cluster warehouse helps process *more requests*; it is not an automatic acceleration of one request.

2.6.3 Why more clusters do not guarantee faster single queries

If one large aggregation is CPU- or memory-bound, making *its assigned cluster* larger may help. Adding another cluster generally addresses simultaneous requests instead. Even scale-up can plateau if the request is restricted by poor pruning, network I/O, skew, or the available parallelism. Record at least median latency, tail latency, queueing, concurrency, and credits for the actual workload before and after change.

Scale the identified bottleneck
More memory and CPU per cluster may help a demanding single query. *More clusters* may help concurrency. *Better pruning or SQL* may reduce work for every cluster. These are competing investments, not synonyms.

Failure mode. Compare a busy weekday on a single cluster with a quiet Sunday on a multi-cluster setup and attribute the difference solely to the cluster count. Without controlled concurrency and cost, this is not convincing evidence.

Project reference: P018: Snowpark Python Performance Guide. *Not independently executed; rights review pending.*

2.7 Time Travel, Fail-safe, and data retention

2.7.1 Time Travel recovery paths

Storage architecture is also a recovery decision. If a transformation accidentally overwrites a table at noon, can the team inspect the previous state and restore it? Snowflake Time Travel can expose historical versions within an applicable retention window, including supported recovery operations on database objects. It is designed for *user-accessible historical querying and recovery*, not as a substitute for a complete backup, replication, or disaster-recovery strategy [7].

In the energy project, imagine that one daily load mistakenly interpreted missing values as zero. A query of the prior table state can help diagnose the change if that history remains inside the configured window. The responsible engineer should first pause the faulty process, identify the relevant change time or statement, and validate the historical result before restoring or copying data.

Listing 5: A read-only historical query; an eligible table and retention window are required.

```
-- Example only: inspect a table's prior state.
SELECT COUNT(*) AS row_count_before_change
FROM BOOK_DEV.CH02_LAB.ENERGY_READING
  AT (OFFSET => -60 * 30);
```

The example requests a point approximately thirty minutes earlier. Its success depends on current retention, privileges, object type, and the table's history. It is not executed in this manuscript, and the existence of an earlier version must not be assumed. A recovery operation should be rehearsed on test objects before being applied to production data.

Recovery question	Evidence required
When did wrong data appear?	Load/job history and first incorrect output
What was the last correct state?	Time Travel query or trusted source snapshot
What else changed after that?	Downstream writes, consumers, and dependencies
Can we recover safely?	Retention availability, restore plan, validation tests

Recovery principle. An impressive UNDROP command does not guarantee restoration of every business outcome. Materialized reports, external consumers, cached extracts, and downstream updates may need separate correction. Runbooks should define owners, communication, and repeatable validation.

2.7 Retention is a design parameter

2.7.2 Fail-safe scope and operational boundaries

Fail-safe and Time Travel solve different problems. Time Travel is a customer-facing historical access feature inside configured limits. Fail-safe is a Snowflake-operated recovery mechanism for eligible permanent-table history after Time Travel, with a seven-day period; it is not an ordinary query interface or an on-demand replacement for Time Travel. Snowflake Support involvement and recovery scope differ from self-service restoration [7,8].

2.7.3 Retention choices by object type and edition

Under documented standard-table behavior, permanent tables have a seven-day Fail-safe period. Standard Edition generally supports a one-day Time Travel period; Enterprise Edition and above permit eligible longer retention, up to 90 days. Transient and temporary tables have at most one day of Time Travel and no Fail-safe; temporary-table lifetime is also tied to session behavior [7,8]. These rules have object-type and feature-specific exceptions, so check the current documentation for the specific asset before relying on a retention number.

Table category	Time Travel (typical)	Fail-safe / caution
Permanent, Standard	0–1 day configured	7 days; historical storage may be billed
Permanent, Enterprise+	0–90 days when supported	7 days; longer history increases storage exposure
Transient	0–1 day	No Fail-safe; unsuitable as only protection for valuable data
Temporary	0–1 day while eligible	No Fail-safe; session lifecycle constrains recovery

Retention has an economic dimension. Old versions and deleted data can consume storage. Selecting the longest permitted period for every large intermediate table is not necessarily the safest or most economical architecture. Choose retention according to recovery-point requirements, ability to recreate data from source, legal obligations, and storage cost [8].

Incident design exercise
A curated daily metric table must be recoverable after accidental modification discovered three days later. A one-day Time Travel policy alone does not satisfy that user-accessible recovery requirement. Propose a supported longer-retention option when eligible, plus source snapshots or another recovery mechanism and a tested process. State which options depend on account edition.

Failure mode. Calling Fail-safe a standard self-service backup; choosing transient raw tables without ensuring the raw files can be replayed; assuming an Iceberg table stored outside Snowflake has exactly the same protection as a permanent native table.

Project reference: P019: Time-Travel Table dbt Package. *Not independently executed; rights review pending.*

2.8 Cloud regions, deployment models, and limitations

2.8.1 Cloud provider, deployment region, and data residency

A Snowflake account resides in a selected cloud-provider region. That decision affects available capabilities, latency to producers and consumers, data movement, resilience options, and applicable organizational or legal constraints. *Data residency* is not established merely by writing the word "Finland" in a schema name. The engineer must identify where bytes are stored, where computations run, which connected services receive information, and whether replication or external integrations cross borders [9,10].

For a Finland-based publisher's example project, the geographical name refers to the *subject of the measurements*; it does not guarantee the data is hosted in Finland. A realistic architecture record separately states: source-system region, Snowflake account region, external storage location, BI user regions, and any cross-region processing. For personal or regulated data, the choice must be assessed against applicable policies and legal obligations; public energy data may have different constraints.

2.8.2 Availability differences across regions

The name "Snowflake" does not guarantee every table type, external integration, model, or AI feature is usable in every account. Features may depend on cloud, region, edition, preview enrollment, or contract. For example, Snowflake's published limitations describe hybrid-table availability in commercial AWS and Azure regions, with restrictions for other environments and trial accounts [11]. Such boundaries are subject to change and need a dated check when the book is finalized.

Deployment decision	Questions to record
Region and provider	Where must data reside; where are users and source systems?
Edition and access	Does the exact feature exist for this account and role?
External integrations	Which credentials, endpoints, regions, and trust boundaries?
Continuity design	Which region failure must be tolerated; what recovery objectives?

A useful design rule
Write the region/edition compatibility matrix *before* choosing a critical feature. A hands-on book must never require an unavailable feature without offering a lower-cost, region-compatible fallback.

Failure mode. Replicate data to a different region to improve availability without checking who is allowed to receive the data, how it will be recovered, and what recurring transfer costs arise.

Project reference: P020: Snowflake MCP and Bedrock AgentCore Guide. *Not independently executed; rights review pending.*

2.8 Locality, transfer, and environment separation

2.8.3 Data transfer costs and environment separation

Moving data between services is not always a zero-cost internal operation. Compute in Snowflake may be billed separately from storage and from relevant network or cross-region transfer activities. When Iceberg tables use external volumes, costs and responsibilities depend on who owns the storage, what crosses cloud or region boundaries, and which engine reads or writes files [9,12]. Always consult current documentation and account terms instead of treating one provider's example cost as a universal tariff.

Example: a cross-region architecture. A source system writes Parquet files to a cloud bucket in Region A. Snowflake computes in Region B while BI users are mainly in Region C. The analyst sees only a table; the engineering team must examine three different kinds of locality: data-location and access permissions, physical transfer flows, and the placement of compute. Moving the BI endpoint alone will not necessarily remove storage transfer costs.

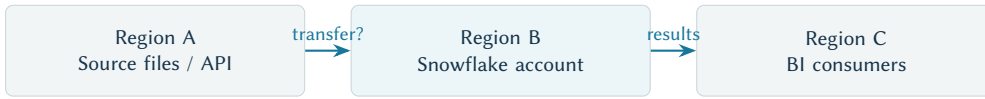


Figure 2.5. A conceptual regional dependency map. Charges and restrictions vary with operation and service; not every arrow incurs the same Snowflake fee.

Separate environments. Development, test, and production should not share unrestricted operational authority merely for convenience. Physical separation might use schemas, databases, separate warehouses, accounts, or more than one of these. The appropriate level follows the sensitivity and blast-radius requirements. A learning lab should never overwrite production objects because a default database happened to be selected.

Energy capstone checks:

1. Are we using only public, correctly licensed inputs or any identifiable operational data?
2. Where do each source, managed table, optional external volume, and dashboard operate?
3. Can this run in a sandbox without unexpected service charges?

Deployment evidence to retain
Cloud/provider/region; feature availability; least-privilege connectivity; source and destination locations; data-reuse license; cross-border processing; expected credit, storage, and network categories; account configuration and date checked.

Project reference: P021: Openflow Deployment Guide. *Not independently executed; rights review pending.*

2.9 Reference architectures and design trade-offs

2.9.1 Batch-first versus streaming designs

A well-designed architecture is proportional to its requirements. For a validated daily electricity report, scheduled file/API ingestion and an incremental transformation may be sufficient. A near-live operations screen might justify a different ingestion approach. Neither choice is inherently superior: freshness, replay, observability, error recovery, and cost must be considered together. Do not choose a stream merely to make the architecture diagram look modern.

2.9.2 Snowflake-managed versus external storage

Standard Snowflake tables use platform-managed columnar micro-partitions. Iceberg tables adopt an open table format and can be backed by Snowflake-managed storage or by customer-managed external volumes, with responsibility and recovery differences. External files may make multi-engine integration attractive but also introduce catalog, access, lifecycle, and egress questions. Evaluate the *whole operating model*, not only the nominal storage price [1,12].

2.9.3 Native, Iceberg, and hybrid table considerations

Hybrid tables use an indexed row-oriented engine for low-latency reads and writes and have different feature/region limitations. They should not be marketed as drop-in replacements for scan-optimized analytical tables. Conversely, open table formats may be valuable where portability and interoperability matter more than tightly integrated managed behavior [1,11,12].

Choice	Good first reason	Main design caution
Standard Snowflake table	Managed SQL analytics and transformation	Proprietary storage organization; compute/cost model
Iceberg table	Open-format or external-lake interoperability	Catalog/storage ownership and feature scope
Hybrid table	Indexed operational reads/writes	Region/edition, workload fit, operational limitations

Reference decision for the capstone
Use a *batch-first, managed standard-table architecture* as the starting hypothesis for the Finland Energy Analytics Platform. Keep raw-source provenance and a reproducible transformation path. Treat streaming, external Iceberg storage, and hybrid tables as alternatives to be justified by new requirements, not mandatory components of a basic energy dashboard.

Trade-off checkpoint. The recommended baseline minimizes deployment dependencies and leaves room to test BI performance. It is not a proof that this architecture is globally cheapest or fastest. The exact API source, daily interval counts, regional setup, licensing, and cost must be determined before implementation.

Project reference: P002: Snowflake Iceberg Tables Guide. *Not independently executed; rights review pending.*

Project reference: P022: Data Application with Hybrid Tables. *Not independently executed; rights review pending.*

2.L Guided lab: follow an analytical query

2.L.1 Prerequisites, permissions, and cost controls

Goal. Draw the capstone's architecture and inspect a Snowflake Query Profile *if* an authorized training account exists. The offline route needs only a text editor and the chapter's supplied data contract; the optional SQL route uses a dedicated test role, an existing X-Small training warehouse, and a new throwaway schema. Never run the SQL against a production database or with broad default privileges.

Cost and security checkpoint
Ask an administrator to provide a sandbox database and a role authorized for schema and table creation only within it. Confirm current warehouse auto-suspend and account cost alerts. End the experiment by suspending the training warehouse if your role and shared-user policy allow it. No hard-coded password, API key, personal data, or customer dataset is required.

2.L.2 Step-by-step procedure and a bounded sample query

Offline path. Copy the conceptual architecture in Figure 2.1 to `architecture_v1.svg`. Add an external source, UTC validation and quality gate, a raw table, a curated summary table, and a BI reader. Label separate BI/ELT warehouses as *proposed*, identify each trust boundary, and annotate which layer owns metadata, persistence, and query execution.

Optional Snowflake path. In an approved development database, run the companion SQL script after filling in safe identifiers. It uses a small synthetic ENERGY_READING table with a known UTC day, and produces counts and aggregates for a query you can find in Query History. The extract below is a representative core, not an assertion of observed execution.

Listing 6: Core of the optional teaching query; synthetic records only.

```
ALTER SESSION SET QUERY_TAG = 'BOOK_CH02_LAB';
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
SELECT source, COUNT(*) AS intervals,
       ROUND(AVG(power_mw), 1) AS mean_power_mw
FROM BOOK_DEV.CH02_LAB.ENERGY_READING
WHERE event_utc >= '2026-01-15'::TIMESTAMP_NTZ
  AND event_utc <  '2026-01-16'::TIMESTAMP_NTZ
GROUP BY source ORDER BY source;
```

Locate the statement in *Query History* using the session or query tag, open *Query Profile*, and record the Query ID, execution time, queue time, warehouse, any TableScan, partitions scanned/total, and cache status where displayed. A tiny test table may not show useful pruning: that absence is an expected limit of the lab, not a failure.

Deliverables. Architecture SVG, SQL script, and `tradeoffs.md`.

The companion README explains setup, cleanup, and expected output.

2.L Validate, decide, and review

2.L.3 Expected results, limitations, and cleanup

For the companion's four illustrative fifteen-minute interval readings, the expected row counts are `SOLAR=2` and `WIND=2`, with mean powers `SOLAR=0.0 MW` and `WIND=490.0 MW`. These values follow from the *synthetic* rows; they are not observations from a live Snowflake run. The query is not intended to estimate total national electricity production. Query-profile timing, partitions scanned, and total credits are *unknown until your actual lab runs*; never replace them with invented screenshots.

If setup fails, record the error and active role; do not elevate privileges without authorization. If a requested object already exists, stop and check ownership rather than overwrite it. Restore `USE_CACHED_RESULT`, clear the query tag if appropriate, and remove only objects that the lab created in its dedicated namespace. In a shared account, obtain approval before warehouse suspension or teardown.

2.L.4 Exercises and scoring rubric

Foundation: label the storage, compute, and service responsibilities in a query path. **Practitioner:** describe why one cached run cannot establish an optimization claim; write a controlled comparison plan. **Production design:** evaluate whether to separate BI and ELT compute for thirty users and a nightly transformation, including cost and recovery risks.

Acceptance checklist and rubric

- **Layer model (25%):** the diagram assigns responsibilities correctly and marks trust boundaries.
- **Evidence (25%):** Query ID and relevant metrics are recorded or explicitly marked not run.
- **Decision quality (25%):** the ADR compares at least two plausible options and names constraints.
- **Safety and honesty (25%):** no credentials or claimed measurements without tests; cleanup is bounded.

Chapter summary. Persistent storage organization, execution capacity, and managed cloud services cooperate but address different problems. Query Profile guides changes; cache and queue effects must be separated; recovery retention and region/edition limits shape architecture. The correct answer is not *always use more compute*, but *identify the controlling constraint and document why the selected design fits.*

SELECTED TECHNICAL REFERENCES (DOCUMENTATION CHECK: OCTOBER 2026)

[1] Snowflake, Key Concepts and Architecture. [2] Micro-partitions and Clustering.
[3] Warehouse Considerations. [4] Multi-cluster Warehouses.
[5] Persisted Query Results. [6] Query History and Profile.
[7] Time Travel and Fail-safe. [8] Historical Storage Costs.
[9] Cross-region Data Sharing. [10] Data Transfer Costs.
[11] Hybrid Table Limitations. [12] Apache Iceberg Tables.

CHAPTER 03 / DEVELOPMENT ENVIRONMENTS

Setting Up Snowflake and Development Environments

A safe first environment is a design decision, not an administrative afterthought.

It is Friday afternoon. An engineer has a promising energy dashboard and wants to test one more transformation before sharing it with colleagues. The fastest option appears to be an unrestricted account, a warehouse that never suspends, and SQL copied directly into a browser worksheet. The demonstration works. On Monday, however, nobody can explain who owns the tables, how much the experiment cost, why a colleague can change the raw data, or which version of the SQL created the results.

The mistake was not a missing Snowflake feature. The team had built a *working query* without building a *repeatable environment*. This chapter establishes a small, inspectable workspace for the Finland Energy Analytics project. We shall separate administration from everyday use; treat credentials as secrets; make data objects predictable; and design clean teardown before the first chargeable query.

The design question
How can another engineer reconstruct our environment, run only the work they are authorized to run, establish that it behaves correctly, and shut it down without risking production data or uncontrolled charges?

By the end of this chapter you will be able to:

- distinguish organizations, accounts, editions, regions, and trial limits;
- navigate Snowsight and record reproducible session evidence;
- provision an X-Small test warehouse, database, schema, and small sample table;
- grant least-privilege reader access and verify a denied operation;
- connect Snowflake CLI and Python without committing passwords;
- distinguish budgets from warehouse resource monitors; and
- produce reviewable bootstrap, verification, and cleanup artifacts.

Prerequisites. Chapters 1–2 and elementary SQL. A Snowflake account is optional for the *offline* design exercises; execution of cloud SQL requires an authorized account, billable credits or trial allowance, and a suitably privileged administrator. All displayed cloud outputs are *expected examples*, not claims of a live run.

3.1 Accounts, organizations, editions, and regions

3.1.1 Organizations versus accounts

A Snowflake *organization* groups one or more accounts under an administrative umbrella. An *account* is a distinct Snowflake environment with its own users, roles, warehouses, databases, billing and administrative settings. It belongs to a cloud provider and geographical region. These terms are not interchangeable: a database is an object *inside* an account, whereas an organization can span several accounts. A typical account identifier is formed from an organization name and an account name; copy the authoritative identifier from the account interface rather than constructing one from memory [1].

An educational account should be logically distinct from a company's production workload. Separate accounts provide stronger boundaries than merely prefixing every database name with DEV; they also require distinct identities, billing oversight, deployment routes, and potentially data replication controls.

3.1.2 Choosing edition and region deliberately

Choose the cloud provider, region, and Snowflake edition *before* creating tables. Edition choices can change feature availability, security capabilities and credit rates; region selection affects service availability, network latency, regulatory requirements, and transfer costs. A Finnish electricity dataset being public does not automatically make any processing region appropriate for private data that may later be joined to it.

Selection record for the running project
Account purpose: isolated book laboratory. Region: a supported EU region selected from Snowflake's current availability list. Edition: the lowest approved edition supporting the exercises. Credentials: institutional sign-in or appropriate protected identity. Financial owner: explicitly named. Trial status: recorded before any resource creation.

3.1.3 Trial restrictions and cost estimate

Snowflake documents a time-limited trial whose current terms and restrictions must be read before use. In the documented October 2026 snapshot, the standard self-service trial ends after 30 days or after its free balance is exhausted; some features, including AI services, may have extra requirements, and trials can exclude certain features [2]. An invitation to experiment is not a promise that every chapter lab is available without charge.

Initial cost is driven by *active compute time*, relevant serverless services, stored bytes and data movement. A suspended virtual warehouse is not a whole-account spending stop. Begin with an X-Small warehouse, short bounded queries, and a written spending ceiling. **Decision checkpoint:** before following a tutorial, confirm its region, edition, identity requirements and estimated credit exposure.

Project reference: P004: Getting Started from Zero to Snowflake. *Not independently executed; rights review pending.*

3.2 Getting started with Snowsight

3.2.1 Interface orientation and object browser

Snowsight is Snowflake's web interface for exploring data and working with worksheets, monitoring screens, and other account features. Layout and menu names can change, so use a workflow rather than memorizing screen coordinates. Start by inspecting the account name and region; then locate the current role, warehouse, and data objects. Confirm each context item before copying SQL from a tutorial into a shared account.



Figure 3.1. The safe inspection sequence; conceptual workflow, not a screenshot of a specific Snowsight release.

3.2.2 Worksheets and Query History

A worksheet is useful for exploration but not sufficient as the only record of a production-minded change. Assign a descriptive title, keep commands in a companion SQL file, and run an identity/context check before making changes. Start with statements that require no warehouse execution:

Listing 1: Read-only session identification; actual result depends on your account.

```
SELECT CURRENT_ACCOUNT(), CURRENT_REGION(),
       CURRENT_ROLE(), CURRENT_WAREHOUSE(),
       CURRENT_DATABASE(), CURRENT_SCHEMA();
```

Query History can help identify a statement, its execution state and duration, the active warehouse and an inspectable query identifier, subject to your role and the account's retention and visibility rules. Do not treat Query History as the sole source-control or long-term audit mechanism: it cannot replace a reviewed commit, a release tag, or recorded test evidence [3].

3.2.3 Capture evidence without leaking information

If screenshots are needed, label them with the capture date, relevant UI version if available, role and feature context. Redact account identifiers, email addresses, customer information and tokens. Preserve *what was observed* separately from *what the book expects a new reader to observe*. A changed button location is a minor editorial update; a changed authorization rule is a technical correction.

Failure mode. A user runs `CREATE TABLE` in the wrong database because a worksheet inherited an earlier context. **Prevention:** check context and use fully qualified names in deployment scripts. **Success check:** another engineer can identify the environment and locate the SQL file that produced the worksheet result.

Project reference: P023: Snowpark in Worksheets and Notebooks. *Not independently executed; rights review pending.*

3.3 Databases, schemas, tables, and warehouses

3.3.1 The object hierarchy and its consequences

The chapter's demonstration uses the following nested identifiers: BOOK_CH03_DEV for the database, LAB for its schema, and ENERGY_READING for one small table. This makes it possible to distinguish objects from session defaults. It also gives cleanup an explicit, narrow scope: remove only these objects *if they were created for this lab and have no other owners or consumers.*

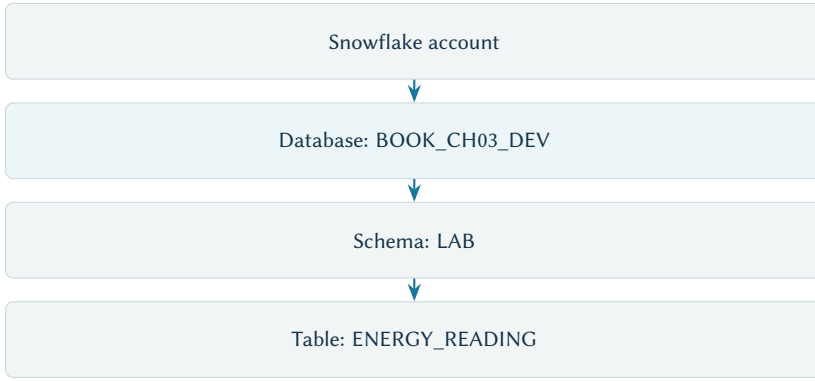


Figure 3.2. Logical containment; the virtual warehouse is account-level compute, not a child of the database.

A table is not the same thing as the warehouse that executes queries against it. Creating a database or schema is largely a metadata operation; scanning table data typically uses compute. Object creators commonly become owners, which determines who can later change objects and manage grants. The final access model must be established deliberately, not inferred from whichever administrator created the first objects [4].

Listing 2: DDL excerpt for a dedicated training account. Administrator review required.

```
-- Prerequisite: an approved provisioner role.
CREATE DATABASE IF NOT EXISTS BOOK_CH03_DEV;
CREATE SCHEMA IF NOT EXISTS BOOK_CH03_DEV.LAB;
CREATE TABLE IF NOT EXISTS
  BOOK_CH03_DEV.LAB.ENERGY_READING (
    OBS_ID NUMBER(38,0),
    SOURCE VARCHAR(40),
    EVENT_UTC TIMESTAMP_NTZ,
    POWER_MW FLOAT
  );
```

Idempotency caveat. IF NOT EXISTS prevents a duplicate-object error; it does not validate that an existing object's columns, owner, retention policy or grants match this lesson. Compare actual definitions before declaring the environment reproducible. A production migration system handles drift explicitly.

3.3 Ownership, naming, and predictable context

3.3.2 Naming conventions and object ownership

Names are operational controls. Prefer plain uppercase, unquoted identifiers for this lab. Use a stable purpose prefix, an environment suffix, a schema with a clear boundary, and descriptive columns with units. Avoid ambiguous names such as `DATA`, `TABLE1`, or `TEMP_FINAL2`. Establish consistent conventions for object name length, plurals, timestamps, and measurements. For example, `POWER_MW` is preferable to `POWER` because it communicates the quantity and unit.

Proposed naming policy
Account: organization-governed and environment-specific. Database: `BOOK_CH03_DEV`. Schema: `LAB`. Table: `ENERGY_READING`. Warehouse: `BOOK_CH03_XS`. Reader role: `BOOK_CH03_READER`. Query tag: `book/ch03/manual`. Every resource is disposable only when the provisioning manifest names it as such.

Ownership should not be conflated with access to read data. A role with `OWNERSHIP` can exercise much broader control than a role with `SELECT`; an engineer who only analyzes readings should not also own the raw table. In a normal schema, the owner controls grants on objects it owns, but a managed-access schema changes who may grant privileges. Avoid universal recipes that omit schema mode or session role [4].

Operational rule. Use the privileged provisioning role only to establish resources. Analysts should switch to narrower reader roles for routine SQL. A separate cleanup approver should confirm which names can be dropped. This keeps administrative actions out of screenshots and tutorial examples addressed to everyday users.

Context inspection before every write

The following query is a low-cost sanity check for role and namespace, but its expected output depends on the active session:

Listing 3: Context checkpoint for the exercise.

```
SELECT CURRENT_ROLE() AS active_role,
       CURRENT_DATABASE() AS active_db,
       CURRENT_SCHEMA() AS active_schema,
       CURRENT_WAREHOUSE() AS active_wh;
```

For an authoring workflow, add a *human-readable evidence record*: date, account alias, purpose, expected role, SQL file checksum, reviewer and acceptance result. Do not store secret account credentials in the record.

Failure mode. `CREATE OR REPLACE TABLE` destroys an unexpectedly named object. **Safer alternative.** In instructional code use `CREATE TABLE IF NOT EXISTS` and a preflight schema check; reserve replacement for consciously reviewed migrations and test environments.

Project reference: P016: Snowflake Python API Getting Started. *Not independently executed; rights review pending.*

3.3 Compute that stops when the experiment does

3.3.3 Small warehouses and auto-suspend

A virtual warehouse executes many compute-bound SQL operations. For a short first lesson, an X-Small warehouse is an appropriate starting *candidate*, not an assertion that all accounts or workloads can execute economically at that size. Configure auto-resume for convenience and auto-suspend to reduce time spent running while idle. A user-managed warehouse can still incur compute charges when it resumes, including minimum billing behavior documented for warehouse startup [5].

Listing 4: A bounded virtual warehouse with a suspended initial state.

```
CREATE WAREHOUSE IF NOT EXISTS BOOK_CH03_XS
  WAREHOUSE_SIZE = 'XSMALL'
  AUTO_SUSPEND = 60
  AUTO_RESUME = TRUE
  INITIALLY_SUSPENDED = TRUE
  COMMENT = 'Disposable training warehouse';
```

Never assume the command altered an existing warehouse: `IF NOT EXISTS` does not reconcile its size or suspension settings. Use `SHOW WAREHOUSES` and inspect the matching resource settings. If an existing warehouse does not match, obtain approval before changing it. If the training account provides a managed shared warehouse, reuse it instead of allocating an additional one.

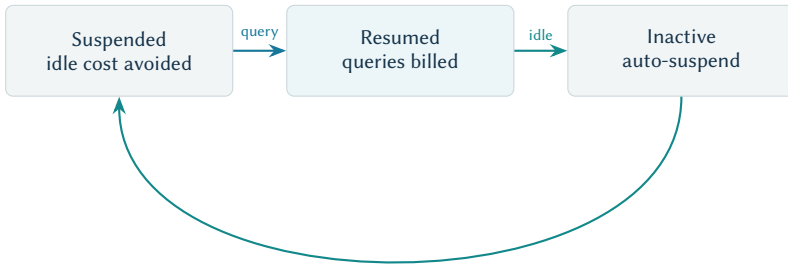


Figure 3.3. Warehouse lifecycle; conceptual only. Compute stop does not suspend independent serverless services or remove stored objects.

Risk check. A warehouse with `AUTO_SUSPEND=0` may remain active indefinitely; a training resource should not be configured this way without explicit justification. A 60-second auto-suspend does not mean a one-second query always costs one second. Billing, caching, and background service cost require separate inspection.

> **Performance is not the first success criterion**
> This chapter's success is a known role, known cost control, repeatable table definition and safe cleanup. Later chapters will investigate Query Profile, ELT execution and performance using real measurements.

Evidence: size, suspension settings, administrator approval.

Project reference: P016: Provisioning Objects with the Python API. *Not independently executed; rights review pending.*

3.4 Users, roles, and basic grants

3.4.1 Role-based access and the grant chain

Snowflake access control separates a *user identity* from a *role* that carries privileges. A typical reader needs USAGE on the compute warehouse and the appropriate containing database and schema, plus SELECT on the table. A privilege on the warehouse alone does not authorize data access. Conversely, having table SELECT without available compute is generally insufficient to run a scan [4].

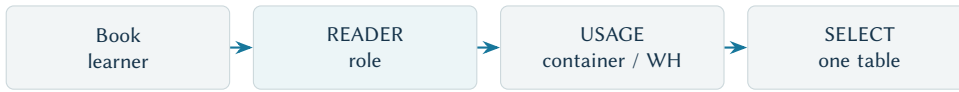


Figure 3.4. A least-privilege reader path; arrows express the teaching concept, not a literal internal authorization sequence.

3.4.2 Execution roles versus ownership roles

Separate *provisioner*, *reader*, and any future *writer* permissions. An approved security administrator can create the reader role; an authorized object owner or grant manager can assign object privileges. The grant sequence below is a *provisioning example*, not SQL for unprivileged users to run. Role assignment to named users must follow an identity review.

Listing 5: Administrator-authorized role and object grants.

```
-- Security administrator or approved delegate:
CREATE ROLE IF NOT EXISTS BOOK_CH03_READER;
-- Authorized grant manager, after lab objects exist:
GRANT USAGE ON WAREHOUSE BOOK_CH03_XS
  TO ROLE BOOK_CH03_READER;
GRANT USAGE ON DATABASE BOOK_CH03_DEV
  TO ROLE BOOK_CH03_READER;
GRANT USAGE ON SCHEMA BOOK_CH03_DEV.LAB
  TO ROLE BOOK_CH03_READER;
GRANT SELECT ON TABLE
  BOOK_CH03_DEV.LAB.ENERGY_READING
  TO ROLE BOOK_CH03_READER;
```

3.4.3 Negative tests matter

With the reader role active and assigned, the learner must be able to SELECT from the sample table but should *not* be able to INSERT into it or create new tables. Use a read-only negative test when possible, such as checking documented grants; a deliberately denied CREATE TABLE should be executed only in an isolated sandbox after approval. **Do not** use ACCOUNTADMIN to "fix" a missing student permission.

A role can inherit privileges from other assigned roles, and secondary role behavior can affect access. Before interpreting a successful negative test, capture the primary and any active secondary roles. An expected denial is useful evidence only when the intended identity context is known.

Project reference: P024: Terraform Snowflake RBAC Infrastructure. *Not independently executed; rights review pending.*

3.5 SQL worksheets, Snowflake CLI, and Git

3.5.1 Worksheets for exploration; files for the durable record

Worksheets lower the barrier to investigating a table, testing a WHERE clause or evaluating a CREATE statement. Their weakness is not their interface; it is the temptation to let the worksheet become the only source of truth. A colleague cannot review a reproducible deployment from a screenshot of a successful command. The book therefore stores the *same SQL* in named files, with comments explaining prerequisites and acceptance criteria.

A useful working directory is:

Listing 6: Proposed Chapter 3 companion-lab repository.

```
chapter03-environment/
  README.md
  sql/
    01_bootstrap.sql
    02_grants.sql
    03_verify.sql
    99_cleanup.sql
  python/connection_probe.py
  .env.example
  .gitignore
  evidence/   # no credentials / private screenshots
```

Make execution explicit. A file that creates a database is not safe to run under every role or account. Each SQL file should declare its intended environment, approval authority, expected object names, impact, rollback procedure and related documentation. Keep identity-specific overrides out of version control.

A reviewable change, not merely an executed change
`01_bootstrap.sql` defines the lab resources. `02_grants.sql` is reviewed by an access administrator. `03_verify.sql` asserts the intended role and data visibility. `99_cleanup.sql` removes only named resources after a separate approval check. Evidence records both the script revision and observed results.

Version the interpretation. If a lab step works today but fails next year because a CLI option has changed, a source-control tag and documented tool version make the discrepancy diagnosable. Update the book's *companion release*, not silently overwrite an old edition with a new command. The history is part of the educational value.

Failure mode. A worksheet contains both exploratory cleanup and a valid production query; exporting and executing everything as one file damages unrelated objects. **Prevention:** separate read-only verification, controlled provisioning and destructive teardown into distinct reviewed files.

Project reference: P025: Official Snowflake CLI Templates. *Not independently executed; rights review pending.*

3.5 A secure CLI workflow

3.5.2 Configuring Snowflake CLI

Snowflake CLI provides a local `snow` command for testing a connection and executing SQL files. It is a client, not an extra data warehouse. Start with the officially maintained CLI installation instructions appropriate to your operating system and record the installed version. The current documentation describes configuration in `config.toml` and, for shared developer-tool connections, `connections.toml`; parameters may be overridden by the environment [6].

For a human using organizational single sign-on, a connection with an external-browser authenticator may be appropriate *if supported by the identity policy.* A local configuration example *without secrets* is:

Listing 7: Illustrative CLI config.toml; adapt account and user locally.

```
[connections.book_dev]
account = "your_org-your_account"
user = "your_login"
authenticator = "EXTERNALBROWSER"
role = "BOOK_CH03_READER"
warehouse = "BOOK_CH03_XS"
database = "BOOK_CH03_DEV"
schema = "LAB"
```

Do not commit a `config.toml` containing session tokens, password fields or locally identifying connection information to a public repository. Protect the configuration file's filesystem permissions. Automated CI usually requires a different, explicitly authorized noninteractive authentication method (for example, managed federation or a protected key), with its privileges no broader than the job requires [6].

Listing 8: Documented CLI command forms; requires approved account access.

```
snow --version
snow connection test --connection book_dev
snow sql --connection book_dev \
  --query "SELECT CURRENT_ROLE()"
snow sql --connection book_dev \
  --filename sql/03_verify.sql
```

The first line runs locally. The remaining lines contact Snowflake and may open browser authentication or resume compute for certain statements. Do not interpret a connection test as proof that permissions for every chapter exercise are correct: authentication and access to a specific table are separate claims.

Common failure patterns: wrong organization-account identifier; incorrect connection name; token/MFA problems; nonexistent default database; role not granted to the identity; warehouse privileges missing. Diagnose with recorded error category and session context, not by putting secrets in a screenshot. **Success check:** CLI authentication works and `CURRENT_ROLE()` reports the expected least-privileged role.

Project reference: P026: Official Snowflake CLI Action. *Not independently executed; rights review pending.*

3.5 Version control and delivery boundaries

3.5.3 Git changes, code review, and pull requests

The purpose of Git is to make the *definition* of the environment reviewable. Give one pull request a narrow scope: "introduce Chapter 3 DEV bootstrap," not "fix all Snowflake settings." The change includes tests, explanations, cleanup criteria and a manifest of the resources the scripts will affect. Reviewers should be able to reject a grant that gives inappropriate ownership without needing a live account to spot the error.



Figure 3.5. Suggested change-control sequence. Automated deployment is an optional advanced step, not part of this chapter's required lab.

Review checklist for a pull request:

- Resource names are scoped to `BOOK_CH03_DEV`; no production defaults appear.
- Code has explicit prerequisite roles, effects, and expected verification output.
- No live secret, private key, access token, customer data, or account URL is committed.
- An authorized reviewer approves grants and cost-bearing resource creation.
- Cleanup names exact resources and refuses ambiguous or shared-object deletion.
- A passing check refers to the submitted commit, not an earlier local file.

Continuous integration can lint SQL, validate file encoding and check for accidentally committed secrets *without* deploying any Snowflake resources. A separate restricted workflow may later perform tests in a dedicated account. Treating GitHub Actions as an administrator by default is a security anti-pattern: runner compromise would otherwise become cloud-administration compromise.

> **A release is reproducible when its identity is known**
> Record the book edition, repository URL, version tag, relevant package versions, and tested commit SHA. A screenshot of a green build indicator without the commit and execution context is weak evidence.

Failure mode. A maintainer merges an unreviewed `GRANT OWNERSHIP` because the script happens to be in a repository. **Decision rule:** separate code review, privilege approval and deployment authority. These may be performed by different people even in a small team.

Project reference: P026: Snowflake CLI CI action. *Not independently executed; rights review pending.*

3.6 Connecting Python and SQL tools

3.6.1 Python Connector and connection verification

Python is useful when an ingestion job must call an API, validate a file or schedule a query. Snowflake provides an official Python Connector; a successful connection establishes a session, not a proof that a pipeline's transformations are correct. Install the connector in an isolated environment and pin a version in the companion project after testing [7].

Listing 9: Short connection-probe pattern; no embedded password.

```
import os
import snowflake.connector

with snowflake.connector.connect(
    account=os.environ["SNOWFLAKE_ACCOUNT"],
    user=os.environ["SNOWFLAKE_USER"],
    authenticator="externalbrowser",
    role="BOOK_CH03_READER",
    warehouse="BOOK_CH03_XS",
    database="BOOK_CH03_DEV",
    schema="LAB",
) as conn:
    with conn.cursor() as cur:
        cur.execute("SELECT CURRENT_ROLE(), CURRENT_REGION()")
        print(cur.fetchone())  # expected: role and account region
```

This example targets a *human interactive* development session in an environment where external-browser sign-in is configured. Do not use it as the default for unattended automation; interactive login can block a headless process. The project also supplies an offline Python preflight checker that validates file and environment conventions without opening a cloud connection.

Error taxonomy. Distinguish DNS/network failures, authentication failures, unauthorized roles, suspended or unauthorized warehouse access, nonexistent objects, SQL compilation errors, and queries that run but return wrong values. Changing one category's configuration cannot be assumed to repair another.

No confirmed live test
The connector snippet is based on documented API patterns. This manuscript has not authenticated to the reader's Snowflake account; it therefore gives *expected session fields*, not invented query identifiers, timings, or copied UI results.

Evidence to capture: Python version, connector version, the selected connection mode, the active role and successful clean connection closure. If the account employs MFA or federation policy, follow the approved identity guidance rather than bypassing security requirements.

Project reference: P027: Official Snowflake Python Connector. *Not independently executed; rights review pending.*

3.6 Secrets and the session lifecycle

3.6.2 Secret storage and environment variables

Do not put credentials in SQL text, book screenshots or `.env.example`. Environment variables are convenient for a developer but are not an encryption scheme: operating-system processes, diagnostic reports or improperly configured CI logs can still expose them. Use a managed secret store or an approved authentication flow for real deployments. In Git, ignore local secret files; examine prior commits because deleting a leaked key from the newest revision does not revoke it.

Listing 10: Committable .env.example: values deliberately left blank.

```
SNOWFLAKE_ACCOUNT=
SNOWFLAKE_USER=
SNOWFLAKE_ROLE=BOOK_CH03_READER
SNOWFLAKE_WAREHOUSE=BOOK_CH03_XS
SNOWFLAKE_DATABASE=BOOK_CH03_DEV
SNOWFLAKE_SCHEMA=LAB
```

Threat model: committed secrets, leaked log output, and excessive privileges. Rotate any exposed credential; erasing a public example does not revoke it.

3.6.3 Controlled lifecycle and clean exit

Open a connection only when needed, close it after failure, and understand transaction boundaries. For this chapter's SELECT, no rollback is required. Ingestion writes in later chapters need retry-safe behavior.

Distinguish three kinds of termination:

1. *Client closure* ends the Python session or cursor. It is not an account deletion action.
2. *Warehouse suspension* stops eligible user-managed compute. It does not delete tables or terminate every serverless service.
3. *Lab teardown* explicitly removes authorized disposable objects after validation and handover.

> **Decision: interactive versus machine identity**
> Choose interactive SSO for an identified human if organization policy permits; choose federation or another approved machine-to-machine method for scheduled pipelines. Static password-in-code is not an acceptable shortcut for either mode.

Failure mode. A Python task returns correct results but hangs in a container because the authenticator tries to open a browser. **Response:** change the authentication architecture through the identity administrator, not by switching to an unprotected plaintext password.

Checkpoint. Document the identity, secret source, exit path and effective role [7].

Project reference: P027: Snowflake Python Connector. *Not independently executed; rights review pending.*

3.7 Naming and DEV/TEST/PROD boundaries

3.7.1 Object naming as an operational signal

A well-named object makes its environment and intended owner visible. The convention `BOOK_CH03_DEV.LAB.ENERGY_READING` tells a reviewer substantially more than `DATA.PUBLIC.T1`. Naming does not confer security, yet it supports automated checks and reduces human error. Use uppercase unquoted names for this training environment; do not quietly introduce case-sensitive quoted names in the next migration.

3.7.2 Separating development, testing, and production

Development is a place to change code and inspect experiments. *Testing* is a place to evaluate a reviewed artifact against representative, controlled data. *Production* serves real users under availability, access and change-control obligations. Three schemas with different prefixes inside one administrator-owned account are *not equivalent* to three isolated accounts. Choose boundaries based on impact, organization scale, data sensitivity and cost.

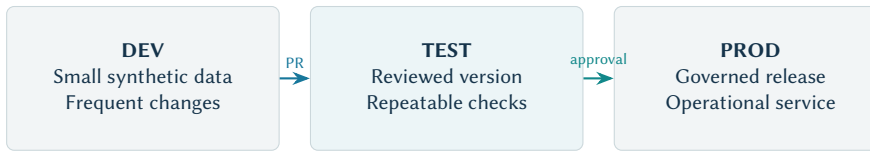


Figure 3.6. Promotion of an artifact, not blind copying of development data into production.

3.7.3 Preventing accidental production operations

Three safeguards complement one another: a context guard in deployment scripts, policy-based authorization that rejects unauthorized actions, and a human-reviewed deployment gate. A guard alone is insufficient if the same service account holds `ACCOUNTADMIN`. Avoid production credentials in local notebooks. Require separate identities or permissions for production changes, and require explicit approval for destructive migrations.

Listing 11: A safe, read-only preflight query; check results before running DDL.

```
SELECT CURRENT_ACCOUNT() AS account_id,
       CURRENT_ROLE() AS active_role,
       CURRENT_DATABASE() AS database_name,
       CURRENT_WAREHOUSE() AS warehouse_name;
```

Common error. The deployment succeeded against the wrong account because both environments used identically named schemas. **Remedy.** Pin and report the account identity in the pipeline configuration; make the deployment stop if it differs from the approved target. Exclude secrets from the resulting log. More complete delivery and rollback patterns appear in Chapter 10.

Project reference: P028: Snowflake DevOps Quickstart. *Not independently executed; rights review pending.*

Project reference: P029: Snowflake Terraform CI/CD Demo. *Not independently executed; rights review pending.*

3.8 Budgets, resource monitors, and cost guardrails

3.8.1 Budgets and supported cost monitoring

A Snowflake *Budget* monitors projected credit usage for supported compute consumption and sends notifications when projected use exceeds a configured monthly limit. Account-level and custom budgets have different scopes; supported service types, AI charges, and additional budget behavior may change over time [8]. A budget is primarily *visibility and notification*, not an automatic universal suspension switch.

Billing vocabulary to keep separate
Credit consumption measures certain compute usage. *Monetary cost* depends on the contract, region, edition and other charges. *Budget threshold* is a monitoring target. *Resource-monitor quota* is a limit configured for assigned user-managed warehouses. These quantities must never be treated as identical in a book's worked example.

Budgets can themselves generate small serverless measurement and metadata costs. Credit data may not update instantaneously. Do not present an alert screenshot as proof that all current costs have been captured, or assume a notification will be received if no valid alert delivery channel has been set up [8].

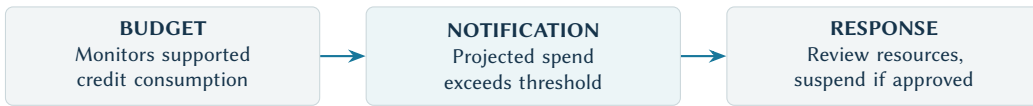


Figure 3.7. Budget response model. No automatic universal charge-stop is implied.

For our demonstration, the recommended order is: ask an account administrator whether a budget already exists; request an appropriate warning threshold; set notification recipients; and confirm which services the budget covers. Do not activate chargeable features just to obtain a screenshot for the book. A trial account may have its own separate balance controls.

Failure mode. A learner believes "the budget is set to five credits, so the account cannot spend six." This is false: a threshold notification is not a hard spending cap. **Decision checkpoint:** who responds to the warning, and what will they stop first? Document a named owner and escalation path.

Project reference: P030: Snowflake Usage App. *Not independently executed; rights review pending.*

3.8 Controls that can interrupt a warehouse

3.8.2 Resource Monitor scope and suspension actions

A Resource Monitor tracks credit usage of *user-managed virtual warehouses* assigned to it. It can notify and, at a threshold, suspend assigned warehouses. It does *not* cover every serverless or AI workload and cannot replace a full account cost strategy [8,9]. A monitor can use a recurring quota period; resets and notification timing should be checked against current documentation. Use dedicated monitor objects only with administrative approval.

Listing 12: OPTIONAL admin-only Resource Monitor configuration; not executed.

```
CREATE RESOURCE MONITOR BOOK_CH03_MONITOR
  WITH CREDIT_QUOTA = 5
  FREQUENCY = MONTHLY
  TRIGGERS
    ON 50 PERCENT DO NOTIFY
    ON 100 PERCENT DO SUSPEND;
-- Assigned by an authorized warehouse admin:
ALTER WAREHOUSE BOOK_CH03_XS
  SET RESOURCE_MONITOR = BOOK_CH03_MONITOR;
```

This is an *illustrative configuration*, not a guarantee of five credits as a total account cap. SUSPEND normally lets running queries complete; SUSPEND_IMMEDIATE has a stronger interruption effect and needs particular care. Only approved administrators should create monitors or change warehouse assignments; these actions might affect teammates if the warehouse is shared [9].

3.8.3 Cost response and emergency-stop runbook

Before: check ownership, auto-suspend, budget and service scope. **During:** stop unexpected retries. **After:** suspend dedicated compute and inspect remaining objects.

Listing 13: Approved stop action; only use against a dedicated lab warehouse.

```
ALTER WAREHOUSE BOOK_CH03_XS SUSPEND;
SHOW WAREHOUSES LIKE 'BOOK_CH03_XS';
```

Do not suspend a shared production warehouse to complete a book exercise. Suspending the warehouse can prevent new user-managed SQL work, but it cannot be treated as an account-wide freeze; some consumption continues independently. If the account is unexpectedly billed, escalate to its administrator and billing owner.

An accurate rule of thumb
Use *budgets* to discover and be warned about supported credit use; use *resource monitors* when warehouse-level suspension is required; use least privilege, guardrails and human response for the remaining risks.

Check: warehouse suspension cannot cap all serverless or AI costs.

Project reference: P031: dbt Snowflake Monitoring. *Not independently executed; rights review pending.*

3.9 Reusable environment bootstrap scripts

3.9.1 Idempotent DDL—and what it does not guarantee

A bootstrap should create the same *intended* lab resources on repeat execution, report whether they already existed and refuse to modify shared objects silently. Simple `IF NOT EXISTS` clauses reduce accidental duplicate creation but do not reconcile column types, warehouse settings or ownership. A robust workflow therefore has a *preflight*, a *controlled apply*, *verification* and *cleanup*. In production, schema migrations and infrastructure-as-code tools extend this pattern with state, drift detection and review [10].

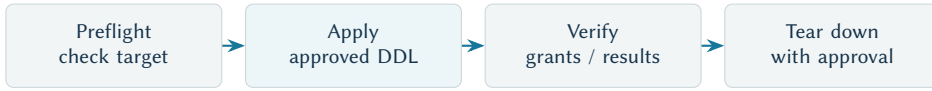


Figure 3.8. Four stages of a repeatable lab, with separate administrative permissions.

3.9.2 Parameterization and environment files

A reusable script should not depend on undocumented local worksheet defaults. Use fully qualified objects or explicitly set the context, and keep account identifiers and authentication settings in protected local configuration. An `.env.example` can enumerate required variable names without publishing values. A deployment tool may substitute validated environment identifiers, but never template arbitrary user strings into privileged SQL object names without validation.

3.9.3 Safe teardown and audit trail

Destructive cleanup is a deployment, not housekeeping. Require a written manifest of lab-owned resources, a confirmation that no other consumers depend on them, and a verified account/role check. `DROP DATABASE` removes an entire hierarchy, so the sample cleanup file defaults to individually removing its *named table and schema* only after authorization. It leaves the database and roles untouched for a responsible administrator to review. The resource monitor and warehouse likewise require ownership checks before deletion.

Definition of a reproducible first environment
A peer can read the manifest, verify permissions, provision isolated objects, use a reader role successfully, observe an expected denial for an unauthorized operation, stop compute, and identify precisely which resources remain. Every step has a documented stop condition and an owner.

Failure mode. A cleanup script deletes `BOOK_CH03_DEV` even though another chapter's dataset has been added. **Prevention:** treat the original lab manifest as a whitelist and refuse broad deletion when objects have drifted.

Project reference: P032: Terraform Sample. *Not independently executed; rights review pending.*

Project reference: P033: Official Terraform Provider. *Not independently executed; rights review pending.*

3.L Guided lab: a governed first workspace

3.L.1 Prerequisites, permissions, and cost boundary

Goal. In a dedicated test account, define a small database and sample table, grant a read-only role, and show that the role cannot write. Use the *offline track* first if you do not have authorized cloud access. No step asks for a real customer dataset. The synthetic readings below are demonstrative only and do not claim to describe Finland's actual electricity production.

Preparation: (a) record the selected account and role; (b) obtain a provisioning approval; (c) choose X-Small or reuse an approved suspended warehouse; (d) enable an available monitoring/alert plan; (e) keep a 10-minute time box; (f) open the companion scripts and verify that all object names start with `BOOK_CH03`. Restrict roles and grants to the training account.

3.L.2 Create, observe, and validate

Provisioning uses separate files: `01_bootstrap.sql` for objects, `02_grants.sql` for administrator-reviewed access, and `03_verify.sql` for the reader. Do not combine them into an unrestricted student deployment.

Listing 14: Selected rows from the standalone bootstrap lab; educational values only.

```
-- Run only after administrator authorization.
INSERT INTO BOOK_CH03_DEV.LAB.ENERGY_READING
  (OBS_ID, SOURCE, EVENT_UTC, POWER_MW)
SELECT 1, 'synthetic_wind',
       '2026-01-15 12:00:00'::TIMESTAMP_NTZ, 82.0
WHERE NOT EXISTS (
  SELECT 1 FROM BOOK_CH03_DEV.LAB.ENERGY_READING
  WHERE OBS_ID = 1);
```

The conditional insertion makes this *small example* reasonably repeatable, but it is not a general exactly-once ingestion protocol. The subsequent check deliberately specifies the table's namespace and verifies the intended role. *Expected*, not observed, read-only output for the one synthetic row: a row count of one, `SOURCE='synthetic_wind'`, and `POWER_MW=82.0`; these values are examples only.

Listing 15: Read-only verification excerpt.

```
SELECT CURRENT_ROLE() AS role_name;
SELECT COUNT(*) AS row_count,
       MIN(POWER_MW) AS min_power_mw
FROM BOOK_CH03_DEV.LAB.ENERGY_READING;
```

Negative access test. After approval, run `04_negative_test.sql` with only the reader role. The write should fail. Record the redacted error and stop if the write succeeds.

Offline: `pythonlab_preflight.py` checks files and names, not cloud behavior.

3.L Evidence, cleanup, and review

3.L.3 What success looks like—and how to recover

A passing cloud session demonstrates *four separate facts*: the authorized provisioner created expected objects; the reader can query the one synthetic row; the unauthorized write test is denied; and the assigned warehouse can be suspended by its authorized operator. A successful offline checker demonstrates *only* syntactic preflight properties. If permissions fail, inspect grants and secondary roles. If objects differ from the manifest, stop; do not overwrite them in place. If any billing warning appears, suspend dedicated compute and escalate.

Cleanup. Stop clients; inspect named objects, dependencies and the active account; apply `99_cleanup.sql` only to the authorized disposable table. The script intentionally leaves the database, warehouse, roles and monitor for an administrator to review. Record remaining objects and their owner. A clean *exit* is not necessarily a blank account.

3.L.4 Exercises and a scored acceptance rubric

1. **Foundation:** describe the account/database/schema/warehouse hierarchy and name two reasons a schema prefix is not a substitute for account separation.
2. **Applied:** adapt the read-only verification for a second synthetic energy source and explain how to avoid duplicating its seed row.
3. **Architecture decision:** an analyst requests permanent `ACCOUNTADMIN` to simplify development. Design a narrower role and list the grants it needs.
4. **Cost incident:** a scheduled job keeps running after the warehouse is suspended. Name two possible causes and two distinct controls to investigate.

Rubric (10 points): identity/namespace correctness (2); least privilege and denied write (3); resource and budget scope explained accurately (2); versioned reproducibility evidence (2); safe cleanup decision (1). *Passing standard:* at least 8 points with no unauthorized access or destructive-change failure.

Chapter takeaway

The first secure environment is not the one with the most services enabled. It is the one whose identity, objects, grants, cost exposure, evidence and teardown are explicit. Chapter 4 will use this boundary to build testable SQL models instead of relying on an unreviewed worksheet.

References and further reading

[1] Snowflake: Account identifiers. [2] Snowflake: Trial accounts. [3] Snowflake: Snowsight. [4] Snowflake: Access control. [5] Snowflake: CREATE WAREHOUSE. [6] Snowflake: CLI connections. [7] Snowflake: Python Connector. [8] Snowflake: Controlling cost. [9] Snowflake: CREATE RESOURCE MONITOR. [10] Snowflake: Terraform provider.

CHAPTER 04 / SQL & DATA MODELLING

Snowflake SQL and Data Modeling

The query can be correct and the dashboard can still be wrong.

At 07:30 a Finnish energy analyst notices that yesterday's generation total increased after a supplier corrected an earlier file. A straightforward `SUM` now reports a larger value, although the physical electricity production has not changed. In the raw observations, both the original and the corrected measurement remain. The database has faithfully retained both versions; the analytical model has accidentally counted both.

This chapter moves from knowing where Snowflake stores and executes data to understanding *what a row means*. Correct SQL is more than grammatical SQL. It is a precise contract between business events, timestamps, units, identifiers, and consumers. We will first learn expression semantics, then build relational transformations, and finally establish an auditable dimensional model for the Finland Energy Analytics platform introduced in Chapter 1.

Our engineering decision
What is the unique observation we promise to deliver, and which SQL transformations and independent tests prove that the resulting energy number is neither duplicated nor silently discarded?

By the end of the chapter, you should be able to:

- choose appropriate Snowflake numeric, string and temporal data types;
- predict join multiplicity, null handling, and aggregation results;
- use common table expressions, windows, and `QUALIFY` defensibly;
- distinguish UTC observations from Finland's local reporting calendar;
- specify a fact-table grain, dimensions, surrogate keys, and history rules;
- build SQL data-quality checks and reconcile source and target totals; and
- document, review, and test an energy-model implementation.

Prerequisites. Chapters 2–3, a non-production Snowflake schema, and basic `SELECT`. The accompanying CSV validator runs offline; Snowflake-specific SQL is prepared for a reviewable live test and is *not* presented as already executed.

Reading path. Sections 4.1–4.4 establish query semantics; 4.5–4.7 define durable data objects and grain; 4.8–4.9 develop evidence and maintainability. The guided laboratory creates the first analytical gold-layer model.

4.1 Snowflake SQL syntax and data types

4.1.1 Primitive types and temporal types

A column type is an assertion about meaning. VARCHAR can preserve an incoming reading exactly, but does not establish that it is a usable quantity. A NUMBER(18,3) expresses a fixed decimal scale; a FLOAT describes approximate numerical representation and should not be chosen for monetary reconciliations or comparisons that require exact decimal equality. Measurement systems may still use floating-point computations, but the trade-off must be explicit.

Snowflake's NUMBER(p,s) has configurable precision and scale. Choose the scale from the resolution of the source measurement rather than from the number of decimal places that happen to appear in one demonstration file. For our energy case, NUMBER(18,3) is an illustrative representation of MWh reported to three decimals; it is not a universal rule for energy markets. The distinction between MWh (energy over an interval) and MW (power at an instant or averaged over an interval) will become important when defining the fact.

Type family	Typical modelling decision
VARCHAR	Raw identifiers, source text, source-system codes; define normalization rules.
NUMBER(p,s)	Exact fixed-scale measures and surrogate integer identifiers.
FLOAT	Approximate calculations when their numerical limitations are acceptable.
BOOLEAN	Validated flags; distinguish false from missing.
DATE	Civil/calendar day with no instant or timezone.
TIMESTAMP_NTZ	Wall-clock value with no timezone semantics encoded; document convention.
TIMESTAMP_LTZ	Instant displayed relative to the session timezone.
TIMESTAMP_TZ	Instant with stored numeric offset, not a permanent zone name.

Listing 1: Type choices; intended for a dedicated book schema.

```
CREATE TABLE energy_reading_example (
    source_code       VARCHAR(32) NOT NULL,
    interval_start_utc TIMESTAMP_NTZ NOT NULL,
    energy_mwh        NUMBER(18,3),
    is_estimated      BOOLEAN
);
```

Decision test. If a source provides 2026-10-25 02:30 without offset or timezone, no temporal type can tell you which occurrence of that local clock reading it meant. Preserve raw evidence and demand a timezone contract before treating the value as an instant. Official type and conversion definitions should be checked against Snowflake's SQL reference [1, 2].

4.1.2 Snowflake SQL dialect distinctions

A useful working assumption is that Snowflake uses familiar relational SQL plus dialect-specific constructs. The danger is not obvious syntax differences; it is code that appears portable but changes meaning. `QUALIFY`, for instance, filters after window expressions have been evaluated. `LIMIT` controls a returned row count but does not imply a repeatable row sample without `ORDER BY`. An identifier can also be case-sensitive when double-quoted, whereas unquoted identifiers are normally normalized by Snowflake. Build a stable naming policy instead of depending on implicit rules.

A *session* supplies defaults such as role, warehouse, database, schema, and timezone. The same SQL text can therefore behave differently in separate sessions when it relies on implicit context. For reproducible examples, set or check the context first. A teaching query should favor explicit column names and an explicit order when result order matters.

Listing 2: Inspect session context before reproducing a lesson.

```
SELECT CURRENT_ROLE()       AS active_role,
       CURRENT_DATABASE()   AS active_database,
       CURRENT_SCHEMA()     AS active_schema,
       CURRENT_WAREHOUSE()  AS active_warehouse,
       CURRENT_SESSION()    AS session_id;

-- Inspect the current timezone setting separately:
SHOW PARAMETERS LIKE 'TIMEZONE' IN SESSION;
```

Three SQL statements should be distinguished: `CREATE TABLE` defines a durable contract; `CREATE VIEW` defines a query-backed interface; and `CREATE TABLE AS SELECT` materializes a query result at a particular moment. They are not interchangeable simply because each can produce a tabular result. Similarly, `WHERE` applies row-level filtering before aggregation; `HAVING` filters grouped output; `QUALIFY` filters results of window functions. Confusing their logical positions creates subtle errors.

> **Maintainability rule.** Every published listing identifies its expected database/schema, assumed permissions and whether it mutates data. A command taken out of a chapter should not accidentally target a reader's production dataset.

Snowflake lets analysts express much of the platform's logic declaratively. That is a strength when the data model and tests are clear: transformations become inspectable, reviewable artifacts. It is a liability when a team treats a successful run as proof of business correctness. Later sections will formalize a model test that is independent of the transformation under test.

Project reference: P035: SQL and Python analytics guide. *Not independently executed; rights review pending.*

A reproducibility check. When copying SQL from another database, review every cast, timestamp literal, identifier and null predicate rather than assuming the two dialects are equivalent. A good migration test runs the same small input through both implementations and compares data types, row grain and null values as well as displayed results. This catches cases where a query returns the expected number of rows but subtly changes their meaning.

4.1.3 CAST, TRY_CAST, and NULL behavior

Data enters analytics in forms chosen by the source, not by the warehouse. A reading may arrive as `'12.500'`, `'N/A'`, the empty string, or a JSON null. `CAST` (and the shorthand `::`) expresses a conversion that fails when an incompatible value is encountered. `TRY_CAST` returns SQL `NULL` rather than an error for supported conversions of *string* expressions. For other inputs, use an appropriate `TRY_TO_*` conversion function. Silent conversion failure is an exception that must be counted, not a data-cleaning strategy [2].

Listing 3: Do not equate an invalid value with zero.

```
WITH incoming(raw_mwh) AS (
  SELECT column1 FROM VALUES
    ('12.500'), ('N/A'), (''), (NULL)
)
SELECT raw_mwh,
       TRY_TO_DECIMAL(raw_mwh, 18, 3) AS parsed_mwh,
       IFF(raw_mwh IS NOT NULL AND
           TRY_TO_DECIMAL(raw_mwh,18,3) IS NULL,
           'REJECT', 'REVIEW') AS parse_status
FROM incoming;
```

SQL has three-valued logic. A predicate comparing a number to `NULL` is neither true nor false; it evaluates to unknown. `WHERE` retains rows for which its predicate is true. Thus `energy_mwh <> 0` excludes rows with null energy as well as zeros. To test missingness, use `IS NULL`; to supply a fallback, use `COALESCE` only when that fallback has an explicit business meaning. Replacing an unknown electricity reading with zero can manufacture an apparent outage.

Failure path: a quiet conversion error
A source sends `'1,25'` in a locale that uses the comma as decimal separator. A generic numeric conversion may return null or fail, depending on its expression and format. The engineering response is to specify the source contract, normalize deliberately, retain the original value, and measure rejection counts—not to silently drop the row.

Verification check. Given an input list containing a valid decimal, a textual flag, an empty string and a missing field, predict the `TRY_TO_DECIMAL` results before executing the query. Explain why the proportion of rejected rows should be monitored separately from the aggregate produced from valid rows.

What to do with failed parses. Keep a rejection table containing the raw value, source row, reason code, ingestion batch and original timestamp string. Report a rejection rate as rejected source events divided by evaluated events, and distinguish completely missing fields from invalid, present text. A corrected record should be able to enter the accepted dataset without erasing the rejected input, so that investigations can reconstruct what the system actually received.

4.2 Filtering, joins, aggregations, and subqueries

4.2.1 SELECT, JOIN, and filters

For the remainder of the chapter, suppose the engineering team has a record for each source's delivered hourly energy: source, metric, start timestamp, MWh, ingestion timestamp, and source-row identity. The *source-row identity* is an audit key; it is not automatically the key of the business observation. A correction may arrive with a different row identity but refer to the same source, metric and hour.

Start with a query that narrows both columns and time range. A bounded UTC interval is easier to reason about than local-date text when clocks change. Half-open intervals, `>= start` and `< end`, also compose without boundary double-counting.

Listing 4: Bounded fact query: no implicit all-time scan.

```
SELECT source_code, interval_start_utc, energy_mwh
FROM book_ch04.fact_energy_hourly
WHERE interval_start_utc >=
      '2026-01-15 00:00:00'::TIMESTAMP_NTZ
  AND interval_start_utc <
      '2026-01-16 00:00:00'::TIMESTAMP_NTZ;
```

4.2.2 GROUP BY, HAVING, and subqueries

`GROUP BY` changes the grain of a query. Every grouping key is part of the output grain, and aggregated measures describe that set of rows. `HAVING` applies conditions after grouping. A subquery can express a reusable threshold or compare a group to a scalar derived from another relation. Before optimizing a query, write its intended *result key* in plain language.

Listing 5: Daily energy by source, after quality filtering.

```
SELECT source_code,
       TO_DATE(interval_start_utc) AS utc_day,
       SUM(energy_mwh) AS total_mwh
FROM book_ch04.fact_energy_hourly
GROUP BY source_code, utc_day
HAVING SUM(energy_mwh) > 0
ORDER BY utc_day, source_code;
```

Boundary. This query is correct only if the fact already contains one accepted value for each promised observation. Aggregating raw retries first and deduplicating later cannot generally recover the true total. That is why the key is part of query design rather than an optional performance optimization.

Project reference: P036: dbt TPC-H relational example. *Not independently executed; rights review pending.*

Worked question. Suppose two independent sites produce 10 and 20 MWh during the same UTC hour. Grouping by hour alone yields 30 MWh; grouping by site and hour yields two separate values. Both queries are mathematically consistent, but they answer different business questions. Before choosing the aggregation keys, decide whether sites can represent overlapping boundaries or duplicated market reporting; if so, summation across sites requires additional business logic.

4.2.3 Duplicate rows and incorrect join cardinality

A join combines rows based on a predicate. It does *not* guarantee one result per row from the left relation. If one energy source joins to two history records in a dimension, one fact row becomes two result rows; SUM then double-counts its energy even if every input was individually valid. This is called a *fanout*. It is among the most expensive semantic mistakes in analytics because it produces believable numbers.

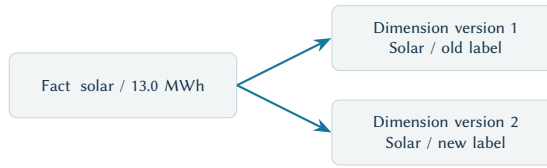


Figure 4.1. A join on `source_code` alone duplicates a fact when a dimension has multiple historical versions.

Check cardinality *before* relying on the join. A source dimension for type-2 history needs an effective-date join, or a fact that already stores the appropriate version's surrogate key. A type-1 dimension should have a unique natural key. Use two independent tests: one for dimension-key uniqueness and one for result-count preservation when a one-to-one relationship is expected.

Listing 6: Detect a dimension key that is not unique.

```
SELECT source_code, COUNT(*) AS versions
FROM book_ch04.dim_source
GROUP BY source_code
HAVING COUNT(*) > 1;
-- Only valid as a defect test for Type 1 dimensions.
```

A join to a dimension can also *lose* facts when written as an inner join and a dimension key is missing. For a reconciliation check, use a left join and count unmatched rows first. Only after the quality gate passes should the presentation layer assume all required dimension matches exist.

Investigation prompt. If six fact rows yield eight rows after a join, do not repair the total using DISTINCT. Identify which key has more than one match and decide whether history or upstream duplication caused the fanout. DISTINCT may discard legitimate repeated measurements while leaving business logic unresolved.

Independent diagnostic. For each source dimension, compare the total row count against the count of distinct source codes where a type-1 rule applies. For type-2 history, test that effective intervals for the same natural key do not overlap. The latter requires interval logic rather than a simplistic uniqueness check. A fact-to-dimension join is acceptable only when its matching behavior is determined by the model's temporal semantics.

4.3 Common table expressions and window functions

4.3.1 Readable common table expressions

Common table expressions (CTEs) let a complex query communicate a sequence of analytical decisions: parse, reject, rank corrections, resolve dimensions, then aggregate. They are a readability and composability technique, *not* a guarantee that intermediate results are physically materialized. Snowflake's optimizer determines the execution plan. CTE names should describe an invariant—`validated_readings` is more informative than `cte2`—and each stage should have a testable row grain.

Listing 7: Decompose correctness decisions explicitly.

```
WITH parsed AS (
 SELECT source_code,
        TRY_TO_TIMESTAMP_NTZ(interval_text)
          AS interval_start_utc,
        TRY_TO_DECIMAL(mwh_text,18,3)
          AS energy_mwh,
        loaded_at_utc
 FROM book_ch04.raw_energy
), valid AS (
 SELECT * FROM parsed
 WHERE interval_start_utc IS NOT NULL
   AND energy_mwh IS NOT NULL
   AND energy_mwh >= 0
)
SELECT source_code, COUNT(*) AS valid_rows
FROM valid GROUP BY source_code;
```

4.3.2 ROW_NUMBER, LAG, and rolling averages

Window functions preserve individual rows while computing values relative to a partition and order. `ROW_NUMBER` is useful for corrections only when its ordering is deterministic. Sort by an authoritative revision timestamp *and* a stable tie-breaker. If two source records share the same timestamp and no authoritative revision rule exists, the result should be flagged rather than chosen at random.

`LAG` compares a row with its predecessor: for energy monitoring, it can reveal suspicious jumps. A three-row rolling mean answers a different question from a three-hour rolling mean when time intervals are missing. Write `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW` only when exactly three consecutive observations are expected and validated. Otherwise build a regular time spine or use an appropriate time-based frame. Snowflake recommends explicit frames over relying on function-specific defaults [3].

Subtle failure case. A window partition keyed only by `source_code` will rank a wind reading against solar and consumption observations if source names or metrics are reused. The partition must express the promised business key, not just the easiest column. A second failure occurs when two source revisions have identical ingestion timestamps; if the provider has no authoritative sequence identifier, quarantine the conflict rather than assert that a larger arbitrary ID is always the correct reading.

4.3.3 QUALIFY and query-plan awareness

QUALIFY filters after window functions. It is especially useful when a deduplication query would otherwise require a nested SELECT. The pattern below chooses the latest raw record for each *business* observation. The source-row identifier breaks ties, but only after ingest-time ordering; in a production model, replace this with the source's actual revision semantics.

Listing 8: Deduplicate by observation key and deterministic order.

```
SELECT source_code, metric_code, interval_text,
       mwh_text, loaded_at_utc, raw_id
FROM book_ch04.raw_energy
QUALIFY ROW_NUMBER() OVER (
 PARTITION BY source_code, metric_code,
              interval_text
 ORDER BY loaded_at_utc DESC, raw_id DESC
) = 1;
```

The grouping here uses the raw timestamp text because the teaching fixture already requires standardized UTC timestamps. In production, parse timestamps into validated instants *before* deduplication; otherwise 2026-01-15T01:00Z and an equivalent offset rendering may be treated as different observations. QUALIFY is Snowflake-specific syntax and appears in the official query reference [4].

Listing 9: Explicit rolling frame on a complete hourly series.

```
SELECT source_code, interval_start_utc,
       energy_mwh,
       LAG(energy_mwh) OVER (
         PARTITION BY source_code
         ORDER BY interval_start_utc
       ) AS prior_mwh,
       AVG(energy_mwh) OVER (
         PARTITION BY source_code
         ORDER BY interval_start_utc
         ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
       ) AS rolling_3_rows_mwh
FROM book_ch04.fact_energy_hourly;
```

Plan-awareness checkpoint. Correctness comes before performance, but not after it. Run a bounded query; inspect the Query Profile for unexpected scan volume or join fanout, and compare actual warehouse time only under controlled cache and concurrency conditions. Do not assert that a CTE, QUALIFY, or a single SQL formatting pattern is inherently faster than an equivalent query without measuring your own workload.

Project reference: P037: Pandas on Snowflake pipeline examples. *Not independently executed; rights review pending.*

A reliable experiment. First construct a three-hour sequence with a single controlled correction. Record the six accepted keys and the chosen revision for the corrected hour. Then introduce a deliberately duplicated timestamp and inspect whether the ranking still selects one deterministic row. Finally remove a middle interval and compare a three-row average to the three-hour period you originally intended. A passing query does not excuse an incomplete time axis.

4.4 Working with dates, time zones, and time series

4.4.1 UTC versus Finland local time

An energy measurement has an instant, but a regulator's daily report uses a *civil date*. Store or normalize the observation instant in UTC; derive `Europe/Helsinki` dates for display and market reporting. A `TIMESTAMP_NTZ` column called `interval_start_utc` is only safe if the input contract guarantees UTC: the type itself does not enforce that convention. Use explicit conversions at the system boundary and track source timezone evidence.

Listing 10: Convert an explicitly UTC wall-clock timestamp.

```
SELECT interval_start_utc,
       CONVERT_TIMEZONE(
          'UTC', 'Europe/Helsinki',
          interval_start_utc
       ) AS helsinki_local_time
FROM book_ch04.fact_energy_hourly;
```

4.4.2 Daylight-saving transitions

A Finland local day can contain 23, 24, or 25 hours when daylight-saving rules change. Local clock times repeat during the autumn transition and skip forward during the spring transition. Therefore the pair `(local_date, local_hour)` is not always a unique key for an hourly fact. Use a UTC instant, preserve the original offset if supplied, and attach local reporting attributes as derived dimensions. Never reconstruct a distinct event solely from a local string such as `02:30` on a transition date.

4.4.3 Time buckets, lagged features, and irregular intervals

`DATE_TRUNC('HOUR', ...)` yields an hour-aligned timestamp; `TIME_SLICE` can express fixed-duration buckets. Before aggregating 15-minute readings into hourly MWh, confirm complete and non-overlapping intervals. Summing MWh across four disjoint quarter hours is meaningful; summing MW measurements without duration weighting is generally not.

Unit checkpoint. If mean power is constant at 40 MW for a 15-minute interval, interval energy is $40\,\text{MW} \times 0.25\,\text{h} = 10\,\text{MWh}$. The reverse calculation assumes the interval is complete and describes average rather than instantaneous power.

Project reference: P038: Time-series analytics on Snowflake. *Not independently executed; rights review pending.*

Modelling consequence. A month-end analytical report may be defined by local civil dates, while its ingestion logs and deduplication keys must remain based on UTC instants. These layers should not overwrite one another. When reporting time-of-use electricity rates, retain the market's authoritative interval definitions, including any provider-specific rules for local-hour identifiers. Deriving an hour solely from formatted local time may merge two different autumn measurements or produce a nonexistent spring measurement.

4.5 Tables, views, and materialized views

4.5.1 Permanent, temporary, and transient tables

Choose persistence according to recoverability. A permanent table supports Snowflake's standard data-protection lifecycle; a temporary table is session-scoped, while a transient table persists across sessions but lacks Fail-safe protection. The latter two are appropriate only for reconstructible intermediates. Their storage is still billable while it exists, and Time Travel behavior depends on table type and retention settings [5].

4.5.2 Views versus materialized views

A normal view gives the model a stable interface without storing its own result set. A materialized view stores a precomputed result and incurs maintenance. Snowflake materialized views are edition-dependent (Enterprise Edition according to current documentation), and present limitations: for example, they are based on a single table rather than arbitrary joins. An energy report that joins multiple dimensions may therefore be better represented by a standard view or prebuilt gold table, depending on frequency and cost [6].

Listing 11: A narrow semantic view, with no stored cache.

```
CREATE OR REPLACE VIEW book_ch04.v_daily_energy AS
SELECT source_code,
       TO_DATE(interval_start_utc) AS utc_day,
       SUM(energy_mwh) AS generation_mwh
FROM book_ch04.fact_energy_hourly
GROUP BY source_code, TO_DATE(interval_start_utc);
```

4.5.3 Maintenance cost and feature restrictions

A materialized view can reduce repeated scan work but introduces storage, incremental refresh work, and serverless maintenance billing. The right comparison is *total workload cost*: underlying ingestion, transformation, storage, read traffic and operational complexity. Record the query and data-change rate before attempting to optimize. For small educational fixtures a normal view is more transparent; no materialized view is required to finish this lab.

Decision rule. Prefer a normal view when the operation is cheap and changes frequently; consider a persisted gold table when a multi-table transformation must be reviewed and tested; evaluate a materialized view only for supported patterns and measured repeated-query benefit.

Project reference: P003: Hybrid table performance examples. *Not independently executed; rights review pending.*

Operational case. Imagine a gold query requested three times each day while its base data changes every five minutes. Maintaining a physical derivative throughout the day may cost more than executing the query a few times. Conversely, a heavily reused stable daily slice may benefit from materialization. Document the workload's modification rate and scan cost before optimization, and check that the proposed materialized definition is actually supported in the selected edition.

4.6 Fact tables, dimensions, and star schemas

4.6.1 Choosing fact-table grain

Write the grain *before* naming any columns: one accepted hourly interval-energy observation per `(source, metric, UTC interval start)`. A raw correction is another version of that observation, not automatically another fact. A table that mixes 15-minute and hourly records without an interval-length field violates that grain even when its SQL schema is valid. Include source provenance so the accepted version can be traced back to a raw record.

4.6.2 Fact/dimension modeling and star schemas

Facts carry measures at a declared grain; dimensions provide descriptive context. The first model uses `dim_source`, `dim_metric`, and `dim_date_utc` with the fact table. Keeping an explicit metric dimension prevents a power reading from being summed as though it were energy. A date dimension describes UTC date here; a separate Finland-local reporting calendar may be introduced later for policy-specific analysis.

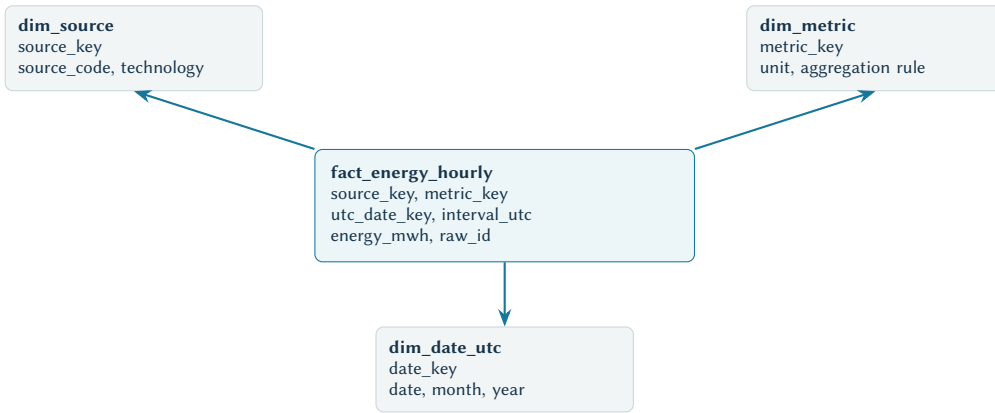


Figure 4.2. A minimal analytical star schema with clearly separated measures and descriptors; all nodes are author-created teaching designs.

4.6.3 Energy/power units and additive measures

MWh is additive across disjoint time intervals and distinct contributing sources *when* the sources do not describe the same physical production twice. MW is a rate and usually needs a duration-weighted aggregation. A ratio such as renewable share is nonadditive: sum the component MWh values and *then* divide, rather than averaging daily ratios without weighting. Every metric dictionary entry must state unit, time grain, population and admissible aggregation.

Project reference: P039: Medallion and dimensional example. *Not independently executed; rights review pending.*

Grain is a promise. If the pipeline later accepts quarter-hour meter data, do not merely insert the new rows into this hourly fact. Either aggregate four complete disjoint intervals into one hour or introduce a separate quarter-hour fact with an explicit duration. A clear grain prevents a quiet semantic regression when a source changes its publishing frequency.

4.7 Keys, surrogate identifiers, and slowly changing dimensions

4.7.1 Natural versus surrogate keys

A source code identifies an entity in the supplying system; a surrogate key identifies a row in our analytical dimension. The distinction matters when source systems change identifiers or when descriptive history must be retained. Keep both keys. Surrogate keys can be generated with sequences or identity mechanisms, but their numbers have no business meaning and should never appear as customer-facing descriptions.

4.7.2 Slowly changing dimensions: types 1 and 2

Type 1 overwrites a descriptive attribute when only the current correct description is needed. Type 2 preserves changes as separate effective-dated rows. If a generation facility changes its technology label, choose based on the reporting contract: should a 2024 report reproduce the label believed in 2024, or use today's corrected category? Neither answer is universally right. Type 2 requires an explicit validity range and a rule for late-arriving corrections.

Listing 12: Type 2 lookup: half-open validity interval.

```
SELECT f.raw_id, s.source_key
FROM book_ch04.fact_energy_hourly f
JOIN book_ch04.dim_source_history s
  ON f.source_code = s.source_code
 AND f.interval_start_utc >= s.valid_from_utc
 AND f.interval_start_utc < s.valid_to_utc;
-- A valid_to_utc sentinel or explicit NULL handling
-- must be declared in the dimension contract.
```

4.7.3 Constraint enforcement caveats by table type

In Snowflake standard tables, primary-key, unique and foreign-key declarations are typically *informational*, not enforced. By contrast, NOT NULL and supported CHECK constraints are enforced; hybrid tables apply additional key-enforcement rules. Snowflake's current constraints documentation must be verified for the edition and table type used [7]. Tests for duplicate natural keys and missing dimension matches remain essential even if a model documents a primary key.

Don't mistake metadata for evidence. A declared primary key on a standard fact table is not proof that no duplicate rows exist. The release gate should count duplicate business keys over the actual data and fail if it finds any.

Project reference: P042: Data Vault and dbt Snowflake modelling example. *Not independently executed; rights review pending.*

History integrity. For type-2 dimensions, reject overlapping effective intervals and document open-ended validity. Retroactive corrections may require replaying historical fact-to-dimension mappings.

4.8 Data validation and SQL testing patterns

4.8.1 Null, uniqueness, and range checks

A useful data test states a falsifiable assertion with a known severity and an owner. In the energy platform, accepted observations must identify a source and metric, have a usable UTC interval start, hold a nonnegative interval energy value for the selected generation metric, and contain at most one accepted observation per source/metric/hour. A test that returns zero offending rows is easy to automate and review.

Listing 13: Failing rows: missing or negative accepted energy.

```
SELECT raw_id, source_code, interval_start_utc,
       energy_mwh
FROM book_ch04.fact_energy_hourly
WHERE source_code IS NULL
   OR interval_start_utc IS NULL
   OR energy_mwh IS NULL
   OR energy_mwh < 0;
```

Listing 14: Failing keys: more than one fact per grain.

```
SELECT source_code, metric_code,
       interval_start_utc, COUNT(*) AS n
FROM book_ch04.fact_energy_hourly
GROUP BY source_code, metric_code, interval_start_utc
HAVING COUNT(*) > 1;
```

Range rules need domain justification. Negative *consumption* or net-flow metrics can be legitimate depending on the definition; a naive generic `value >= 0` test could erase meaningful sign conventions. Separate the rule for the *generation_mwh* metric used in this exercise from any generalized future metric that might have a different valid range. Likewise, a null `estimated_flag` might mean "unknown" rather than "no".

Test specification. Each check must document its column scope, expected value or range, evidence of the business rule, result semantics (zero rows versus one summary row), severity, and remediation step.

Expected fixture result. After applying the declared correction rule to seven raw rows, the demonstration contains six accepted unique hourly observations. No accepted record has null or negative energy. These are *expected results for the synthetic fixture*, not measurements from an actual Snowflake execution.

Project reference: P043: Retail data engineering pipeline example. *Not independently executed; rights review pending.*

An operational metric. A test that discovers 1 invalid row out of 7 has a different impact from one that discovers 1 invalid row out of 7 million. Preserve both absolute exception counts and normalized rates, and do not allow percentage dashboards to conceal a small number of serious defects. Define freshness and completeness checks as independent requirements: valid values from only one of two expected source systems still produce an incomplete national picture.

4.8.2 Transformation reconciliation

A model can pass uniqueness and null checks yet still lose a valid source during a join. Reconciliation compares what entered a controlled transformation with what emerged, at matched business grain. For our fixture, deduplication legitimately removes one superseded reading. Therefore *raw row count = fact row count* is the wrong invariant. The correct checks are *one accepted version per business observation*, complete coverage of expected observations, and preservation of totals calculated from those accepted versions.

Listing 15: Independent control total on accepted raw versions.

```
WITH ranked AS (
 SELECT source_code, metric_code, interval_text,
        TRY_TO_DECIMAL(mwh_text,18,3) AS mwh,
        ROW_NUMBER() OVER (
          PARTITION BY source_code, metric_code,
                       interval_text
          ORDER BY loaded_at_utc DESC, raw_id DESC
        ) AS rev_rank
 FROM book_ch04.raw_energy
), control AS (
 SELECT COUNT(*) AS n, SUM(mwh) AS mwh
 FROM ranked WHERE rev_rank = 1
)
SELECT * FROM control;
-- Synthetic fixture: expected n=6, mwh=97.500.
```

An independent target check should compare accepted source keys and row counts, then verify the energy sum. Use tolerances for approximate floating-point metrics; here exact scale-3 decimal totals are suitable. A failure should identify the source and hour, not merely report FALSE.

Failure mode: two wrongs reconcile.
If the pipeline and its "test" both deduplicate with the same erroneous partition key, they can agree perfectly while both being wrong. A credible control calculation should use independent logic, fixture expectations, or a separately verified source contract.

Tests are operational agreements. A pipeline should stop publishing new gold data on a critical reconciliation failure, retain rejected rows and raw inputs for investigation, and avoid presenting partial totals as verified values. For a long-running platform, save the acceptance metrics with batch IDs and source snapshots for audit and replay.

Reconciliation at scale. Global totals are useful but insufficient when large positive and negative discrepancies cancel. Add per-source, per-metric and per-date reconciliation; include count and total, plus a list of missing or unexpected keys. Use stable batch identifiers to compare the same snapshot at each stage. Reconciliation must also consider whether a late revision is expected to restate a historical total or merely append a new publication event.

4.8.3 SQL testing anti-patterns and corrections

The most dangerous tests are those that look decisive but do not describe a business invariant. `COUNT(*) > 0` proves only that a table is nonempty. `DISTINCT` in the final query can hide a join fanout. `COALESCE(value,0)` converts missing measurements into numeric events. Choosing the latest row without deterministic ordering can produce different accepted values on otherwise similar runs. An assertion that joins two differently aggregated datasets can itself double-count and conceal a discrepancy.

Fragile pattern	More defensible test
"Table has rows"	Check expected source/date coverage and freshness.
"No NULL after COALESCE"	Check raw parse failures before fallback is applied.
"DISTINCT fixed duplicates"	Enforce and test business-key uniqueness upstream.
"Latest row wins"	Specify revision authority and deterministic tie-breaking.
"Total matches yesterday"	Reconcile to accepted source versions and batch IDs.
"Test passed once"	Test every publication and track historical failures.

A *negative test* intentionally introduces a defect and verifies that the pipeline catches it. Insert a duplicate accepted business key in an isolated disposable copy; the uniqueness test must fail. Change one generated MWh value to a negative number; the domain check must identify it. Remove a required dimension row; the unmatched-fact check must detect a missing join. Never inject faults into shared or production tables, and always script cleanup.

Listing 16: Left anti-join: facts without matching source dimension.

```
SELECT f.source_code, COUNT(*) AS missing_matches
FROM book_ch04.fact_energy_hourly f
LEFT JOIN book_ch04.dim_source s
  ON f.source_code = s.source_code
WHERE s.source_code IS NULL
GROUP BY f.source_code;
```

Release gate. Define the failure categories `BLOCKER`, `WARNING`, and `INFORMATIONAL`; specify alert owner, replay strategy, and known accepted exceptions. The gate should be testable offline with synthetic fixtures even when cloud credentials are unavailable. The companion bundle includes such a validator rather than claiming a live Snowflake integration run.

Avoid self-confirming tests. When the transformation relies on a timezone conversion, a test that calls the exact same conversion on the same field can repeat its error. Write edge-case fixtures containing known UTC instants around both clock transitions and manually verified expected local labels. When a join uses effective dates, include a fixture on each boundary. Deliberately failing tests are often more informative than repeatedly passing the happy-path sample.

4.9 Naming, documentation, and maintainable SQL

4.9.1 Readable SQL and explanatory comments

A SQL transformation is a maintained engineering artifact, not a one-time interactive answer. Maintainability starts with a predictable structure: name each CTE by its role, list only needed columns, place a join predicate beside the join, keep filters easy to audit, and expose grain-changing operations. A code review is easier when the business key appears consistently across staging, tests, and documentation.

Listing 17: A readability-oriented transformation sketch.

```
WITH accepted_readings AS (
    SELECT source_code, metric_code,
           interval_start_utc, energy_mwh
    FROM book_ch04.stg_energy_accepted
    WHERE metric_code = 'generation_mwh'
), matched_sources AS (
    SELECT r.*, s.source_key
    FROM accepted_readings r
    LEFT JOIN book_ch04.dim_source s
      ON r.source_code = s.source_code
)
SELECT source_key, interval_start_utc,
       energy_mwh
FROM matched_sources;
```

Do not use comments to restate the obvious (`-- select source key`). Explain assumptions that the SQL cannot express: why a late revision supersedes an earlier record, which physical sources may overlap, why a local date is used, and which source metadata authorizes a unit conversion. Comments should not disagree with the query; a stale comment is worse than none.

Naming convention for this book
`raw_*`: appendable source evidence; `stg_*`: typed validated source-shaped data; `dim_*`: descriptive context; `fact_*`: measures at declared grain; `v_*`: reader-facing views; `test_*`: assertions returning violating rows. Names are descriptive, unquoted, and consistently snake_case.

Avoid enforcing arbitrary style preferences as facts. Whether leading commas or trailing commas are used is much less consequential than a clear grain, a testable result and consistent formatting. Use SQL linting where feasible, but distinguish stylistic lint rules from semantic correctness checks.

An editing criterion. Prefer meaningful names over abbreviations that save a few characters but hide a contract. A future maintainer must know whether `energy_mwh` represents final adjusted energy or raw preliminary energy. Make the column name and dictionary agree, then include one test that demonstrates the semantics. A SQL comment about a physical formula should identify its source and units so that it remains defensible if a provider modifies its measurement specification.

4.9.2 Data dictionary and lineage

A model is not documented until a new engineer can state what its measures mean without reading the entire transformation script. A *data dictionary* records entity purpose, primary business grain, column types, descriptions, units, nullability expectations, acceptable value ranges, sources, and ownership. A *lineage* record identifies the upstream inputs and transformations used to construct each important output.

Column	Type	Meaning and rule
`source_code`	VARCHAR	Source natural key; not null in accepted data.
`interval_start_utc`	TIMESTAMP_NTZ	UTC hour start by contract; half-open interval.
`energy_mwh`	NUMBER(18,3)	Accepted generation energy for one hour; additive when sources do not overlap.
`raw_id`	NUMBER	Audit pointer to authoritative latest raw reading.
`loaded_at_utc`	TIMESTAMP_NTZ	Source-record ingestion time, not measurement time.

The distinction between `interval_start_utc` and `loaded_at_utc` is fundamental. A late source correction should change which raw record is accepted for a January measurement; it should not reclassify that measurement into October simply because the correction arrived then. Lineage must also preserve the revision-selection rule and make retained raw values inspectable.



Figure 4.3. Model lineage identifies contracts between layers, not merely arrows between database objects.

For a production-ready project, store documentation beside versioned SQL and register the owner who can resolve disputes about meaning. The `data_dictionary.md` file in the lab records assumptions and deliberately flags synthetic data so no example figures can be mistaken for Finnish national energy statistics.

Field-level lineage. For the fact table energy measure, trace the originating raw decimal text, the decimal parsing policy, the latest-revision ordering, the source and metric lookup, and the fact table's publication batch. For `interval_start_utc`, document the provider's UTC declaration and the parser used. These two traces allow a reviewer to answer: "Which exact input established this amount, and which transformation made it reportable?"

Change management. Store the dictionary in version control together with SQL, tests and review decisions. If the source updates its unit from MWh to kWh, do not silently rename the same numeric column and continue. Record the new contract, add a unit conversion with explicit factor and validation, and backfill or distinguish historical partitions. The dictionary should specify the effective date of the change and the person who approved it.

4.9.3 Naming policy and peer review

A small team can produce a large operational risk by allowing multiple, incompatible definitions of "daily generation." Agree on naming and semantic conventions before publishing dashboards. For example, `generation_mwh_utc_day` is explicit about the interval basis; `total_energy` may not be. The data contract should state whether the figure is gross or net generation, the role of imports and exports, the geographic coverage, and the day boundary.

A meaningful pull-request review tests five questions. *Grain*: can two rows legitimately share the declared key? *Units*: are the numbers additive or rates? *Time*: which timezone, interval boundaries, and late-arrival rules apply? *Correctness*: where are the independent controls and defect fixtures? *Operations*: can the transformation be replayed safely and cheaply without broad privileges? These questions are more durable than review comments about capitalizing SQL keywords.

Review record: model version CH04-1
Source shape: two synthetic generation sources, three UTC hours, one superseding revision. Accepted grain: one value per source, metric and UTC hour. Accepted observations: six. Expected generation: 97.500 MWh. Evidence: raw CSV checksum, offline validator output, SQL test queries, reviewer initials and Snowflake live-run date (blank until actually executed).

All example repositories in the external catalogue are *research references*, not automatically licensed source files. The teaching implementation supplied with this chapter is original. Before importing external code, check its license and record the applicable commit. If a GitHub repository is missing, archived, or incompatible, the chapter should remain understandable and the companion lab should still run with its own disclosed test inputs.

Project reference: P044: dbt Snowflake utilities. *Not independently executed; rights review pending.*

Project reference: P045: dbt constraints examples. *Not independently executed; rights review pending.*

End-of-model checklist. A reader should be able to reconstruct a valid six-row fact, explain why an apparent seventh record is superseded, identify a faulty join before aggregation, and justify the timezone and units. Section 4.L provides the exact acceptance artifacts to verify these claims.

Practical peer-review procedure. Ask a second engineer to execute the offline fixture, inspect the accepted correction, and explain why it changes the aggregate by 0.500 MWh relative to the earlier record. Then ask that reviewer to delete a required dimension row from a disposable copy and demonstrate the orphan test failing. Record findings in a pull request rather than editing the expected result to make a failure disappear. Reviewers should also confirm that published SQL contains no credentials and that each destructive statement is scoped to an explicitly disposable schema.

Edition control. A published book inevitably outlives some vendor interfaces. Maintain an errata page and versioned companion code for API differences, but preserve the chapter's durable rationale: grain, units, joins, corrections and validation. Readers should be able to apply that reasoning even if a repository or user-interface menu changes.

4.L Guided lab: a tested energy star model

4.L.1 Prerequisites, inputs, and cost guardrails

Inputs. A synthetic CSV containing seven source events: six distinct source-hour observations and one corrected revision. The original reading for one solar hour is 12.500 MWh; its later authoritative version is 13.000 MWh. The accepted total is 97.500 MWh. These numbers are fictional and are not measurements of Finland's grid. Use only the dedicated `BOOK_CH04` schema provisioned under Chapter 3's bounded laboratory account.

Safe order: (1) run the offline Python validator, (2) inspect its six accepted keys, (3) create a disposable transient raw table in the isolated schema, (4) execute staging and fact scripts, (5) run failure-returning tests and inspect row counts, (6) explicitly suspend the small warehouse and delete only approved disposable objects. A file labelled `cleanup.sql` is not permission to delete any table outside this lab.

4.L.2 Procedure: parse, select revision, and materialize

Listing 18: Illustrative model transformation; full script in bundle.

```
CREATE OR REPLACE TRANSIENT TABLE
 book_ch04.stg_energy_accepted AS
SELECT raw_id, source_code, metric_code,
       TRY_TO_TIMESTAMP_NTZ(interval_text)
          AS interval_start_utc,
       TRY_TO_DECIMAL(mwh_text,18,3)
          AS energy_mwh,
       loaded_at_utc
FROM book_ch04.raw_energy
WHERE TRY_TO_TIMESTAMP_NTZ(interval_text)
          IS NOT NULL
  AND TRY_TO_DECIMAL(mwh_text,18,3)
          IS NOT NULL
QUALIFY ROW_NUMBER() OVER (
 PARTITION BY source_code, metric_code,
              TRY_TO_TIMESTAMP_NTZ(interval_text)
 ORDER BY loaded_at_utc DESC, raw_id DESC
) = 1;
```

Important. The SQL above is a selected teaching excerpt. It intentionally uses a dedicated schema and an explicit parsing boundary, but production ingestion also needs format, rate, deduplication tie and source-identity contracts. Full bundle scripts add stable dimensions and an output fact. Nothing in these pages implies that a Snowflake account was connected or queried while this manuscript was prepared.

Required deliverables. `model.sql`; `sql-tests/`; `data_dictionary.md`; `energy_sample.csv`; `validate_fixture.py`; and a README with account prerequisites, actual run evidence fields, troubleshooting, and cleanup controls.

Why the fixture includes a correction. A six-row flawless example would teach syntax but not the engineering reason for version selection. The seventh source event forces the reader to distinguish the raw audit log from the accepted observation table. Its known 0.500 MWh adjustment creates an independent numerical oracle for the test suite.

4.L.3 Expected results, failures, rollback, and cleanup

The seven-row synthetic fixture contains six accepted observations and 97.500 MWh after applying one solar correction. The offline validator checks these expectations independently. A live Snowflake run succeeds only when it matches these results and all blocker tests return zero violations. Record its edition, revision, query IDs and timestamp; do not present expected output as measured evidence.

Listing 19: Acceptance checkpoint for the isolated Snowflake lab.

```
SELECT COUNT(*) AS accepted_rows,
       SUM(energy_mwh) AS accepted_mwh
FROM book_ch04.fact_energy_hourly;
-- Expected on the synthetic fixture:
-- accepted_rows=6, accepted_mwh=97.500
```

If results differ: inspect rejected casts, timestamps, revisions, join fanout and filters before changing an expected value. Preserve raw input and logs. Restrict cleanup to named disposable objects and suspend the lab warehouse; otherwise costs may continue.

4.L.4 Exercises and assessment rubric

1. **Foundation.** Explain the difference among `NULL`, zero and a failed text-to-number conversion. Predict the result of a `LEFT JOIN` when the dimension row is missing.
2. **Practitioner.** Add a second source revision and design a test that fails if two accepted records share the same business key. Show the expected accepted-count change (or lack thereof).
3. **Architecture.** Decide whether daily national output belongs in a materialized view, normal view or persisted gold table. Defend your answer using read rate, update rate, feature limitations, costs and auditability.

Rubric (10 points). Grain/units (3); deterministic revisions/time semantics (2); independent reconciliation/tests (3); safe operation/docs (2). **Pass:** at least 8/10 and no missing blocker.

Chapter 4 summary. Correct analytics needs grain, cardinality, time and unit semantics, safe conversions, and independent validation. Chapter 5 will ingest external files without losing this evidence.

Documentation and further reading.

[1] Snowflake, Data Types. [1] Snowflake, Table Design.
[2] Snowflake, TRY_CAST. [3] Snowflake, Window Frames.
[4] Snowflake, QUALIFY. [5] Snowflake, Temporary / Transient Tables.
[6] Snowflake, Materialized Views. [7] Snowflake, Constraints.

CHAPTER 05 DATA ENGINEERING / INGESTION

Batch Data Ingestion and Loading

From trustworthy source files to an auditable bronze layer

A plausible number is not necessarily a trustworthy number.
An energy analyst opens Monday's dashboard and notices that Finnish electricity generation has apparently increased by 14 percent overnight. A colleague celebrates; another asks whether the meter network changed. The actual cause is less exciting: Sunday's CSV file was loaded twice after a failed scheduler retry. Nothing crashed. Every SQL statement succeeded. The pipeline was wrong.

Loading is not merely a file transfer. The warehouse must reconcile a publisher's export, preserved bytes, loaded rows, and the observations safe to count. An engineer preserves the evidence connecting them.

We build the ingestion boundary for the *Finland Energy Analytics* project from Chapters 1 and 4. A public dataset does not automatically permit republication, so the initial fixture is explicitly *synthetic*. Real Finnish data require separate rights, schema and provenance checks.

By the end of this chapter, you can:

- specify source contracts, freshness and licensing checks before writing a loader;
- select file formats, stages and safe Snowflake `COPY INTO` settings;
- retain file and row provenance, reject bad input, and reconcile a batch;
- explain what duplicate-file prevention does *not* guarantee;
- prepare a repeatable Python-assisted transfer, monitoring and recovery plan.

Prerequisites. Chapters 3–4, elementary SQL and optional Python 3. A real Snowflake lab requires an authorized account, role with object privileges, and a small virtual warehouse. An offline validation path is provided in the companion materials. None of the example outputs below is presented as a measured cloud benchmark.



Figure 5.1. The ingestion boundary preserves both data and evidence.

5.1 Data source assessment and ingestion requirements

5.1.1 Source endpoints, formats, and refresh intervals

Before choosing between a Python script and a managed connector, build a *source inventory*. At minimum, record the publisher, access method (HTTPS API, SFTP, database export or cloud bucket), file format, date/time semantics, grain, expected size, nominal cadence, revision policy and a human contact. The source inventory is a technical contract with your future self: it tells you what to investigate when yesterday's source files stop looking like today's.

A useful distinction is between *publication time*, when a source claims to have released a revision; *event time*, when an observation actually occurred; and *ingestion time*, when your platform received it. These clocks are not interchangeable. A monthly correction published today can legitimately update a measurement from the previous year. If you partition only by ingestion date, such late-arriving corrections still need a business-key reconciliation.

Contract field	Example / decision for a synthetic energy source
Source identifier	`fi_energy_demo`; accountable source owner
Primary access	HTTPS batch download or reviewed local fixture
Payload	UTF-8 CSV, comma delimiter, one header row
Observation grain	source + metric + UTC interval start
Publication cadence	Hourly export, checked daily (illustrative)
Correction rule	Same business key may be republished with revision
Required evidence	URL, retrieval time, license, SHA-256, row count

An endpoint can remain reachable while delivering stale content. Conversely, an HTTP timeout does not establish that the publisher has no new data. Treat access success and dataset freshness as separate health checks. In file-based systems, the *inventory* should also capture whether a batch is a complete snapshot, a changed-row extract or a sequence of disjoint partitions; the recovery process depends on that meaning.

Design decision. For the book's first runnable load, prefer two small immutable CSV files. A reader can inspect their bytes, calculate expected rows by hand, and reason about retries. The later streaming chapter will address cases where files cannot meet the service objective.

Project reference: P046: Snowflake Google Analytics connector guide. *Not independently executed; rights review pending.*

5.1.2 Availability, API quota, and dataset licensing

Public accessibility is not the same as legal permission to republish a dataset. A responsible ingestion design records where the data comes from, its permitted uses, attribution conditions, retention restrictions and whether redistribution in a commercial technical book is allowed. A direct public URL is evidence of discoverability; it is *not* proof of a reusable license. For a Finnish energy source, preserve a dated copy of the source's own terms and link to the official publisher. Do not infer permissions from generic national open-data policies.

API constraints belong in the same review. Record authentication, per-minute or daily quotas, pagination, maximum response size, support for conditional requests, changes to endpoints, and how HTTP status codes are used. Never retry a permanent authorization failure as though it were a transient outage. For retryable failures such as temporary throttling, honor `Retry-After` when present, use bounded exponential backoff and cap the total retry duration.

Source admission checklist

1. **Legal and ethical:** dataset rights, personal-data exposure, appropriate attribution and acceptable use.
2. **Operational:** uptime assumptions, rate limit, payload pagination, contact and incident channel.
3. **Semantic:** units, time zone, revisions, missing intervals, negative values and category definitions.
4. **Audit:** stable source ID, retrieval log, checksum, immutable archive and manifest.

An unreliable publisher may still be useful if the consumer can tolerate a larger freshness window and an explicit "data delayed" state. A high-value dashboard does not justify concealing missing observations by copying the previous value and presenting it as new. If the publisher changes the meaning of a metric, the pipeline must version its contract and potentially recompute previous aggregates.

Failure example. A provider switches from MWh to kWh without updating the column name. The next daily total appears to increase a thousandfold. A schema check that only tests column counts passes. A comprehensive contract must also test units, plausible ranges and declared revisions. Unit conversion must be explicit and attributable, not an undocumented magic factor inside `COPY`.

Reader check. Can you answer who owns a source dataset, how the license was verified, whether a historical correction is expected, and what must happen if its hourly release is two hours late? If not, the source is not ready for production ingestion.

Project reference: P047: API enrichment framework. *Not independently executed; rights review pending.*

5.1.3 Freshness requirements and service objectives

An ingestion objective should be a measurable contract rather than the instruction "load the data quickly." Define a freshness service-level objective (SLO) separately from a loader's runtime. A successful copy may complete in 20 seconds after the source data has already been delayed for an hour. What matters to an analyst is the age of the latest *accepted business observation*, not just the age of the newest file.

For source s at evaluation time t, a basic age-of-data indicator is

$$A_s(t) = t - \max\{\, event_time(r) : r \in \text{accepted}(s) \,\}.$$

This expression assumes comparable UTC timestamps and a known observation cadence. It is not a substitute for checking missing intervals: a fresh reading from one meter may coexist with a week-long gap for another.

Metric	Illustrative target	Evidence required
Arrival delay	p95 below 2 h	publication-to-stage times
Accepted freshness	under 3 h	max accepted UTC interval, per source
Completeness	at least 99%	expected vs observed source/hour keys
Reconciliation	exact row counts	manifest, copy result, bronze
Incident response	alert within 30 min	monitoring timestamps

All numerical targets above are design examples, not verified requirements of a Finnish grid operator. Record a tolerance for scheduled maintenance and define whether percentiles are computed over successful loads only or over all expected windows. Excluding failed batches makes the reliability metric meaningless.

Trade-off: avoid paying for a promise nobody needs.
If analysts make decisions every morning, a stable scheduled batch may be more dependable and less costly than a continuous connector. If operations require action within seconds, batch may be structurally unsuitable. Decide the latency and recovery objectives before selecting Snowpipe, external orchestration or periodic COPY.

The acceptance gate for this chapter is intentionally narrow: every input file has a known origin and checksum; the target contains the expected number of rows; every row retains a retrievable source-file identity; and a rerun does not silently change the accepted analytical result. We will build those invariants before scheduling automation.

5.2 File formats: CSV, JSON, Parquet, and others

5.2.1 Creating reusable file-format objects

A file format is part of the contract between a source's bytes and the warehouse's parser. A named Snowflake `FILE FORMAT` makes that contract visible, reusable and reviewable. This matters when several loads share the same source but have separate tables: without a named object, a "temporary" override of delimiter, quote or null parsing can silently cause inconsistent interpretations. Snowflake supports formats including CSV, JSON and Parquet; their options differ [1].

Listing 1: Named formats for a controlled CSV and JSON source.

```
CREATE FILE FORMAT IF NOT EXISTS ff_energy_csv
  TYPE = CSV
  FIELD_DELIMITER = ','
  SKIP_HEADER = 1
  FIELD_OPTIONALLY_ENCLOSED_BY = '"'
  ENCODING = 'UTF8'
  EMPTY_FIELD_AS_NULL = FALSE
  NULL_IF = ('__MISSING__')
  ERROR_ON_COLUMN_COUNT_MISMATCH = TRUE;

CREATE FILE FORMAT IF NOT EXISTS ff_energy_json
  TYPE = JSON
  STRIP_OUTER_ARRAY = FALSE;
```

Here, `SKIP_HEADER=1` skips one header line; it does not compare header names to expected column names. The fixture contract therefore checks the header separately in Python. The source-specific sentinel in `NULL_IF` and `EMPTY_FIELD_AS_NULL=FALSE` are deliberate: blank text is preserved for semantic review rather than silently converted to a null. Check the file format options against the Snowflake version used for a released edition.

Two valid CSV strategies. A positional schema with `SKIP_HEADER` is appropriate for a tightly controlled export. Alternatively, CSV header parsing with `PARSE_HEADER=TRUE` is used in supported inference and column-matching workflows. These two header strategies are *not interchangeable*; do not set `SKIP_HEADER` together with `PARSE_HEADER` [1].

Design principle. The bronze layer should make unexpected source values inspectable, not magically convert them to the nearest convenient type. A later quality transformation will decide which textual timestamps and decimal values are valid. Preserve the input representation at the trust boundary and record parser settings alongside the load.

Project reference: P049: Terraform Snowflake file-format example. *Not independently executed; rights review pending.*

5.2.2 Encodings, headers, delimiters, and quotations

CSV is a family of conventions, not a fully self-describing schema. The byte sequence `A,"1,200",2026-01-15` contains three fields only if quotation marks are interpreted correctly. A leading byte-order mark, an embedded newline, a semicolon separator or a source locale that uses comma decimals can all change the result. Sample more than the first line: edge cases often appear at file boundaries or in unusually long text fields.

In production, explicitly test encoding; the header's exact spelling and order; newline conventions; quote escapes; Unicode characters; empty versus null; and whether compression applies to the complete file. For untrusted files, reject mismatched column counts rather than quietly shifting a time value into a metric field. For sources with deliberate evolving schemas, use a separately versioned contract instead of disabling every error check.

5.2.3 Choosing CSV versus columnar Parquet

Criterion	CSV	Parquet
Schema	External contract required	Types and metadata embedded
Human inspection	Easy with text tools	Requires reader / library
Compression	Often gzip or equivalent	Columnar compression / encoding
Best fit	Simple portable exports	Larger typed analytical extracts
Common hazard	quoting, drift, locale	incompatible type/logical annotation

Parquet often performs well for large, typed analytical extracts; it is not automatically the right format for every tiny hourly upload. File size, batch count, publisher support, read characteristics and operational tooling can matter more than a generic claim that "Parquet is faster." For Snowflake loading, verify the current Parquet logical-type and scanner settings before benchmarking; do not silently compare different data types after parsing [1].

JSON excels when payloads carry variable nested structures, but an array, JSON Lines and one document per file can require different parsing choices. Chapter 6 develops semi-structured modelling in detail. Here, the invariant is simpler: every source format must map to a versioned contract with an independent parser test.

Diagnostic: one valid file, one surprising total.
A CSV with two embedded commas may be parsed into six fields instead of four if a quotation option is wrong. Loading all fields as text postpones numeric conversion errors, but it does not protect against a shifted column. Header checks, field-count checks and semantic range checks must cooperate.

Project reference: P048: Twitter auto-ingest sample. *Not independently executed; rights review pending.*

5.3 Internal and external stages

5.3.1 Internal versus external stages

A *stage* describes where Snowflake reads the files to be loaded. A named internal stage stores files in Snowflake-managed storage; a named external stage references supported customer-managed cloud object storage. User stages and table stages exist as well, but a named stage generally makes an instructional pipeline easier to reason about and grant permissions to [2].

Listing 2: Create and inspect an internal ingestion stage.

```
CREATE STAGE IF NOT EXISTS stg_energy
  FILE_FORMAT = ff_energy_csv;
LIST @stg_energy;
-- Run PUT from SnowSQL or another supported client,
-- using an OS-local absolute file path:
-- PUT file:///absolute/path/energy_part_01.csv
--     @stg_energy AUTO_COMPRESS=FALSE OVERWRITE=FALSE;
```

The PUT line is shown as a *client command*, not as a command guaranteed to run from a browser worksheet. The companion README identifies an appropriate client; only authorized lab users should upload. Prefer OVERWRITE=FALSE and versioned file paths for an auditable exercise. A stage listing helps establish that the expected files exist before the copy begins, but the file name alone does not prove content integrity.

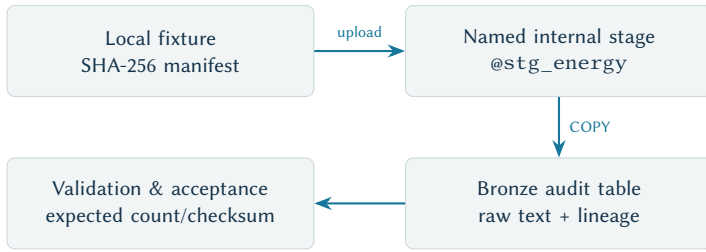


Figure 5.2. The stage is a transport boundary, not proof of correctness.

Operational distinction. Snowflake-managed internal storage reduces the need to configure another cloud bucket for a small lab. In a larger organization, the external stage may fit an established landing zone and independent retention policy. Both approaches still need an access audit, naming standard, encryption review, lifecycle rules and predictable cleanup.

5.3.2 Storage integrations and object access

An external stage delegates read access to a cloud location such as Amazon S3, Azure Blob Storage or Google Cloud Storage. Prefer Snowflake storage integrations and cloud-native trust policies over embedding long-lived access keys inside SQL. The integration has to be configured by a sufficiently privileged administrator and limited to expected locations. The object store's own policy must authorize the relevant Snowflake identity; the SQL object alone cannot grant access outside Snowflake.

Listing 3: External-stage shape; placeholders are not runnable credentials.

```
-- Administrator-reviewed example, NOT lab bootstrap:
CREATE STAGE energy_external
  URL = 's3://<approved-bucket>/energy/incoming/'
  STORAGE_INTEGRATION = <approved_integration>
  FILE_FORMAT = ff_energy_csv;
```

A secure integration is not merely "the connection works." It must be narrow enough that a mistaken path cannot expose unrelated company objects. Review region, cross-cloud egress, object-store request charges, identity rotation and who may replace an object after the manifest is signed.

5.3.3 Credential safety and file path conventions

Treat object paths as stable identifiers with clear semantics. For example, a path grouped by publisher, metric and date helps operational navigation, but does *not* replace the business observation key. Do not use a filename like `latest.csv` as a permanent identity: it can mean different bytes on two days.

Recommended landing layout

`landing/source=fi_energy_demo/contract=v1/`
`  published_date=2026-01-15/part-001.csv`
`manifest/source_manifest.csv`
`quarantine/batch_id=.../` (restricted access)

Authentication material belongs in approved secrets storage or short-lived identity providers; never in Git, notebooks, screenshots or example command history. For a demo, prefer an internal stage and local synthetic files. An external stage can be introduced once the permissions and cloud cost have been reviewed independently.

Incident question. What should happen when a file is replaced at the same path? The ingestion process must detect the checksum change, determine whether it is an authorized correction, and avoid assuming that a successful previous COPY makes the replacement safe.

Project reference: P050: AWS storage-integration infrastructure example. *Not independently executed; rights review pending.*

5.4 Loading data with COPY INTO

5.4.1 COPY INTO with named formats

`COPY INTO <table>` is the core SQL command for loading staged files into Snowflake. A reliable tutorial states the expected input and target shapes before running it. For this chapter each record has four fields: source code, metric, UTC interval and MWh value. Bronze accepts text as text so later validation can distinguish source defects from parser errors.

Listing 4: An auditable bronze target for unchanged source fields.

```
CREATE TABLE IF NOT EXISTS bronze_energy (
  source_code VARCHAR,
  metric VARCHAR,
  interval_text VARCHAR,
  value_text VARCHAR,
  source_file VARCHAR,
  source_row NUMBER,
  scanned_at TIMESTAMP_LTZ
);

COPY INTO bronze_energy (
  source_code, metric, interval_text, value_text,
  source_file, source_row, scanned_at
)
FROM (
  SELECT t.$1, t.$2, t.$3, t.$4,
         METADATA$FILENAME,
         METADATA$FILE_ROW_NUMBER,
         METADATA$START_SCAN_TIME
  FROM @stg_energy/ (FILE_FORMAT => ff_energy_csv) t
)
ON_ERROR = ABORT_STATEMENT;
```

This is a *COPY transformation*: the select list combines file fields and Snowflake virtual metadata columns. Use an explicit column list to protect against accidental target-column order changes. The example assumes an existing authorized database and schema selected in the session; the companion project includes a complete scoped setup. The `METADATA$START_SCAN_TIME` value records a scan boundary, not the publisher's original observation time [3].

Why text in bronze?
If `value_mwh` is initially loaded directly as `NUMBER`, a nonnumeric source token may abort the whole load or be skipped under a permissive error policy. Landing it as text makes the original value visible; a subsequent test can reject it with an attributable reason. This design is not a recommendation to skip validation indefinitely.

Project reference: P006: Snowflake Northstar data engineering guide. *Not independently executed; rights review pending.*

5.4.2 Duplicate-file loading behavior

Snowflake keeps load metadata for file-based COPY into a table and normally avoids loading a previously recorded file again. That mechanism is useful for operational reruns, but it is not a universal exactly-once guarantee: files can be renamed, changed, staged in another way, or reprocessed after historical metadata becomes uncertain. The FORCE=TRUE option deliberately bypasses normal duplicate-file avoidance and can create duplicate rows [4].

Scenario	Expected engineering response
Same staged file, repeat COPY	Usually skipped while load status is known; inspect results
Same content, different path	Treat as possible duplicate at business-key layer
Changed file under same path	Compare content key and publisher revision contract
FORCE=TRUE	Explicit repair only, with pre/post reconciliation
Old file with uncertain history	Follow documented older-file policy; do not guess

The older-file behavior is subtle. When load metadata is no longer decisive, Snowflake provides LOAD_UNCERTAIN_FILES and FORCE controls with different intents. Neither replaces a durable source manifest or a business-level deduplication rule [4]. An ingestion system should know which files it intended to process; the platform's recent history is supporting evidence, not its sole ledger.

Failure scenario. A scheduler crashes after Snowflake commits a load but before the orchestrator records success. Retrying exactly the same immutable file may be safely skipped by COPY's remembered status, but retrying a repackaged or renamed export may be processed again. The stable *business key* (source, metric and interval) is therefore checked in the accepted layer. File-level identity and observation-level identity solve different problems.

Guardrail.
Do not add FORCE=TRUE to a standard retry script. If a backfill genuinely requires reloading, isolate a batch, record its approval, choose a new versioned target or run a controlled reconciliation, and preserve the old manifest and load history.

Project reference: P052: Community end-to-end Snowflake pipeline. *Not independently executed; rights review pending.*

5.4.3 Loading history and parsing options

COPY INTO returns a result describing the files processed, rows parsed and loaded, and errors. Retain that statement's query ID and output as part of the orchestrator's batch record. A success message means the statement met its parser and load rules; it does not establish that the supplier provided every expected hour, used the correct unit or sent the latest revision.

For recent operations, Snowflake exposes an Information Schema COPY_HISTORY table function covering the past 14 days; a separate Account Usage view has a broader, delayed history [5]. Access rights, object names and display latency must be reviewed before designing a production monitoring dashboard.

Listing 5: Investigate recently processed files; example scope.

```
SELECT file_name, status, row_count, row_parsed,
       error_count, last_load_time
FROM TABLE(INFORMATION_SCHEMA.COPY_HISTORY(
  TABLE_NAME => 'BRONZE_ENERGY',
  START_TIME => DATEADD('day', -1, CURRENT_TIMESTAMP())
))
ORDER BY last_load_time DESC;
```

A meaningful *copy reconciliation* compares the expected files in the approved source manifest, the files that the load history reports, and the file/row counts actually present in the bronze table. These can disagree for legitimate reasons: a loaded file can have no valid business rows; a repeated file can be skipped; and an infrastructure failure can occur after the last file was committed but before the application received the result.

What to capture per ingestion attempt
`batch_id` and immutable manifest version; source/publisher; expected files, expected byte sizes and SHA-256; current Snowflake role/warehouse; started/completed UTC times; query ID; copy status; rows parsed/loaded; rejected rows; run result; a pointer to preserved logs.

Performance warning. Staging tens of thousands of tiny files can impose overhead even if their combined bytes are modest. Conversely, one extremely large compressed CSV can limit throughput for some parsing configurations. Choose file sizes using measured work with the real workload, not a universal number from a blog. Remember that COPY_HISTORY is not a permanent substitute for an organization-owned audit ledger [4,5].

Verification. After loading two immutable files, identify their filenames, expected line counts, the COPY query ID and how you will prove whether a second invocation added any rows. Record data and control-plane timestamps in UTC.

5.5 File validation and rejected-record handling

5.5.1 VALIDATION_MODE and ON_ERROR

A parser can fail because a file is malformed; a business rule can fail even when the file parses perfectly. Test both independently. The ON_ERROR COPY option controls what happens to rows or files with *load-level* errors. ABORT_STATEMENT gives this small bronze exercise an auditable all-or-nothing default; CONTINUE trades that property for partial progress [3,6].

Listing 6: Optional structural dry run for a four-column file.

```
-- Separate validation table with only source columns.
CREATE TEMP TABLE csv_shape_check (
  source_code VARCHAR, metric VARCHAR,
  interval_text VARCHAR, value_text VARCHAR
);
COPY INTO csv_shape_check
FROM @stg_energy
FILE_FORMAT = (FORMAT_NAME = ff_energy_csv)
VALIDATION_MODE = RETURN_ALL_ERRORS;
```

Critical restriction: Snowflake does *not* support VALIDATION_MODE in a COPY statement that uses a SELECT transformation (such as the provenance-enriched load on the preceding pages). Neither VALIDATION_MODE nor the VALIDATE table function is a general semantic data-quality engine [3,6]. In the companion lab, the validation check is separate, and a second query checks accepted numbers, times, keys and source revisions.

5.5.2 Rejected-record quarantine

Keep a quarantine record with the raw row, file path, row number, reason code and check version. Suggested reason codes distinguish bad timestamps, nonnumeric MWh, unknown metrics and unexpected sources. Do not equate a negative number with corruption until the domain contract says whether signed net flows are legitimate.

5.5.3 Actionable ingestion error reports

A useful incident report says which batch failed, which rule rejected it, how many rows are affected and whether publication is blocked. It includes a small redacted sample, a reproducible fixture and the relevant query ID. Alerting on *rejected rows divided by evaluated rows* helps distinguish a one-off defect from a schema-wide change. Source credentials and personal data never belong in the error report.

Project reference: P053: Data engineering with CoCo. *Not independently executed; rights review pending.*

5.6 Schema evolution, metadata, and provenance

5.6.1 Source-file and ingestion timestamp metadata

Metadata transforms a disconnected row into an attributable observation. A bronze table should answer at least three questions: which file supplied this value, where was it in that file, and when did Snowflake read it? Snowflake exposes virtual filename, file-row-number, content-key and scan-start-time fields for supported staged-file workflows [7]. Their exact SQL names appear in the example below.

Listing 7: Preview row identity and staged-file metadata.

```
SELECT METADATA$FILENAME AS source_file,
       METADATA$FILE_ROW_NUMBER AS source_row,
       METADATA$FILE_CONTENT_KEY AS content_key,
       METADATA$START_SCAN_TIME AS scanned_at,
       t.$1 AS source_code,
       t.$4 AS value_text
FROM @stg_energy (FILE_FORMAT => ff_energy_csv) t;
```

Do not confuse `FILE_CONTENT_KEY`, a Snowflake metadata value, with the SHA-256 calculated by a local Python manifest generator. They answer related but not necessarily identical questions, and should not be assumed comparable or interchangeable without an explicit contract. The `START_SCAN_TIME` value is a warehouse observation, not a guarantee of when a publisher produced the original file.

Field	What it actually establishes
Event time	Time interval represented by source data
Published/revised at	Publisher claim, if captured independently
Retrieved at	When our downloader received the bytes
Staged at	When the client placed the object in the stage
Scanned at	Snowflake stage-file scan timestamp
Accepted at	Time a semantic quality gate accepted a version

Preserving a single generic `created_at` timestamp sacrifices this distinction. If a regulator or analyst asks why an energy total changed, a complete evidence chain includes the original file, the publisher's revision, the ingestion job, and the acceptance decision. This is more important than any one cloud feature.

5.6.2 Schema drift and versioned contracts

A schema can change syntactically (new field, renamed column, reordered CSV), semantically (MWh becomes kWh), or operationally (the publisher stops sending weekend files). A naive schema-evolution setting addresses only a subset of those changes. For production, version both the *expected data contract* and the *mapping code*; record the contract version with each batch's manifest.

Change	Recommended initial action	Reason
Added optional field	Review and version mapping	Might change business meaning
Renamed identifier	Quarantine / block	Breaks business-key semantics
Unit change	Block until mapped	Can corrupt totals silently
Column reorder	Fail positional header test	Shifts values across fields
Long source outage	Publish stale-state alert	Freshness SLO at risk

Snowflake provides schema inference and automatic table schema evolution for supported load patterns, subject to configuration and privileges. Automatic schema evolution requires a table setting such as ENABLE_SCHEMA_EVOLUTION, column-name matching in the load, and an appropriate privilege; for CSV matching, header parsing and other parser settings matter [8]. These capabilities are not active by default in this deliberately narrow bronze load, and enabling them does *not* make unreviewed changes safe.

Decision: evolve or hold?
An added optional source description may be compatible with a contract update. A newly nullable key, changed time zone or altered unit cannot be accepted as a harmless extra column. Whenever a change could alter the meaning of an already-published metric, hold publication, preserve the new raw file, and require an explicit mapping version.

For a small teaching case, the companion Python validator checks the exact header and number of columns before upload. A separate semantic rule evaluates timestamps, metric codes and values after the bronze load. Readers can therefore distinguish a change in file *shape* from a change in *meaning*.

Independent check. Imagine the new export adds `source_timezone` but retains timestamps in UTC. Does the current contract still mean the same thing? Possibly—provided the new field does not change the definition of `interval_start_utc`. Now imagine those timestamps silently switch to Helsinki local time. A shape-only test passes; a data-contract review must fail.

5.6.3 Auditable provenance of transformed records

Provenance is a chain of evidence rather than a note attached to a dashboard. A credible pipeline can trace a published result backward through an accepted observation, its selection among revisions, the bronze row, the staged file, and the source manifest. At each transition, the system should name the transformation and the invariant it preserved.

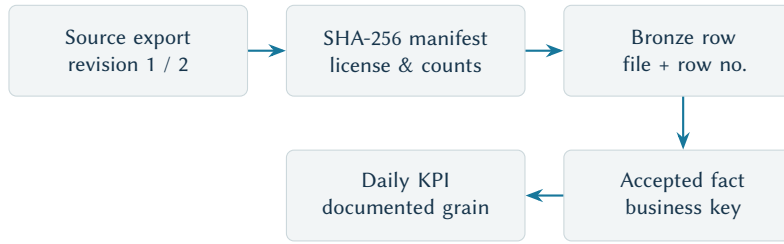


Figure 5.3. Traceability follows the selected revision into the reporting model.

A candidate load can pass its parser checks but fail the acceptance rule because the values are inconsistent with domain expectations. Retain both decisions: *loaded into bronze* and *accepted for analysis*. If a record is superseded, keep the earlier row traceable and mark the winner in a separate derived layer or documented view. Destroying revisions makes both reproducibility and incident investigation harder.

A practical source manifest has a stable batch ID, relative filename, SHA-256 digest, byte count, expected data rows, source revision, publisher and retrieval timestamps, contract version and source URL. A warehouse load ledger adds the COPY query ID, loaded count and status. Tie the two with a batch key rather than assuming the file name remains globally unique forever.

Data-lineage acceptance question
For the national generation KPI displayed at `2026-01-15 01:00` UTC, can you identify the source codes included, the accepted revision for each source, each staged file and the reason that an earlier reading was superseded? If the answer requires checking an engineer's private laptop, the publication process is not auditable.

Design trade-off. Keeping immutable source files has storage and retention costs, but can be significantly cheaper than explaining an irreproducible report. Apply a documented retention policy consistent with contracts and applicable data-protection duties; do not retain sensitive information merely because storage is easy.

Project reference: P047: API source enrichment and provenance. *Not independently executed; rights review pending.*

5.7 Idempotency, deduplication, and load auditing

5.7.1 Exactly-once expectations versus practical idempotency

Exactly-once is often used casually to mean "my job can run twice without creating an incorrect result." That is a useful business objective, but it is not achieved solely by file-level COPY history. A stronger, testable invariant is: *given the same approved manifest and source revisions, running the acceptance pipeline again produces the same set of accepted business keys and values.* This property is idempotency at the reporting boundary.

5.7.2 Detecting duplicates and replayed loads

Define the business key before removing duplicates. Our exercise uses `(source_code, metric, interval_start_utc)`; a revision is a new candidate for the *same* observation, not a seventh independent hour. The accepted result must choose the authorized highest revision deterministically. Sorting by arbitrary ingestion timestamp is insufficient if delayed files arrive out of order.

Listing 8: Find repeated business observations before publication.

```
WITH parsed AS (
 SELECT source_code, metric, interval_text,
        TRY_TO_DECIMAL(value_text,18,3) AS mwh
 FROM bronze_energy
)
SELECT source_code, metric, interval_text,
       COUNT(*) AS candidate_count
FROM parsed
GROUP BY source_code, metric, interval_text
HAVING COUNT(*) > 1;
```

This query detects multiple candidates; it does not decide which wins. The versioned manifest supplies publisher revision order; the acceptance transformation applies that order before aggregation. When a source lacks an explicit revision number, write a defensible tie-breaking policy and flag ambiguity rather than silently choosing the most convenient value.

5.7.3 Audit identifiers and reconciliation checks

Three invariants: (1) uploaded file count equals the approved manifest count; (2) bronze row count matches source rows plus any explicitly recorded partial-load deviations; (3) accepted business keys are unique, and their measure totals match the independently calculated fixture expectation. In the synthetic case used below, eight physical records contain one malformed value and one superseded revision: six current observations sum to **97.500 MWh**. These figures come from the companion fixture, not from a Finnish energy authority.

Project reference: P043: Retail data-engineering pipeline reference. *Not independently executed; rights review pending.*

5.8 Python-assisted batch ingestion

5.8.1 Python scripts for retrieval and staging

Python is a useful orchestration boundary when a publisher exposes an HTTP API but not a Snowflake-native export. Keep it small: fetch or open a source file, validate its declared structure, calculate a checksum, write an immutable manifest, upload via an authorized client, and invoke a reviewed load. Do not embed data-cleaning surprises inside an "easy" helper function. The offline fixture uses only Python's standard library and needs no credentials.

Listing 9: Core idea: a local file identity is byte-based.

```
import hashlib
from pathlib import Path

def sha256_file(path: Path) -> str:
    digest = hashlib.sha256()
    with path.open('rb') as stream:
        for chunk in iter(lambda: stream.read(1 << 20), b''):
            digest.update(chunk)
    return digest.hexdigest()
```

5.8.2 Batching, retry/backoff, and rate limits

HTTP download code should classify failures. A 401 or 403 usually requires a credential or authorization fix; a 429 calls for rate-limit handling; a transient 5xx might be retried with backoff and jitter. Set connect/read timeouts, a maximum attempt count and a maximum elapsed time. Respect the provider's `Retry-After` header where relevant. Resume only when the protocol and file identity make partial downloads safe; otherwise discard the incomplete file and start a new temporary download.

5.8.3 Secret handling and idempotent invocations

Promote a temporary file to its immutable, checksum-named destination only after integrity checks pass. A manifest update should be atomic from the orchestration perspective; publish it only after all listed files are present. Keep tokens in a secrets manager or narrowly scoped environment variables, never in the dataset, logs or repository. Restrict local staging permissions.

The retry trap.
Retrying a GET request may download the same logical revision twice under different local filenames. If those filenames become new staged objects, Snowflake file-level duplicate detection may not recognize a repeated business event. Always reconcile the publisher revision ID or stable business key independently of local file naming.

Hands-on task. Run `python validate_fixture.py`. Change one CSV header, then rerun. The test must fail before any upload. Restore the header, set a value to `N/A`, and observe structural success but semantic quarantine.

Project reference: P054: AWS-to-Snowflake batch ETL example. *Not independently executed; rights review pending.*

5.9 Recovery procedures and ingestion monitoring

5.9.1 Recovery point selection and reprocessing

A recoverable ingestion process knows the last *approved manifest*, not merely the timestamp of the last scheduler success. On failure, establish whether the source was retrieved, the bytes staged, the copy committed and the acceptance output published. Retrying every step blindly can be wasteful or unsafe; a partial run should resume from the most recent verified boundary.

Observed failure	First safe diagnostic
Source fetch timeout	Compare downloaded checksum, retry policy, publication ID
Upload incomplete	Check stage listing vs approved manifest
COPY aborted	Preserve query ID; examine parser error, do not force reload
Bronze loaded, audit missing	Query COPY history and target before retrying
Acceptance test fails	Hold gold publishing, inspect quarantine and revisions

5.9.2 Observability dashboards and alerts

Track five independent indicators: expected versus arrived files, successful versus rejected rows, accepted-key uniqueness, data freshness, and estimated ingestion cost. Alerting must distinguish no new data from no new *valid* data. A dashboard that updates despite a failed source contract is not proof of pipeline health. Use incident severity and operational ownership appropriate to the organization rather than inheriting a demo's arbitrary thresholds.

5.9.3 Cleanup, retention, and operational handover

Stop or suspend lab compute when finished, remove only disposable chapter-scoped objects after explicit review, and keep the evidence needed for a reproducible acceptance test. Do not issue unrestricted `REMOVE @stage` or `DROP DATABASE` commands in a shared environment. An operations handover should contain the source owner, schema contract version, runbook, escalation route, cost controls, backfill procedure and rollback trigger.

Recovery decision checklist
Record the incident ID; manifest ID; approved file list; target bronze count before/after; relevant Snowflake query IDs; failed acceptance tests; selected replay approach; authorization for any forced reload; and a post-recovery comparison of the expected business-key set.

Project reference: P055: Snowflake Alert Hub reference. *Not independently executed; rights review pending.*

5.L Guided lab: a trustworthy bronze ingest

5.L.1 Prerequisites, inputs, and cost guardrails

This lab continues the same *synthetic* Finnish-energy-themed fixture introduced in Chapter 4. It intentionally contains a source revision and an invalid value so that a passing parser is not confused with a passing data-quality gate. Input: `energy_r1.csv` (six records), `energy_r2.csv` (a later correction and one invalid reading), and an automatically generated `source_manifest.csv`. Expected raw rows: eight; accepted unique observations: six; accepted total: **97.500 MWh**.

An optional cloud run requires a disposable database/schema already authorized by the user, a limited ingest role, a small warehouse with auto-suspend, and a named internal stage. Assess credit charges before execution. The offline fixture verification requires no Snowflake account and no paid API calls. Never use production credentials or ACCOUNTADMIN as an everyday lab role.

5.L.2 Procedure and selected code

Step 1: Generate and verify the manifest before uploading; assert the header, row counts, file hashes and revision order. **Step 2:** Create the named CSV format, stage and bronze table using `sql/file_formats.sql` and `sql/load_batch.sql`. **Step 3:** Use a supported client to PUT the two files to the stage, leaving the immutable names unchanged. **Step 4:** Run the transformed COPY once; preserve its output and query ID. **Step 5:** Run quality queries, build the authorized revision selection and compare expected against actual.

Listing 10: The essential reconciliation: physical vs accepted rows.

```
SELECT COUNT(*) AS bronze_rows,
       COUNT(DISTINCT source_file) AS files_seen
FROM bronze_energy;
-- Expected after one successful copy: 8 rows, 2 files.

SELECT COUNT(*) AS invalid_numeric_rows
FROM bronze_energy
WHERE TRY_TO_DECIMAL(value_text,18,3) IS NULL;
-- Expected: 1 invalid numeric row in the fixture.
```

Step 6: Rerun the same COPY *without* FORCE; inspect returned status and count changes. If the status is uncertain, stop rather than making repeated exploratory writes. **Step 7:** Confirm that the accepted layer chooses the revised solar value of 13.000 instead of the older 12.500. The manifest's revision order is authoritative for this synthetic exercise.

Evidence requirement.
A screenshot of "query completed" is not a test. Preserve the input manifest, COPY query ID, target counts, rejected-record report, deterministic revision rule and the six accepted business keys. The companion archive includes an offline validator and expected results.

5.L.3 Expected results, failure diagnosis, and cleanup

The synthetic fixture produces a reproducible acceptance story: **two** unique files, **eight** physically loaded rows, **one** invalid numeric reading, **one** superseded revision, **six** accepted source/hour observations, and **97.500 MWh** across those accepted observations. These are fixture-derived assertions, not a claim that an actual Snowflake account was used in preparing the manuscript. A cloud reviewer should run the same queries in a clean account and save the real COPY results before release.

If the counts differ, investigate in order: (1) manifest file name/hash and record count; (2) stage contents and CSV parser; (3) copy history and query ID; (4) raw bronze row identity; (5) invalid timestamp/numeric rule; (6) source-revision precedence; (7) group-by business grain. Do not start by changing `FORCE` or removing duplicate rows with a blind `DISTINCT`. The lab's cleanup file contains *commented* destructive statements only and an explicit warehouse-suspension reminder.

5.L.4 Exercises, design review, and assessment

1. **Foundation:** Explain why an 8-row bronze table does not imply eight valid energy observations. Name at least three distinct time fields in an ingestion ledger.
2. **Practitioner:** Introduce a third revision to one observation. Modify the manifest and test so the accepted count remains six, while the total reflects exactly the authorized replacement value.
3. **Architecture:** Compare one daily immutable CSV file with 24 small hourly files. Defend a choice using freshness, cost, recovery, source rights and operational complexity.

Scoring rubric (10 points). Source contract and rights (2); parser/shape fidelity (2); manifest and provenance (2); deterministic acceptance (2); safe recovery and evidence (2). **Pass criterion:** at least 8/10 and no missing security or correctness gate.

Chapter 5 in one idea. Ingestion is a repeatable claim about which source bytes arrived, which records were accepted, and why a rerun will not rewrite the business story. Chapter 6 will extend this discipline to nested and semi-structured payloads.

Official reference notes (accessed 22 October 2026).

[1] Snowflake, CREATE FILE FORMAT. [2] CREATE STAGE. [3] COPY INTO table. [4] Loading considerations. [5] COPY HISTORY. [6] Bulk-load troubleshooting. [7] Staged-file metadata. [8] Schema evolution.

CHAPTER 06 DATA ENGINEERING / SEMI-STRUCTURED

JSON and Semi-Structured Data

Turning changing API payloads into dependable analytical facts

The dashboard did not break. Its meaning did.
At 08:00, the energy dashboard reports the correct daily total. At 09:00, a supplier adds a nested `unit` field, changes `observations` to `measurements`, and represents a missing reading as JSON `null`. The ingestion job still says *success*. A naive flattening query silently returns zero observations for the new payload. The next chart is empty, not because production stopped, but because the data contract changed.

In Chapter 5, we preserved exact file provenance before accepting source records. This chapter extends the same discipline to nested data. A JSON document is useful because it preserves the source's hierarchy; a data warehouse is useful because it gives that hierarchy consistent analytical meaning. Connecting the two requires more than a cast and a FLATTEN.

What you will learn

- decide when to retain flexible JSON in VARIANT and when to normalize it;
- distinguish parse errors, missing paths, JSON `null`, SQL NULL and wrong types;
- traverse objects and arrays and reason about the number of rows produced;
- handle publisher schema revisions without hiding invalid observations;
- turn mixed-version energy API messages into typed, tested silver-layer records.

Prerequisites. Chapters 4–5, basic SELECT/JOIN, and familiarity with a JSON object. The companion includes synthetic fixtures and an offline Python validator, requiring no cloud account. Optional Snowflake SQL requires an approved limited role and a small, auto-suspending warehouse. No Snowflake execution or performance benchmark is claimed for this manuscript.

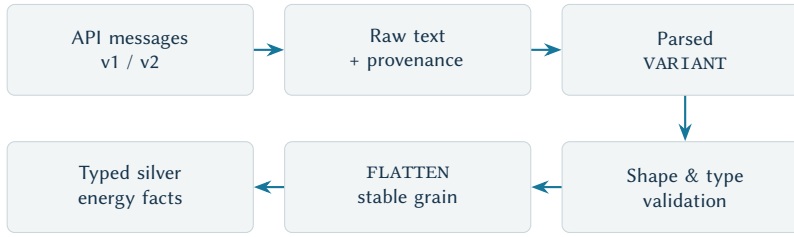


Figure 6.1. Preserve the original message while promoting only validated readings.

6.1 Semi-structured data and the VARIANT type

6.1.1 Semi-structured payloads and VARIANT

A relational table gives each column a declared type and each row an expected shape. API documents often do not. A payload may combine an envelope, a provider identifier, a timestamp, an optional quality flag and a variable number of measurements. Schema variation is not necessarily a data-quality failure: the publisher may be introducing an optional field legitimately. The engineering challenge is to preserve useful flexibility without abandoning predictable outputs.

Snowflake's VARIANT type stores a single semi-structured value, including a JSON-derived object, array, scalar or null. It is not the same as putting opaque JSON *text* into a VARCHAR. Parsing interprets the hierarchy, so queries can navigate paths and work with nested values. Yet the original text may remain important for legal provenance, replay, cryptographic comparison or investigating a parse failure. A robust bronze boundary can preserve both the exact text and a parsed representation, with separate source identifiers.

Listing 1: Inspect the type after parsing a synthetic API message.

```
SELECT TYPEOF(PARSE_JSON(
  '{"provider":"synthetic-fi","observations":[{"mwh":12.5}]}'
)) AS document_type;
-- Expected type: OBJECT (illustrative, not cloud-tested).
```

The most important distinction is **schema flexibility versus semantic ambiguity**. A publisher can change the arrangement of fields without altering the meaning of *energy measured during the UTC interval.* But if a field called `value` changes from MWh to kWh, a successful PARSE_JSON does nothing to prevent a thousandfold reporting error. Structure and meaning require separate contracts.

A two-layer contract

- *Envelope contract:* source ID, retrieval time, schema version, valid JSON, payload hash, publisher terms and message identity.
- *Observation contract:* metric identity, UTC observation time, unit, numeric value, acceptable domain and deduplication key.

The first contract helps you replay a message accurately; the second tells you which rows should count in a report. Do not confuse storing a document with accepting every measurement it contains. Snowflake's documented guidance permits keeping the flexible structure in VARIANT while promoting high-use attributes into typed columns when workloads are understood.

Project reference: P058: Document AI introductory reference. *Not independently executed; rights review pending.*

6.1.2 Parsing types and nested values

JSON's small type system creates surprisingly large modelling decisions. Its values may be objects, arrays, strings, numbers, booleans or `null`; it does not define a native timestamp, binary data type or the business meaning of a measurement. The string `"12.50"` and numeric value `12.50` may look identical in a dashboard, but a typed transformation should treat them differently until an explicit contract authorizes conversion.

Incoming form	Example and engineering response
Numeric JSON value	`12.50`; check range and units before using
Numeric string	`"12.50"`; parse deliberately and record conversion
Timestamp string	`"2026-01-15T10:00:00Z"`; normalize to UTC
Object or array	Nested provider information; validate expected structure
JSON null	Explicitly supplied null; apply missing-data policy
Missing key	No supplied value; distinguish from explicit null

`TYPEOF` and type predicates help diagnose unexpected values. Consider a measurement property that was a number yesterday but becomes an object with `amount` and `unit` keys today. A blind cast may raise an error, or a try-cast may yield SQL `NULL`. Neither outcome tells the analyst *why* it happened. Your schema-drift report should record the encountered path and type, not simply discard the row.

Listing 2: Inspect the nested value before applying a numeric contract.

```
WITH src AS (
  SELECT PARSE_JSON('{"value":{"amount":12.5}}') AS v
)
SELECT TYPEOF(v:value) AS value_kind,
       TYPEOF(v:value:amount) AS amount_kind,
       v:value:amount::VARCHAR AS raw_amount
FROM src;
```

The shape `OBJECT` is expected for the outer `value`; the inner `amount` is a number. Cast only after the path and shape have been checked. Native JSON numeric literals may preserve fixed-point values, but the parsing engine can use floating representation for scientific notation or values outside fixed-point bounds. Never infer measurement precision from a screenshot of pretty-printed JSON.

Practical judgment. The bronze schema should not be redesigned every time a nonessential source field appears. The silver schema *should* change through an explicit review whenever a published business metric changes meaning. Keep this separation throughout the book.

6.1.3 Choosing schema-on-read versus early normalization

Schema-on-read means preserving a relatively flexible representation and interpreting selected paths when queried. *Early normalization* means parsing, validating and promoting the important attributes into relational columns soon after ingestion. Neither approach is a universal winner. The design should follow the consumer's contract and the likely failure modes.

Use a flexible bronze object when a new source is exploratory, nested fields change frequently, or replay of previously unseen attributes is valuable. Use typed silver columns for stable KPIs, joins, constraints, timestamps and large repeated workloads. In the Finland Energy Analytics platform, bronze retains the provider's JSON, while silver represents one row per `provider + technology + UTC interval`. An analytical aggregate must never depend on whether the provider called an array `observations` or `measurements`.

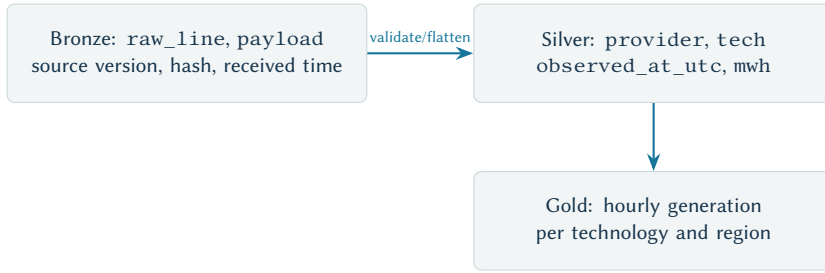


Figure 6.2. Promote a well-defined business grain, not an accidental JSON shape.

A less obvious issue is *reprocessing*. A schema correction might require recreating a silver table from last week's original messages. If bronze keeps only a destructive transformation and discards the payload, the repair may be impossible without retrieving the source again. Preserving exact source text and publishing a migration plan makes that recovery feasible.

Decision rule.
Choose `VARIANT` as a flexible boundary, not an excuse to delay contracts forever. Materialize typed, frequently queried fields after measurement, and retain original content long enough for your documented audit and replay obligations. The appropriate retention period depends on rights, cost and organizational policy.

Check. Identify a new optional provider tag that should remain flexible, and a unit-of-measure change that must block publication until approved. Explain why they deserve different treatment.

6.2 OBJECT and ARRAY structures

6.2.1 Constructing OBJECT values

An OBJECT represents named attributes, not a positional record. It is useful for packaging a set of related fields while preserving natural source hierarchy. Snowflake provides OBJECT_CONSTRUCT; its treatment of SQL NULL values is significant: key/value pairs with a SQL-null value are omitted, whereas OBJECT_CONSTRUCT_KEEP_NULL retains them as JSON nulls. This is more than a formatting choice when a downstream consumer distinguishes *not provided* from *explicitly cleared*.

Listing 3: Construct a compact provider object with explicit null behavior.

```
SELECT OBJECT_CONSTRUCT(
         'provider', 'synthetic-fi',
         'revision', 2,
         'note', NULL
       ) AS omitted_note,
       OBJECT_CONSTRUCT_KEEP_NULL(
         'provider', 'synthetic-fi',
         'revision', 2,
         'note', NULL
       ) AS retained_note;
```

An object has named keys, but do not assume any stable order of keys when printed or converted back to JSON text. Source hashes should be calculated over the original source bytes, not by assuming TO_JSON(PARSE_JSON(x)) reproduces those bytes exactly. Whitespace, key order and normalization can change.

6.2.2 Constructing ARRAY values

An ARRAY represents a sequence. In our payload it contains readings for one observation interval. Arrays are zero-indexed in JSON-style path operations, and position can matter when order encodes source priority or telemetry sequencing. Use ARRAY_CONSTRUCT to build illustrative sequences; use ARRAY_SIZE to inspect expected cardinality, and GET to retrieve elements dynamically.

Listing 4: Object versus array: names compared with positions.

```
SELECT ARRAY_CONSTRUCT('wind','solar','hydro') AS techs,
       GET(ARRAY_CONSTRUCT('wind','solar','hydro'),1)
         AS second_technology;
-- Index 1 refers to the second array element.
```

For ingestion, an empty array is not equivalent to a missing array or a malformed array. An empty array may be a legitimate “no readings” batch; a missing field may signal schema drift; an object where an array was promised is a contract violation. Capture each case separately.

Project reference: P059: Unstructured-data extraction reference. *Not independently executed; rights review pending.*

6.2.3 Safe traversal and array/object type inspection

Paths are not a substitute for shape checks. Suppose a supplier sends a `meter` object for one row and an array of `meter` objects for another. Both parse as valid JSON, but the query `payload:meter:id` only expresses the first structure. A robust mapper either supports both variants by version or rejects the unexpected version explicitly.

For diagnostic exploration, ask three questions in order: Does the path exist? What type is its value? Is the value allowed by the business contract? A path can exist and contain JSON null; it can be a number when a string is expected; or it can be an array of objects instead of one object. These distinctions determine whether to default, quarantine or migrate.

Listing 5: Non-destructive shape inspection of an array field.

```
WITH src AS (
 SELECT PARSE_JSON('{"observations":[{"tech":"wind"}]}') v
)
SELECT TYPEOF(v) AS document_kind,
       TYPEOF(v:observations) AS readings_kind,
       ARRAY_SIZE(v:observations) AS readings_count
FROM src;
```

Array cardinality is a data contract. If a publisher guarantees one reading per technology, duplicated technologies in an array can double totals unless the consumer enforces a uniqueness rule. Conversely, a missing technology might mean the source did not report it; automatically writing zero would make absence look like measured zero. For this book, each accepted observation is keyed by provider, technology and UTC interval. An array position is retained as provenance, not used as a stable business key.

Which default is safe?
A missing `comment` can normally remain `NULL`. A missing `unit` is unsafe when MWh and kWh may coexist. An empty `observations` array should trigger a source completeness check; a JSON null `mwh` must never be summed as though it were an actual zero.

An automated test fixture should cover valid object, empty array, missing array, wrong field type, duplicate reading key and explicit null. These cases cost little to construct and detect a surprising range of production failures.

6.3 Parsing, inspecting, and validating JSON

6.3.1 PARSE_JSON and TRY_PARSE_JSON

PARSE_JSON interprets JSON text and returns a VARIANT; a malformed nonempty input raises an error. TRY_PARSE_JSON returns SQL NULL when parsing fails. That behavior is valuable for quarantine, but it is not a complete validity test: null input, empty or whitespace-only input, and invalid JSON require explicit handling. In Snowflake, the text `'null'` parses to a JSON null VARIANT, which differs from an SQL-null parse result.

Listing 6: Observe four distinct parser outcomes.

```
SELECT TRY_PARSE_JSON('{"a":1}') AS valid_object,
       TRY_PARSE_JSON('{bad json}') AS invalid_text,
       PARSE_JSON('null') AS json_null,
       PARSE_JSON(NULL) AS sql_null;
```

6.3.2 Invalid payload detection

Do not simply discard rows where TRY_PARSE_JSON returns null. Preserve the raw input, manifest, position, error class and failed contract version. A failure may be due to invalid syntax, an empty body, unsupported shape, duplicate object key or a valid document with invalid measurements. Snowflake's strict JSON parser rejects duplicate object keys by default; permissive duplicate-key parsing is a deliberate trade-off because the last value wins and information is lost.

6.3.3 Test fixtures for valid and malformed JSON

The companion file `data/sample_api_payload.jsonl` includes two valid envelope revisions, one syntactically valid but semantically unacceptable envelope and one malformed line. That arrangement tests both parsing and business acceptance. The offline verifier reports the original line number without sending payloads to an external service.

Testing invariant.
Every physical input line must have exactly one recorded outcome: *accepted envelope*, *quarantined syntax*, or *quarantined contract*. The sum of these categories must equal the physical input line count. Never confuse "the parser did not throw" with "the data are fit for analytics."

Project reference: P060: Community JSON pipeline. *Not independently executed; rights review pending.*

Project reference: P061: Streams-and-tasks JSON pipeline. *Not independently executed; rights review pending.*

6.4 Querying nested keys and paths

6.4.1 Path notation for nested attributes

A path such as `payload:source:provider` is shorthand for selecting a nested attribute. Array positions use brackets, for example `payload:observations[0]:tech`. A valid path can return a VARIANT, not yet the typed string or number your model needs. Quoted key syntax and GET_PATH are helpful when key names contain punctuation or are dynamic. Document a versioned path map rather than scattering dozens of fragile path expressions across dashboards.

6.4.2 Casting nested values

Explicit casts serve two purposes: they provide a predictable SQL type and make an unapproved publisher change visible during transformation. Where unexpected input must be quarantined rather than halt every row, convert to a string and use the appropriate TRY_TO_* function, then record both the attempted input and the failure reason. A silent null is not a sufficient audit event.

Listing 7: Access a nested energy value and convert deliberately.

```
WITH src AS (
 SELECT PARSE_JSON(
   '{"reading":{"mwh":"12.500","unit":"MWh"}}') v
)
SELECT v:reading:unit::VARCHAR AS unit,
       TRY_TO_DECIMAL(v:reading:mwh::VARCHAR,18,3)
         AS energy_mwh
FROM src;
```

6.4.3 Null, missing, and unexpected type handling

A missing key produces SQL NULL during path extraction. A present JSON null can be distinguished with IS_NULL_VALUE; a value present with the wrong type must be handled as a separate contract failure. Consider how each state should appear in audit statistics, and avoid a single "failed to cast" counter that hides materially different root causes.

The decision at the dashboard boundary
For a measured energy total, SQL NULL, JSON null and a missing key may all yield no numeric value, but their cause is different. Preserve the cause in a quarantine reason code. For an optional comment field, a null result may be acceptable; for a required MWh measurement, it is not.

Check. If a supplier starts sending an `mwh` object instead of a numeric `mwh` value, identify the failing path and explain why applying COALESCE(...,0) before contract validation would conceal the error.

Project reference: P062: Nested multimodal extraction example. *Not independently executed; rights review pending.*

6.5 Unnesting arrays with FLATTEN and LATERAL

6.5.1 The FLATTEN table function

An array is often the natural API representation for a collection of measurements. An analytical fact table normally uses one row per observation. Snowflake's FLATTEN function bridges those shapes by emitting rows for the elements of a semi-structured container. Among the output columns are VALUE, INDEX, PATH and KEY. For array elements, INDEX records the original position: it is useful when auditing whether a publisher reordered or omitted a reading.

6.5.2 LATERAL usage and row cardinality

The LATERAL join permits FLATTEN to reference a field from the envelope row to its left. One envelope with three readings produces three flattened rows; two envelopes with three readings each produce six. An ordinary inner-style lateral flatten on an empty array produces no child rows. Consequently, counting resulting rows alone cannot prove that no input envelopes were lost. A separate envelope-level validation must reconcile accepted, empty and invalid arrays.

Listing 8: One output row per measurement in an envelope.

```
WITH src AS (
 SELECT PARSE_JSON(
  '{"observations":[{"tech":"wind","mwh":21.25},'
  || '{"tech":"solar","mwh":12.50}]}') AS doc
)
SELECT f.index AS source_array_index,
       f.value:tech::VARCHAR AS technology,
       f.value:mwh::NUMBER(18,3) AS mwh
FROM src s,
     LATERAL FLATTEN(INPUT => s.doc:observations) f;
```

6.5.3 Deep arrays and array position preservation

Chained LATERAL FLATTEN calls are appropriate for nested arrays only when the mapping needs both levels. For example, a response may contain regions, each with hourly readings. Record the outer region identifier and inner reading position before aggregating. Recursive flattening can explore irregular documents, but exploratory recursion is not a substitute for a specific, reviewed business grain.

Cardinality check.
If the contract promises three unique technologies per hourly envelope, require `COUNT(*) = 3` and `COUNT(DISTINCT technology) = 3` after flattening each envelope. A row count of six over two envelopes is necessary but not sufficient: a duplicate technology could replace a missing one.

Failure case. A publisher includes two arrays: readings and quality checks. Flattening both together may multiply rows. Flatten only the needed path; aggregate separate arrays at the correct grain.

Project reference: P063: Nested arrays from multimodal analysis. *Not independently executed; rights review pending.*

6.6 Handling schema drift and missing values

6.6.1 Compatible schema changes

Schema evolution is a contract negotiation, not an automatic casting trick. A new optional metadata field can be compatible with existing consumers. Renaming a required observations array may be compatible only if the mapper knows which provider schema versions use the old and new paths. Changing a unit or redefining the observation interval is potentially breaking even if the JSON structure remains identical. Version your publisher-to-canonical mapping and document the effective date.

Our synthetic energy feed illustrates this. Version 1 contains `observations`; version 2 contains `measurements`. Both map to the same canonical business grain. The mapper must inspect the declared `schema_version`; a naive COALESCE of two paths is convenient for exploration but not a complete contract strategy when both fields can appear or have different semantics.

6.6.2 Defaulting versus quarantining records

Defaulting is appropriate only when the missing value has a documented and low-risk interpretation. An absent optional quality comment may be defaulted to an empty display string. Missing `mwh`, a non-numeric reading, or an unrecognized unit must be quarantined. Setting missing measurements to zero suppresses errors and fabricates a measurement. Record at least the offending envelope, path, expected type, observed type and failed rule.

6.6.3 Tracking missing fields and attribute drift

Maintain an observable schema signature: version, required-key coverage, keys seen, type distribution, unknown paths and null frequency. Compute it over received envelopes *before* publishing silver rows. Differences between yesterday and today are signals for review, not always automatic failures. A whitelist of explicitly supported versions is often more reliable than guessing based on the presence of individual keys.

Change	Assessment	Outcome
New optional tag	Nonbreaking if documented	Preserve in bronze
`observations` renamed	Versioned mapper needed	Normalize after review
`mwh` becomes `null`	Required numeric absent	Quarantine reading
`unit` changes to kWh	Semantic contract change	Convert only if authorized

Production lesson. Successful query completion is not evidence of unchanged semantics. Monitor both the physical event stream and the accepted record stream. An increasing percentage of quarantined rows should raise a source-quality alert before the dashboard quietly loses coverage.

Project reference: P064: ShareWatch schema-drift reference. *Not independently executed; rights review pending.*

6.7 Parquet, mixed-format sources, and normalization

6.7.1 Columnar Parquet payloads

JSON is convenient for transporting variable nested records. Parquet is a column-oriented file format designed for efficient storage and analytics, with schema and encoding metadata. It commonly appears in cloud landing zones and lakehouse architectures. Parquet can itself contain nested structures; do not assume a Parquet file is already a clean relational table. Snowflake supports loading and transforming staged Parquet data through appropriate file-format and COPY INTO workflows.

6.7.2 Reconciling mixed schemas

Suppose two publishers provide conceptually identical electricity readings: API A sends JSON with a numeric `mwh`; export B sends Parquet with a field named `energy_kwh`. Both should map through source-specific adapters into one canonical model. Normalize units, timestamps, source IDs and technology vocabulary explicitly. Combining files first and hoping implicit casts fix their differences makes lineage hard to explain and mistakes hard to reverse.

Listing 9: Illustrative typed mapping after a reviewed conversion.

```
-- For an authorized source that declares kWh:
SELECT TRY_TO_DECIMAL(energy_kwh::VARCHAR,18,3)
         / 1000 AS energy_mwh
FROM approved_parquet_staging;
-- This query assumes a previously created typed staging
-- relation. It is not the statement that loads Parquet.
```

6.7.3 Typed silver-layer normalization

Define a transport-independent silver contract: provider, technology, UTC interval, energy in MWh, source envelope ID, array position, and schema version. Persist validated measurements with decimal precision suitable for the use case. For a production system, retain a documented rounding policy and reject non-finite or out-of-range values; do not trust automatic format conversion to determine financial or scientific precision.

Compatibility is a property of the contract, not the extension.
A well-defined JSON adapter and a well-defined Parquet adapter may produce identical silver rows. A badly documented Parquet file can be less trustworthy than a carefully documented JSON feed. Data quality is demonstrated by tests and provenance, not guaranteed by a file format.

Reader check. Explain how you would compare two publishers with different currency-like unit scales, duplicate correction policies and timestamp conventions. Which rules belong in the source adapters, and which belong in the canonical energy fact contract?

Project reference: P002: Iceberg table reference. *Not independently executed; rights review pending.*

Project reference: P065: Data crawler reference. *Not independently executed; rights review pending.*

6.8 Performance and modeling considerations

6.8.1 Selective projection and flattening cost

The most expensive semi-structured transformation is sometimes the unnecessary one. A single API document with hundreds of attributes may be stored and inspected without flattening every nested array. Select only paths required by the consumer and preserve the rest in bronze. Flattening an array with n elements produces n child rows; flattening two independent arrays with m and n elements in the same row can produce $m \times n$ combinations. Multiplication is a correctness problem before it becomes a performance problem.

For the energy application, suppose 100,000 hourly envelopes each hold ten technology readings. The theoretical full flattening produces one million rows before filters and later grouping. This is an illustrative cardinality calculation, not a measured Snowflake benchmark. If a report needs only wind power, project that technology deliberately and compare filtering in the nested representation with filtering a typed silver table. The best implementation depends on the data distribution, workload and storage layout.

Listing 10: Project only the contracted columns needed downstream.

```
SELECT e.envelope_id,
       f.index AS source_position,
       f.value:tech::VARCHAR AS technology,
       TRY_TO_DECIMAL(f.value:mwh::VARCHAR,18,3)
         AS energy_mwh
FROM accepted_envelopes e,
  LATERAL FLATTEN(INPUT => e.payload:observations) f
WHERE f.value:tech::VARCHAR = 'wind';
-- Illustrative query: accepted_envelopes is a lab view.
```

In an environment with frequent queries, repeated casting and flattening can add latency and cost. It can be cheaper operationally to materialize selected, validated fields into a stable silver table. But materialization introduces its own refresh, storage, reconciliation and deletion responsibilities; make that trade-off explicit rather than assuming duplication is free.

Measure the real workload.
Performance claims require comparable query text, warehouse settings, warm/cold cache disclosure, data volume, time window and correctness checks. Never describe a faster result as an optimization if it returns fewer valid readings. Chapter 12 develops a controlled measurement method.

Project reference: P066: Document extraction modelling reference. *Not independently executed; rights review pending.*

6.8.2 Pruning limits with nested attributes

Snowflake stores data using columnar optimizations, including extraction of some semi-structured paths. Nevertheless, not every nested attribute behaves like a dedicated relational column. In particular, inconsistent types, JSON null values and complex arrays can interfere with efficient extraction of individual paths. Snowflake's published considerations recommend promoting some frequently filtered or non-native fields into native columns after evaluating a representative workload.

The implication is practical: `payload:observed_at` is often a string in source JSON, whereas `observed_at_utc TIMESTAMP_TZ` is a typed column in silver. A query filtering one UTC day may perform differently across these layouts. It is not enough to infer from SQL readability that the storage engine will scan identical amounts of data.

Design	Consequence to evaluate
Raw `VARIANT` only	Maximum source flexibility; repeated path access and casting
Nested path projection	Avoids irrelevant fields; still depends on nested representation
Typed silver columns	Predictable types, joins and filters; additional refresh/storage
Flattened child table	Reusable one-row-per-reading grain; expansion and maintenance

The correct comparison measures the same answer over the same accepted business keys. If `VARIANT` contains seven records but silver quarantines one invalid record, raw and silver totals cannot be compared as equivalent workloads until the eligibility filter is aligned. Explain this invariant before teaching Query Profile: otherwise readers may accidentally reward the query that silently drops bad data.

The limits of intuition. Nested paths are not automatically slow; Snowflake documents cases where relational and well-behaved semi-structured data show similar performance. Nor is a flattened schema automatically superior for sparse, high-cardinality optional attributes. The storage and compute decision should be revisited if query patterns shift, source versions multiply, or analysts repeatedly use the same three nested fields.

Experiment specification
Compare (A) `VARIANT` path filtering and (B) a typed silver table with identical validation rules. Record row counts, checksum or totals, scanned data, elapsed time, warehouse size, cache state, and estimated cost. Run repetitions and report the distribution rather than picking the fastest run.

6.8.3 Documented modeling and performance decisions

A production design decision becomes more useful when recorded as a small architecture decision record (ADR). For our synthetic energy platform, write down the context: a publisher sends nested hourly documents, a reporting team needs reliable aggregates, optional metadata evolves, and invalid records must remain traceable. Then state the decision: retain raw text and parsed JSON for audit; apply explicit versioned adapters; create typed silver readings for repeatable joins and reporting.

ADR-006 — Normalize high-value energy readings
Context: Nested API envelopes change over time; dashboards require stable UTC hourly facts.
Decision: Store immutable source text and `VARIANT`; reject unknown required versions; flatten approved observation arrays; materialize typed readings.
Consequences: Stronger audit and consumer stability; additional storage, change-control and pipeline maintenance.
Revisit when: Schema change frequency, workload, costs or required freshness differ materially.

Monitoring matters as much as speed. Track parse failure rate, unsupported-version rate, arrays with zero or excess elements, missing business keys, invalid MWh count, accepted observation count and data freshness. For every metric, agree on an owner and a response. A spike in unknown source fields should create a schema-change review; an unexplained fall in the accepted observation count should stop publication rather than merely produce a smaller chart.

The upstream source contract also affects recovery. If a malformed message is later republished correctly under the same publisher message ID, the new revision should be preserved rather than overwriting the earlier bytes. If the provider does not expose stable IDs, define an author-owned fingerprint from source, retrieval metadata and content hash, and record its limitations. Such a fingerprint identifies a received message, not necessarily a unique real-world observation.

What we can and cannot conclude. The examples in this chapter are original teaching material with deterministic synthetic expectations. They do not establish the best performance profile for a particular Snowflake edition, region, warehouse size or production source. External GitHub repositories listed as references are candidates for further study, not proof that every dependency, license or execution path has been independently checked.

Chapter decision summary.
Preserve raw bytes to know what arrived; parse without hiding failures; approve schema-version mappings; flatten with explicit row grain; cast with validation; and publish stable silver facts only when acceptance evidence is complete.

Project reference: P067: Automated document-derived pipeline example. *Not independently executed; rights review pending.*

6.L Guided lab: energy API payloads to silver facts

6.L.1 Inputs, prerequisites, and cost guardrails

This lab continues the *synthetic* Finland Energy Analytics case from the earlier chapters. The data are not observations from Fingrid or another real publisher. Four JSONL physical lines represent: a valid v1 document with three technology measurements; a valid v2 document with a renamed observations array; a semantically invalid document whose energy value is JSON null; and a syntactically malformed document. The lab tests whether these four outcomes can be traced without turning invalid readings into zero.

Expected offline fixture: 4 physical lines; 2 accepted envelopes; 1 contract-quarantined envelope; 1 syntax-quarantined envelope; 6 accepted observations; accepted total **97.500 MWh**. Exact values are stored in the supplied `data/sample_api_payload.jsonl` and checked by `tests/test_payloads.py`. This is a deterministic learning fixture, not a published energy statistic.

Cloud execution is optional and billable. Use a disposable schema, a limited role and auto-suspending warehouse. Do not commit credentials. Save query IDs and results for review.

6.L.2 Step-by-step procedure and selected code

Step 1: Inspect each raw line and its line number. **Step 2:** Execute the offline Python validator; check accepted, quarantined, and normalized counts. **Step 3:** In an approved Snowflake lab, create `bronze_json_raw` and load the same raw lines as text, preserving the line index. **Step 4:** Use `TRY_PARSE_JSON`, supported versions and expected array path to classify envelopes. **Step 5:** Flatten accepted arrays and cast the values to a typed silver relation.

Listing 11: The core extraction; full script and guards in companion lab.

```
WITH parsed AS (
 SELECT source_line, TRY_PARSE_JSON(raw_line) AS payload
 FROM bronze_json_raw
), approved AS (
 SELECT source_line, payload,
        CASE payload:schema_version::NUMBER
          WHEN 1 THEN payload:observations
          WHEN 2 THEN payload:measurements
        END AS readings
 FROM parsed
 WHERE IS_OBJECT(payload)
)
SELECT source_line, f.index AS array_position,
       f.value:tech::VARCHAR AS technology,
       TRY_TO_DECIMAL(f.value:mwh::VARCHAR,18,3)
         AS energy_mwh
FROM approved a,
     LATERAL FLATTEN(INPUT => a.readings) f
WHERE IS_ARRAY(a.readings);
```

This extraction is *not* the full acceptance gate: the companion adds time, units, uniqueness and rejection tests. Preserve actual query IDs in a real account.

6.L.3 Expected results, failure diagnosis, and cleanup

The offline validator produces the following expected outcome from its four synthetic physical lines: **2 accepted envelopes**, **1 semantic rejection**, **1 syntax rejection**, **6 accepted technology readings**, and **97.500 MWh**. These expectations were calculated from the fixture and are cross-checked by the Python tests. They are not presented as output from an executed Snowflake warehouse.

If Snowflake counts disagree, investigate in this order: (1) raw-line count and content hash; (2) parser classification, including JSON versus SQL null; (3) approved schema versions; (4) selected array path; (5) number of elements and array positions; (6) numeric/unit/time contracts; (7) duplicate accepted business keys; (8) reconciliation to the published silver table. Do not replace a failed assertion with a blind `DISTINCT`, nor use `COALESCE(mwh, 0)` as an error correction.

Cleanup. Keep the original fixture and test report for reproduction. The companion SQL cleanup file contains only commented statements affecting chapter-specific objects; review ownership and object names before manually enabling anything destructive. Suspend lab compute after use. Optional query logging should preserve IDs and schema version, not credentials or personal data.

6.L.4 Exercises and assessment

1. **Foundation:** Explain why JSON null, a missing `mwh` key, and invalid JSON need separate error classifications. Provide one example of each.
2. **Practitioner:** Add a valid v3 envelope using `readings` instead of `measurements`. Implement an explicit v3 mapper and update the expected total and tests; preserve the earlier fixtures unchanged.
3. **Architecture:** Design a bronze-to-silver migration when a source changes from MWh numbers to `{"value":12500,"unit":"kWh"}` objects. Identify what is backward-compatible and what requires approval.

Scoring (10 points). Raw provenance (2); parser/shape classification (2); typed conversion and units (2); array cardinality and uniqueness (2); recovery and proof of verification (2). Passing requires at least 8 points and no hidden null-to-zero replacement.

> **The chapter in one principle.** JSON flexibility belongs at a controlled ingestion boundary. Analytical meaning must remain stable even when the source representation evolves. Chapter 7 extends that boundary from batch messages to streaming arrival and freshness guarantees.

Official documentation and practical references (checked October 2026).

[1] PARSE_JSON and TRY_PARSE_JSON; [2] VARIANT design considerations; [3] FLATTEN; [4] GET_PATH; [5] OBJECT_CONSTRUCT_KEEP_NULL; [6] Transforming data during load; [7] Parquet loading tutorial.

CHAPTER 07 DATA ENGINEERING / CONTINUOUS INGESTION

Snowpipe, Streaming, and Real-Time Ingestion

Build a pipeline whose freshness claims survive failures and replay

The alert was early. The evidence arrived late.
At 10:05 an energy operator receives a grid-balancing alert. The dashboard still shows 09:45 generation figures, although sensors have transmitted readings every second. The team asks whether Snowflake is "slow." But the missing time may be in a message broker, an object-storage batch, a cloud notification, a paused consumer, or the transformation after ingestion. The engineering question is not merely how rapidly bytes enter a table. It is *how soon a trustworthy answer becomes visible to its intended reader.*

Chapter 5 designed reproducible file loading; Chapter 6 taught us to preserve, inspect and validate evolving event payloads. We now ask when files are enough and when continuously arriving rows justify a new ingestion contract. The Finland Energy Analytics Platform will retain its bronze–silver boundary, immutable event identity and UTC observation grain, but introduce event clocks, delivery offsets, operational visibility and recovery tests.

After this chapter you can:

- choose scheduled `COPY INTO`, event-triggered Snowpipe or Snowpipe Streaming from measurable service needs;
- explain cloud notifications, stages, pipes, streaming channels and checkpoint responsibilities;
- separate source ordering, arrival ordering, revision precedence and query visibility;
- exercise duplicate delivery, late observations, schema drift and transient failure safely;
- present a defensible freshness and cost measurement plan without inventing cloud results.

Prerequisites: Chapters 3, 5 and 6; basic SQL and Python. The core lab is offline, with synthetic data. Live Snowflake examples are reviewed against vendor documentation but *not executed in a customer account.* Region, edition, SDK version and consumption pricing must be rechecked before deployment.



Figure 7.1. Freshness is an end-to-end property, not the ingestion API's name.

7.1 Batch versus near-real-time versus streaming

7.1.1 Throughput and latency definitions

A pipeline accepts units of work and exposes results. **Throughput** is the number of events, rows or bytes successfully processed per unit time; **latency** measures elapsed time for one particular work item. A system can exhibit excellent throughput while individual messages wait in a queue. Conversely, a tiny live demo can show low latency but collapse under an actual peak. Never infer one metric from the other.

Distinguish at least four clocks: *event time*, when the device says the measurement applies; *source publish time*, when the sender hands it to transport; *ingest visibility time*, when the accepted bronze record can be queried; and *business readiness time*, when validation and aggregation make it safe for the dashboard. A reliable SLA explicitly chooses the interval it measures, including its treatment of late source publication and failed arrivals.

Operational vocabulary
Event-to-visible age: $t_{\text{visible}} - t_{\text{event}}$. **Ingestion delay:** $t_{\text{visible}} - t_{\text{publish}}$. **Business freshness:** how old the most recent *accepted* observation is for a required key. **Backlog:** work accepted upstream but not yet committed and validated downstream.

Listing 1: Measure received-to-event age when both UTC timestamps are recorded.

```
SELECT technology,
  DATEDIFF('second', event_utc, received_utc)
    AS observed_source_age_seconds
FROM energy_telemetry_raw;
```

That query does *not* measure actual Snowflake ingest service latency unless the receive timestamp was recorded at a reliably defined visibility boundary. The lab uses synthetic times precisely to exercise the formula; it makes no vendor speed claim. In production, collect enough rows for percentile statistics and stratify by provider, technology and hour: one median can hide an unacceptably slow tail. **Decision check.** A stakeholder says "we need real time." Ask: Freshness of which metric? Measured from which clock? What proportion must meet the target? How long may corrected values lag? These four answers determine architecture more usefully than the phrase itself.

Project reference: P007: Snowflake time-series streaming guide. *Not independently executed; rights review pending.*

7.1.2 Batch, micro-batch, and continuous designs

Scheduled batch loads move source snapshots or files at fixed intervals. They are easy to inspect, replay and reconcile; they add waiting time between arrivals. Micro-batching reduces that wait by processing frequent small groups, but every batch still needs discovery, serialization, commit and monitoring. Row-oriented streaming accepts new events without first requiring an object-storage file. None of these modes automatically produces a correct analytical table.

Pattern	Strong fit	Primary burden
Scheduled COPY	Hourly/daily immutable exports	Scheduler; batch boundaries
Snowpipe files	Frequent cloud-object arrivals	Notification plumbing; file lifecycle
Streaming API	Continuous event feeds	Checkpoints, retries, channel health

Table 7.1. Select against workload requirements, not product branding.

Suppose a market report refreshes once an hour and source files arrive every fifteen minutes. Scheduled loading may be sufficient even if the source calls its service a "stream." If operators need a sub-minute view of sensor fault indicators, waiting for objects to be created and discovered may be inappropriate. If the source only publishes once per hour, Snowpipe Streaming cannot invent more recent observations.

A subtle architectural trap: event frequency and decision frequency are different. Thousands of sensor events per second may be aggregated into five-minute operational windows, yet incident detection may depend on an instantaneous threshold. Split the design when workloads have distinct freshness and quality demands; do not send every raw event through the most expensive path merely because some consumers need fast alerts.

Design against two SLAs.
The Finland energy example maintains *bronze ingest visibility* and *validated silver reporting readiness* as separate service objectives. Fast bronze ingestion with an hours-old silver watermark is still an operational failure for the analyst.

Check. Sketch two plans for a source producing one 2-MB object per minute: a scheduled load every fifteen minutes and event-triggered Snowpipe. Name every place a file can wait, and state which additional moving parts the second design introduces.

7.1.3 Costs, failure modes, and selection discipline

Cost has both a metered and an engineering component. Batch loading typically consumes virtual-warehouse credits while the scheduled load runs. File-based Snowpipe uses Snowflake-managed ingestion services, with billing rules that are not identical to a user-managed warehouse. High-performance Snowpipe Streaming has a throughput-oriented ingestion billing model; actual charges depend on current account pricing and the volume measured by the service. Downstream SQL transformations, retained storage, notification infrastructure and monitoring may add costs in every option.

A useful comparison fixes the *same accepted records* and the *same business freshness objective*. Otherwise a cheap hourly batch and a continuously validated streaming system are solving different problems. Record: expected events/day; peak-to-mean ratio; average payload and file size; maximum acceptable loss and duplicate rate; replay horizon; human on-call time; retention; network egress; and idle-resource billing.

Why fast ingestion can be expensive operationally
The fragile implementation fails when a client restarts, processes the same broker offset twice, mistakes "sent" for "committed," loses its schema version, or publishes a late correction without adjusting the aggregate. Recovery and observation are part of streaming architecture, not optional polish.

Listing 2: Design-time arithmetic; illustrative numbers, not vendor prices.

```
events_day = 86400 * 4          # four events per second
mean_bytes = 280                # application data estimate
estimated_GB_day = events_day * mean_bytes / 1e9
print(round(estimated_GB_day, 3))  # 0.097 GB/day
```

Even that number is not a bill: the accounting basis can exclude JSON structural bytes and include additional service classes. Avoid extrapolating a small simulator into a dollar forecast without the account's rate card.

Decision procedure. First confirm the source can deliver incremental data. Second define a p95/p99 freshness requirement and replay guarantee. Third run an identical input through two feasible patterns. Fourth measure downstream query visibility, data quality and metered usage. Finally document why the extra complexity is warranted. This chapter's controlled experiment provides the metrics framework; it does not claim to settle the choice for every team.

7.2 Snowpipe architecture and auto-ingest concepts

7.2.1 Files, stages, and the PIPE object

File-based Snowpipe is a managed service that uses a `PIPE` object to define a `COPY INTO` load from a stage. It loads new staged files without requiring you to schedule a virtual-warehouse `COPY` operation for each delivery. The pipe records loading instructions; storage integration and event publication are separate concerns. The pipeline still needs a stable file naming policy, predictable file formats and a contract for partial or rejected records.

Listing 3: File-based pipe definition; requires an approved external stage.

```
-- ENERGY_EVENTS_EXT and storage permissions are
-- provisioned separately. No cloud queue is created.
CREATE OR REPLACE PIPE ENERGY_FILE_PIPE
  AUTO_INGEST = FALSE AS
COPY INTO ENERGY_TELEMETRY_RAW
FROM @ENERGY_EVENTS_EXT
FILE_FORMAT = (TYPE = JSON);
```

7.2.2 From cloud event to a visible row

For auto-ingest, supported object-storage notifications announce new file objects; cloud-specific queue or subscription plumbing makes these announcements available to Snowflake. Snowpipe matches notifications to relevant staged paths and executes the pipe's load definition using managed compute. Visibility occurs later than a cloud's "object created" event; track both instants when diagnosing delay. An event is not the file itself, and a pipe declaration alone does not configure the notification service.

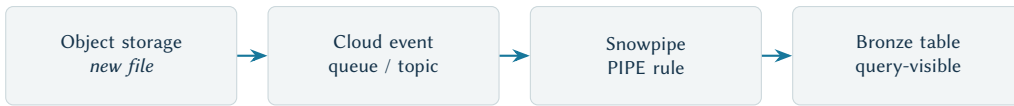


Figure 7.2. Events trigger file discovery; files hold the source records.

7.2.3 Arrival and load observability

`SYSTEM$PIPE_STATUS` exposes operational state for a pipe, including notification-related fields for auto-ingest where available. Load history and error notifications give different views: do not treat an empty error log as proof of complete source delivery. Reconcile file manifests against accepted rows and keep rejected-record provenance.

Project reference: P048: Auto-ingest demonstration. *Not independently executed; rights review pending.*

7.3 Cloud event notifications and staged files

7.3.1 Event topics, queues, and subscriptions

An object-store notification announces that a file has appeared or changed. Depending on cloud provider, configuration can involve S3 events and queues, Azure event infrastructure or Google Cloud Pub/Sub; Snowflake's integration instructions are cloud-specific. Specify an event subscription restricted to the intended path and event type. Broad notifications can waste work, expose sensitive object names or route unrelated files to the same ingestion path.

7.3.2 Storage integration as a trust boundary

A stage tells Snowflake *where* to look; storage integration determines which permitted identity may read it. Principle of least privilege applies to both the Snowflake role that creates or operates pipes and the cloud identity authorized to access objects. Prefer dedicated paths per environment, and avoid embedding cloud access keys in book code. Before deployment, check region constraints and organizational data-residency policy.

Common failure: notifications succeed, access fails.
Cloud monitoring shows the object-created event was published, yet no data appear. The pipe may lack rights to read the object, the stage may target a different prefix, the file format may reject records, or a queue permission may block delivery. Diagnose *object access*, *notification receipt*, *copy execution*, and *silver validation* as separate gates.

7.3.3 Duplicates, replays, and delivery assumptions

Cloud notifications may be repeated or delayed; object keys may be reused; producers may overwrite existing keys. A safe publication policy uses immutable object names that encode source partition and batch identity. Do not claim that a repeated notification always adds duplicate table rows; Snowflake tracks file loading in supported workflows, but its load metadata retention and reloading behavior have limits. Your source-level deduplication and reconciliation policy remains necessary.

Listing 4: Inspect a configured pipe without changing data.

```
SELECT SYSTEM$PIPE_STATUS('ENERGY_FILE_PIPE');
-- Inspect notification queue state and error fields
-- in context of this account's cloud provider.
```

Recovery drill. A cloud notification is lost while the source file remains in object storage. Demonstrate a bounded, audited backfill procedure based on a manifest and ingestion history, not a blind "force reload every file" script. Document who may approve a replay and which analytical keys are protected from double-counting.

Project reference: P056: Terraform Snowpipe cloud plumbing. *Not independently executed; rights review pending.*

7.4 Snowpipe Streaming fundamentals

7.4.1 Architecture: classic and high-performance paths

Snowpipe Streaming accepts event records directly through a supported ingest client rather than first discovering a staged file. As of this manuscript's documentation checkpoint, two architectural families matter. The *classic* approach uses its original SDK, channel conventions and operational limitations. The *high-performance* approach uses newer SDK/REST interfaces and a server-side `PIPE` to accept and, where configured, transform streamed rows. They are not merely two names for identical code. Follow the documentation for the client and architecture actually selected.

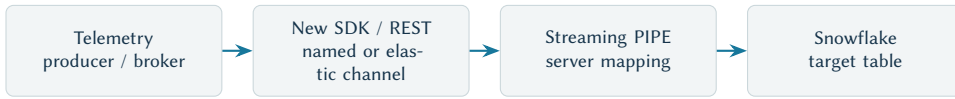


Figure 7.3. High-performance streaming uses a server-side pipe entry point.

In the high-performance architecture, you can define a named pipe for custom mapping, or use a supported default-pipe workflow. Where the architecture and SDK support them, *named channels* allow explicit offset tracking. Some interfaces also offer *elastic channels* with a different delivery model. Do not transfer an exactly-once claim from one channel mode to another. Those terms have consequences for retries and deduplication.

The database boundary matters. A successful SDK submission, a committed channel offset and a business-valid silver row are three distinct states. For the energy platform, raw telemetry carries event identity, source partition/offset, original event time, producer schema version and processing status. Transformations that compute publication-ready MWh totals occur downstream, where duplicates and revisions can be resolved with domain rules.

Project reference: P068: Snowpipe Streaming SDK examples. *Not independently executed; rights review pending.*

7.4.2 Rows versus files: the engineering boundary

File-based Snowpipe and row-based Snowpipe Streaming can both produce bronze records, but the delivery units differ. A file pipeline needs object naming, object lifecycle, cloud notification integration and file-load auditing. A row pipeline needs channel identity, source offsets, client resilience, schema mapping and row-error behavior. Shared downstream obligations include time semantics, privacy, reconciliation, data quality and cost attribution.

Dimension	Snowpipe files	Streaming API
Transport unit	Cloud object	Row / client batch
Primary identity	File path + load state	Source event and channel offset
Discovery	Notification or refresh	Client submission
Recovery	Manifest, load history, backfill	Checkpoint, replay, row errors
Typical source	Periodic exports	Broker, IoT, CDC event stream

Table 7.2. Both options need explicit correctness contracts.

Do not confuse Snowpipe Streaming with the SQL STREAM object introduced later in Chapter 8. The former is an *ingestion interface*; the latter tracks changes to a Snowflake table for downstream consumption. Likewise, receiving CDC messages from an upstream database is not the same as using a Snowflake change-tracking stream. Similar vocabulary hides different ownership and failure modes.

Why not stream everything?
An hourly regulator report that must be signed off against immutable batches benefits from file manifests and bounded reconciliation. A high-frequency sensor feed requiring rapid anomaly identification may prefer a streaming client, but if the business reprocesses daily for reporting, it can still retain an immutable archive for audit. Mixed paths can coexist when their acceptance and reconciliation rules are clear.

Check. Compare a source sending 10,000 tiny files per hour with a source sending 10,000 events per hour over a broker. Which ingestion unit is natural in each case? What changes in operational burden when you force one source into the other's delivery unit?

Project reference: P070: Official Snowflake Kafka connector. *Not independently executed; rights review pending.*

7.4.3 Supported interfaces and deployment checks

For the high-performance architecture, official Snowflake materials document a newer Java/Python SDK family and REST endpoints, with interface-specific behavior for channel operations. Availability, implementation limits, SDK APIs, region coverage and throughput ceilings are versioned product facts, not permanent design axioms. Freeze exact library versions and check the current documentation immediately before execution or publishing.

Listing 5: Illustrative high-performance streaming pipe syntax.

```
CREATE OR REPLACE PIPE ENERGY_EVENTS_STREAM_PIPE AS
COPY INTO ENERGY_TELEMETRY_RAW
  (EVENT_ID, TECHNOLOGY, EVENT_UTC,
   ENERGY_MWH, REVISION)
FROM (SELECT
  $1:event_id::VARCHAR,
  $1:technology::VARCHAR,
  TRY_TO_TIMESTAMP_NTZ($1:event_utc::VARCHAR),
  TRY_TO_DECIMAL($1:energy_mwh::VARCHAR,18,3),
  TRY_TO_NUMBER($1:revision::VARCHAR)
  FROM TABLE(DATA_SOURCE(TYPE => 'STREAMING')));
```

This is a *pipe definition*; it does not send a single event and does not start a client. It also assumes a compatible target table, approved role, supported region and validated schema contract. Real schemas must specify whether malformed values are rejected, quarantined or represented as NULL. To reduce accidental data loss, reserve business validation for a downstream controlled step when appropriate.

Preflight before a paid integration test
Record cloud/region/edition, account identifier (not credential), selected SDK or REST version, table types, privilege grants, client endpoint, pipe mode, channel model, event logging, expected ingest volume and maximum bill. Confirm target behavior for schema mismatch, retries, and non-supported data types. Set a stop condition and a cleanup plan.

A companion repository in this book is a research reference, not a tested deployment. The linked Iceberg example can help readers understand where streaming intersects open table formats, but copying it into production without checking licenses and version constraints would violate the book's own editorial policy.

Project reference: P069: Streaming to Iceberg reference. *Not independently executed; rights review pending.*

7.5 SDK-based and API-based streaming workflows

7.5.1 SDK setup and integration responsibilities

A useful ingestion client is more than an API loop. It initializes a secure session, discovers pipe/channel capability, accepts source events, serializes a supported type representation, sends bounded batches, tracks per-record outcomes and preserves a restartable source checkpoint. For a managed broker, the source partition and offset form a natural ordering domain. For a custom REST source, the producer must define an equivalent stable sequence or cursor.

Reference architecture contract
Source adapter owns source reading and backpressure. **Mapper** owns schema-version selection and type conversion. **Ingest client** owns supported Snowflake API calls. **Checkpoint manager** owns source partition and last safely committed position. **Quarantine** owns invalid inputs with reason codes. **Observability** owns counts, retry age, clock deltas and costs.

Illustrative pseudocode (not an SDK API):

```
for source_partition in partitions:
    channel = open_supported_channel(source_partition)
    start = committed_offset(channel) + 1
    for event in broker.read_from(start):
        validate_envelope(event)
        submit_to_ingestion(channel, event)
        wait_for_confirmed_commit(channel, event.offset)
        broker.checkpoint(source_partition, event.offset)
```

Do not mistake those intentionally abstract function names for calls in the Snowflake Python SDK. Real SDKs may commit asynchronously, batch internally or expose separate methods for channel status. The point is the ordering of *source advancement* relative to *verified destination commit*. A source checkpoint written too early can lose data; one written too late can cause repeats. Implement a deterministic replay policy rather than demanding that a network never fails.

Security boundary. Prefer organization-approved short-lived identity mechanisms and supported Snowflake authentication. Secrets never appear in code examples, notebooks, printed screenshots, logs or the repository history. Restrict the ingest principal to its dedicated target and apply separate privileges for metering and administration.

Project reference: P071: Python ingestion interface reference. *Not independently executed; rights review pending.*

7.5.2 Authentication, channels, and offsets

A *channel* allows a client to relate a stream of submissions to durable progress. Named-channel workflows in the current high-performance design support offset-token tracking, while elastic-channel endpoints have at-least-once delivery behavior and avoid some lifecycle responsibilities. The ingestion guarantee describes a component boundary; exactly-once *business totals* still depend on event identity and downstream logic, especially across source corrections and multiple partitions.

An offset token belongs to the source application's chosen position scheme. Treat it as an opaque marker unless your source defines comparable numeric offsets; do not sort arbitrary offset strings lexicographically to infer broker position. Persist source partition, token, schema-version mapping and client identity together in a checkpoint ledger. When opening a channel after restart, reconcile destination committed status and source checkpoint before producing a new sequence.

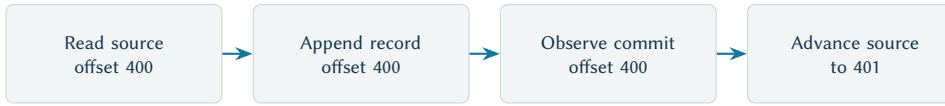


Figure 7.4. Durable acknowledgement must precede safe source advancement.

Correctness invariant
For a single source partition, no source position greater than the last acknowledged committed position is permanently discarded from retry storage. The application may retry an uncertain submission; that must not silently multiply silver-layer records.

To test offsets, inject a failure at three boundaries: before submission, after submission but before acknowledgement, and after acknowledgement but before the upstream checkpoint. The first is a straightforward retry; the second and third test idempotence. Capture the exact destination and source states so a future maintainer can reproduce the outcome rather than rely on folklore.

Project reference: P072: Java ingestion reference. *Not independently executed; rights review pending.*

7.5.3 Errors, disconnections, and reconnect design

The client needs two failure classes. *Retryable faults* include an interrupted network call, temporary throttling or unavailability. *Non-retryable contract faults* include malformed or forbidden payloads, an invalid schema version or denied permissions until configuration changes. Infinite retries on a permanent error hide the cause and block useful work; immediate discard after a temporary timeout risks loss.

A sensible strategy applies bounded exponential backoff with jitter, a retry budget, explicit quarantine for poison records, and preserved source offsets. Enforce maximum queue memory: if the destination cannot keep up, pause or throttle the source rather than letting a process grow indefinitely. Log at least producer partition, source token, pipeline step, reason code, attempts, first-failure time and eventual outcome, without logging credentials or sensitive payloads.

Listing 6: Illustrative retry decision, not a Snowflake SDK call.

```
if error.kind == 'TRANSIENT' and retries < limit:
    delay = min(max_wait, base_wait * 2 ** retries)
    schedule_retry(event_id, delay)
elif error.kind == 'CONTRACT':
    quarantine(event_id, error.reason)
else:
    stop_partition_and_alert(event_id)
```

Runbook: the channel reports an error
(1) Freeze or slow upstream offsets. (2) Inspect channel error/status and event log without exposing payload secrets. (3) Find last committed offset. (4) Distinguish rejected rows from unacknowledged rows. (5) Reopen with the documented SDK recovery method. (6) Replay bounded source positions. (7) Compare raw and silver counts. (8) Record final incident timeline.

In the simulator, source offset 8 fails once and then succeeds. We record *two attempts but one committed event.* That simple test is stronger than a screenshot showing a green connection. In a cloud exercise, preserve the client version, run identifier and actual channel-status output before asserting the system recovered. **Check.** What happens if one poison event blocks a strictly ordered source partition? Propose a policy that preserves its identity and error while making later healthy events available, without lying about contiguous commit progress.

7.6 Event ordering, duplicates, and late-arriving data

7.6.1 Arrival order is not event order

Source systems produce *event time*, transport imposes an arrival sequence, and the ingest client may commit asynchronously. These orderings are different. A sensor reading for 09:55 may arrive after 10:00 data because a device was offline, a retry waited, or a cloud region recovered. Sorting rows by ingest timestamp would make yesterday's correction appear newer than today's physical observation. Instead retain event time, original source partition/offset, ingest time, producer revision and an application-defined identity.

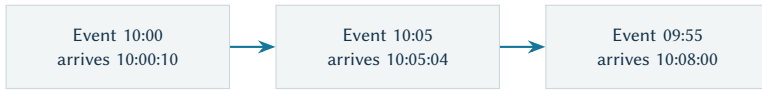


Figure 7.5. Physical observations can arrive out of event-time order.

7.6.2 Duplicates and idempotent analytical writes

A repeated network acknowledgement, a retried event or two overlapping source extracts may cause an input to appear more than once. Define the deduplication key by *business identity*: for our case `(provider, technology, event_utc)` together with a revision ordering. Raw bronze may intentionally retain duplicate deliveries; silver must represent the latest approved revision once. A source event ID alone is insufficient when a legitimate correction uses a new message ID.

Listing 7: Select approved latest revisions at the analytical grain.

```
SELECT * EXCLUDE rn FROM (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY provider, technology, event_utc
    ORDER BY revision DESC, received_utc DESC
  ) rn FROM energy_events_bronze
  WHERE energy_mwh >= 0
) WHERE rn = 1;
```

Tie-breaking policy matters. If two events carry the same business key and revision but different values, select neither arbitrarily: quarantine the conflict or escalate based on the publisher contract.

Project reference: P073: Kafka streaming demonstration. *Not independently executed; rights review pending.*

7.6.3 Late corrections, watermarks, and honest freshness

A *watermark* is a declared bound on how far event-time processing is expected to lag. It is not simply the maximum observed timestamp. Consider a five-minute aggregation that waits up to two additional minutes for stragglers. If a reading for 10:00 arrives at 10:09, the system needs an explicit correction policy: revise a historical aggregate, publish an adjustment record, or reject it after a documented business cutoff. Every choice affects downstream trust.

For Finland energy reporting, retain UTC timestamps in the transport and fact grain; convert to `Europe/Helsinki` only for display or reporting periods with daylight-saving awareness. A local hour may be skipped or repeated around the daylight-saving transition. A source event with a local clock string but no offset is ambiguous on such dates and needs publisher rules, not a guessed conversion.

Three separate notions of late
Transport late: it was published promptly but queued. *Publisher late:* the source only sent it after the measurement. *Correction late:* the publisher issues a higher revision after downstream consumers had finalized a value. A single ingestion-lag number does not diagnose all three.

Listing 8: Synthetic event age (seconds), not a service benchmark.

```
event_time = 0       # baseline UTC epoch in a toy clock
visible_time = 485   # illustrative late observation
age_seconds = visible_time - event_time
assert age_seconds > 300
```

Our lab retains one synthetic 09:55 observation arriving after 10:00 data. That late event increases the maximum event-to-bronze-visibility age even though it might have been ingested promptly after publication. For KPI reporting, measure the lag from the appropriate contractual clock and report late-data revisions separately.

Production check. Define a publishable hourly total when 99.9% of expected interval keys arrive by minute 4, but a late correction is permitted for 48 hours. Record whether the displayed result is preliminary or finalized, and ensure analysts can identify version changes. *Fast and correct* can mean versioned, not frozen.

Project reference: P074: CDC plus Snowpipe pattern. *Not independently executed; rights review pending.*

7.7 Schema changes and failure recovery

7.7.1 Versioned payload mapping

A new optional quality flag may be backwards compatible, while changing `energy_mwh` into `energy_kwh` is a semantic breaking change despite valid JSON. Accept only documented schema versions, preserve the untouched message and keep conversion rules in a tested mapper. Route unknown versions to a quarantine table until reviewed. Chapter 6's missing-versus-null checks remain essential in a continuous feed, where a bad schema can arrive thousands of times before somebody notices.

7.7.2 Interrupted ingestion and state recovery

A restart needs a known good source cursor and an independent destination commit record. Restore client identity, inspect named-channel status if supported, reconcile last committed positions, and replay the overlap needed to cover uncertain acknowledgement. Never advance the upstream cursor based solely on a client-side "request sent" message. For multi-partition sources, track independent progress; a single global counter can mask one stalled partition.

7.7.3 Replay, quarantine, and safe cutover

Backfills can interact with live traffic. Choose a bounded event range, preserve original event IDs and revision numbers, and make the silver merge idempotent. Quarantine records require an explicit disposition: reprocess after a mapper fix, replace from an authoritative snapshot, or mark irrecoverable with a signed reason. Do not silently drop them to make the dashboard healthy.

Failure injection matrix
Disconnect before send: retry from source. **Commit uncertain:** verify channel progress, then replay safely. **Schema mismatch:** quarantine with version and field. **Duplicate delivery:** retain bronze provenance but prevent duplicate silver totals. **Late revision:** update the relevant analytical interval with an audit trail.

Check. A live feed begins publishing `unit='kWh'` without incrementing its schema version. Design the alarm, containment, replay and backfill steps that prevent an unnoticed thousandfold reporting error.

Project reference: P075: Stream data generator. *Not independently executed; rights review pending.*

Project reference: P076: Multi-service streaming reference. *Not independently executed; rights review pending.*

7.8 Openflow and connector-based ingestion overview

7.8.1 When connectors are useful

A purpose-built connector can own source polling, authentication, snapshot/CDC semantics, schema discovery and transformations that would otherwise take many lines of custom code. Its value is a maintained operational contract, not a magical guarantee that every row is correct. Evaluate connector lag, replay support, source permissions, observability, dependency lifecycle and change handling just as you would a bespoke consumer.

7.8.2 Openflow deployment boundaries

Snowflake Openflow provides managed integration workflows with deployment options including Snowpark Container Services-hosted runtimes and bring-your-own-cloud arrangements. A deployment contains runtimes, and a runtime executes one or more integration flows according to supported configuration. Do not assume the same integration features, preview status or runtime controls across deployment generations and cloud regions. The currently documented platform includes generation-specific limitations and some preview configuration features.

7.8.3 Infrastructure, observability, and hidden costs

Openflow can add running control-plane/compute-pool costs, per-runtime compute, ingestion credits, logging/telemetry and external transfer or CDC warehouse charges. An idle-looking flow may still incur compute-pool costs if its management resources remain running. Separate an *integration demo* from the cheapest way to ingest a small periodic dataset. Before choosing Openflow, compare a minimal direct Snowpipe integration on identical acceptance criteria.

Connector review questions
Does it support our database and precise CDC version? Can it recover after a multi-hour outage without snapshots duplicating rows? Who owns schema evolution? Which components keep billing when paused? What credentials are required? Can we export complete source offsets and raw events if the service is replaced?

Example choice. A PostgreSQL transactional source with ongoing CDC and multiple tables may justify an Openflow connector. A single weekly public energy CSV probably does not. Prefer the design with the least operational burden that meets measurable freshness, security and traceability requirements.

Project reference: P021: Openflow on SPCS reference. *Not independently executed; rights review pending.*

Project reference: P077: Openflow structured pipeline. *Not independently executed; rights review pending.*

Project reference: P078: Openflow PostgreSQL CDC. *Not independently executed; rights review pending.*

7.9 Latency, monitoring, throughput, and cost

7.9.1 Measure the full event-to-table path

Measure pipeline stages separately: source observation, source publish, broker receipt, client read, Snowflake query visibility and silver readiness. Set clocks to UTC; record their precision and synchronization error. A message that is three minutes old at source publication cannot be fixed solely by changing the ingest product. Conversely, if publish-to-visible grows under pressure, inspect queue depth and client backlog before changing SQL transformations.

Listing 9: Freshness query against an approved raw table.

```
SELECT technology,
 DATEDIFF('second', event_utc, received_utc)
   AS source_age_seconds
FROM energy_telemetry_raw
WHERE event_utc IS NOT NULL;
```

Measurement caveat: `received_utc` is a stored column, not proof of the query-visible timestamp. Use a separately recorded acknowledgment, controlled probe query, or documented channel metric to bound ingest visibility in a real benchmark. Instrument silver completion independently. For p95/p99, report sample count and the observation window, not only a representative screenshot.

7.9.2 Throughput, backlog, and backpressure

Define upstream offered rate λ and sustained downstream service rate μ. When $\lambda > \mu$ for long enough, backlog grows regardless of optimistic average latency claims. Little's law relates long-run mean items-in-system L, throughput λ, and residence time W under stable conditions: $L = \lambda W$. Use it diagnostically, not as an excuse to assume stationarity during an outage.

Alert example
Alert if the oldest uncommitted source offset ages beyond the freshness objective for three consecutive evaluation windows, or if one partition stops advancing while others continue. Join this to acceptance/quarantine rates; a silent reduction in accepted rows can make throughput look "fast" while dropping data.

Check. A client handles 500 messages/s but the source produces 650 messages/s for 10 minutes. What backlog accumulated *ignoring retries*? Why is an average ingest latency measured before the peak uninformative about recovery time?

7.9.3 Separating ingestion billing from pipeline economics

Compare costs using the account's actual service consumption table and a time-bounded meter export. File-based Snowpipe usage can be examined in `PIPE_USAGE_HISTORY` in the Snowflake Account Usage schema, which has reporting latency. High-performance Snowpipe Streaming is metered separately according to its documented throughput-oriented model. Traditional warehouses, connector runtimes and downstream transformations are separate billable resources. Resource monitors that stop user-managed warehouses do not, by themselves, cap every serverless ingestion or connector charge.

Listing 10: Auditable file-based Snowpipe meter query; requires privileges.

```
SELECT TO_DATE(start_time) AS usage_day,
       pipe_name,
       SUM(credits_used) AS credits_used,
       SUM(bytes_billed) AS bytes_billed
FROM snowflake.account_usage.pipe_usage_history
WHERE start_time >= DATEADD(day,-7,CURRENT_TIMESTAMP())
GROUP BY 1,2 ORDER BY 1,2;
```

This query reports service usage, not end-to-end energy-dashboard cost. Add storage, query warehouses, stream processing, orchestration, network and on-call cost before comparing architectures. Document reporting lags; do not describe incomplete last-hour metering as zero charges.

Five metrics to keep
(1) Accepted input events/s; (2) p95 event-to-bronze age; (3) p95 event-to-silver readiness; (4) partition offset lag and oldest retry age; (5) ingestion *and* downstream credits per million approved observations. Always report quarantines separately.

Reading trail and status. Snowflake documentation checkpoint: 22 October 2026. Full links appear in the companion README and the references here; functionality, prices and edition limitations may change. Primary references: Snowpipe auto-ingest; CREATE PIPE; streaming architectures; named-channel recovery; Openflow costs; and file Snowpipe costs. No live cloud ingest or cost benchmark was performed.

Project reference: P079: Event streaming presentation reference. *Not independently executed; rights review pending.*

Project reference: P080: Community end-to-end streaming demo. *Not independently executed; rights review pending.*

7.L Guided lab: a failing energy telemetry feed

7.L.1 Inputs, environment, permissions, and cost

The companion's `streaming_simulator.py` runs with Python 3.10+ and the standard library only. It models one ordered source partition over nine synthetic events. An exact duplicate, a higher-revision correction, a late arrival, a negative measurement and a retry after transient failure are deliberately included. No network access, Snowflake account, API key or credits are required for the offline experiment.

The optional SQL files are *integration templates*, not executed database scripts. Only run cloud components after an administrator approves a disposable schema, a narrowly scoped role, storage integration, region and cost limit. A file-based Snowpipe definition needs an actual authorized external stage; a high-performance streaming pipe needs a matching current SDK/REST client. Never publish secrets or assume a commented pipe template provides an event queue.

7.L.2 Execute and inspect the deterministic experiment

Listing 11: From the extracted Chapter 7 lab folder.

```
python streaming_simulator.py --out output
python -m unittest -v test_streaming_simulator.py
```

Inspect `output/bronze.csv` and `output/silver.csv`. Bronze preserves eight accepted deliveries, including the duplicate and the older wind revision. Silver selects one approved revision for each (provider, technology, UTC interval) key. The negative reading goes to `output/quarantine.json`; the failure at source offset 8 is recorded in `attempts.json` before replay succeeds. The final source offset is committed only after a successful attempt.

Deterministic simulator output (offline)	Result
Source records / delivery attempts	9 / 10
Accepted bronze / quarantined	8 / 1
Unique latest-revision silver facts	6
Silver total, MWh	66.000
Median / maximum event-to-visible age, seconds	27 / 485
Last safely committed synthetic offset	8

Table 7.3. Measured from synthetic fixture clocks, not Snowflake.

Failure interpretation. The large maximum reflects a deliberately late observation, not necessarily a slow ingest service. Reporting it as Snowflake's p99 latency would be a serious measurement error. Distinguish accepted source delivery from published business readiness and document the assumed clock domains.

7.L.3 Assertions, diagnosis, replay, and cleanup

The eight offline tests check row accounting; higher-revision precedence; total MWh; quarantine; retry/commit order; late-event detection; deterministic replay; and the no-failure baseline. The strongest invariants are: accepted bronze plus quarantined equals unique source messages; silver contains at most one approved latest revision per business key; replay does not increase the business total; and the committed source offset never skips an unprocessed failure.

If the count differs, inspect `attempts.json`. If energy total increases on a second run, inspect business-key deduplication and revision precedence. If freshness is negative, validate event and visibility clocks before treating the message as fast. **Cleanup:** remove the local `output/` directory; the lab creates no cloud resources. Cloud teardown must be approved separately after actual integration testing.

7.L.4 Practice, architecture decision, and assessment

Foundation exercise. Add a second duplicate delivery and assert that bronze count increases while silver count and total remain stable. **Practitioner exercise.** Add a late higher-revision correction for hydro and prove that the final energy total changes exactly once. **Production decision.** Design a rollout from hourly file loads to row-based streaming for an energy alarm. Specify source partitions, channel mode, credentials, safety limit, backpressure, clock measurements, reconciliation and rollback. Decide whether the extra speed is worth its operational obligations.

Assessment rubric (10 points)
Architecture selection with measurable freshness (2); explicit source/target acknowledgment invariant (2); duplicate and revision-correction handling (2); quarantine and replay evidence (2); cost, privacy and cleanup plan (2). An architectural choice without source-offset safety cannot receive full marks.

Key takeaways. "Continuous" describes delivery, not guaranteed freshness; exactly-once semantics have a specified boundary; event-time ordering is distinct from arrival order; poison records need a named disposition; and costs include service-side compute, cloud plumbing and operating responsibility. Chapter 8 will consume this trusted bronze feed through Snowflake Streams, Tasks and Dynamic Tables, keeping ingestion and subsequent change processing as separate concerns.

Publishing verification gate.
At print approval, confirm all cited APIs, region/edition constraints, ingest and channel costs, repository licenses and client versions; run selected SQL in a bounded Snowflake test account; capture execution IDs, date, actual usage and cleanup proof. The present chapter does *not* claim those cloud tests were completed.

CHAPTER 08 DATA ENGINEERING / INCREMENTAL SYSTEMS

Streams, Tasks, and Dynamic Tables

Make changed data useful without rebuilding the world

A correction is not another megawatt-hour.
An operator opens the Finland Energy Analytics dashboard at 10:15. Wind generation for 10:00 is shown as 12 MWh, although a corrected reading of 14 MWh arrived five minutes earlier. A naive pipeline appends both observations to a daily aggregate and displays 26 MWh. The ingestion service did its job, but the analytical result is wrong. The system needs not only rapid delivery, but an unambiguous rule for *which version of a business fact is current*, together with a dependable method for maintaining that rule as changes arrive.

Chapter 7 established a safe bronze ingestion boundary: events carry identity, UTC interval, revisions, and records of retry and validation. This chapter consumes that boundary. We compare two styles of downstream maintenance: *procedural* change consumption using Snowflake Streams and Tasks, and *declarative* materialisation using Dynamic Tables. Both solve meaningful problems; neither can repair an unspecified business contract.

Learning outcomes
After this chapter, you should be able to: (1) reason about insert, update, delete and replay semantics; (2) explain stream offsets, transactional consumption and staleness; (3) build a deterministic MERGE with a triggered Task; (4) specify a Dynamic Table with an intentional refresh mode and target lag; (5) compare consistency, freshness, observability, cost and recovery; and (6) reconcile two implementations on a controlled energy fixture.

Prerequisites. Chapters 5–7, SQL joins and window functions from Chapter 4, and a basic understanding of change data capture. The offline laboratory uses Python 3.10+ without external packages; SQL templates require an authorised Snowflake account and have *not* been executed there.

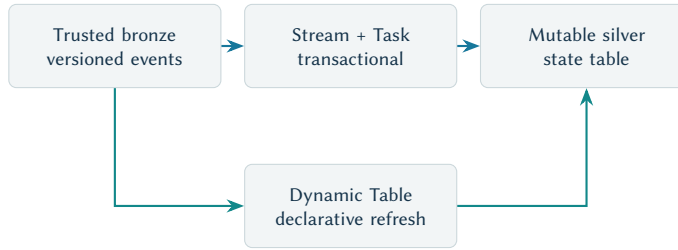


Figure 8.1. Alternative paths to a comparable current-state view; not one pipeline requiring both.

8.1 Change data capture and incremental updates

8.1.1 Change data capture as an information contract

Change data capture (CDC) describes changes to a source dataset rather than transmitting a complete current snapshot after every alteration. In a database log, CDC may identify committed INSERT, UPDATE and DELETE operations. In an event feed, what looks like CDC can instead be a collection of immutable *revision events*: the original measurement remains in the log and a later event states a better value. These two encodings are not automatically interchangeable. Snowflake Streams expose change tracking over a supported source object; they do *not* magically supply business meaning when an upstream event is a correction rather than a SQL UPDATE.

For the energy example, define business key $K = (provider, technology, interval_utc)$. The provider identifies the source authority, technology identifies the energy category and the interval represents a UTC measurement grain, not a wall-clock label in Europe/Helsinki. A new event contains `revision`, `event_id`, `operation` and its measurement. The business contract is *highest approved revision wins*. This is different from last record received wins: delayed networks may deliver a newer revision first and an obsolete correction later.

Time	Message	Required result
10:00	Wind r1: 12	Current wind = 12 MWh
10:05	Wind r2: 14	Replace; current wind = 14 MWh
10:06	Wind r1: 12	Ignore obsolete replay; still 14 MWh
10:07	Wind r2: 14	Duplicate: one effective version

Table 8.1. Arrival order and revision order are not equivalent.

A CDC feed should publish a documented rule for transaction boundaries, primary keys, deletes and retention. Without it, two consumers can independently produce incompatible totals while both pipelines appear operational. The rule for energy is especially important: summing raw received versions creates double-counting, so the silver table must resolve state before an analytical aggregate is calculated.

Decision test
Is the source a database table emitting row-level mutations, an append-only audit log, or a revisioned measurement API? Identify the difference first. If you cannot state what a DELETE means for a business key and whether the next lower revision may reappear, do not choose the ingestion operator yet.

Project reference: P081: PostgreSQL CDC to Snowflake reference. *Not independently executed; rights review pending.*

Check. Which key prevents the 10:00 wind reading from being counted twice, and why is `received_at` an inadequate sole conflict-resolution field?

8.1.2 Inserts, updates, deletes, and tombstones

In a state table, INSERT creates a previously unknown row, UPDATE changes attributes of an existing row, and DELETE removes it from that table's current state. Yet these physical operations do not necessarily preserve an audit trail. The bronze event log should normally retain the original delivery for traceability, within an approved retention and privacy policy. Silver can store one current state per key, with a flag or operation marker indicating that the latest approved revision is a deletion.

A *tombstone* is an explicit deletion record whose version remains visible to the maintenance logic. Suppose TEST energy has revision 1 with value 1 MWh and revision 2 marked DELETE. Removing the silver row entirely looks correct until an old revision 1 is replayed into an empty target; the row may be reinserted. A retained revision 2 tombstone prevents such resurrection. Downstream visible views exclude tombstones; reconciliation retains them.

Listing 1: State representation, not a complete executable pipeline.

```
CREATE TABLE CH08_SILVER_STATE (
  provider VARCHAR, technology VARCHAR,
  interval_utc TIMESTAMP_NTZ,
  revision NUMBER, energy_mwh NUMBER(12,3),
  operation VARCHAR, event_id VARCHAR,
  ingested_at TIMESTAMP_NTZ
);
-- The consumer-facing view must exclude DELETE state.
SELECT technology, energy_mwh
FROM CH08_SILVER_STATE
WHERE operation = 'UPSERT';
```

Soft delete versus hard delete. A soft delete preserves the tombstone in the current-state table and hides it from ordinary consumers. A hard delete may be required by a separate retention or erasure policy, but then a version ledger or upstream snapshot reconstitution must prevent reintroduction. Do not confuse technical event replay with the legal requirements for personal-data erasure; these are different governance decisions.

Corrections are not negative deltas by default. If a reading changes from 12 to 14 MWh, the corrected *state* is 14 MWh. A delta-based feed might instead publish +2 MWh, but that is a different contract. This chapter uses absolute-value revisions; downstream aggregates sum the latest approved states rather than raw change amounts.

Invariant worth defending
For every business key there is at most one latest approved state; deleting a key never allows an older revision to become current. Implementations may differ internally, but their published view must satisfy this invariant after recovery and replay.

Check. If the only record retained after a DELETE is the absence of a row, what additional durable information is needed to reject an old INSERT?

8.1.3 Incremental correctness and replay assumptions

Incremental processing is attractive because it touches only what changed, but correctness depends on more than an efficient execution plan. Assume a source may deliver a message at least once; ingestion may retry after an ambiguous acknowledgment, while later corrections may arrive out of order. A replay-safe consumer requires deterministic precedence, stable keys and an explicit commit boundary. If only the target update commits while a consumer cursor advances too early, data can be lost. If only the target commits and the cursor does not advance, replay must be safe.

Separate three identities: a *delivery identity* distinguishes transport attempts, an *event identity* names a logical source version, and a *business key* identifies the fact to present. The first may change on retry; the second should remain stable; the third intentionally groups multiple revisions. Reliable deduplication depends on choosing the correct identity for each layer. A SHA-256 file checksum from Chapter 5 is useful for input integrity, but it is not the business identity of an energy reading.

Acceptance criteria for the running case
(1) A duplicate event may increase bronze physical rows but not silver business totals. (2) A higher revision updates exactly one key. (3) A lower revision cannot roll the result back. (4) A tombstone hides the key and blocks resurrection. (5) A rejected negative value is quarantined instead of changing an approved row. (6) Reprocessing an already committed batch leaves the approved result unchanged.

An *incremental equivalence test* compares the incremental result with a reference snapshot recomputed from all *accepted* events. Let E be the accepted event history, $F(E)$ the current-state result obtained by full reduction, and $I(E_1, \ldots, E_n)$ the state after sequential incremental batches. The requirement is

$$I(E_1, \ldots, E_n) = F\left(\bigcup_{j=1}^{n} E_j\right)$$

for the stated conflict-resolution contract. This equality is about logical records, not matching warehouse execution plans, scheduling decisions or costs. The offline companion tests it over synthetic events; it does not simulate Snowflake's internal stream metadata.

An additional assumption must be stated: if a provider can issue two conflicting events with the same revision, precedence requires a reproducible tie-breaker, preferably specified by the source. This book uses the persisted UTC ingest timestamp and event ID. In production, investigate why conflicting same-revision values were emitted instead of silently accepting arbitrary arrival order. **Decision check.** Would a `MERGE` keyed on `event_id` maintain the correct current energy state? Explain why the *business key* belongs in the target match condition.

Project reference: P082: Incremental data warehouse community study. *Not independently executed; rights review pending.*

8.2 Change tracking with Streams

8.2.1 Stream offsets, source objects, and consumption

A Snowflake Stream is a change-tracking object over a supported source table or view. It tracks a *position* in the source's version history and returns the *net changes between offsets* as a queryable relation. It is not a separate queue storing one independent copy of every historical event. A stream on an append-only event log can be configured `APPEND_ONLY = TRUE` to expose inserted records; a standard stream is appropriate when source row updates and deletes must be captured.

Listing 2: Create an insert-only change boundary for an append-only source.

```
CREATE STREAM CH08_BRONZE_STREAM
  ON TABLE CH08_BRONZE_EVENTS
  APPEND_ONLY = TRUE;
SELECT event_id, revision, METADATA$ACTION
FROM CH08_BRONZE_STREAM; -- observation only
```

Crucially, a `SELECT` from a stream does *not* consume its offset. Successful DML using the stream in a transaction advances that stream's offset when the transaction commits; a rolled-back transaction does not. For an append-only source, reading a stream in a target `MERGE` is a common transactional boundary: the target data and the stream offset move together after success. If multiple independent consumers are needed, give each its own stream and monitor it separately.

8.2.2 Metadata columns and change semantics

Standard streams expose action metadata (INSERT or DELETE), an update marker for change pairs, and a source-row identifier. Their exact column names are documented in the SQL reference. An UPDATE on the source table is commonly represented as linked DELETE and INSERT entries; that does *not* mean that an immutable measurement revision appears as an UPDATE. A stream on our append-only bronze log returns INSERTs for new *events*, including business corrections and tombstones. Interpretation of `operation='DELETE'` remains our *payload contract*.

Common misconception
Two consecutive stream queries can return identical rows because observation alone does not commit consumption. Snowflake row tracking IDs are also not stable business keys. The first belongs to Snowflake source-row tracking; the second must come from the domain model.

Check. Why can an incoming event with `operation='DELETE'` still have stream metadata `METADATA$ACTION='INSERT'`?

8.2.3 Retention, stale streams, and safe recovery

A stream's offset depends on historical change tracking being retained on its source. If the consumer falls behind beyond the effective data retention window, a stream may become *stale*; it can no longer safely reconstruct changes since its saved offset. Monitor SHOW STREAMS or DESCRIBE STREAM, including STALE_AFTER, and set alarms well before the predicted threshold. Do not treat the displayed timestamp as a perpetual guarantee independent of retention configuration, cloning or source replacement.

Listing 3: Inspect stream status and potential staleness.

```
SHOW STREAMS LIKE 'CH08_BRONZE_STREAM';
SELECT SYSTEM$STREAM_HAS_DATA(
  'CH08_BRONZE_STREAM') AS possible_change;
```

The system predicate is intended for Task gating. It is designed to avoid *false negatives*, but may report TRUE for an effectively empty net change set. A task that continually reacts to such a result without consuming the stream can waste credits. A committed DML operation consuming the empty delta advances its offset; investigate the situation rather than repeatedly resuming tasks without a plan.

Operational recovery playbook: pause the affected task, preserve diagnostic metadata, identify the last committed business watermark and source version, then determine whether the source event log or an earlier trustworthy snapshot can reconstruct the required interval. Establish a new stream boundary only after reconciling the rebuilt target with the source of truth. Simply dropping and recreating the stream loses the original offset; it is not evidence that missing events were recovered.

Staleness budget
Retention and outage tolerance must be designed together. A source retaining sufficient history for only a short period is incompatible with a longer unattended recovery target. Calculate monitoring lead time, manual response window and worst-case backfill duration; test these before calling the system recoverable.

For a prolonged outage, a complete accepted-event archive in durable external storage is a valuable independent recovery source. The chapter's small offline fixture acts as that archive only for illustration; it is not a substitute for an actual audited retention plan. **Check.** Does SYSTEM$STREAM_HAS_DATA returning TRUE prove that at least one publishable energy row exists? Why not?

Project reference: P083: Snowflake-Labs stream-creation utility. *Not independently executed; rights review pending.*

8.3 Scheduling jobs and task dependency graphs

8.3.1 Scheduled and triggered tasks

A Task runs SQL statements or supported procedures on user-managed or serverless compute. A root task can run on a schedule; a triggered task can wake when a stream reports possible changes. A schedule represents an execution opportunity, *not* a guarantee of eventual business freshness: a long-running query, paused warehouse, restricted role or failed prerequisite may cause its result to arrive late. Choose a trigger for irregular incoming events, and a fixed schedule when a batch contract or predictable reporting window matters.

Listing 4: An initially suspended triggered Task; adapt warehouse before use.

```
CREATE TASK CH08_ENERGY_MERGE_TASK
  WAREHOUSE = CH08_WH
  WHEN SYSTEM$STREAM_HAS_DATA('CH08_BRONZE_STREAM')
AS
  MERGE INTO CH08_SILVER_STATE ... ;
-- The full executable MERGE is in the lab SQL file.
-- ALTER TASK CH08_ENERGY_MERGE_TASK RESUME;
```

The ellipsis is a *teaching placeholder*, not executable SQL; the companion file supplies the complete Task statement. A newly created task is suspended by default. Resuming it can initiate billable processing. For a serverless triggered task, the current creation syntax requires `TARGET_COMPLETION_INTERVAL` instead of a `WAREHOUSE`; its billing and scaling characteristics differ from a user-managed warehouse.

8.3.2 Dependencies and failure domains

A task graph expresses causal order with `AFTER` predecessor relationships. A sensible chain might *load approved bronze* → *merge silver* → *validate published metrics*. Every arrow must encode a genuine dependency. A downstream task should not publish data if a required predecessor failed. Multiple parents, finalizer tasks and overlapping graph runs create additional semantics that need testing on the relevant account edition and service version.

8.3.3 Monitoring, retries, and runaway schedules

Capture task execution ID, start/end timestamps, state, error messages and number of affected business keys. Set a policy for repeated failures, suspension and operator notification. Re-execution is not safe merely because a Task can be retried: any external call or non-idempotent side effect in the body still needs its own retry contract.

Design rule
If the silver merge finishes in 20 seconds but a reconciliation query takes five minutes, they should not necessarily share one continuously triggered transaction. Separate the correctness-critical commit from slower diagnostic work, and specify how the validation observes the committed state.

Project reference: P084: Task graphs getting-started guide. *Not independently executed; rights review pending.*

8.4 Incremental MERGE and upsert design

8.4.1 Match conditions determine the meaning of a row

The MERGE command compares each source row to the target using an explicit match condition. For our energy state, `provider`, `technology` and `interval_utc` determine the current row. `revision` must *not* be included in the match key: doing so would insert a new state row for every correction. `event_id` is likewise an audit identity, not a key for the current business fact.

8.4.2 Deterministic deduplication before matching

Snowflake documents that a MERGE can be nondeterministic when several source rows match the same target and request conflicting actions. The default protection against nondeterministic MERGE results detects some conflicts, but duplicate source rows can also produce duplicate INSERTs into an empty target. Reduce the incoming batch to one chosen event per business key *before* the match. A ranking function can encode the same revision and tie-break policy used in the full-state reference.

Listing 5: Core of the laboratory's deterministic source reduction.

```
SELECT event_id, provider, technology,
       interval_utc, revision, energy_mwh,
       operation, ingested_at
FROM CH08_BRONZE_STREAM
WHERE METADATA$ACTION = 'INSERT'
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY provider, technology, interval_utc
  ORDER BY revision DESC, ingested_at DESC,
           event_id DESC
) = 1;
```

8.4.3 Idempotency, version tests, and tombstones

The target update should proceed only when the incoming version is strictly better according to the stated ordering. The DELETE payload is stored as an updated tombstone, not a physical deletion that would allow an older revision to return. Reject malformed or negative measurements in an upstream quality gate; otherwise a filtered source may be *consumed* without ever reaching a quarantine table. In a real pipeline, a single transaction can stage invalid and valid rows before committing the final state; the textbook SQL assumes bronze contains already-approved events.

Testing requirement
Insert a row, apply a correction, replay both deliveries, then present a deletion and an older replay. Assert one stable latest state per key and a stable aggregate. Matching row counts alone cannot establish correctness.

Project reference: P085: Streams-and-Tasks ETL example. *Not independently executed; rights review pending.*

Project reference: P086: SCD Type 1 upsert example. *Not independently executed; rights review pending.*

8.5 Defining transformations with Dynamic Tables

8.5.1 A query definition instead of a manual update loop

A Dynamic Table stores the materialised result of a SQL definition and is maintained by Snowflake according to supported refresh policies. The author declares *what the result means*: for every energy key, retain the most recent approved revision. Snowflake manages the update plan and dependencies. This can remove procedural bookkeeping for transformations that are naturally expressible as SQL, but it does not make source contracts, semantic validation or operational responsibilities disappear.

Listing 6: Declaration of a latest-revision materialisation; requires approved warehouse.

```
CREATE OR REPLACE DYNAMIC TABLE CH08_DT_LATEST
  TARGET_LAG = '20 minutes'
  WAREHOUSE = CH08_WH
  REFRESH_MODE = INCREMENTAL
AS
SELECT event_id, provider, technology,
       interval_utc, revision, energy_mwh,
       operation, ingested_at
FROM CH08_BRONZE_EVENTS
WHERE revision >= 0
  AND (operation = 'DELETE' OR
       (operation = 'UPSERT' AND energy_mwh >= 0))
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY provider, technology, interval_utc
  ORDER BY revision DESC, ingested_at DESC,
           event_id DESC
) = 1;
```

8.5.2 Declarative semantics and consumer views

The declaration yields *latest state including tombstones*. A simple ordinary View over the dynamic result exposes only `operation='UPSERT'` to reports. Importantly, a Dynamic Table is not a writable ordinary table; a consumer cannot arbitrarily `UPDATE` its rows to repair one reading. Repair the source history or change the definition and allow refresh to propagate. Direct DML and stored-procedure side effects require a different procedural boundary.

For this source, `QUALIFY ROW_NUMBER() ... = 1` is documented as a supported and often efficient incremental-refresh pattern. Nevertheless, creating a dynamic table with an explicit unsupported construct returns an error rather than silently changing the requested refresh mode. Always capture the actual refresh mode from `SHOW DYNAMIC TABLES`, and check service feature availability for the account.

Economic consequence
Creating a dynamic table may initialise it immediately and begin warehouse-based refresh work. Do not execute a production-like definition in a trial account simply to inspect its SQL. Review privileges, warehouse name, region and expected runtime first.

Project reference: P087: Declarative pipelines using Dynamic Tables. *Not independently executed; rights review pending.*

8.5.3 Eligibility, source dependencies, and support checks

A Dynamic Table supports a broad subset of query expressions, but *incremental refresh* has stricter requirements than merely running an ordinary SELECT. Window functions, joins, aggregation, VARIANT operations, policies and input types need to be checked against the current supported-query matrix. For the reader's model, use this sequence: start with a minimal query, confirm explicit incremental refresh on a disposable object, inspect refresh history, and then incrementally introduce additional operators while preserving a known-good comparison result.

Snowflake documentation in October 2026 also describes ADAPTIVE refresh and custom incrementalisation capabilities. These are not interchangeable with INCREMENTAL: their reinitialisation conditions, support status and operational semantics require separate evaluation. This chapter deliberately uses INCREMENTAL to make the lab's assumption explicit. An AUTO choice may resolve to FULL at creation, so it is not a reliable shorthand for an incremental performance promise.

Gate	Evidence required before promotion
Query support	Confirmed syntax and eligible operators for chosen refresh mode
Privileges	Owner role can read source and use approved refresh compute
Initialisation	Refresh status shows completion; consumers know when data is ready
Change volume	Sampled distribution of inserted, revised and deleted business keys
Correctness	Zero bidirectional set differences versus approved full-state reference
Cost	Measured warehouse/Cloud Services usage for bounded workload

Table 8.2. Declarative maintenance still needs an acceptance gate.

Subtle boundary. Dynamic Tables can support Snowflake-managed source tables and supported views; not every external, directory or specialised table structure is an eligible input. A pipeline involving side effects, external API calls or procedural branching may need a Task, a connector, or a hybrid architecture. Choosing one technology for every stage is not a goal.

> **Correctness before optimisation**
> A Dynamic Table refresh that costs fewer credits but chooses the wrong revision is still defective. Hold the input fixture, business key, tie-break order and assertions constant while comparing execution mechanisms.

Check. What does a successful CREATE DYNAMIC TABLE prove? What additional evidence is required before claiming that incremental refresh stays within a 20-minute business freshness objective?

Project reference: P088: Pipelines for agent workflows (advanced reference). *Not independently executed; rights review pending.*

8.6 Refresh modes, target lag, and dependencies

8.6.1 Target lag is a goal, not a cron expression

For a Dynamic Table, `TARGET_LAG='20 minutes'` asks Snowflake to maintain a certain freshness relationship between the materialised output and its source. It does not mean "run exactly every 20 minutes," nor is the target a guaranteed bound. If refresh work takes longer than the declared interval, actual staleness may exceed it. Teams should monitor the achieved freshness distribution and treat target lag as an optimisation input, not as a contractual latency measurement.

8.6.2 Incremental, full, auto, and adaptive choices

`INCREMENTAL` processes source changes according to eligible operators. `FULL` recomputes the result; it can be appropriate when most rows change each cycle or unsupported operators are required. `AUTO` selects a mode at creation and does not continually reconsider the choice. The current documentation also describes `ADAPTIVE`, which can favour incremental work while permitting a reinitialisation when bulk changes make it more economical; confirm availability and preview/GA status for the account at print time. No mode means *zero cost*; change detection and scheduling can have metering implications even when output rows do not change.

8.6.3 Pipeline graphs and the DOWNSTREAM setting

The target lag of an intermediate Dynamic Table can be `DOWNSTREAM`, allowing a downstream consumer with an explicit lag to drive its refresh timing. A terminal table with `DOWNSTREAM` and no dynamic consumer may never refresh automatically. Conversely, a downstream table cannot demand a smaller explicit lag than an upstream dependency allows. When a multi-stage graph is configured correctly, Snowflake coordinates refresh ordering; the designer must still validate end-to-end lag measured from the ultimate source.

Listing 7: Inspection after creation; outputs require interpretation.

```
SHOW DYNAMIC TABLES LIKE 'CH08_DT_LATEST';
-- Check refresh_mode, scheduling_state and lag.
-- For a separate downstream model:
-- TARGET_LAG = DOWNSTREAM on intermediate only.
```

Selection exercise

The dashboard is read once per hour; operators need validated measurements no older than 20 minutes. Give the consumer table an explicit 20-minute goal. Before shortening it to one minute, measure refresh duration and change-detection costs; a faster target has no value if the upstream provider publishes hourly.

Project reference: P089: Introductory Dynamic Tables reference. *Not independently executed; rights review pending.*

8.7 Dynamic Tables versus Streams/Tasks versus views

8.7.1 Imperative boundaries and declarative results

The key architectural distinction is *who describes the update procedure.* A Stream plus Task explicitly states when to run an operation and how to modify the target. A Dynamic Table states what the target relation should contain and delegates refresh planning to Snowflake. A standard View stores SQL text and computes its result when queried, usually without persisting a separately refreshed result. Each can be appropriate for a different consumption profile.

8.7.2 Compare constraints, not feature labels

Dimension	Stream + Task	Dynamic Table
Business logic	Explicit DML/procedure	SQL query definition
State writes	Target is mutable	Managed output read-only
Scheduling	Task schedule/stream gate	Target-lag refresh orchestration
Retry boundary	Task and transaction design	Managed refresh and history
Side effects	Possible via supported procedural tools	Not in declarative query
Recovery	Stream offset + target + archive	Source replay + refresh/reinitialise
Cost evidence	Task run, warehouse, checks	Refresh warehouse, service work

Table 8.3. Decision criteria for maintained transformations.

A Stream plus Task is often natural when a durable `MERGE` target must be updated conditionally, when several statements form an approved transaction, or when a downstream service requires an explicit action. A Dynamic Table is often more maintainable for a pure SQL relation that should track evolving base data. A View can be simplest when the dataset is small, reading is infrequent, or the cost of query-time computation is acceptable.

An important refinement: different mechanisms may be combined. For example, a Dynamic Table can prepare a validated intermediate relation while a triggered Task performs imperative downstream publishing, subject to the documented stream-on-dynamic-table restrictions and the actual refresh mode. A hybrid system should have a crisp ownership boundary rather than two unsynchronised jobs modifying the same target.

Decision principle
Choose the lowest-maintenance approach that expresses the required semantics, meets a measured freshness objective, and permits transparent failure recovery. Do not migrate a working procedural pipeline merely because a newer declarative feature exists.

Check. Which pattern would you use for (a) a frequently read hourly energy summary, (b) a workflow that conditionally calls an external incident service, and (c) a one-off analyst query?

8.7.3 Maintenance, operational cost, and the hidden work

A useful comparison accounts for more than the number of lines in a SQL definition. Streams/Tasks can require stream-lag monitoring, Task retry policies, warehouse sizing, execution-order controls and manual recovery procedures. Dynamic Tables shift some of that orchestration into a managed service, but still require source-quality management, refresh troubleshooting, ownership privileges, business freshness alerts and explicit handling of unsupported operators. Views avoid refresh compute but move work to every eligible query. Total ownership cost depends on workload, staffing, and failure frequency as well as Snowflake charges.

For a controlled experiment, hold constant: source rows, row sizes, key cardinality, arrival rate, corrections, deletes, source retention and customer query frequency. Measure *equivalent* business results before costs. Then examine task compute credits; dynamic-table refresh compute and Cloud Services; bytes scanned; staleness distribution; retries; and operational response time. Use the account's official metering views with reporting lag disclosed. A resource monitor that suspends a virtual warehouse is not a universal cap on every serverless service charge.

Question	Measurement	Required disclosure
Correct result?	Bidirectional `MINUS`; sum	Null and delete semantics
Fresh enough?	p95/p99 event to silver	Clock boundaries; missing keys
Efficient?	Credits / 1M accepted facts	Full cost categories and volume
Recoverable?	Simulated outage and backfill	Time, data-loss boundary, run IDs
Maintainable?	Incidents + engineer effort	Workload and observation window

Table 8.4. A reproducible comparison checklist.

False economy example
A 60-second target on an unchanged hourly table can create frequent change-detection overhead with no additional business information. A low-lag feature setting should follow a user requirement and actual refresh evidence, not serve as a speed badge in documentation.

Security and governance. Ownership and execution privileges differ by resource. Tasks execute according to their task ownership and security context; Dynamic Tables refresh using their configuration and owner access. Privilege drift, revoked source grants or a changed masking policy can affect downstream refresh behaviour. Record ownership, permitted data visibility and alert routing at deployment rather than relying only on successful initial CREATE statements.

Project reference: P090: dbt and Snowflake pipeline deployment reference. *Not independently executed; rights review pending.*

8.8 Recovery, backfills, and pipeline observability

8.8.1 Designing backfills that preserve history

When a pipeline breaks, the first instinct is often to rerun yesterday's job. That may recover an append-only file load, but an incremental change consumer requires more care: what source interval is missing, which stream offset was committed, and which corrections arrived after the failure? A backfill must reconstruct an *approved state*, not merely append old events for a second time. Historical repair should also preserve evidence of the original run and the reason for reprocessing.

For the Finland energy case, use a repair ledger recording provider, UTC interval bounds, source snapshot hash or event offsets, intended target keys, invocation ID and post-repair assertions. Stop affected publishing before repair if readers could see inconsistent mixed versions. With Streams and Tasks, a prolonged outage may require rebuilding target state from the trusted bronze archive and then establishing a fresh, valid stream offset. With Dynamic Tables, options include resuming failed refresh, manual refresh, or approved recreation/reinitialisation depending on source and schema changes. A full refresh is not a guarantee of recovery if the source archive is incomplete.

Seven-step incident playbook

1. Confirm the incident and freeze further destructive changes.
2. Capture SQL/query IDs, Task history, stream state and dynamic refresh history.
3. Define expected source cutoff and acceptance invariants.
4. Select authoritative event archive or snapshot.
5. Rebuild in isolation and compare against a deterministic oracle.
6. Publish only after acceptance checks pass and monitoring is restored.
7. Preserve the incident log, access approvals and cost record.

Backfill example. A publisher corrects the 10:00 wind measurement from revision 1 to revision 2 and a 10:05 downstream run fails. If the source event is still available, successful retry must publish 14 MWh exactly once. If the stream is stale, it is not safe simply to create a new stream and assume that the unseen historical correction will reappear. Reconstruct the silver state from retained accepted history first.

Recovery time objective versus data-loss objective

An RTO of one hour says how rapidly useful service should return. An RPO of zero accepted observations says what state cannot be lost. A design without adequate source history may meet one objective while failing the other. State both in the architecture decision record.

Check. What must be proven before a repaired silver table replaces the current production table? How does the answer change if source corrections can arrive a day late?

8.8.2 Freshness regression and silent failure detection

A pipeline may be "green" while still publishing old or incomplete data. A Task may finish successfully but process zero approved rows because its source was empty, a filter was wrong, or every input failed validation. A Dynamic Table may be running but miss its target lag. Measure multiple independent signals: *source high-watermark*, *last accepted bronze event*, *last published silver interval*, *age of oldest missing correction*, and *quarantine rate*. A sudden drop in valid output is an incident even when system errors remain at zero.

The key alert for a provider should distinguish no fresh source data from delayed transformation. Compute $L = t_{\text{observation}} - t_{\text{latest approved source event}}$ as a business signal with documented clocks. Track an additional event-to-visible measure for individual records. If a provider legitimately publishes hourly, a 20-minute target for the refresh mechanism does not make the public dataset itself up-to-the-minute. Avoid reporting the tool's scheduler setting as achieved freshness.

8.8.3 Lineage, Query History, and audit evidence

Record the chain: source file or event ID → bronze key/revision → stream consumption or dynamic refresh → silver state → dashboard query. Snowflake Query History, Task History and Dynamic Table refresh history can help explain a failure, subject to privileges and reporting latency. Capture the relevant IDs and timestamps in the incident log, and keep a narrow reproducible diagnostic SQL query.

Listing 8: Illustrative history inspections; permission requirements apply.

```
SELECT * FROM TABLE(
  INFORMATION_SCHEMA.TASK_HISTORY(
    TASK_NAME => 'CH08_ENERGY_MERGE_TASK',
    RESULT_LIMIT => 20));
SHOW DYNAMIC TABLES LIKE 'CH08_DT_LATEST';
SHOW STREAMS LIKE 'CH08_BRONZE_STREAM';
```

Refresh and Task history fields can change; validate the approved account version before copying these queries into operational dashboards. Do not confuse delayed account-usage reporting with absence of cost. Never disclose customer data or credentials in screenshots or monitoring exports intended for a public repository.

Three complementary alerts
Freshness: one source's last approved interval is older than its service objective. **Correctness:** business-key duplicates, invalid revisions or aggregation mismatch appear. **Liveness:** a Task or refresh fails repeatedly, or the stream approaches staleness. Suppressing one must not silence the others.

Project reference: P091: Snowflake-Labs engineering toolbox. *Not independently executed; rights review pending.*

Project reference: P055: Alert Hub reference. *Not independently executed; rights review pending.*

8.9 Incremental transformation design exercise

8.9.1 Choose a pattern for two distinct workloads

Workload A, energy dashboard: immutable revision events arrive irregularly, visible rows represent the latest approved state, consumers read results every ten minutes, and SQL can describe the transformation. Prefer a Dynamic Table if its operators are eligible, refresh cost is acceptable, and measured output freshness meets the business requirement. Choose Streams/Tasks if an explicit, tightly controlled mutable-state transaction or a side-effecting approval step is necessary.

Workload B, incident response: a corrected high-voltage alarm must be enriched, deduplicated, written to a ledger and then passed through an external action with an acknowledgment. A declarative table alone is not an adequate workflow engine. Keep a transactional state boundary and an independently idempotent outbound action; a Task or external orchestrator is a better fit for procedural steps.

8.9.2 Acceptance contract

For either workload, state a business key, tie-break order, deletion policy, invalid-event disposition, latest acceptable data age, maximum recovery time, archival retention, exact downstream exposure point and cost measurement method. An assertion such as "all records processed" is too weak: define exactly which records are accepted and how corrections replace earlier values. Require the two result implementations to agree under replay, late arrival, deletion and intentional fault injection.

8.9.3 Correctness, latency, and cost compared fairly

Use the same synthetic event history to prove logical equivalence offline. Then, only after authorisation, run both Snowflake implementations over a controlled cloud fixture, collecting query IDs, refresh histories and actual credits. Distinguish measured percentiles from arbitrary *target lag* settings; report all source and silver timestamps in UTC and clearly explain the measurement clock.

Chapter argument
The real choice is not "Streams or Dynamic Tables?" It is "Which responsibility should our team own, and which should Snowflake manage, without surrendering correctness, evidence or recovery?" Choose the implementation whose risks you can test and explain.

Further reading, verified technical documentation (22 October 2026): Streams; stream staleness; triggered tasks; MERGE; dynamic refresh modes; target lag; and eligible constructs. Availability, costs, previews and behaviour require another check before release.

Project reference: P092: Community Dynamic Table comparison. *Not independently executed; rights review pending.*

Project reference: P093: dbt, Streams and Tasks exploration. *Not independently executed; rights review pending.*

8.L Guided lab: two ways to maintain energy state

8.L.1 Prerequisites, inputs, permissions, and cost controls

The companion archive provides an event fixture, Python simulator and tests, procedural and declarative SQL templates, plus a comparison memo. Python 3.10+ and the standard library suffice for the *offline* portion. No cloud account, credentials, network or billed resources are required. The Snowflake scripts are integration templates and have *not* been executed in Snowflake. Before any optional cloud test, obtain approval for a disposable schema, a narrowly scoped role, a known warehouse and a fixed teardown plan.

Our fixture delivers 12 synthetic rows: six initial technologies, a wind correction, its repeated delivery, a temporary TEST measurement and subsequent tombstone, an old hydro revision, and one invalid negative solar measurement. The unique accepted event archive has ten events; the negative reading is quarantined. One event is a duplicate delivery. All observations use a UTC source interval, and the provider name is illustrative rather than a download from any real service.

8.L.2 Run the model and inspect the results

Listing 9: Reproducible offline commands from the companion lab directory.

```
python incremental_model.py
python -m unittest -v test_incremental_model.py
```

The script independently computes a full latest-state snapshot and a transactional-batch-style state reduction. The latter models one aborted attempt followed by a retry that must not lose data. Both outputs are compared after all accepted events, including a DELETE tombstone, have been evaluated. Files are written to `output/` as a JSON comparison ledger and small CSV exports.

Actually observed offline result	Value
Delivered rows / invalid quarantined	12 / 1
Unique accepted logical events / duplicate deliveries	10 / 1
Latest keys including one tombstone	7
Visible latest-revision energy facts	6
Approved energy total (MWh)	66.000
Simulated aborted batch subsequently retried	1
Procedural-model equals full snapshot	True

Table 8.5. Deterministic offline result, not a live Snowflake measurement.

Optional cloud path. Run the Table/Stream bootstrap and complete Task DDL in a permitted schema. The Task is intentionally left suspended. If it is explicitly enabled, its MERGE must commit before comparing the silver output against a completed Dynamic Table initialisation. A successful CREATE statement alone does not prove either data path completed.

8.L.3 Assertions, troubleshooting, rollback, and cleanup

Eleven offline tests exercise input accounting; duplicate event IDs; rejection of negative energy; corrected wind precedence; resistance to older replay; tombstone-based delete; energy sum; equality of full and incremental reductions; simulated failed-batch retry; replay idempotence; and the event-validation contract. A passing offline suite proves those *model* invariants for the supplied fixture, not live transaction or refresh guarantees.

If the sum is not 66.000 MWh, inspect the revision key and tie-break. If TEST reappears, confirm that its highest-version tombstone was preserved. If the two models diverge, reduce the event sequence to the first differing key and compare its full revision history. If a real Task fails, preserve the query ID, statement error, stream status and committed target contents before retry. If a Dynamic Table appears empty, first check that its initial refresh has completed and the owner role can read its sources.

Cleanup. Delete the local `output/` folder. SQL teardown statements are commented out, as are Task activation statements. Suspending active cloud tasks and dropping objects requires explicit approval and a record of outputs, credentials and consumption. Never call DROP on a shared schema based on a textbook instruction.

8.L.4 Exercises, architectural decision, and scoring

Foundation: Add a duplicate corrected-wind event with a new delivery timestamp but the same event ID. Show which counters change and why the approved sum does not. **Practitioner:** Add a higher-revision late hydro correction; prove the new sum changes once, and that a replay of the original revision cannot undo it. **Architecture task:** Recommend either Streams/Tasks, Dynamic Tables or a View for an hourly Finnish market dashboard and a side-effecting fault alarm. Specify the tested correctness invariant, freshness metric, cost guardrail and recovery action.

Assessment rubric (10 points)
Versioned business key and tombstone semantics (2); deterministic reconciliation (2); transactional or declarative refresh explanation (2); failure/replay evidence (2); cost, least privilege, monitoring and cleanup plan (2). A design with no rollback mechanism cannot receive full marks.

Chapter review. (1) What advances a stream offset? (2) Why can the stream-data predicate yield a false positive? (3) Why is TARGET_LAG not a cron schedule? (4) Why must MERGE deduplicate before matching? (5) Which tasks still require human design when a Dynamic Table maintains SQL output?

Looking ahead. Chapter 9 adds Snowpark Python for transformations that benefit from a programmatic DataFrame interface; the need for stable keys, validation, lineage and replay discipline remains unchanged.

Print-release verification gate
Before print, verify feature status, access, credits and external project rights. Test both SQL paths in an approved Snowflake account and retain query IDs, costs and teardown proof. No live-cloud execution is claimed.

CHAPTER 09 DATA ENGINEERING / SNOWPARK PYTHON

Data Engineering with Snowpark Python

Write transformations in Python without losing the data platform

A correct transformation written in the wrong place can still be a bad design. The Finland Energy Analytics service receives a wind correction. The incoming event has a trusted business key and revision, but a junior engineer copies the entire bronze table to a laptop, filters the data in pandas, and uploads the result again. The calculation is right for six records. At national scale, the same pattern transfers sensitive rows across a trust boundary, creates an unbounded memory dependency, and makes retries harder to audit. There is a better question than "Could Python express this calculation?" It is "Where should the calculation execute, and what proof of correctness survives deployment?"

Chapters 4–8 established data modelling, batch loading, JSON processing, delivery semantics and incremental state. Snowpark adds a *Python interface to relational work* while preserving Snowflake's ability to execute supported operations near its data. It does not repeal the rules of secure access, deterministic business keys, idempotent updates or measured cost.

Learning outcomes

After this chapter, you can (1) explain the difference between Python client orchestration and server-side execution; (2) manage least-privilege sessions; (3) build and inspect lazy DataFrame transformations; (4) evaluate joins, casts and aggregates; (5) decide between SQL, UDFs and stored procedures; (6) test without incurring cloud charges; and (7) design a maintainable, observable Snowpark pipeline.

Prerequisites. Chapters 3–6 and competent basic Python. SQL examples and production integration require access to a permitted Snowflake account. The bundled *pure-Python offline fixture* runs on Python 3.10+ without network access; a Snowpark integration run is not claimed.

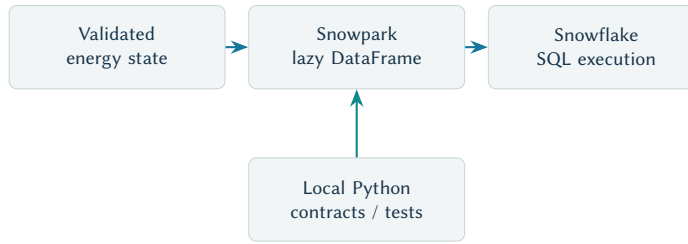


Figure 9.1. Local control code defines a plan; supported relational work runs on Snowflake compute.

9.1 Where Snowpark fits in the platform

9.1.1 Snowpark's role in data engineering

Python has two distinct roles in an analytical platform: it can *orchestrate* work and it can *express* transformations. Snowpark provides a DataFrame API that lets Python code describe projections, filters, joins, windows and aggregations over Snowflake tables. For operations supported by the DataFrame API, the engine generates SQL and executes the relational plan in Snowflake. The Python process is not ordinarily reading every row to evaluate a `filter` expression itself. This division is both the central power of Snowpark and the source of many misconceptions.

Imagine an energy table with one approved row per provider, technology and UTC interval. To compute an hourly total, SQL is an excellent fit. A Snowpark DataFrame can express the same logic while sharing names, parameters and reusable business helpers with a broader Python codebase. The platform still performs the scan and aggregation. Snowpark is not a requirement to do analytics correctly; it is a reasonable interface when programmatic composition is valuable.

Listing 1: Declarative query construction; cloud execution is not performed here.

```
from snowflake.snowpark.functions import col, sum as sf_sum
energy = session.table('ENERGY_SILVER')
plan = (energy.filter(col('ENERGY_MWH') >= 0)
        .group_by('INTERVAL_UTC')
        .agg(sf_sum('ENERGY_MWH').alias('TOTAL_MWH')))
# plan is a DataFrame, not a list of downloaded rows.
```

Why it matters. A transformation function can receive a session and return a DataFrame without writing a table, triggering data transfer, or choosing the consumer's schema. That separation makes a pipeline testable and lets an orchestrator own when an action should happen.

Architectural decision
Use Snowpark when Python organisation, complex reusable expression-building, or Python runtime integration provides concrete value. If a small, stable SQL model already expresses the business rule transparently, introducing a Python application and dependencies may increase maintenance without improving correctness. Compare models against the same acceptance tests before introducing an extra runtime.

Project reference: P094: Getting started with Snowpark Python. *Not independently executed; rights review pending.*

Check. Is a DataFrame returned by `session.table()` an in-memory pandas DataFrame? Why does the distinction matter when a source table grows one hundredfold?

9.1.2 Python execution boundaries

A Snowpark program crosses more than one boundary. The *client* authenticates, creates a Session and constructs a logical DataFrame. *Virtual warehouse compute* ordinarily executes supported query plans generated from that DataFrame. Python code inside a registered *user-defined function* (UDF) or *stored procedure* runs in a Snowflake-managed Python environment, subject to its applicable runtime and sandbox restrictions. These environments have different deployment, permissions and package availability; describing all three simply as "Python runs in Snowflake" obscures important operational detail.

Boundary	Typical action	Risk to control
Client process	Construct DataFrame, call actions	Credential handling, accidental data collection
Snowflake SQL engine	Filter, join, aggregate	Compute consumption, scan volume, permissions
Python UDF/runtime	Execute Python handler	Package, sandbox, row processing overhead
External compute	Process exported data	Network exposure, duplication, transfer cost

Table 9.1. Execution location determines data, security and cost obligations.

The code below *looks* local, but the expensive relational operation belongs to a virtual warehouse when an action is called. Conversely, `collect()` returns a *local list* and may materialise many rows in client memory. `to_pandas()` similarly transfers a result to the Python process. Protect these actions with a narrow selection, a row limit for inspection, and explicit assertions about expected cardinality.

Listing 2: Safe inspection versus unbounded client materialisation.

```
preview = plan.sort('INTERVAL_UTC').limit(5).collect()
# CAUTION: do not call plan.collect() on an unbounded dataset.
```

A Python worksheet in Snowsight, a notebook, a command-line application and a stored procedure may all use Snowpark, but their session ownership and deployment semantics differ. In a Python worksheet, the handler receives an existing session. A local command-line application normally constructs its own session. These examples must not be conflated when copied into production code.

Failure mode: hidden extraction
An engineer writes a clean-looking function that calls `to_pandas()`, then silently performs all grouping in local memory. Queries can appear fast in a six-row demonstration and later fail because the result is too large. Inspect the data movement as carefully as the formula.

Check. Which step transfers data: `group_by(...)` or `collect()`?

9.1.3 Choosing SQL, Snowpark, or external compute

A technology choice should be derived from the location of data, type of operation, governance boundary and operational ownership. SQL remains the clearest expression for many set-oriented joins, aggregations and window transformations. Snowpark is valuable when Python functions help assemble many consistent relational plans or when approved server-side Python algorithms are necessary. External compute is justified when a specialised library, runtime or integration cannot be deployed appropriately within Snowflake, but it increases the burden of moving data safely and proving reproducibility.

Choice	Strong fit	Common downside
SQL model	Stable declarative transformation	Complex Python control logic awkward
Snowpark DataFrame	Composed, reusable relational code	Additional Python dependency and tooling
Python UDF / procedure	Specialised logic near stored data	Runtime limits, packaging, row-level overhead
External Python	Custom integrations and runtimes	Extraction, network, identity, observability

Example. Convert unit-labelled synthetic meter readings to MWh with a simple expression. Writing this as an external Python loop would move data for no business reason. If the transformation instead requires a proprietary simulation package unavailable in the managed runtime, local or other cloud compute may be justified after a security and cost review.

Listing 3: A readable SQL baseline for equivalent Snowpark logic.

```
SELECT interval_utc, SUM(energy_mwh) AS total_mwh
FROM energy_silver
WHERE operation = 'UPSERT' AND energy_mwh >= 0
GROUP BY interval_utc;
```

An engineering comparison must hold the workload constant: identical input, warehouse configuration, output schema, cache conditions and concurrency. Do not claim “Snowpark is faster” or “SQL is cheaper” from a language label. If both interfaces lower to a similar SQL plan, the observed difference may come from action choices, input volume or collection to the client rather than fundamental engine speed.

Decision record exercise
For the Finland dashboard, write three sentences: the computation to perform; the execution boundary you choose; and how a reviewer can verify whether data ever leaves Snowflake. Include why the rejected options are less appropriate for this workload.

Transition. Once an interface is chosen, its credentials and session lifecycle become part of the production design, not a notebook afterthought.

9.2 Python sessions and connection management

9.2.1 Session creation and explicit options

A Snowpark `Session` represents the authenticated context used to submit statements, obtain DataFrames and manage certain client-side integrations. Local applications create and close sessions; some managed execution environments provide the session. Keep creation in the program's entry point and pass the session to pure transformation functions. Do not create a new connection inside each mapping function or hold a global authenticated session in a library imported by tests.

Listing 4: Connection configuration is injected, never hard-coded.

```
from snowflake.snowpark import Session

def run(config: dict):
    session = Session.builder.configs(config).create()
    try:
        return session.sql('SELECT CURRENT_ROLE()').collect()
    finally:
        session.close()
```

9.2.2 Authentication, role, and warehouse selection

Use approved authenticator methods such as interactive authentication for development, or an organisation-approved noninteractive identity for automation. Credentials belong in a secret manager, protected environment or supported connection configuration, *not* in a manuscript, example CSV or committed file. Select the least-privilege role and a dedicated, cost-limited warehouse. A developer should not use a powerful administrative role for routine reads and transforms.

Preflight query, run only with permission
Record `CURRENT_ACCOUNT()`, `CURRENT_ROLE()`, `CURRENT_WAREHOUSE()` and `CURRENT_DATABASE()` before a cloud test. A successful login does not imply permission to create permanent procedures or write into analytical schemas.

9.2.3 Connection shutdown and security controls

Always release a locally owned session in a `finally` block or a documented application lifecycle. Never log tokens, key material, connection strings with secrets or raw personal records. Decide how concurrent runs isolate sessions and statement timeouts. A retry after an authentication failure should not repeatedly disclose credentials or exhaust a login rate limit. Where code is executed in a Snowflake-hosted worksheet, use the injected session instead of duplicating connection setup.

Project reference: P095: Snowpark Python application template. *Not independently executed; rights review pending.*

Project reference: P096: Snowflake SQLAlchemy integration. *Not independently executed; rights review pending.*

Check. What closes the session if a query raises an exception, and who grants permissions to the service role?

9.3 DataFrames, expressions, and lazy evaluation

9.3.1 DataFrames as composable relations

A Snowpark DataFrame is best understood as a description of a relation and its transformations, not a snapshot of rows stored in Python memory. Its schema names, expression types and semantics correspond to Snowflake operations. A `select()` returns a new DataFrame; it does not mutate the existing DataFrame object. This functional style makes it natural to construct reusable functions that accept a DataFrame and return an enhanced DataFrame.

Listing 5: Composable transformations: no output action yet.

```
from snowflake.snowpark.functions import col, lit

def approved_energy(df):
    return (df.filter(col('OPERATION') == lit('UPSERT'))
              .filter(col('ENERGY_MWH').is_not_null())
              .filter(col('ENERGY_MWH') >= lit(0))
              .select('PROVIDER', 'TECHNOLOGY',
                      'INTERVAL_UTC', 'ENERGY_MWH'))

base = session.table('ENERGY_SILVER')
filtered = approved_energy(base)
```

Important precondition. The incoming table is already a *conformed latest-state table.* If it still contains old revisions, an `OPERATION='UPSERT'` filter will count obsolete versions. Chapter 8 established how to resolve revision order and tombstones before an aggregate is exposed. State this precondition explicitly in both code comments and data contracts.



Figure 9.2. Transformations build a query plan; an action triggers execution.

Schema convention
Prefer stable uppercase unquoted warehouse identifiers in the introductory lab; document exact casing and quoted-name exceptions for actual systems. Never rely on an accidental join column name or implicit cast without a schema assertion.

Project reference: P097: Snowpark data engineering introductory demo. *Not independently executed; rights review pending.*

Check. Why should `approved_energy` return a DataFrame rather than call `collect()` within itself?

9.3.2 Lazy evaluation and action triggers

Lazy evaluation separates the definition of a transformation from its actual evaluation. This allows a pipeline to reuse a shared filter and select only the columns required by a downstream query. It also creates a trap: a developer can mistake a constructed DataFrame for a completed side effect. For example, `session.sql('CREATE TABLE ...')` produces a DataFrame object; the command is submitted when an action such as `collect()` executes it.

Listing 6: Actions are deliberate execution boundaries.

```
source = session.table('ENERGY_SILVER')
valid = source.filter(col('ENERGY_MWH') >= 0)
print(valid.count())              # Executes a count query
valid.limit(3).show()             # Executes and displays
rows = valid.limit(3).collect()   # Moves at most 3 rows
# valid.collect()                  # Avoid if dataset unbounded
```

Correctness risk. Evaluating the same lazy plan at different times can observe different committed source data. A DataFrame does not always freeze a table at the instant the Python variable is created. When a pipeline requires repeatable reconciliation, define a stable input snapshot, staging boundary or explicit temporal query. Do not compare two queries performed minutes apart and assume their source versions matched.

Cost risk. Three actions may run three queries, not one cached computation. A quick `count()` for a debugging message may cause a large scan. Instrument every action point and distinguish exploratory diagnostics from tasks required by the production contract.

Practical action checklist
Before calling an action, ask: How many rows can return? Which account and warehouse executes the statement? Which source snapshot is visible? Is this an essential result or merely a debug print? Could it be replaced by a smaller deterministic assertion or an aggregate?

Design alternative. If a consumer only needs a total, return an aggregate DataFrame; do not collect the base data and sum it in Python. If a downstream pipeline must persist the result, use an intentional approved write path and verify its committed contents, not simply the existence of the plan object.

Project reference: P098: Snowpark Python official source. *Not independently executed; rights review pending.*

Check. If a script prints the first five rows twice, what additional work might the script be requesting from Snowflake?

9.3.3 Inspecting execution plans and generated SQL

A DataFrame pipeline is readable only if a reviewer can connect Python expressions to work performed by the database. Snowpark offers plan-inspection functionality, including `DataFrame.explain()` and query-generation interfaces that may vary with API release. These are *diagnostics*, not guarantees of a particular optimiser plan or runtime. Use Query History and Query Profile after an authorised cloud execution to investigate actual scans, pruning and compute.

Listing 7: Development-only plan inspection.

```
from snowflake.snowpark.functions import col
q = (session.table('ENERGY_SILVER')
     .filter(col('ENERGY_MWH') >= 0)
     .select('TECHNOLOGY', 'ENERGY_MWH'))
q.explain()  # Displays an execution-plan representation
```

Query inspection must distinguish generated SQL text from the physical execution plan selected by the service. A logical join may be reordered; a filter may be pushed down; caching may eliminate some work on a repeated query. A plan printed on an earlier API release or warehouse configuration is not a permanent contract.

For reproducible investigation, record a query tag, warehouse name and size, account edition, input row count, data profile and query ID. When comparing two plans, use the same data version and logical output. Comparing a cached aggregate against an uncached full-table export teaches little about language choice.

Evidence	What it can support
Python code review	Intended filter, join and projected columns
Generated SQL / explain	How a DataFrame expresses a query
Query Profile / History	Actual operator behaviour and billed execution context
Output assertions	Business correctness for the tested fixture

Diagnostic discipline
Start from the incorrect result, trace the data contract and output assertions, then inspect its physical plan when performance is the problem. Optimising a broken join can make the wrong total arrive more efficiently.

Project reference: P099: dbt Labs Python/Snowpark Formula 1 example. *Not independently executed; rights review pending.*

Check. What evidence would distinguish a costly row export from a well-pushed-down aggregation?

9.4 Transformations, joins, and aggregations

9.4.1 Filtering, joining, and grouping

The energy case requires a technology-to-category lookup. A left join preserves unmatched technologies for quality review; a strict inner join could silently discard facts when a lookup value is missing. Verify that the dimension contains exactly one row per TECHNOLOGY, otherwise a many-to-many join can multiply MWh totals. Group only after accepted latest-state data have been selected.

Listing 8: Core Snowpark plan used by the companion project.

```
from snowflake.snowpark.functions import col, sum as sf_sum
fact = session.table('ENERGY_SILVER')
dim  = session.table('TECHNOLOGY_DIM')
report = (fact.filter(col('OPERATION') == 'UPSERT')
          .join(dim, ['TECHNOLOGY'], 'left')
          .group_by('CATEGORY', 'INTERVAL_UTC')
          .agg(sf_sum('ENERGY_MWH').alias('TOTAL_MWH')))
```

9.4.2 Data types and casting checks

Treat quantity as a numeric type with suitable precision; record units explicitly. A string `'18.000'` is not automatically equivalent to a correctly validated measurement in every transformation. Use deliberate type conversion and rejection rules before an aggregate. Timestamps should represent explicit UTC source instants; local Finnish wall-clock reporting belongs in the presentation/model layer with daylight-saving behaviour specified.

9.4.3 Validation of transformation output

For the synthetic fixture, approved values are wind 14, solar 6, hydro 18, nuclear 16, bioenergy 8 and other 4 MWh, totalling **66.000 MWh**. Under the sample dimension, Renewable contains wind, solar, hydro and bioenergy (46 MWh); OtherLowCarbon contains nuclear (16 MWh); and Other contains the remaining 4 MWh. Those names are *teaching categories*, not an official energy-sector classification.

Output contract
Assert nonnegative measures, one current business fact per key, no unrecognised category, six visible latest-state rows, three expected category groups, and a conserved total of 66.000 MWh. A successful API call is not a passing business test.

Project reference: P100: Snowpark Python demonstrations. *Not independently executed; rights review pending.*

Check. Which join mistake doubles the total even when every source measurement is valid?

9.5 User-defined functions and stored procedures

9.5.1 UDFs, UDTFs, and stored procedures

A *scalar user-defined function* (UDF) returns a scalar value for its input; a *user-defined table function* (UDTF) can produce multiple rows; a *stored procedure* encapsulates a callable operation with control flow and, where authorised, data-management effects. These tools are useful, but not interchangeable. A built-in SQL expression should be preferred when it is sufficiently expressive, easier to optimise, and requires no bespoke Python runtime.

Listing 9: Pure handler idea; deployment and account rights not demonstrated.

```
def normalise_label(value: str) -> str:
    return (value or '').strip().upper()
# Register as a UDF only after testing and a deployment review.
```

9.5.2 Execution environments and appropriate use

A Python UDF is appropriate for a deterministic, small transformation not already well supported by SQL functions. Stored procedures are better suited to multi-step application logic requiring a defined invocation and structured error reporting. A UDTF is appropriate for specialised row expansion. Consider supported runtime versions, execution permissions, scaling characteristics and the possibility that Python boundary overhead dominates a simple SQL expression.

9.5.3 Permissions, packaging and limitations

The deployment role may need permissions to create functions or procedures and use an internal stage for permanent handler artefacts. Installed packages must be available through supported mechanisms; an arbitrary `pip install` on a local laptop does not make a dependency available inside a managed Python handler. External network access, secrets and memory behaviour require feature-specific configuration and policy review. For regulated datasets, deliberately review whether a handler may process raw inputs.

Decision checklist
Before registering Python code, document: output determinism; handling of nulls and exceptions; type contract; runtime/package versions; data-access and external-access permissions; test evidence; migration path; and fallback when a package version becomes unavailable.

Project reference: P101: UDF and UDTF guide examples. *Not independently executed; rights review pending.*

Project reference: P102: External functions examples. *Not independently executed; rights review pending.*

Check. Why is a procedure not simply a UDF with a longer Python file?

9.6 Dependencies, packaging, and code organisation

9.6.1 Dependency pinning and compatibility

The same notebook can behave differently months later if Snowpark or its transitive dependencies change. Keep a versioned package specification, lock exact versions before a reproducible publication release, and test against a declared Python interpreter. Constraints such as `>=1.18,<2` are a *development range*, not a final reproducibility lock; the distribution of this book's eventual companion artefacts must include resolved version records.

Listing 10: Project metadata; optional integrations are not installed for offline tests.

```
[project]
name = "snowflake-chapter09-energy-lab"
version = "0.1.0"
requires-python = ">=3.10"
dependencies = []
[project.optional-dependencies]
cloud = ["snowflake-snowpark-python>=1.18,<2"]
```

9.6.2 Modular Python project layout

Separate *business rules* (version ordering, time grain, units and invalid-data policy) from *connectors* (session credentials), *query builders* (Snowpark DataFrames), *application entry points* and *tests*. Prefer functions with explicit DataFrame inputs and outputs. Avoid notebooks that rely on hidden cell order, global state or personal file paths. The companion project includes `snowpark_pipeline.py`, `test_snowpark.py`, `data/` fixtures and optional SQL bootstrap templates.

9.6.3 Environment and artefact management

Never commit credentials, downloaded personal data or run logs with secrets. Record artefact hashes where they matter, link tests to tagged source revisions, and document how to recreate the environment from a clean checkout. For deployed procedures, treat source code, package manifest and stage artefacts as a controlled release rather than an ad hoc click sequence.

Reproducibility note
The printed text demonstrates a stable idea. The companion archive records the specific code and fixture used for the 22 October 2026 editorial check. A later edition should retest all feature-dependent statements and update its release tag rather than quietly changing the archive behind a printed claim.

Project reference: P103: Snowpark on Jupyter reference. *Not independently executed; rights review pending.*

Project reference: P015: Snowflake Python API reference. *Not independently executed; rights review pending.*

Check. What additional artefact is needed to make a permissive package constraint reproducible?

9.7 Local development and automated testing

9.7.1 Unit tests independent of live services

Most energy-data rules can be proved without a Snowflake account: a revision must supersede an older revision, a duplicate event must not double-count, a tombstone must remove a visible fact, and a negative MWh reading must be rejected. Pure Python contract tests are fast, deterministic and free of cloud consumption. They do not establish that the Snowpark DataFrame API correctly generates the intended SQL or that account permissions are sufficient.

Snowflake also documents a *local testing framework* for a subset of Snowpark DataFrame APIs. Its emulation is useful for earlier feedback but does not implement every function or exactly reproduce all warehouse behaviour. For example, the documentation distinguishes unsupported `Session.sql()` operations in local testing. Consequently, label the exact test type in the report rather than calling every successful offline test a Snowflake integration test.

9.7.2 Integration tests in a test account

A real integration check creates a small synthetic fixture under a disposable schema, builds the Snowpark plan and submits a deliberate action. Assert schema and numeric results, log the query ID, and remove only objects created by that test run. Repeat in the account edition and region relevant to readers if a feature varies by deployment. A test that succeeded for a six-row mock cannot prove the engine's throughput or package deployment limitations.

9.7.3 CI secrets and expensive-test controls

Split CI into mandatory offline tests and permission-gated cloud tests. Never enable cloud integration on every untrusted pull request, and never expose a high-privilege password to forked code. Gate paid tests behind an approval step, short-lived credentials, spending and timeout controls, and an auditable cleanup mechanism.

Listing 11: The reproducible default is an offline test.

```
python snowpark_pipeline.py
python -m unittest -v test_snowpark.py
# Live integration: deliberately not automated by default.
```

Project reference: P104: Testing Snowpark tutorial. *Not independently executed; rights review pending.*

Check. Which two assertions cannot be proved by an offline business-rule unit test alone?

9.8 Snowpark performance and execution trade-offs

9.8.1 Data movement and execution locality

A central aim of Snowpark is to let developers describe set-based work without extracting the entire dataset into a Python process. Consider two implementations of a regional generation total. Implementation A selects several million rows into pandas and groups locally. Implementation B groups by region in a Snowpark DataFrame and collects only the dozen rows of resulting totals. Both may produce correct MWh figures for a fixture; their network transfer, privacy exposure and client-memory demands are radically different.

Listing 12: Prefer a bounded aggregate over collecting all facts.

```
from snowflake.snowpark.functions import sum as sf_sum
summary = (session.table('ENERGY_SILVER')
           .group_by('TECHNOLOGY')
           .agg(sf_sum('ENERGY_MWH').alias('MWH')))
small_result = summary.collect()  # bounded by technology count
```

Not an unconditional guarantee. An aggregate can still be expensive within Snowflake, especially with poor clustering, excessive cardinality or skew. The benefit is that the client receives only the result, not every source row. If grouping by an unconstrained free-text user ID, the returned result might again be huge; boundedness is a domain assertion that the test must verify.

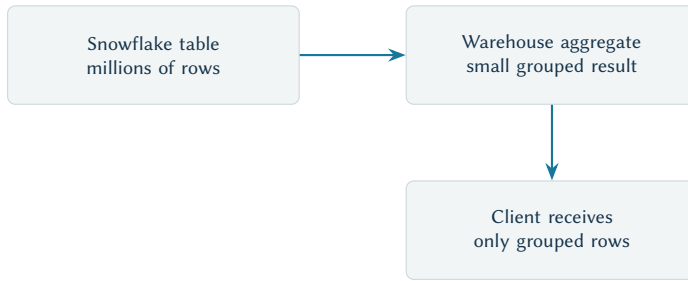


Figure 9.3. Push relational work to compute close to storage; transfer only necessary output.

Operational boundary
Before an application collects a DataFrame, define an expected maximum number of output groups, a query timeout and a failure response. If the resulting cardinality is unexpectedly large, alert and stop rather than serialising unreviewed records to a local file.

Check. Which implementation exposes the larger volume of raw data outside Snowflake, and how would you prove it?

9.8.2 Cost and latency comparisons without misleading benchmarks

Snowflake compute consumption depends on the services and resources actually used, not on whether an author writes `.group_by()` or `GROUP BY`. A benchmark must control the input snapshot, output equivalence, warehouse size, concurrency and cache conditions. It must also separate client time, queueing, execution duration, data-transfer time, Python handler overhead and billed credits. Without that breakdown, a wall-clock speed comparison encourages incorrect architectural conclusions.

The energy dataset in this book is intentionally tiny: twelve synthetic deliveries and six latest-state rows. Its measured local test runtime is not meaningful evidence about production-scale warehouse latency. To compare at scale, construct an approved, suitably sized synthetic workload and publish repeatable queries, query IDs, actual credit consumption and variance across runs. A fabricated price estimate is not a benchmark.

Measurement	Required context
Wall-clock latency	Start/end boundary, network, queueing, repetitions
Credits consumed	Warehouse and service category, idle time, time window
Rows scanned / output	Input version, predicates, pruning, group cardinality
Result correctness	Exact comparison against a reference oracle
Transfer volume	Bytes and number of records returned to client

A sound experiment might execute both SQL and Snowpark representations of the same approved hourly aggregate in a dedicated warehouse with controlled result caching. It should report *median and tail durations* only after collecting real measurements. Cost optimisation then targets the observed bottleneck: fewer repeated actions, narrower scans, efficient joins, or right-sized compute. Changing languages alone is not a validated strategy.

Benchmark release rule
Label every performance number as measured, simulated or illustrative. Store scripts, input generation method and raw run evidence. Never convert an unexecuted performance hypothesis into a production claim merely to make a chapter more persuasive.

Project reference: P018: Snowpark Python performance tips. *Not independently executed; rights review pending.*

Check. Why is it invalid to compare a warm-cache aggregate with a cold-cache full export?

9.8.3 Avoiding Python-level anti-patterns

Three common problems recur. **First, accidental eager collection:** an unbounded `collect()` or `to_pandas()` pulls records into a limited client process. **Second, row-at-a-time work:** repeated queries inside a Python loop add round trips and overhead where a set-based join would suffice. **Third, unnecessary UDFs:** wrapping built-in SQL arithmetic in Python can introduce runtime and deployment cost without changing business logic.

Listing 13: Avoid a query for every energy category.

```
# Anti-pattern (illustrative only):
# for name in thousands_of_categories:
#     session.table('ENERGY_SILVER').filter(
#         col('TECHNOLOGY') == name).collect()

# Better: one grouped relational plan.
summary = (session.table('ENERGY_SILVER')
           .group_by('TECHNOLOGY')
           .agg(sf_sum('ENERGY_MWH').alias('MWH')))
```

Optimisation does not license changing correctness requirements. If the source contains revisions, a smaller filter that discards a necessary tombstone might make a query faster while returning the wrong total. If a join's category dimension is not unique, aggregation may magnify every fact. Build tests for these risks before investigating operator-level tuning.

Another mistake is to rely on implicit ordering. Relational tables are unordered unless the query specifies an order. A preview sorted by event timestamp is a presentation choice, not a stable business resolution rule. Use explicit deterministic tie-breaking for revision events and an agreed metric grain.

Improve the system in this order
(1) Validate the business result; (2) inspect data movement; (3) measure actual query operators; (4) reduce redundant actions; (5) tune warehouse and table design; (6) rerun correctness tests. Document any trade-off that makes the code harder to reason about.

Project reference: P013: Pandas on Snowflake starting point. *Not independently executed; rights review pending.*

Check. How many remote queries are issued by a loop that performs one `collect()` per source identifier?

9.9 Production-oriented Python pipeline patterns

9.9.1 Reusable functions, configuration and logging

Production code should separate four kinds of responsibility: connectivity, query construction, validation and publication. A `build_report(session, ...)` function can return a lazy DataFrame. A `validate_contract(rows)` function can run entirely offline. An orchestrator chooses when to execute and how to persist data. Each responsibility receives explicit parameters rather than importing configuration from an undocumented notebook variable.

Logs should record release version, run ID, source identifier, row-count expectations, query ID, outcome, elapsed time and object names. They should *not* contain raw API secrets, passwords or unminimised personal records. A query tag can connect Snowflake Query History to a pipeline run, but it does not replace application-level data lineage or approved data retention.

Listing 14: An original, testable function boundary.

```
def build_category_report(session, source, dimension):
    """Return a lazy, read-only DataFrame plan."""
    from snowflake.snowpark.functions import col, sum as sf_sum
    fact = session.table(source).filter(col('ENERGY_MWH') >= 0)
    dim = session.table(dimension)
    return (fact.join(dim, ['TECHNOLOGY'], 'left')
            .group_by('CATEGORY', 'INTERVAL_UTC')
            .agg(sf_sum('ENERGY_MWH').alias('TOTAL_MWH')))
```

In this abbreviated illustration, the `source` must already be latest-state, with a single approved version per business key. Production callers should use a protected table name from allowlisted configuration rather than accept unchecked SQL identifiers from external requests.

Logging design
Persist a small JSON run ledger: input dataset version; code commit; query IDs; expected and observed row counts; aggregate checksum; error category; retry count; and final publication status. Distinguish *attempt failed* from *run committed*. This evidence supports both incident investigation and reader reproducibility.

Project reference: P105: Snowpark DevOps reference. *Not independently executed; rights review pending.*

Check. Which elements of an execution log are essential to reproduce a faulty run without printing sensitive data?

9.9.2 Retries, idempotency and cleanup

Retries are normal because transient failures occur: connections break, warehouses suspend, jobs time out and source services return errors. A retry is safe only if the intended work is idempotent or protected by an explicit transaction and publication boundary. For the energy example, replaying a previously accepted event must not create a second visible measurement. Higher revisions may change a result once; an older revision must not undo that correction.

Failure point	Safe recovery decision
Before query action	Rebuild plan from versioned inputs
After query, before record	Check committed target / query ID; never guess
Partial application output	Publish through atomic or versioned boundary
Duplicate delivery	Deduplicate stable event ID, preserve revision order
Uncertain third-party effect	Use external idempotency key / reconciliation

A Python exception does not necessarily prove that an earlier warehouse statement rolled back. If a network connection fails after a write completes, blindly replaying the write may duplicate data. The correct response is to inspect transaction and target-state evidence and to use deterministic `MERGE` or version-based publication patterns where they fit. This is the same operational lesson as the Streams/Tasks example in Chapter 8, now expressed through a Python-controlled process.

Cleanup should be explicit. Temporary output artefacts may be deleted by a local test after it has preserved required assertions. Cloud objects, however, may contain other users' data or active tasks. Destructive teardown statements should remain commented until a human confirms object ownership and permissions. Never hide production cleanup inside `finally` if failure diagnostics require the data to remain for investigation.

Idempotency invariant
For a fixed accepted event set, rerunning the pipeline must produce the same latest-state facts and exact MWh aggregate, regardless of repeated delivery. This is an observable property, not merely an implementation style.

Project reference: P106: Data engineering with Snowpark Python. *Not independently executed; rights review pending.*

Check. Why is "the Python function raised an exception" not sufficient proof that no database changes occurred?

9.9.3 Monitoring, versioning and production operations

An operational service needs signals for *liveness*, *freshness*, *correctness* and *expense*. Liveness asks whether a run finished. Freshness asks whether the latest approved interval is available within its agreed service objective. Correctness asks whether business invariants pass. Expense asks whether the observed credit and storage use remain inside a budget. A successful task status does not prove the energy aggregate is correct; conversely, a correct historical snapshot is not necessarily fresh enough for a dashboard.

Example monitoring contract (design targets, not measured data)
Expected latest source interval: explicit UTC; max desired end-to-end freshness: 30 minutes; visible business-key duplicates: zero; rejected negative readings: quarantined and reported; aggregate reconciliation: exact for the synthetic fixture; periodic cost review: consumption by pipeline and warehouse. These thresholds are proposals that operators must approve before production use.

Release management should bind the transformation logic, fixture, dependency lock, schema definition and test output to one version identifier. Schema drift and library upgrades require an explicit compatibility test. When stored procedures are involved, account runtime and package resolution can change deployability; a successful local `pip install` is not deployment proof.

A production readiness review should also ask: What happens if the *dimension table* is updated during a processing run? Can two jobs publish to the same destination at once? Is the source snapshot consistent? Are run IDs unique? Is the rollback strategy reversible? Is time-zone conversion delayed until a well-defined presentation layer? Which role owns the process when the original developer leaves?

Three questions that close the chapter
Where does this Python operation actually run? How would a new engineer prove that the latest energy value is the right one? What evidence would remain after a failed or expensive run? A maintainable Snowpark design makes all three answers inspectable.

Official technical references, inspected 22 October 2026: Working with DataFrames; local testing framework; stored procedures; Python UDFs. Recheck version-sensitive details prior to print release.

Project reference: P107: Snowpark data engineering and ML. *Not independently executed; rights review pending.*

9.L Guided lab: clean and enrich energy observations

9.L.1 Prerequisites, fixtures, permissions and cost guardrails

The companion archive provides `snowpark_pipeline.py`, `test_snowpark.py`, `pyproject.toml`, a synthetic CSV, optional SQL bootstrap and a review worksheet. The default path uses Python 3.10+ and the standard library; it needs no account, credentials, cloud access or paid compute. Optional Snowpark DataFrame execution requires an approved Snowflake account, a limited-role warehouse and compatible package versions. No such cloud execution is claimed in this edition.

The event fixture has **12 delivered rows**, including one duplicate delivery, one superseded wind measurement, one temporary TEST reading and subsequent tombstone, plus an invalid zero revision and negative solar reading. The model enforces explicit UTC, finite nonnegative quantities, event-ID deduplication and highest revision per provider/technology/interval key. This is a *synthetic* teaching dataset, not statistics from Fingrid or another external source.

9.L.2 Step-by-step transformation and evidence

Listing 15: Default reproducible steps from the lab directory.

```
python snowpark_pipeline.py
python -m unittest -v test_snowpark.py
```

The first command writes three files to `output/`; the second checks twelve invariants. The Snowpark helper accepts a Session and returns an *unexecuted* join-and-aggregation plan over approved source tables. It performs no write and is not imported by the offline tests.

Actually observed offline fixture result	Value
Delivered rows / invalid rows	12 / 2
Identical event deliveries skipped	1
Distinct accepted valid events	9
Visible latest-state facts	6
Latest-state tombstones	1
Approved energy aggregate (MWh)	66.000
Offline unit tests passed	12 / 12

Table 9.2. Deterministic Python result; no cloud-performance or Snowpark API test is implied.

Optional integration extension: Load exactly the six silver facts into a disposable table and a `TECHNOLOGY_DIM` with one row per technology, then call the lazy Snowpark plan. Expect three category groups and the same 66.000 MWh after grouping. Retain query IDs and environment versions; the optional route is *not executed* in this review.

9.L.3 Assertions, troubleshooting, rollback and cleanup

Twelve tests verify row accounting, six approved facts, the 66.000 MWh total, wind revision precedence, tombstones, duplicate/replay handling, malformed data, revision conflicts and UTC enforcement. They prove only the tested offline Python rules, not live Snowpark execution.

If the total differs, inspect the business key and revision order. If TEST reappears, preserve tombstones during latest-version selection. If category totals multiply, inspect dimension uniqueness. For cloud connection errors, inspect role, warehouse and policy without exposing secrets. A local mock limitation is not evidence of an engine bug.

Rollback and cleanup. The default artefacts are inside `output/`; preserve evidence, then delete that local folder. No cloud teardown is active. For optional integration, approve object ownership, removal and billing evidence.

9.L.4 Foundation, practitioner and production-design exercises

Foundation: Add a duplicated wind correction with the same EVENT_ID. Identify which counter changes and prove the approved total is unchanged. **Practitioner:** Add a later solar correction with a higher valid revision, then show both the new result and its invariance to replay; explain why arrival time is not the ordering key. **Production design:** Compare a single SQL model, a Snowpark DataFrame service and an external pandas service for a governed hourly dashboard. Specify data locality, role, test strategy, version pinning, latency SLO and teardown.

Assessment rubric (10 points)
Data grain and revision semantics (2); accurate Snowpark execution-boundary explanation (2); six-row and 66.000 MWh reconciliation (2); tests and replay handling (2); cost, least privilege, evidence and cleanup (2). Full credit requires stating what was not executed.

Chapter review. (1) What triggers a Snowpark DataFrame action? (2) Where can Python handlers execute? (3) How would you detect an exploding join? (4) What is the difference between a local mock test and a warehouse integration test? (5) Which operational evidence should accompany a production release?

Looking ahead. Chapter 10 places such transformations inside dbt models and repeatable ELT delivery workflows. Python does not remove the need for declared lineage, testable contracts, or principled production releases.

Print-release verification gate
Before print, recheck Snowpark runtimes and packages; execute and document a real Snowflake integration; substantiate any measured costs; and review external GitHub licensing.

CHAPTER 10 ENGINEERING / TRANSFORMATION DELIVERY

ETL/ELT Pipelines, dbt, and Delivery Workflows

A passing transformation is not yet a deployable data product

An accurate dashboard can be one deployment away from a silent failure. At 08:15, Finland Energy Analytics reports 66.000 MWh from six accepted technology readings. At 08:20, a developer adjusts an incremental filter. The next deployment succeeds, but a late correction is ignored; the dashboard becomes stale while every task remains green. The team has a SQL query, but not yet a trustworthy delivery workflow. Who owns the source contract? Which test would have failed? Can an operator restore the last accepted result without erasing the corrected records?

The previous chapters built ingestion, revision handling, incremental state and Python transformations. This chapter adds an engineering discipline around them: *declarative transformation models, source contracts, documented lineage, controlled execution and safe release.* We use dbt to make these concerns inspectable. Neither dbt nor an orchestration service automatically guarantees correctness; the guarantees come from explicit tests, sound keys, reliable source semantics and operational procedures.

Learning outcomes

After this chapter you can: contrast ETL and ELT; organise bronze, silver and gold models; configure a minimal Snowflake dbt project; implement source/model tests; distinguish incremental state from history; evaluate Snowflake Tasks and external orchestrators; configure a conservative CI pipeline; apply freshness and quality gates; and document a reversible release.

Prerequisites. Chapters 4, 8 and 9, Git, SQL and basic Python. Cloud exercises need a restricted Snowflake role, approved warehouse, and dbt adapter. The companion includes a **zero-cloud Python oracle**; the dbt project is prepared for a separately approved integration test, not claimed executed here.

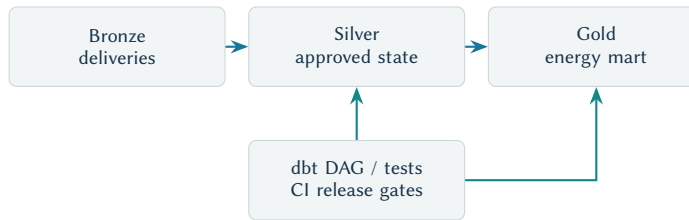


Figure 10.1. Correctness evidence and change controls surround the transformation path.

10.1 ETL versus ELT: architecture and trade-offs

10.1.1 ETL versus ELT execution paths

Extract–transform–load (ETL) transforms data before it enters the analytical destination; **extract–load–transform (ELT)** lands permitted source data and performs subsequent transformations within or adjacent to the destination. These acronyms describe *execution order*, not the quality of the pipeline. Both can validate, protect privacy and maintain lineage; both can also do those things poorly.

Take twelve synthetic revision events from Chapter 9. An ETL service could parse the CSV, reject invalid measurements, apply winning revisions, and then load six approved energy facts. Under ELT, the same event deliveries are first written into a bronze relation, while Snowflake SQL models perform checks, deduplication and curation. The business outcome must be compared at a specified grain, not inferred from architecture labels.

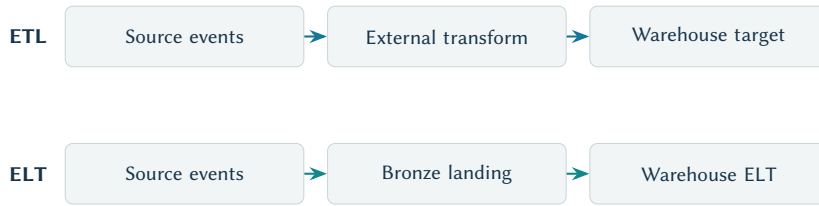


Figure 10.2. The key distinction is where the transformation boundary falls.

Preserve an audit trail. Loading a raw event does not mean every field should be retained indefinitely. A bronze zone should enforce approved retention, access and masking requirements. Where the incoming payload contains sensitive information, extract-and-redact before persistence may be a better architecture. An analytical platform is not a justification for unrestricted raw-data accumulation.

> **Decision rule**
> If a source must be cleansed or de-identified before crossing a legal or contractual boundary, ETL or a privacy-preserving hybrid may be mandatory. If data can be landed safely and the main transformations are relational, ELT often simplifies lineage and makes compute elastic. Decide from constraints; do not select an acronym first.

Check. Which stage holds the first durable copy of a source record in each design? Where would you quarantine an invalid negative MWh value?

10.1.2 Pushdown and platform trade-offs

Pushdown means delegating operations to a compute engine near the relevant data rather than exporting intermediate rows to a separate process. A dbt SQL model normally executes through the destination adapter; Snowflake evaluates the SQL on its compute. This avoids unnecessary client materialisation for ordinary joins and aggregates. It does *not* mean external engines are obsolete: specialised libraries, contractual boundaries, or source-side API restrictions may require upstream processing.

For the energy example, aggregating `SUM(ENERGY_MWH)` by `INTERVAL_UTC` is naturally pushed into a Snowflake SQL model. Parsing a proprietary binary instrument message, by contrast, may warrant a narrowly scoped extraction service. An engineer should record the trust boundary and data movement in either design.

Listing 1: Warehouse-side gold calculation; assumes accepted silver rows.

```
SELECT interval_utc, SUM(energy_mwh) AS total_mwh
FROM analytics.silver_energy
GROUP BY interval_utc;
```

The main trade-offs are **transfer cost**, **replay capability**, **compute isolation**, **governance**, and **operational complexity**. A cloud warehouse can scale compute separately from storage, yet badly selected warehouse sizes and repeated full scans can still be expensive. An external compute engine may be cheaper for a particular transformation but create operational expense in scheduling, credentials, logs and recovery.

Dimension	ETL before loading	ELT within Snowflake
Data movement	Potential pre-load processing and copies	Raw landing, then in-platform queries
Lineage	Must connect two execution systems	dbt graph plus source/load evidence
Failure recovery	Replay external transform and upload	Rebuild downstream objects from retained bronze
Compliance	Can redact before destination	Needs landing permissions and retention policy
Economics	External runtime and transport charges	Warehouse and storage charges

Failure case: the appealing six-row benchmark
Do not compare a local process that reads six rows with a warehouse pipeline that checks and processes a million. Benchmark the same input, identical business semantics, matching output, relevant data-transfer boundaries and fully itemised cost.

Check. When would replacing an ELT SQL aggregate with a pandas download worsen observability even if elapsed time falls in a tiny demo?

10.1.3 Selecting a design for maintainability

A maintainable design places each rule where its meaning is obvious and its behaviour is independently testable. For our energy project, separate *delivery validation* (may this event be accepted?), *state resolution* (which revision wins?) and *analytical representation* (which measure and grain do consumers see?). Mixing all three inside a long SQL statement may deliver the expected number but makes drift, deletions and late corrections difficult to investigate.

Write a short architecture decision record (ADR). State the requirements, alternatives, reasons, constraints and a trigger to revisit the decision. Do not claim that ELT always reduces complexity: dbt can create its own configuration, dependency, deployment and observability work. The objective is an explainable path to a stable output.

ADR-010: illustrative decision
Context: permitted synthetic events arrive in small batches; engineers own Snowflake SQL and Git. **Decision:** retain an audited bronze history, transform approved revisions with dbt, publish a gold mart; use a local Python oracle in CI. **Rejected default:** exporting all bronze rows to a local Python job. **Revisit when:** required latency, data volume or regulatory obligations change materially. **Risk:** source freshness and Snowflake SQL validation must be checked independently.

Listing 2: A concise acceptance contract for the running example.

```
Grain:     provider + technology + interval_utc
Revision:  highest valid positive revision wins
Delete:    latest DELETE makes its key invisible
Value:     finite and >= 0 MWh for UPSERT
Oracle:    6 rows; SUM(energy_mwh) = 66.000
Proof:     source tests + model tests + offline oracle
```

An acceptance contract is also a *boundary for change*. If the business later wants both corrected and original measurements displayed, that is a new semantic requirement, not a one-line enhancement. Its consequences may include an event-history table, revised gold grain and new tests.

Project reference: P108: Astronomer ETL/ELT with Airflow and Snowflake. *Not independently executed; rights review pending.*

Check. What would make an ELT-first design inappropriate even if the Snowflake code is technically correct?

10.2 Bronze, silver, and gold transformation layers

10.2.1 Raw bronze ingestion

A *bronze layer* preserves approved source deliveries, their provenance and enough detail to explain what happened. In this book the small input is a revisioned energy-event CSV. It deliberately includes a repeated event ID, an older hydro reading, a logical deletion, a negative quantity and a zero revision. A source delivery is not automatically a business fact: bronze records may be incomplete, invalid or superseded.

A useful bronze contract records the source identifier, delivery timestamp, file or batch reference, schema version, ingestion outcome and retention rules. Snowflake file metadata can help trace staged loads; application records should not pretend every connector exposes exactly the same fields. Stable event IDs and batch lineage are especially helpful during replay.

Listing 3: Illustrative bronze landing structure.

```
CREATE TABLE IF NOT EXISTS RAW_ENERGY_EVENTS (
  event_id VARCHAR, provider VARCHAR,
  technology VARCHAR, interval_utc VARCHAR,
  revision VARCHAR, energy_mwh VARCHAR,
  operation VARCHAR, source_batch VARCHAR,
  ingested_at TIMESTAMP_LTZ DEFAULT CURRENT_TIMESTAMP()
);
```

Why text in bronze? Retaining malformed numeric strings can be useful for diagnosis. At silver, a failed cast is a quality event; at bronze, the original payload is evidence. If compliance rules disallow persisting a field, exclude or mask it before this point.

> **Common error: treating ingestion success as analytical validity**
> A batch can load completely while containing a negative MWh value. Loading, parsing, validation, correction handling and publication are separate states. A dashboard must never infer quality solely from the absence of a COPY error.

Practical task. Identify the three fields you would require for answering "which file supplied this incorrect nuclear measurement?" How would you prevent repeated transfers from doubling a reported total?

10.2.2 Validated silver transformation

Silver provides a stable, explicitly defined analytical contract. For our energy stream, the rules are: reject negative quantities and invalid revisions, collapse exact replay events, identify the highest accepted revision for each provider–technology–UTC interval, and exclude a key whose winning event is a DELETE. These steps must be consistent with earlier chapters; this chapter does not quietly alter their meanings.

Listing 4: Core revision-resolution pattern; abbreviated for print.

```
WITH valid AS (
  SELECT * FROM raw_energy_events
  WHERE TRY_TO_NUMBER(revision) >= 1
    AND TRY_TO_DECIMAL(energy_mwh,18,3) >= 0
), latest AS (
  SELECT *, ROW_NUMBER() OVER (
    PARTITION BY provider, technology, interval_utc
    ORDER BY TRY_TO_NUMBER(revision) DESC, event_id DESC
  ) AS rn FROM valid
)
SELECT * FROM latest
WHERE rn = 1 AND UPPER(operation) = 'UPSERT';
```

This is an instructional fragment, not a full conflict policy. A real pipeline must also check identical event IDs, equal-revision disagreements, timestamp format, allowed operations and null keys. The companion project implements those checks around the printed SQL. If two distinct readings claim the same business key and revision, selecting an arbitrary winner risks silent corruption: quarantine the conflict and request source-owner resolution.

10.2.3 Business-ready gold marts

Gold models expose consumer-oriented measures and grain. Our `fct_energy_hourly` contains one row per UTC interval and technology, with a sum of accepted MWh. A separate daily roll-up should use a documented calendar convention; converting UTC to `Europe/Helsinki` may create 23- or 25-hour local days around daylight-saving changes. Never mix local date and UTC interval silently.

Listing 5: A gold measure with explicit grain.

```
SELECT interval_utc, technology,
       SUM(energy_mwh) AS energy_mwh
FROM silver_energy
GROUP BY interval_utc, technology;
```

Project reference: P043: Snowflake retail analytics pipeline. *Not independently executed; rights review pending.*

Check. Why should a downstream BI model query gold rather than recalculate the highest revision from arbitrary bronze deliveries?

10.3 Structuring a dbt project for Snowflake

10.3.1 Core dbt project structure

dbt is a transformation framework in which SQL models, dependency declarations, tests, documentation and configuration form an auditable directed acyclic graph (DAG). The `ref()` function declares a dependency on another dbt model or seed; `source()` refers to a declared external relation. dbt compiles templated SQL, runs the relevant adapter queries and produces execution artifacts. It does not itself ingest files, create a complete incident response process or substitute for an external business contract.

Listing 6: Minimal repository tree in the companion archive.

```
dbt_project/
  dbt_project.yml
  profiles.example.yml
  seeds/energy_revisions.csv
  models/staging/stg_energy_events.sql
  models/intermediate/int_energy_current.sql
  models/marts/fct_energy_hourly.sql
  models/schema.yml
  tests/assert_fixture_total.sql
.github/workflows/offline-checks.yml
.github/workflows/manual-cloud-deploy.yml
```

The file `dbt_project.yml` names the project, sets its profile and groups model defaults. A seed is a tiny CSV that dbt can load to a development database. It is convenient for a deterministic teaching fixture; it is not a recommended high-volume source-ingestion mechanism. Model names and directory structure should communicate responsibilities rather than mirror an organisation chart.

Listing 7: Simplified project configuration.

```
name: 'finland_energy_ch10'
version: '1.0.0'
config-version: 2
profile: 'finland_energy_ch10'
model-paths: ['models']
seed-paths: ['seeds']
test-paths: ['tests']
models:
  finland_energy_ch10:
    staging: {+materialized: view}
    intermediate: {+materialized: view}
    marts: {+materialized: table}
```

Test what changes. A source-file rename, a new macro or a model configuration update may change compiled SQL and dependency selection. Store source and configuration in Git, review both and capture artifacts for reproducibility. Avoid mixing credentials with model code.

Project reference: P109: Getting started with dbt on Snowflake. *Not independently executed; rights review pending.*

Check. Why should a model reference `{{ ref('stg_energy_events') }}` rather than a hard-coded production schema?

10.3.2 Snowflake profiles and targets

A dbt profile connects a selected target name to Snowflake account, database, schema, role and warehouse settings. Keep confidential values in a secret manager or environment variables; never commit tokens or passwords to a book repository. For a local dbt Core-style configuration, a credentials-free `profiles.example.yml` makes required names explicit while leaving values to the operator.

Listing 8: Environment-driven profile excerpt; no secrets provided.

```
finland_energy_ch10:
  target: dev
  outputs:
    dev:
      type: snowflake
      account: "{{ env_var('DBT_SNOWFLAKE_ACCOUNT') }}"
      user: "{{ env_var('DBT_SNOWFLAKE_USER') }}"
      authenticator: externalbrowser
      role: "{{ env_var('DBT_SNOWFLAKE_ROLE') }}"
      database: "{{ env_var('DBT_SNOWFLAKE_DATABASE') }}"
      warehouse: "{{ env_var('DBT_SNOWFLAKE_WH') }}"
      schema: CH10_DEV
      threads: 2
```

The choice of `externalbrowser` is illustrative for an interactive workstation, not a universal unattended CI authentication strategy. Service identities require approved noninteractive authentication and carefully scoped privileges. Test edition, account policies and adapter support before adopting any recipe from the internet.

10.3.3 Separation of development and deployment

A private development schema protects production objects from experiments. The production deployment identity should access only the schemas and compute needed for its tasks, not run as ACCOUNTADMIN. Define who can approve production changes, how expensive warehouse usage is limited and how a failed build prevents promotion. In a real installation, environment naming must also account for dbt's schema-generation behaviour: a custom schema name may be appended to the target schema rather than replace it, unless project macros change that convention.

Environment contract
Dev: synthetic data, developer role, bounded credits, disposable schema. **CI:** isolated temporary namespace or approved selective-build strategy, no write permission to production tables. **Prod:** approved code, stable role, explicit build window, retained run artifacts and rollback owner.

Project reference: P111: dbt and Snowflake hands-on lab. *Not independently executed; rights review pending.*

Check. Which settings must differ between a developer's laptop and an unattended production job even if both compile identical SQL?

10.4 Models, sources, tests, and documentation

10.4.1 Source declarations and models

The semantic origin of a relation matters. In dbt, a source declaration makes the upstream object an explicit DAG node, while `ref()` gives downstream consumers a stable dependency on a managed model. Use a *seed* only for this chapter's tiny synthetic fixture; a production pipeline would normally declare a landed bronze source whose freshness and ingestion owner are documented.

Listing 9: Selected YAML model properties; dbt Core-style syntax.

```
version: 2
models:
  - name: fct_energy_hourly
    description: "Accepted synthetic MWh by UTC hour and type"
    columns:
      - name: technology
        data_tests: [not_null]
      - name: energy_mwh
        data_tests: [not_null]
```

10.4.2 Data tests and documented lineage

A *generic data test* checks a reusable condition: uniqueness, non-nullness, relationships or accepted values. A *singular test* is an SQL query returning offending rows; an empty result normally means the test passed. Tests must match the model grain and domain. A total-mismatch test tied to an invented teaching fixture is appropriate in this chapter, but would be a harmful permanent rule for real Finnish energy statistics.

Listing 10: A fixture-only dbt singular test; zero rows means pass.

```
SELECT SUM(energy_mwh) AS observed_mwh
FROM {{ ref('fct_energy_hourly') }}
HAVING ABS(SUM(energy_mwh) - 66.000) > 0.0001;
```

Lineage is not simply a diagram. It should identify the business definitions used at each edge: event revision key, accepted values, UTC grain and publication contract. The DAG explains dependencies, while incident evidence identifies which source batch and model invocation produced a visible number.

10.4.3 Build failures and acceptance criteria

`dbt build` runs selected models and relevant tests, and failures must gate promotion rather than merely appear in logs. A SQL syntax error, a relationship-test failure and a missing upstream relation need distinct diagnostic procedures. **Source freshness is a separate operation**; do not claim a normal `dbt build` has necessarily checked it. The command names and supported properties should be verified against the pinned dbt Core/adapter versions used for release.

Project reference: P112: dbt Core repository. *Not independently executed; rights review pending.*

Check. Why does a passing `not_null` test on `technology` not establish that exactly one current revision exists for each key?

10.5 Incremental models and snapshot strategies

10.5.1 Incremental model configuration

A dbt incremental model updates an existing destination rather than rebuilding it in full each time. The first run typically creates the table; subsequent runs apply a developer-defined source filter and adapter-specific incremental strategy. `unique_key` is not a substitute for testing source uniqueness: duplicate rows in the update set can make a merge ambiguous, and repeated versions can silently corrupt current-state semantics.

Listing 11: Minimal incremental pattern; adapt to validated change-feed data.

```
{{ config(materialized='incremental',
          incremental_strategy='merge',
          unique_key='event_business_key') }}
SELECT * FROM {{ ref('stg_changes') }}
{% if is_incremental() %}
WHERE source_updated_at >= (
    SELECT DATEADD('hour',-2,MAX(source_updated_at))
    FROM {{ this }}
)
{% endif %}
```

The two-hour lookback is a *design hypothesis*, not a general guarantee. Late corrections can arrive beyond it, and wall-clock update timestamps may not agree with source revision order. For critical energy data, retain a full history and document replay or rebuild conditions. Aggregate output must be recomputed for all affected keys, not merely append newly arrived increments.

10.5.2 Snapshot and change-history techniques

A snapshot records historical versions when a tracked relation changes. A snapshot is useful for mutable source rows that overwrite previous state and where point-in-time reconstruction matters. If an upstream event log already preserves every revision, a second snapshot may duplicate history and processing costs. The decision depends on audit requirements, source overwrite behaviour and the available change timestamp.

10.5.3 Rebuild, reprocess, and schema drift

`--full-refresh` is a powerful operational tool, not a harmless retry flag: rebuilding can consume substantial compute, alter object state and interact with consumers. A schema change should be evaluated against compatibility, dbt adapter behaviour and downstream contracts before activation. Always stage a full refresh and compare row counts, keys, totals, latency and access grants in an approved environment.

Failure mode: a late wind correction
An incremental filter based only on maximum already-processed timestamp can drop a valid revision arriving today for last week's interval. Business time and ingestion time are separate coordinates. Carry a replay window or explicit affected-key strategy, and test it.

Project reference: P113: incremental dbt reference. *Not independently executed; rights review pending.*

Check. How would you approve a production full refresh?

10.6 Native and external orchestration options

10.6.1 Native orchestration options

Snowflake Tasks can schedule SQL actions and arrange task graphs inside the platform. They can also be triggered under supported conditions. Their strengths include a small operational footprint for platform-local work; their limitations depend on dependencies, retries, deployment workflow and the need to coordinate external systems. A Task does not replace dbt's model graph: one describes when work runs, while the other describes what transformations depend on what inputs.

10.6.2 External schedulers and dependencies

Airflow, Prefect and similar orchestration systems can coordinate downloads, validation, dbt commands, approvals and downstream notifications. They introduce an external state store, worker environment, permissions and another availability boundary. A pipeline that lands an API feed, validates raw contracts, runs dbt, checks freshness and publishes a BI artifact may benefit from such orchestration. A small warehouse-only transformation often does not need this additional platform.

Dimension	Snowflake-native	External scheduler
Primary domain	SQL / warehouse execution	Cross-system tasks and approvals
Dependency view	Task graph and history	Orchestrator DAG and run states
Credentials	Snowflake role boundaries	Secrets across several systems
Failure control	Task retry, history and alerts	Custom retry, backfill and alerts
Operations	Fewer moving parts for local tasks	Additional infrastructure lifecycle

10.6.3 Choosing based on reliability and operations

List actual failure scenarios: network outage during input fetch, partial object deployment, a source schema change, an unexpected dbt test failure and a report-refresh error. Then choose the smallest orchestration design that can detect, stop, resume and explain each failure. *At-least-once orchestration* still requires idempotent data transformations; task retries do not automatically become exactly-once business updates.

Operator question
After an orchestrator resumes a failed run, how will it distinguish completed, safe-to-skip work from a partially published gold table? A transaction boundary, staging-and-swap strategy or release-marker table may be required.

Project reference: P114: Astronomer Airflow example. *Not independently executed; rights review pending.*

Check. Which system should own the schedule if the same release must call a partner API before beginning a Snowflake dbt job?

10.7 Git branching, CI/CD, and environments

10.7.1 Branch-based change controls

A release should be traceable to a reviewed Git commit, a set of model and test artifacts, and a person responsible for authorising the change. Branch-based collaboration separates experimental SQL from approved business semantics. Protect the default branch, require review for gold-grain or access-policy changes, and avoid direct edits to production worksheets that leave the repository out of date.

10.7.2 Automated validation in CI

Continuous integration (CI) can run inexpensive offline checks on every pull request without connecting to Snowflake. The companion runs Python contract tests against the same synthetic input as Chapter 9 and checks the project layout. A separately approved cloud CI job can then run `dbt deps`, `dbt compile` and selected builds in an isolated namespace. Success in an offline oracle cannot establish correct dbt compilation or adapter behaviour; each verification layer should be labelled.

Listing 12: A safe pull-request workflow: no cloud secrets or billing.

```
name: offline-data-contracts
on: [push, pull_request]
jobs:
  checks:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.11'}
      - run: python -m unittest discover -s lab -p 'test_*.py'
```

Action versions shown in print are examples, not immutable supply-chain pins. For an actual release, pin reviewed action commit SHAs, restrict workflow permissions, review dependencies and save test logs as artifacts. Do not store a reusable production password in a public workflow file.

10.7.3 Deployment environments and change approvals

Continuous delivery means a release is reliably *ready* to deploy; continuous deployment also automates the production promotion itself. For this book the recommended default is manual approval for production. The `manual-cloud-deploy.yml` example uses workflow dispatch and an environment approval boundary, yet is deliberately disabled unless the owner configures secrets, roles and a controlled target.

Failure mode: an impressive green workflow
All local tests can pass while the release service account lacks a Snowflake grant or the production source schema has drifted. CI is layered evidence; it is not a magic certificate. Require a separate preproduction integration gate.

Project reference: P028: Snowflake DevOps hands-on guide. *Not independently executed; rights review pending.*

Check. Who approves a unit change in gold?

10.8 Data contracts, quality gates, and observability

10.8.1 Data contracts and schema expectations

A *data contract* specifies what a consumer can assume about a published dataset, who owns that promise and how changes are negotiated. For the energy platform, a concrete contract defines source identity, business key, revision precedence, timestamp representation, non-negative MWh, operation values, expected grain, retention and issue escalation. It should also distinguish invalid deliveries, late but valid corrections and genuine source outages.

Illustrative silver contract: `energy_current_v1`
Owner: Energy ingestion team. **Primary key:** (provider, technology, interval_utc). **Time:** ISO-8601 UTC with a literal `Z`; no implicit Helsinki-local conversion. **Measure:** `energy_mwh`, decimal, finite, nonnegative. **Version:** maximum approved positive revision. **Delete:** winning DELETE removes the current-state row; event history remains governed separately. **SLA:** agreed publication time measured at the gold layer.

An important distinction is *schema compatibility versus semantic compatibility.* A column can remain numeric while its unit silently changes from kWh to MWh; a type test will pass while the result is off by a factor of one thousand. Contract versioning should therefore cover units, grain, timezone, update semantics and permitted nulls, not merely column names.

Listing 13: Compact contract excerpt; companion includes the full version.

```
name: energy_current_v1
key: [provider, technology, interval_utc]
unit: MWh
schema_version: 1
revision_policy: highest_valid_revision
on_conflicting_revision: quarantine
consumer_breaking_change: review_required
```

Acceptance evidence. Confirm each key is unique and non-null, no visible current row is deleted, and all accepted quantities satisfy the unit/value constraints. The data contract is incomplete without an owner and explicit escalation path for violations. Capture example failure records, not merely a numeric pass percentage.

Project reference: P118: dbt data-quality example. *Not independently executed; rights review pending.*

Check. Why could a pipeline pass `not_null`, `unique` and `relationships` yet still publish a semantically incorrect energy total?

10.8.2 Freshness and observability

Freshness measures how recently relevant source data was made available, but different timestamps answer different questions. **Event time** is when energy was measured; **ingestion time** is when a delivery arrived; **publication time** is when a tested gold result became visible. Delayed correction events make these times diverge. For the Finland Energy Analytics platform, end-to-end delay from approved source update to published gold is often the best operational measure.

Metric	What it detects
Source max delivered timestamp	Source has not supplied expected batches
Accepted-row ratio	Sudden growth in schema/value rejections
Gold publication lag	Stale reporting despite successful data arrival
Model run duration	Slow scans, concurrency or warehouse sizing
Failed test count	Contract violations blocking a release
Cost per accepted batch	Changing pipeline economics

dbt freshness checks are not equivalent to `dbt build`. dbt versions and deployment products differ in command naming and features; for a dbt Core v1 workflow, `dbt source freshness` remains a relevant command, while newer platform documentation also describes `dbt freshness`. Pin the engine/adapter before release and verify the specific CLI invocation. A declared `loaded_at_field` should be a trustworthy ingestion timestamp, not the energy measurement's original event timestamp unless that is the intended SLA.

Listing 14: Illustrative source freshness declaration.

```
sources:
  - name: energy_landing
    database: ANALYTICS_SANDBOX
    schema: RAW
    tables:
      - name: ENERGY_EVENTS
        loaded_at_field: ingested_at
        freshness:
          warn_after: {count: 2, period: hour}
          error_after: {count: 4, period: hour}
```

Failure mode: fresh source, stale mart
A source freshness check may pass at 09:00 while a model's last successful materialisation was yesterday. Track source receipt, dbt run outcome and downstream publication independently. A green source check is necessary in some workflows, never sufficient for an end-to-end guarantee.

Check. Which timestamp should be used to alert that a near-real-time gold dashboard is more than thirty minutes behind an approved source correction?

10.8.3 Quality gates, ownership, and alerting

A *quality gate* is a decision point, not simply a dashboard number. Define whether a failure blocks a deployment, quarantines one batch, notifies an owner or merely records a warning. Tests protecting business keys, current-state visibility and published units should be blocking for the gold mart. A temporarily late noncritical reference file may warrant an alert without invalidating otherwise correct published facts.

Example release gate policy
Block: duplicate current business key; a negative accepted MWh; conflicting equal revisions; missing required technology; model or schema compilation error; or a critical source freshness breach. **Warn:** minor delay below the agreed SLA, a documented optional dimension missing or modest runtime increase. **Escalate:** repeated failed retries, prolonged stale gold, unexplained discrepancies with a verified source total.

Observability is also a *human information system.* Each alert should include model and run identifiers, first failed condition, expected versus observed value, the affected data interval, suggested recovery procedure and the responsible contact. An alert that says only `dbt failed` forces the on-call engineer to reconstruct context while the business report continues aging.

Listing 15: Incident record fields for the release runbook.

```
release_id: ch10-demo-r1
status: BLOCKED
failed_gate: silver_current_key_unique
affected_interval_utc: 2026-03-01T10:00:00Z
source_batch: synthetic_fixture_01
owner: data-platform-on-call
next_action: quarantine + inspect offending keys
```

Implement independent verification when feasible. A Python oracle or carefully maintained SQL assertion can validate a critical invariant separately from the dbt model that created it. An independent checker is most valuable when it does not duplicate the same code and therefore the same bug. Store successful and failed outcomes; suppressing test failure logs undermines reproducibility.

Cost and risk. Tests scan data too. Prefer narrowly scoped assertions on high-value contracts, benchmark warehouse consumption and isolate expensive historical reconciliation from ordinary pull-request checks. A skipped test is not a passed test.

Project reference: P117: Data engineering benchmark reference. *Not independently executed; rights review pending.*

Check. Under what circumstances would you publish yesterday's last known-good gold result rather than partially publish today's failed update?

10.9 Deployment, rollback, and release checklists

10.9.1 Release checklists and migration safety

A release is a controlled transition from one accepted data product state to another. The checklist should cover executable code *and* the implied changes to business semantics, table materialisation, grants, refresh behaviour and consumer expectations. "The SQL compiled" cannot answer whether a release is safe: a compiled query can return the wrong grain or change units unnoticed.

Gate	Required evidence before promotion
Source contract	Owner approved schema, unit, grain, revisions
Code review	Reviewed commit, dependency and policy diffs
Independent oracle	Same current-key count and aggregate as fixture
Cloud integration	dbt build/tests in isolated approved target
Operational readiness	Spend guardrails, runbook, monitoring and on-call
Rollback readiness	Last-good artifact and restore procedure
Promotion authorisation	Named reviewer approved release identifier

Schema migration is not always reversible. Adding a nullable field may be compatible with old consumers; dropping a column, changing a numeric unit, replacing a relation or modifying an access role may require a multi-step migration. Prefer *expand–migrate–contract*: introduce compatible structures, move readers with evidence, and remove old structures only after consumer sign-off. Keep a record of which public dashboards, APIs or scheduled jobs depend on a model.

Before promotion, evaluate whether the deployment will rebuild large tables. If it uses `--full-refresh`, estimate storage and compute and check the effect on downstream grants, data retention and scheduled consumers. The safe alternative for a high-impact gold mart can be to build a candidate relation and publish it only after its tests pass. Precisely how to swap or rename objects depends on the account's object types and grants; rehearse that procedure before treating it as a rollback plan.

Hard gate
Do not deploy a breaking model change if the team cannot name (1) its affected consumers, (2) the acceptance queries, (3) the last known-good state, and (4) the person authorised to stop or reverse publication.

Project reference: P119: schemachange SQL migration tool. *Not independently executed; rights review pending.*

Check. Why is a passing pull-request test insufficient proof that a production `--full-refresh` will preserve the intended consumer experience?

10.9.2 Rollback strategy and change logs

Rollback can mean *revert the code*, *restore or repoint the published relation*, or *recompute from retained input*. These are related but distinct operations. Reverting a Git commit does not rewind data written by an incremental model. Copying an old table may restore a dashboard but lose valid late corrections. A robust plan names the intended recovery objective and which state is authoritative.

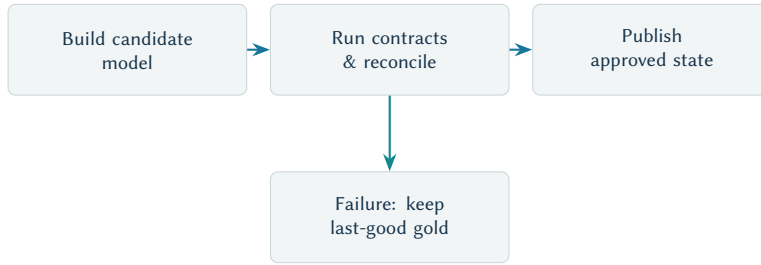


Figure 10.3. Publication is gated independently of candidate computation.

For the energy case, suppose a new model accidentally includes the invalid Solar revision of -5 MWh. The candidate's negative-value gate should fail. Operators leave the last approved six-row, 66.000 MWh gold result visible, mark it stale as appropriate, and investigate. If the incorrect model was already published, the team follows the rehearsed restore procedure and checks every affected reporting time window.

Listing 16: Minimum change-log entry.

```
release: energy-v1.1.0
commit: <reviewed SHA>
model_change: revision filter tightened
approved_by: <named operator>
integration_result: <Snowflake run IDs>
backout: repoint last-good published relation
verify: current_keys=6; fixture_total=66.000 MWh
```

A meaningful rollback record also states what *was not* reverted: the persisted raw event history, consumer caches, notification exports or downstream replicated copies may still require remediation. Snowflake Time Travel can provide recovery options for eligible objects within their retention policy, but it is not a universal substitute for backups, migration design or permission-aware rollback.

Failure mode: revert SQL, forget state
A schema or incremental merge may already have changed target data. Treat code revision and data-state restoration as separate tests, with a business-level reconciliation after both.

Check. Which rollback method preserves a legitimate late correction that arrived after the last code release?

10.9.3 Handover documentation and incident response

The person paged at midnight needs a map of the pipeline, not a list of folder names. A usable handover names the operating owner, source contact, most important data contract, critical model identifiers, schedule, alert thresholds, typical failure modes, escalation path and manual recovery steps. It distinguishes what can be rerun without data corruption from actions that require explicit approval.

Incident scenario: stale daily energy total
Signal: approved gold publication exceeds its freshness objective. **Triage:** inspect source arrival, validation rejects, dbt run results and task/orchestrator history. **Containment:** keep last known-good gold result, visibly mark freshness. **Recovery:** fix input/configuration, stage selective replay, rerun tests, reconcile affected UTC intervals, approve publication. **Postmortem:** record detection delay, data impact, corrective action and new regression test.

Do not automatically retry every failed action. A transient network error may warrant a bounded retry with backoff. A repeated duplicate-key violation is a contract problem, and another ten retries merely spend credits. A permission failure may reflect an intentional policy change; do not escalate to `ACCOUNTADMIN` as an emergency workaround.

The runbook is itself a product artifact. Store it with the reviewed release, link it from monitoring and test its major steps in an isolated environment. Record the last time an operator verified those steps. Keep the incident procedure small enough to follow under pressure, with detailed technical commands in a controlled supplement.

Listing 17: Operator's quick triage, in order.

```
1. Check data contract and expected gold SLA.
2. Identify last successful model and source batch.
3. Inspect failed tests and affected business keys.
4. Determine whether retry/replay is idempotent.
5. Keep or restore last known-good publication.
6. Request approval, rebuild, reconcile, document.
```

Project reference: P120: Snowflake DevOps data projects guide. *Not independently executed; rights review pending.*

Check. What evidence should an on-call engineer capture before deleting a faulty intermediate relation during recovery? How will consumer teams learn whether the report is stale or incorrect?

10.L Guided lab, chapter review, and exercises

10.L.1 Prerequisites, inputs, environment, permissions, and cost guardrails

Goal. Deliver a governed `fct_energy_hourly` model from the twelve synthetic events used in Chapter 9. The offline route is intentionally free: Python 3.10+ with no cloud credentials. The separate *cloud integration route* requires a permitted Snowflake account, an isolated development database/schema, an auto-suspending warehouse and dbt Core plus a compatible Snowflake adapter. Never use production customer data.

Inputs. `seeds/energy_revisions.csv` includes duplicate delivery, valid revision correction, tombstone, invalid negative measure and invalid revision. The contract expects six visible current facts and a total of **66.000 MWh**. These are synthetic assertions, not measurements of Finnish electricity production.

10.L.2 Step-by-step procedure with selected code and actual or clearly illustrative results

1. Open `lab/README.md`. Run `python offline_oracle.py` and the automated tests. Save `output/offline_summary.json`.
2. Inspect the dbt seed and dependency chain: `stg_energy_events` → `int_energy_current` → `fct_energy_hourly`. Identify the rule that prevents deleted keys from appearing in gold.
3. Examine `models/schema.yml` and the singular SQL assertions; distinguish domain, uniqueness and fixture-total checks.
4. Review `offline-checks.yml` and `manual-cloud-deploy.yml`. Explain why no pull-request workflow holds production secrets.
5. *Optional, approved account only:* configure a personal profile, seed in a disposable schema, run selected `dbt build`, inspect the result and capture real query/run identifiers. This step has **not** been performed for this manuscript.

Listing 18: Safe zero-cloud commands.

```
cd lab
python offline_oracle.py
python -m unittest -v test_pipeline.py
# Cloud steps are separately approval-gated.
```

Expected fixture outcome, not a live Snowflake measurement
Delivered events: **12**. Rejected invalid deliveries: **2**. Duplicate deliveries: **1**. Approved current facts: **6**. Grand total: **66.000 MWh**. A release fails if any required invariant differs. Read the generated machine-readable summary rather than copying these figures into an operational report.

Cost guardrail. Offline execution generates no Snowflake spend. For an actual integration test, agree on expected warehouse usage and teardown before enabling credentials; suspend compute and remove only the disposable objects you own.

10.L.3 Expected result, assertions, common failure diagnosis, rollback, and cleanup

The acceptance suite tests revision ordering, duplicate delivery, logical deletion, negative-energy quarantine, missing keys, conflicting revisions, idempotent replay, current-row uniqueness, total reconciliation, unit semantics, output manifest and release-gate behaviour. An offline pass demonstrates the *Python contract implementation*; it does not establish that dbt SQL compiled or executed on Snowflake.

Failure symptom	Diagnosis / action
Duplicate current key	Check event-ID dedupe and revision partition
Incorrect total	Compare approved event set and tombstones
Missing dbt relation	Review profile, schema generation, grants
Stale source	Check ingest time separately from gold publication
CI green, cloud red	Review adapter, permissions, role and SQL compile

Rollback and cleanup. The offline lab only writes files in `lab/output`; deleting that directory restores a clean local state. In a cloud run, use the project-specific approved cleanup plan. Never make a blanket `DROP DATABASE` command the default teaching step. Save generated manifests, run result artifacts and an operator-approved release note first.

10.L.4 Foundation exercise, practitioner exercise, production-design question, and scoring rubric

Foundation (20 pts): describe the dbt dependency graph and explain why a bronze row may be rejected. **Practitioner (35 pts):** modify a valid technology revision, recompute the expected six-row current state and update the contract test without weakening quality rules. **Architecture (30 pts):** propose an environment design, CI gates, freshness thresholds and rollback for an hourly operational dashboard. **Evidence (15 pts):** provide clean test logs, a reviewed commit hash, a concise ADR and explicit limitations. *Pass guideline:* at least 80/100 with no critical correctness or safety failure.

Chapter synthesis

ETL and ELT determine a transformation boundary; bronze, silver and gold determine data responsibilities; dbt makes models and their dependencies inspectable; orchestration schedules work; CI and contracts guard changes; and release operations protect the users of published data. A reliable data product needs all of these responsibilities assigned, even when no single vendor tool implements them all.

Further reading. dbt incremental models; dbt data tests; dbt source freshness; Snowflake Tasks; the research repository URLs noted by section (not independently executed). Verify versions before publication. **Next.** Chapter 11 takes tested gold models into semantic definitions and Power BI consumption without losing provenance or grain.

CHAPTER 11 ANALYTICS / TRUSTED CONSUMPTION

Analytics Engineering and Power BI Integration

Reliable decisions begin where the pipeline meets the reader

The dashboard is green; the decision is wrong.
An energy team receives a polished visual showing a 66.000 MWh production total. At 08:30 one manager calls that number "current power", another treats it as national net supply, and a third compares it with yesterday's demand. Yet the fixture represents one synthetic interval, six technologies, no grid-wide coverage claim, and a separately supplied consumption sample. All three readers can use a technically correct chart to reach a false conclusion. The missing layer is a *contract between data producers and decision makers*.

Chapters 4 and 10 constructed a dimensional grain and a tested gold transformation. This chapter turns that gold output into a defensible *analytical product*: defined metrics, controlled semantics, documented freshness, deliberate connectivity, usable Power BI reports and an owner who knows what to do when numbers disagree. You will learn not just how to connect a database but how to know whether the resulting report deserves trust.

Learning outcomes
After this chapter, you can specify a serving layer; write a versioned KPI contract; configure a read-only Snowflake-to-Power BI connection; choose Import or DirectQuery using measured requirements; plan star-schema and DAX semantics; critique energy dashboard visuals; trace generated queries; establish refresh and incident evidence; and hand off a governed BI product.

Prerequisites. Chapters 4–5 and 10, introductory SQL, Power BI terminology, and an understanding of MW versus MWh. The companion includes a **standard-library Python oracle** and illustrative M/DAX/SQL; there has been **no live Snowflake or Power BI execution**. Any cloud activity needs approved credentials, a restricted role and a pre-agreed spending limit.

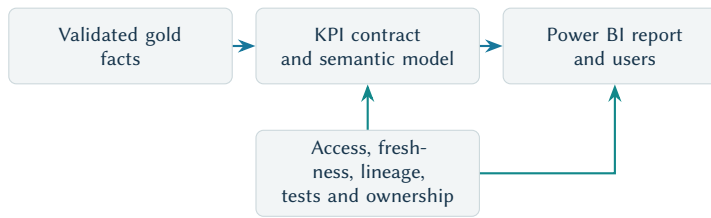


Figure 11.1. A decision is governed across every boundary, not only at ingestion.

11.1 Analytics engineering and serving-layer design

11.1.1 Serving layer and consumption requirements

The *serving layer* is the stable interface between engineered data and its consumers. It may consist of governed Snowflake views, materialised tables, a semantic model, APIs or curated extracts. The interface should expose approved fields and business meanings rather than force each report author to reconstruct upstream rules. For our running case, the upstream system resolves source-event revisions and logical deletes before publishing `FactGeneration` at *one row per UTC interval and technology*. A consumer must never sum the twelve raw delivery rows to calculate production.

A useful serving contract answers five questions: which rows are visible; what is their grain; when is a period complete; which role may access them; and whom should consumers contact after a correction? These questions are more durable than a particular BI product. A Snowflake view can be the public contract; internal bronze tables should not be granted merely to make connector setup easy.

Listing 1: Illustrative read-only inspection of the serving grain; not cloud-tested.

```
SELECT interval_utc, technology,
       SUM(energy_mwh) AS generation_mwh
FROM FINLAND_ENERGY_DEMO.GOLD.FCT_ENERGY_HOURLY
GROUP BY interval_utc, technology
ORDER BY interval_utc, technology;
```

11.1.2 Stakeholder questions and decision support

A dashboard should begin with a decision, not a chart palette. A grid operator may ask whether observed production balances measured consumption at the *same interval and geographic scope*. A finance analyst may ask for an auditable monthly energy total. An engineering lead may ask whether new source deliveries altered a validated KPI. Those are three different questions, potentially requiring different grains, refresh objectives and ownership.

Consumer	Question / grain	Acceptance evidence
Operations	Interval, source scope	Arrival age, coverage and aligned timestamps
Finance	Month, approved revisions	Reconciliation and restatement log
Engineering	Delivery/run, metric contract	Test status, query ID and lineage

Failure case. A chart titled "current Finnish electricity supply" built from our one-interval synthetic fixture invents a scope the source never promised. Replace it with "Synthetic generation by technology, one UTC interval" and publish a completeness note.

Project reference: P121: Snowflake Labs end-to-end Snowflake and Power BI analytics. *Not independently executed; rights review pending.*

Check. Which stakeholder question would justify an hourly published mart instead of record-level DirectQuery?

11.1.3 Semantic and physical design responsibilities

Physical design answers where and how data are stored, partitioned, refreshed and queried. **Semantic design** defines the user-facing meaning of fields, filter propagation, calculations and hierarchies. The two cooperate but are not interchangeable: accelerating a query does not correct an inconsistent definition of "renewable share." The gold model owns validated revisions and units; the Power BI model owns relationships, visual measures, descriptions and report interactions. The data product owner approves the combined contract.

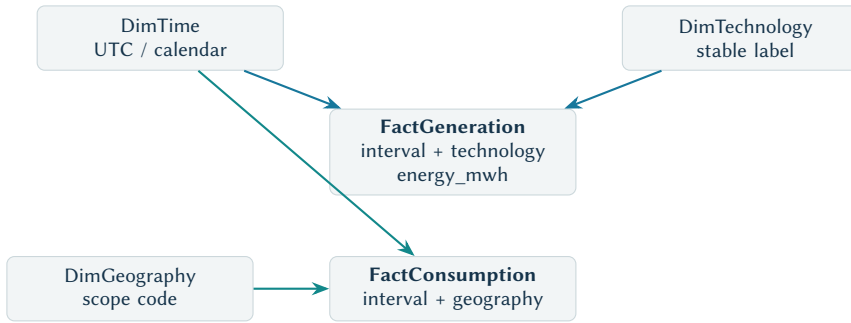


Figure 11.2. Two separately sourced fact tables share conformed time attributes; they are not directly joined.

Who owns which invariant?
Snowflake/dbt: business-key uniqueness, approved revision choice, non-negative energy, source lineage and controlled gold schema. **Semantic model:** valid relationships, measures, formatting, row-level security where required and report-level context. **Business owner:** time and geographic comparability, permitted interpretation and metric approval. **Operations:** refresh health, access review and incident response.

A model at the wrong grain is dangerous even when relationships appear valid. Joining `FactGeneration` directly to `FactConsumption` on interval alone multiplies the consumption observation once per technology and can inflate totals. Publish *separate measures* and let shared dimensions filter each fact. Validate totals under both single-technology and all-technology selections.

Design decision. Begin with a small, dimensional gold interface and an Import semantic model. Reconsider pre-aggregation or DirectQuery only after observing consumer counts, dataset size, acceptable freshness, query patterns and security obligations. Avoid copying source-derived calculated columns into multiple reports. **Check.** Why should consumption not be a column copied onto every technology row?

11.2 Defining trusted metrics and KPI contracts

11.2.1 Metric definitions, units, and aggregation grain

A *KPI contract* is an executable promise about a number. It specifies purpose, business grain, formula, numerator, denominator, unit, time basis, expected source completeness, null behaviour, owner, version and permitted aggregation. Without these fields, a report can display mathematically correct but semantically incompatible values.

Our controlled fixture reuses Chapter 10's *six accepted synthetic generation observations*, all at `2026-03-01T10:00:00Z`: Wind 14, Solar 6, Hydro 18, Nuclear 16, Bioenergy 8 and Other 4 MWh. Their sum is **66.000 MWh**. This chapter adds *one separately synthetic FI consumption observation* of **60.000 MWh** at the same UTC interval. The resulting illustrative net position is **+6.000 MWh**; the wind share of generation is **14/66 = 21.2121%**. None is a real Finnish grid statistic.

KPI	Business rule	Expected fixture
Generation	Sum accepted MWh at identical scope/interval	66.000 MWh
Consumption	Independent, aligned measured MWh	60.000 MWh
Net position	Generation − consumption if both present	+6.000 MWh
Wind share	Wind / all technologies, equal time/filter context	21.2121%
Source coverage	Observed expected technologies / six	6/6

Power is not energy.
A 480 MW instantaneous or averaged power observation contributes 120 MWh *only if* the measurement represents constant or average power over a 15-minute interval. You cannot add values in MW and label the sum MWh, and you cannot infer a daily energy total from one interval without an explicit coverage assumption.

Null is not zero. An unavailable consumption feed means the net position is *unknown*; rendering zero would create an artificial surplus. Likewise, missing technology should be marked incomplete, not silently treated as valid 0 MWh. The report must distinguish *zero observed* from *not observed*. Use explicit time alignment before subtracting independently provided measures.

Project reference: P122: Snowflake Labs dbt semantic view. *Not independently executed; rights review pending.*

Check. Why is the ratio of row-level wind shares not a meaningful daily wind-share KPI?

11.2.2 Ownership and consistency checks

Metric consistency does not emerge from agreeing on a label. Teams need a *single reviewable definition* and automated checks across layers. The source steward guarantees that timestamps and units are correct, the gold maintainer guarantees one accepted fact per business key, and the BI owner guarantees that a measure respects the intended filter context. A dashboard test should compare the same slice with a direct gold SQL aggregate and an independent oracle.

Listing 2: Fixture reconciliation query; illustrative Snowflake relation name.

```
SELECT COUNT(*) AS accepted_rows,
       SUM(energy_mwh) AS generation_mwh
FROM FINLAND_ENERGY_DEMO.GOLD.FCT_ENERGY_HOURLY
WHERE interval_utc =
  '2026-03-01T10:00:00Z'::TIMESTAMP_TZ;
-- Fixture expectation: 6 rows, 66.000 MWh.
```

The *identity of the verifier* matters. A DAX measure compared only against another DAX measure sharing the same flawed relationship is not an independent check. Use independently calculated totals, uniqueness tests and negative tests: add a duplicate generation key, omit Solar, supply a negative energy value, or remove consumption. The test should either reject the bad input or render an honest blank/incomplete state. Our companion Python oracle exercises these cases; it does not claim to run DAX.

11.2.3 KPI contract changes and review

Contract evolution is not synonymous with renaming a visual. Suppose stakeholders change "renewable generation" to include Bioenergy only when a classification criterion is met. The numerator changes, historical reports may need restatement, and dashboard comparisons become non-equivalent across releases. Approve a new contract version that records the effective date, calculation rule, old/new results, change author and downstream acceptance status.

Versioned contract card: `NET_POSITION_MWH / v0.1`
Formula: aligned generation MWh minus aligned consumption MWh. **Grain:** UTC interval and common geographic scope. **Missing data:** return blank and raise a completeness note. **Owners:** grid-source steward and analytics product owner. **Change gate:** two-person review, fixture reconciliation, DAX check, visual caption review and release note.

When the calculation changes, update `metrics_catalogue.md`, semantic model documentation, tests, downstream contracts and training examples in the same reviewed pull request. Do not silently update a production measure while leaving a dashboard title unchanged. Consider using Snowflake semantic views or dbt metric infrastructure where appropriate, but compare feature support and governance boundaries rather than assuming tools enforce business meaning automatically.

Project reference: P123: Snowflake Labs semantic-model-generator. *Not independently executed; rights review pending.*

Check. What evidence would prove that a contract change preserves comparability with the previous monthly series?

11.3 Snowflake connectors and BI integration

11.3.1 Supported Snowflake connectivity

A connector is more than a connection string: it selects a driver, an authentication route, a source role, a compute warehouse, a query mode and a refresh behaviour. The maintained *Power Query Snowflake connector* supports Import and DirectQuery for Power BI semantic models. Microsoft documents connector implementation 2.0 using an Arrow Database Connectivity (ADBC) driver, with new Power BI Desktop connections using this implementation by default; existing installations and the host version still matter. Check the actual release notes before production rather than hard-coding an assumption about a particular tenant.

Listing 3: Power Query M connection template; not run in Power BI.

```
Source = Snowflake.Databases(
  "YOUR_ACCOUNT_IDENTIFIER.snowflakecomputing.com",
  "BI_WH", [Implementation="2.0", Role="BI_READER"]
)
```

This is a *template*, not a turnkey credential. Replace the account host, select a permitted warehouse and verify the exact role and navigator paths in the installed Desktop version. The Power Query dialog normally asks for a Snowflake server and warehouse and provides a choice of storage mode. Always verify which objects Power BI is permitted to discover; prefer curated Gold views or tables instead of raw sensitive layers.

11.3.2 Credential and gateway considerations

Common enterprise authentication patterns include named credentials, approved Microsoft Entra integration and key-pair/service-principal arrangements where supported by the product and connector combination. Authentication capability varies by connector implementation and cloud deployment. Never paste credentials into a PBIX, an M file, Git history or a screenshot. Keep account secrets in organisation-approved credential facilities and rotate them by policy.

Boundary	Required review
Power BI Desktop	Connector version, allowed account and local sign-in
Power BI Service	Stored credentials/SSO, capacity and refresh settings
Snowflake	Role grants, BI warehouse, object exposure and query logs
Network/gateway	Cloud connectivity, proxy/private route, gateway only if required

Avoid a generic gateway assumption. The Snowflake cloud connection can often run without an on-premises gateway; enterprise networking, private endpoints or organisation-specific topology may change that. Test the *actual published-service connection*, not only Desktop success.

Project reference: P124: snowflakedb/snowflake-connector-net. *Not independently executed; rights review pending.*

Check. Which four values should an operator record when a published refresh fails despite working on the developer's machine?

11.3.3 Refresh, user identity, and permissions

Credential success does not prove that the right person is querying the right rows. Power BI workspace membership controls who can interact with a report; semantic model permissions and optional Power BI row-level security (RLS) govern model visibility; Snowflake roles govern the upstream objects. Their policies can diverge. In *Import*, the configured refresh identity retrieves data into the model, and downstream user filtering is primarily enforced within the published semantic model. In *DirectQuery*, report queries reach Snowflake; without supported single sign-on (SSO), a configured identity can serve many viewers. With correctly configured Microsoft Entra SSO, the end user's identity can be delegated for the supported DirectQuery path. Never imply that SSO is automatic.

Grant a reader role only USAGE on the approved database, schema and warehouse and SELECT on curated Gold objects. Keep data engineering write roles separate. Confirm the account's actual grants and ownership; do not provide a blanket privileged role to sidestep a login issue.

Listing 4: Review queries for a permitted reviewer; illustrative.

```
SELECT CURRENT_ROLE(), CURRENT_WAREHOUSE();
-- As an authorised administrator/reviewer only:
SHOW GRANTS TO ROLE BI_READER;
-- Inspect approved grants, not raw credentials.
```

Security test: two real identities, two expected views.
A source-owner account might see all approved rows while a restricted reader should see only their authorised slice. Test the actual published report with both identities, not a single developer account impersonating the other without recording evidence. When SSO is unavailable, design compensating semantic-model RLS and verify it explicitly.

For published refreshes, log at least the model name, run start/end, source readiness timestamp, status, changed row counts, failed stage and owner. DirectQuery may not need an Import refresh but still relies on up-to-date source transformations, warehouse availability and report cache behaviour. A report click may reuse a prior result; observing a visual refresh is not proof of instantaneous source propagation.

Connector migration notes document implementation-specific issues. Check distinct counts and large-model loads with the installed driver; one benchmark does not establish universal correctness or performance.

Project reference: P125: Snowflake Labs marketplace-data-app. *Not independently executed; rights review pending.*

Check. Where would you enforce regional visibility if all DirectQuery consumers share one stored Snowflake identity?

11.4 Power BI Import and DirectQuery considerations

11.4.1 Import versus DirectQuery trade-offs

Import stores a compressed copy inside the Power BI semantic model and queries that cache interactively. **DirectQuery** leaves the bulk of source data in Snowflake and sends one or more source queries as users interact. Neither means "always fast" or "always current". Import depends on refresh; DirectQuery depends on source completion, latency, security, caching and the frequency of visual queries. Microsoft generally recommends starting with Import, then justifying alternatives using requirements.

Dimension	Import	DirectQuery
Typical visual work	Power BI engine	Snowflake and connector
Source change	Visible after model refresh	Visible after subsequent query/-cache refresh
Model capacity	Memory/storage for data	Remote query and concurrency cost
Semantics	Flexible modelling	DAX / transformation restrictions
Security	Model RLS and refresh identity	Source SSO only if configured; otherwise shared identity

11.4.2 Model size, latency, and costs

For Import, evaluate row count, compressed model size, refresh duration, memory, licence limits and historical retention. For DirectQuery, evaluate concurrent viewers, per-page visuals, interaction count, SQL shape, warehouse warm-up, queueing, result caching and cost. More visuals and slicer changes can multiply source queries. No performance estimate follows from our six-row fixture.

11.4.3 Choosing a mode for energy data

If daily operational reporting tolerates periodic refresh, a small Gold aggregate in Import is a defensible baseline. If a control-room report needs a verified sub-refresh freshness requirement and cannot meet it with Import/incremental refresh, evaluate DirectQuery, a hybrid arrangement or another platform design. Beware of calling every fast-updating panel "real time": document *event-time lag*, *source-ready lag*, *model lag* and *visual display lag* separately.

Architecture decision BI-011 (illustrative)
Chosen starting mode: Import of a restricted Gold mart. **Reason:** predictable historical chart interaction and simple operational checks. **Open risks:** refresh schedule, model limits and business-approved SLA are unverified. **Revisit:** when measured data freshness, storage or concurrency breaks acceptance criteria.

Project reference: P126: mattfeks/Snowflake-PowerBI. *Not independently executed; rights review pending.*

Check. Why might a ten-second cold DirectQuery visual cost more and feel worse than an hourly Import model, even with identical Snowflake data?

11.5 Semantic modelling and refresh strategies

11.5.1 Dimensional relationships and DAX semantics

In the semantic model, define a *dimension-to-fact* filter path with stable keys, explicit single-direction relationships and intentionally named measures. `FactGeneration` relates to technology and time dimensions; a separate consumption fact relates to time and geography. Do not connect facts directly or enable bidirectional filtering simply because a total looks wrong. *DAX filter context* determines which rows a measure sees; a card, slicer and bar chart can therefore evaluate the same measure at different slices.

Listing 5: Illustrative DAX, not evaluated by the offline tests.

```
Generation MWh = SUM('FactGeneration'[energy_mwh])
Consumption MWh =
  SUM('FactConsumption'[consumption_mwh])
Net Position MWh =
  IF(ISBLANK([Generation MWh]) ||
     ISBLANK([Consumption MWh]), BLANK(),
     [Generation MWh]-[Consumption MWh])
```

A ratio such as wind share needs a clearly defined denominator. If the dashboard filters technology to Wind, `Generation MWh` returns 14 MWh; for an "overall wind share" tile, the denominator must remove *only* the technology filter while retaining compatible time/geography filters. Never sum per-technology percentages or use a cross-region denominator by accident.

11.5.2 Dataset refresh frequency

The fastest available refresh schedule is not necessarily the appropriate one. Start with the business decision deadline, source publication cycle and data delivery quality. An Import model refreshed every 15 minutes does not become current if Snowflake gold is updated only every two hours. Capacity, licence, incremental-refresh policy, service limits and gateway topology may constrain frequency; verify these in the target environment rather than promising a fixed universal limit.

11.5.3 Freshness and lineage validation

Record a chain of evidence: latest expected source event timestamp → accepted Gold publication timestamp → last successful semantic-model refresh → last visual evaluation. A green refresh indicator means a refresh ran; it does not prove that a source delivered recent or complete data. Every operational report should expose its *as-of* timestamp and data coverage status.

Missing consumption rule.
If the consumption record disappears, the correct net KPI is *blank/unknown*, not +66 MWh. The offline lab asserts this as a contract rule.

Project reference: P128: snowflake-semantic-views-powerbi. *Not independently executed; rights review pending.*

Check. Why does a DAX grand total often need reevaluation of the measure rather than summing displayed row percentages?

11.6 Building energy analytics dashboards

11.6.1 Generation and consumption KPIs

A useful energy dashboard presents *unit, period, scope and completeness* before a striking number. Our synthetic report begins with four cards: generation **66.000 MWh**, consumption **60.000 MWh**, net position **+6.000 MWh**, and source-technology coverage **6/6**. A footnote makes clear that generation and consumption are different synthetic inputs aligned for a single UTC interval. These are teaching fixtures, not observed national production or demand.

11.6.2 Meaningful visuals and filters

Use a horizontal bar chart for the six technology values: Hydro 18, Nuclear 16, Wind 14, Bioenergy 8, Solar 6 and Other 4 MWh. Zero-based bars and explicit MWh labels support comparison. Add technology, UTC time and source-scope selectors only if the underlying model actually supports their values. A trend line needs multiple timestamps; one data point is not a trend, and a synthetic extension must be labelled as such.

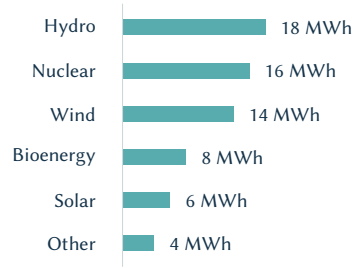


Figure 11.3. Original illustration of the controlled six-technology synthetic snapshot.

11.6.3 Interpretation cautions and data completeness

Every chart should answer "what is excluded?" The six-technology fixture does not establish complete Finnish generation coverage, imports/exports, transmission loss, or the scope of actual measurement providers. A high visual completion score for *expected fixture technologies* must not be presented as nationwide data quality. Add an *as-of* label, dataset identifier, last source success, known exclusions and a short link to definitions.

Dashboard acceptance checklist
Accurate title and units; aligned UTC time; accessible colours and readable labels; no false precision; tested slice totals; visible missing-data state; filter instructions; and documented source, owner and release version.

Project reference: P129: Customer Insights with Snowflake and Power BI (community reference). *Not independently executed; rights review pending.*

Check. Which two annotations would prevent our six-bar chart from being mistaken for a national energy-mix estimate?

11.7 Dashboard performance and query behaviour

11.7.1 Generated SQL and pushdown behaviour

Query performance begins at the consumption boundary. A Power BI visual translates an analytical request into DAX and, in DirectQuery mode, typically sends native queries to Snowflake. The database may see multiple generated SQL statements for one visual interaction. *Query folding* means eligible Power Query operations are delegated to the source rather than processed inefficiently after data retrieval. For DirectQuery tables, scalable folding is essential; for Import, folding can reduce refresh cost and transfer size.

For a six-category bar chart, a source-side grouping is natural. This SQL is *illustrative of the intended result*, not a claim about the exact SQL a specific connector build generates:

Listing 6: Expected pushdown shape for a grouped visual.

```
SELECT technology, SUM(energy_mwh) AS generation_mwh
FROM FINLAND_ENERGY_DEMO.GOLD.FCT_ENERGY_HOURLY
WHERE interval_utc =
  '2026-03-01T10:00:00Z'::TIMESTAMP_TZ
GROUP BY technology;
```

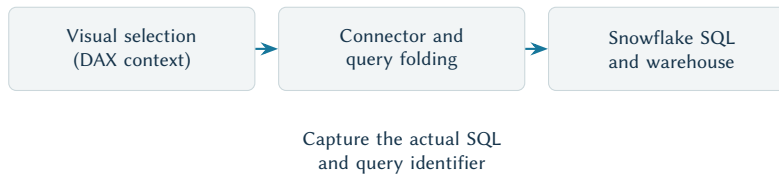


Figure 11.4. The generated query is evidence, not a guess from a visual's appearance.

A folding trap. Complex custom functions, certain native SQL patterns and transformations after a native query can prevent efficient folding. Microsoft guidance warns that DirectQuery native SQL inputs have restrictions and that incremental refresh depends on compatible folding. Prefer validated Gold views for complex source logic. Use the installed Power Query folding indicators and Performance Analyzer to inspect real behaviour.
Check. If a visual performs well with six rows but stalls with six million, which evidence distinguishes bad SQL pushdown from BI visual rendering overhead?

11.7.2 Performance troubleshooting

Do not fix a slow report by blindly increasing the Snowflake warehouse size. First reproduce a single interaction with a specific dataset, user role, filter state and connector version. Measure the time users experience, then separate Power BI visual rendering, DAX calculation, network latency, Snowflake compilation, queueing and execution. Power BI Performance Analyzer and Snowflake Query History/Query Profile provide complementary views. Each measurement needs a timestamp, relevant identifiers and a cache-state note.

Observed symptom	Possible mechanism	First evidence
First click slow, next quick	Warehouse wake-up or cache	Resume time and profile
Slicer creates long waits	Excess source round trips	Analyzer query counts
Slow despite small result	Poor pruning or big join	SQL text and profile
Peak-hour timeouts	Concurrency/queueing	Warehouse load and history
Refresh uses excess memory	Connector/model processing	Refresh log, driver version

A valid experiment holds dataset, filter set, user role and output semantics constant. Run repeated controlled trials and report medians or percentiles with sample sizes. Compare warm and cold behaviour separately; querying a cached six-row result is not a general Snowflake benchmark. Include warehouse time in any cost comparison and document what was actually billed. The companion package does *not* provide fabricated p95 latency or credit consumption.

Power BI modelling decisions affect source load: high-cardinality joins, ambiguous relationships, broad `DISTINCTCOUNT`, cross-highlighting and too many simultaneously refreshing visuals can increase query complexity. Source-side aggregation often helps, but must preserve correct filter semantics. A pre-aggregated table that drops the technology key cannot correctly answer a technology-specific question. Compare a candidate aggregate against the detailed gold model under representative filters before making it the default.

Cost optimisation that silently changes the answer is not an optimisation. The acceptance benchmark is *same users, same filter context, same valid rows, same KPI contract, comparable cache and warehouse settings.* Make correctness the first gate and latency/cost the second.

Project reference: P132: streamlit-page-analytics (related app measurement reference). *Not independently executed; rights review pending.*

Check. Why is comparing Import card response time with cold-start DirectQuery time without reporting mode a misleading benchmark?

11.7.3 Refresh reliability and user-perceived delay

A semantic model can report *refresh succeeded* while the information a human sees is stale. Separate four clocks: t_s is the event interval, t_g is the timestamp when approved Gold was ready, t_r is the last successful model refresh and t_v is when the visual was actually evaluated. An end-user freshness delay can be described as $t_v - t_s$ *only if* the displayed data correspond to event time t_s and every intervening state is known.

$$\text{Source-to-gold age} = t_g - t_s, \qquad \text{Gold-to-model delay} = t_r - t_g.$$

These are explanatory measurements, not guaranteed service-level agreements. In DirectQuery, a fresh interaction may query source data directly, but caching and source completion still matter. In Import, a model refresh defines when changes can become visible, and tiles or browser sessions can retain other cached states.



Figure 11.5. Preserve each timestamp to diagnose which boundary became stale.

An illustrative failure walk-through
At 08:00, Gold still reflects events up to 06:00. An Import refresh at 08:05 succeeds and stamps *08:05 refresh success*; the report opened at 08:10 still contains no events after 06:00. The correct operator response is to investigate source lateness, not force repeated Power BI refreshes.

Set freshness and availability targets from an approved operational requirement, and alert separately on missed upstream arrivals, failed transformations, failed model refresh, invalid KPI reconciliation and inaccessible reports. Record the incident's *first affected period*, not only the first visible error. After recovery, compare corrected output and downstream caches; do not silently backfill without a restatement note when user-facing historical values change.

Check. What two timestamps would you request first if a Power BI model refreshed at 09:00 but the dashboard still shows yesterday's energy values?

11.8 Data-sharing patterns and operational handoff

11.8.1 Governed sharing and access policies

Sharing a report is not the same as sharing unrestricted access to source data. Distribution choices include Power BI workspace/apps, governed semantic models, approved Snowflake views and Snowflake secure data-sharing capabilities. Each moves a different boundary. A Power BI report user may only need permission to view a published report, while a Snowflake data consumer may need a dedicated reader role and object grants. Source permissions, personal-data scope, regional restrictions and retention rules must be checked before choosing the channel.

For the energy case, expose a read-only Gold interface for approved analysts and distribute visual consumption through the BI workspace. If other teams need curated tables directly, use dedicated grants or an approved governed sharing mechanism. Never make a raw ingestion schema public merely because a BI connector can see it. If row-level visibility differs between departments, coordinate Snowflake row access policies, model RLS and source identity; do not rely on workspace visibility alone.

Audience	Allowed surface	Security decision
Executive reader	Published report	Viewer access and approved model RLS
BI developer	Development Gold interface	Least-privilege Snowflake role
Data engineer	Controlled source and test data	Separate write role, audit logging
External consumer	Governed sharing product	Contract, classification and approval

Data residency is not proven by placing Snowflake in one region.
Power BI tenant/capacity settings, exports, caches, user downloads, support operations and any external connections also matter. A compliance claim requires review of the complete flow and the organisation's applicable obligations, not just one region selector.

Before releasing a report, inspect every field exposed in the semantic model, including hidden technical columns that might be accessible through alternate report or export paths. Validate exports, workspace sharing and any Build permissions. Apply sensitivity labels and audit policies where applicable, but verify their enforcement in the configured tenant.

Project reference: P133: Snowflake Labs Horizon intra-organisation sharing. *Not independently executed; rights review pending.*

Check. Why can a report be correctly protected in Power BI while a shared Snowflake reader role remains over-permissive?

11.8.2 Report documentation and ownership

An analytical product needs a human maintenance story. Its README should name a product owner, a technical maintainer and a source steward, then state the metric contract version, Gold objects, refresh mode, normal freshness window, row-level permission model, dependencies and escalation route. Add a report change log, dashboard screenshots with capture dates, and an explicit note about data coverage. *No one owns it* is an architectural defect, not a documentation nicety.

Suggested report manifest (example)
Name: Finland Energy Analytics / Synthetic Demonstrator. **Source:** approved Gold generation + separately supplied synthetic consumption. **Grain:** UTC interval and technology, or interval and geography. **Mode:** proposed Import, pending tenant approval. **KPI version:** 0.1. **Owner:** analytics product owner (to be assigned). **Runbook:** section 11.8.3. **Release state:** design and offline tests only.

Every measure should carry a plain-language description, unit, aggregation restrictions, empty-data handling and the exact version of the upstream contract on which it depends. Provide a curated field list with intended slicers, relationships and hidden columns. The `semantic_model_spec.md` and `metrics_catalogue.md` companion files are examples of this handoff. They are design documents, not exports from a completed Power BI semantic model.

Lineage should be bidirectional: a reader can trace "Net Position MWh" back through DAX, two fact tables, Gold business rules, source intervals and synthetic fixtures; an operator can trace a changed source contract forward into dependent measures, reports and users. Store accountable links and version tags, not only screenshots of a graph. Avoid an opaque multi-report chain in which the final business definition depends on undocumented intermediate calculated columns.

Documentation artifact	Maintenance purpose
KPI catalogue	Business meaning, formulas, effective dates
Semantic model spec	Grain, keys, filter paths, DAX decisions
Refresh/runbook	Failure steps and authorised recovery
Release log	What changed, proof and rollback reference
Access matrix	Who sees which data, reviewed by whom

Check. Which artifact would you consult first if Finance claims the net position changed after a model release but no source delivery changed?

11.8.3 Operational acceptance and maintenance

A handoff is complete when a second operator can detect, explain and recover from a failure *without guessing the designer's intentions.* Before release, a reviewer should validate Gold row-grain uniqueness, KPI totals, missing-data behaviour, SSO/RLS boundaries, refresh schedules, user-visible freshness and dashboard accessibility. Then rehearse one failure: suspend the BI warehouse or block the test source in a disposable environment, confirm the alert, restore only approved resources and demonstrate that the downstream result is correct.

Incident signal	Evidence to collect	First safe response
Missing expected KPI	Contract, source coverage	Freeze distribution; trace lineage
Permission denied	Identity, role, workspace	Review grants; no admin bypass
Unexpected stale data	Last source/Gold/model times	Fix failing stage, then refresh
Slow report	Analyzer and Query Profile	Reproduce before resizing
Wrong value after release	Metric version, independent oracle	Rollback semantic change if approved

Rollback is a controlled data-product operation.
Reverting a DAX measure or report is different from undoing a Gold transformation. Keep source history and prior approved metric contract. Do not drop a production database, delete valid event history or claim a blanket DROP is a safe cleanup strategy. For an actual tenant, use the organisation's version-control and deployment pipeline.

Define a *stop-the-line* gate for critical incidents: broken RLS, incorrect unit, duplicated current key, leaked fields or materially wrong KPI. An attractive visual with such a defect is not fit for decision support. Conversely, a minor label adjustment may follow a lightweight change path if the owner approves and regression tests pass.

Acceptance evidence should include the precise environment, semantic model and connector versions; the approved report or PBIP/PBIX artifact; the dataset version and query/refresh IDs; redacted screenshots; tests of at least two identities; source reconciliation; and the signed handoff checklist. Our laboratory cannot supply live proof for these items and explicitly records that limitation.

Project reference: P134: Snowflake Labs internal marketplace sharing. *Not independently executed; rights review pending.*

Check. When an upstream correction changes yesterday's KPI, who authorises the restatement and what evidence tells users it happened?

11.L Guided lab, chapter review, and exercises

11.L.1 Prerequisites, inputs, environment, permissions, and cost guardrails

Scenario. Extend the Chapter 10 gold energy output into a reviewable reporting contract. The primary fixture has six synthetic generation rows totalling **66.000 MWh** at `2026-03-01T10:00:00Z`. A *separate artificial consumption fixture* contains one FI observation at the same UTC interval, **60.000 MWh**. Neither constitutes an actual Finnish energy dataset. The goal is to specify a Power BI semantic model, document a dashboard and verify the measures offline before anyone authenticates to Snowflake or Power BI.

Files. The companion lab includes generation and consumption CSVs; `metrics.py` and tests; a KPI catalogue; a semantic model specification; DAX, M and SQL templates; and a report README. The Power BI folder is a *design package*, not a finished `.pbix`.

Offline prerequisite. Python 3.10+ and a local terminal; no external dependencies, Snowflake credits or Power BI subscription. **Optional cloud stage.** A current Power BI Desktop installation, approved Snowflake account, read-only BI role and auto-suspending small warehouse. Record estimated cost, responsible approver and cleanup scope before cloud access. Do not run a chargeable environment merely to validate six rows.

11.L.2 Step-by-step procedure with selected code and expected results

1. Run the offline contract oracle and all tests. Keep its machine-readable output.
2. Inspect grain and keys; verify six unique generation rows and one aligned consumption record.
3. Review `metrics_catalogue.md` and challenge the distinction between MWh and MW, blank and zero, fixture coverage and national scope.
4. Draw the proposed dimensional relationships, write DAX measures, and review the dashboard page specification. For a real report, build it manually in Desktop and retain the resulting PBIX/PBIP plus screenshots.
5. If separately approved, point the template at a disposable Snowflake Gold environment, inspect Power Query folding and query history, then verify refresh and access with authorised identities. This step is **not** recorded as completed.

Listing 7: Zero-cloud command sequence from the companion lab.

```
cd lab
python metrics.py
python -m unittest discover -s tests -v
# All source/account credentials are optional and separate.
```

Offline fixture oracle
Generation: 66.000 MWh. **Consumption:** 60.000 MWh. **Net:** +6.000 MWh. **Wind share:** $14/66 \approx 21.2121\%$. **Accepted technologies:** 6/6. These are exact fixture calculations; they do *not* test Snowflake, DAX, Power BI rendering or refresh latency.

11.L.3 Expected result, assertions, common failure diagnosis, rollback, and cleanup

A pass must validate exact fixture totals, non-negative finite MWh, one row per generation key, aligned UTC timestamps, allowed technologies, separate consumption grain and *blank net position* when consumption is absent. The accompanying test suite also injects duplicate and negative records, unexpected technology, non-UTC time, missing technology and duplicate consumption; each must be rejected or handled explicitly. Only the Python fixture semantics have been executed; the M and DAX files are templates that require independent product-specific validation.

Symptom	Investigate first
Total $\neq$ 66 MWh	Wrong revision, duplicate key or filter context
Net equals generation	Consumption missing or blank incorrectly coerced to zero
Visual unexpectedly slow	Query mode, folding, generated SQL and cold start
Refresh green, data stale	Source-ready time versus model refresh time
Restricted user sees all	Stored credentials, RLS, SSO and Snowflake grants

Rollback and cleanup. Remove the locally generated metrics JSON to reset the offline lab. The SQL examples are read-only. For any approved cloud exercise, suspend the learner-owned warehouse and follow an authorised cleanup checklist; never modify shared production objects.

11.L.4 Foundation exercise, practitioner exercise, production-design question, and scoring rubric

Foundation (20 points): explain fact grains and compare Import with DirectQuery. **Practitioner (35):** remove consumption to prove net is unknown; add a second synthetic interval and update trend tests. **Architecture (30):** design a restricted BI connection with freshness and incident rules. **Evidence (15):** provide logs, KPI catalogue, model spec and untested-cloud limitations. *Pass:* 80/100 without critical errors.

Chapter synthesis

A trustworthy dashboard is not a successful connection. It is a governed chain: accepted facts, shared metric definitions, correct filter semantics, appropriate query mode, validated freshness, restricted identities, traceable calculations and maintained operational ownership. A single accurate total becomes decision-useful only when its meaning and limits travel with it.

Further reading. Power Query Snowflake connector; DirectQuery guidance; Snowflake Query Profile. Confirm the installed connector and current product documentation. **Next.** Chapter 12 examines source and warehouse performance with controlled workloads, rather than trusting a fast-looking report.

CHAPTER 12 PERFORMANCE / EVIDENCE / CREDITS

Query Performance and Warehouse Optimization

Make the query correct; diagnose the bottleneck; then spend deliberately

The tempting fix is to double the warehouse.
At 08:30 a Power BI report that normally opens promptly starts taking twenty seconds. The data engineer increases the Snowflake warehouse size. The next screen loads quickly, but the team cannot say whether resizing helped: the first request resumed a suspended warehouse, the second benefited from cache, and the underlying SQL unexpectedly duplicates generation rows after joining a many-to-many lookup. Credits rise while the KPI remains wrong. A faster answer is not an improved system when it answers the wrong question.

This chapter provides an investigation method for the *Finland Energy Analytics* platform. Building on Chapters 2, 4 and 11, it connects the semantics of a query with evidence from Query History, execution operators, micro-partition pruning, queuing, memory pressure and measured cost. Optimisation is presented as an experiment, not as a collection of magic switches.

Learning outcomes
You will: (1) formulate falsifiable tuning hypotheses; (2) compare the same correct result before and after a change; (3) interpret Query Profile and history fields; (4) decide whether pruning, warehouse sizing, clustering, caches, query acceleration or search optimisation addresses the observed bottleneck; (5) report latency and credits honestly; and (6) explain why a result from a six-row teaching fixture cannot predict production performance.

Prerequisites. Chapters 2, 4 and 11; SQL `JOIN`, `GROUP BY` and time filters; access to a permitted Snowflake test environment for optional cloud runs. Offline lab tests use only synthetic data. Cloud benchmarking may consume credits; agree a budget and use a restricted role.

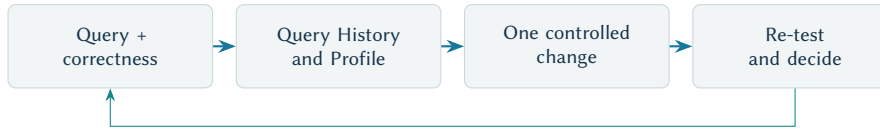


Figure 12.1. Performance work is an evidence loop, not a warehouse-size reflex.

12.1 A structured performance troubleshooting workflow

12.1.1 Hypotheses and reproducible tests

Performance work begins with a statement that can be disproved. "Snowflake is slow" is not testable. "The daily energy report scans most partitions because its timestamp predicate cannot exploit the stored range" is testable: inspect scan ratios, rewrite the filter without changing its meaning, and compare profiles over repeated runs. Record the query text or versioned file, query ID, QUERY_TAG, warehouse size, account/region, inputs, role, result shape, cache policy and competing workload.

A useful triage sequence is **scope** → **correctness** → **wait time** → **operators** → **candidate change** → **repeat**. Distinguish a slow visual from a slow Snowflake statement: report rendering, gateway/network transfer, source SQL compilation, warehouse provisioning and query execution contribute separately to perceived latency. Do not optimise a warehouse on the basis of an uncorrelated stopwatch from the browser.

Listing 1: Illustrative session controls: do not change shared warehouses.

```
ALTER SESSION SET QUERY_TAG =
  'book_ch12_energy_benchmark_v1';
ALTER SESSION SET USE_CACHED_RESULT = FALSE;
SELECT CURRENT_ROLE(), CURRENT_WAREHOUSE();
-- Restore at the end of the isolated experiment:
ALTER SESSION UNSET QUERY_TAG;
ALTER SESSION UNSET USE_CACHED_RESULT;
```

Disabling persisted query-result reuse does *not* eliminate warehouse-local data caching or other environmental variation. A warm-versus-cold experiment must record warehouse suspension/resume state and cannot assert perfect cache isolation without further controls.

Experimental contract
Question: does a half-open timestamp range reduce work compared with an expression around the filtered column? **Invariant:** identical grouping keys and MWh totals. **Evidence:** query IDs, scan/partition counts, execution and queuing components, five or more controlled repeats per treatment. **Decision:** accept only when gains are repeatable and materially exceed operational costs.

Project reference: P017: sfc-gh-bokon/query_performance_deep_dive. *Not independently executed; rights review pending.*

Check. What observation would falsify a theory that pruning is responsible for the delay?

12.1.2 Correctness before speed

An optimisation may reduce runtime while changing the answer. Consider the running case: six accepted technology observations sum to **66.000 MWh** for one synthetic UTC interval. If an engineer joins those rows to a technology alias table with two matching aliases per row, a subsequent `SUM` can silently double production to **132.000 MWh**. Neither a faster join algorithm nor more compute makes that result acceptable.

Listing 2: Confirm the gold grain before joining.

```
SELECT interval_utc, technology, COUNT(*) AS n
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
GROUP BY interval_utc, technology
HAVING COUNT(*) <> 1;
-- Expected on the provided seven-day fixture: 0 rows.
```

For each baseline and proposed rewrite, test **row count, key uniqueness, null handling, unit consistency and aggregates**. A missing period must not become a zero-valued period unless the business contract explicitly defines imputation. When the source includes revisions, benchmark the deduplicated authoritative state, not all raw deliveries. Compare all result rows (or a deterministic keyed digest) rather than only a total that could conceal offsetting errors.

Two kinds of correctness.
Semantic correctness means that generation is measured in MWh, at the intended event-time grain, for the intended geographic and source scope. *Mechanical correctness* means that the query respects uniqueness, grouping and equality of output for a given snapshot. Both must hold.

A practical review pairs each expected result with its computation: `COUNT(*)=42` and `SUM(energy_mwh)=462.000` for the companion's *seven-day expanded synthetic fixture* (six technologies repeated across seven days). These numbers are offline oracle values, not observed Finland generation. **Failure diagnosis.** If a rewritten query disagrees, stop timing tests and inspect join multiplicity, time boundaries, `NULL` behaviour and source versions. **Check.** Why is matching total MWh alone not enough to prove two queries equivalent?

12.1.3 Isolating the performance bottleneck

A single elapsed-time number conceals different problems. A statement can wait for a suspended warehouse to resume (provisioning), queue because concurrency exceeds available capacity (overload), compile complex SQL, scan unnecessary partitions, spill intermediate data or transfer a large result across the network. These demand different remedies. Use the *highest-cost observed stage*, not a guessed bottleneck, to choose the next experiment.

Evidence	Plausible cause	First test
Provisioning wait	Suspended/cold warehouse	Separate resume from steady-state
Overload wait	Concurrent queries	Repeat with controlled concurrency
Many partitions scanned	Weak filter pruning	Preserve sargable ranges
Remote spill	Large intermediate state	Inspect join/aggregation before resizing
Large returned bytes	Excess columns/rows	Return only required fields

Not every indicator is a proof. For example, some workloads legitimately scan nearly all partitions; selective pruning is not meaningful for an unfiltered whole-table aggregation. Similarly, remote spill is usually suspicious but particular services may show nonzero remote-storage metrics for reasons that require profile-level verification. Query compilation can vary independently of scan efficiency.

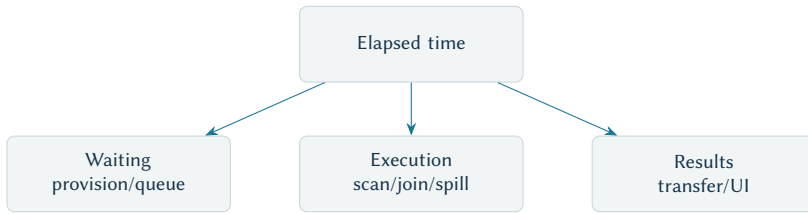


Figure 12.2. One elapsed time may hide three different interventions.

Decision rule. Rewrite an inefficient expression before buying a larger warehouse. Investigate a concurrency bottleneck before changing clustering. Analyse an oversized BI result before enabling a paid acceleration feature. **Check.** Which metric distinguishes queued overload from a long-running hash aggregation?

12.2 Query History and Query Profile

12.2.1 Query History metadata

Snowflake provides recent query details in Snowsight and Information Schema table functions, while SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY supports broader historical analysis subject to privileges and reporting latency. Useful fields include query ID and tag, warehouse name/size, status, total elapsed/compilation/execution times, provisioning and overload waits, scanned bytes, scanned/total partitions, result-cache percentages and spill metrics. Field availability and interpretation should be rechecked at publication time.

Listing 3: Recent per-user history: permitted users only.

```
SELECT query_id, query_tag, warehouse_name,
       total_elapsed_time, execution_time,
       queued_overload_time, bytes_scanned
FROM TABLE(INFORMATION_SCHEMA.QUERY_HISTORY(
  DATEADD('HOUR', -1, CURRENT_TIMESTAMP()),
  CURRENT_TIMESTAMP(), 100))
WHERE query_tag = 'book_ch12_energy_benchmark_v1'
ORDER BY start_time DESC;
```

Account Usage can lag behind the most recent execution. For an immediate diagnosis, inspect Snowsight and the appropriate recent-history function; for a retrospective account-wide trend, use a granted monitoring role. Access to observability data is *not* a reason to grant ACCOUNTADMIN to a lab user.

12.2.2 Query Profile operators and scans

Open the saved query ID in Query Profile. Read its execution graph from scans through joins and aggregates to the final result. Compare *actual scanned partitions*, not the verbal presence of a filter. An explicit filter may still scan most table storage when values overlap broadly between micro-partitions; a low scan fraction may already indicate good pruning. Observe rows entering and leaving nodes: a join expanding six rows into twelve can be the root cause of an incorrect total.

Project reference: P018: Snowpark Python performance tips. *Not independently executed; rights review pending.*

Check. Why must the time at which Query History becomes visible be recorded separately from the execution time?

12.2.3 Reading spilled data and queuing indicators

Spill occurs when an operation cannot retain all required intermediate state in available memory. Storage spill to local disk is usually less damaging than remote spill. Sorting, large joins, distinct aggregations and skewed groups are common places to investigate. A larger warehouse may supply more memory, but fixing a cross join or reducing the intermediate working set can be both faster and cheaper. Check the operator-level profile and compare row expansion before resizing.

Queuing is not execution. A query can run in a few hundred milliseconds once scheduled yet wait several seconds behind other requests. Record provisioning wait and overload wait separately from execution; a multi-cluster warehouse primarily addresses concurrent workload throughput where supported, not necessarily the runtime of one serial query. Do not conclude that a clustering key cures queues.

Listing 4: Historical diagnostic fields; requires monitoring privileges.

```
SELECT query_id, warehouse_name,
       total_elapsed_time / 1000.0 AS elapsed_s,
       queued_overload_time / 1000.0 AS overload_s,
       bytes_spilled_to_local_storage AS spill_local,
       bytes_spilled_to_remote_storage AS spill_remote,
       partitions_scanned, partitions_total
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE query_tag = 'book_ch12_energy_benchmark_v1'
ORDER BY start_time DESC LIMIT 25;
```

Reading a profile without false certainty
Observed: heavy remote spill and large rows entering an aggregation. **Hypothesis:** the intermediate result is too large for the available memory. **Next test:** remove an unintended duplicate join and compare spill and output equality. **Alternative:** if the work is intrinsically memory intensive, compare warehouse sizes under the same workload.

The companion includes an *illustrative metrics CSV* solely to exercise reporting logic. Its invented numbers must never be presented as live Query History measurements or as an empirically proven speedup.

Project reference: P135: Query Performance Streamlit in Snowflake Notebooks. *Not independently executed; rights review pending.*

Check. What evidence would make increasing memory a better option than rewriting the join?

12.3 Micro-partition pruning and clustering

12.3.1 Micro-partition elimination

Snowflake automatically organises table data into micro-partitions and records metadata that the optimiser may use to avoid irrelevant partitions. Pruning works well when a predicate reduces the required value range and stored partition ranges support that exclusion. A filter on `interval_utc` is a candidate when the fact is roughly ordered by time; a filter on an unrelated low-selectivity dimension may scan similar data to the unfiltered query.

Listing 5: Same intended UTC interval; inspect plans, not timings alone.

```
-- Expression around column: may impede pruning.
SELECT SUM(energy_mwh)
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
WHERE TO_DATE(interval_utc) = '2026-10-04';

-- Explicit half-open range preserves boundary intent.
SELECT SUM(energy_mwh)
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
WHERE interval_utc >= '2026-10-04'::TIMESTAMP_NTZ
  AND interval_utc <  '2026-10-05'::TIMESTAMP_NTZ;
```

Both statements intentionally use a UTC-encoded `TIMESTAMP_NTZ` fixture; in a real pipeline verify the source timestamp type and time-zone contract. The rewrite should return the same six records and **66.000 MWh** on the sample for 4 October. Actual Snowflake optimisation is workload-dependent; an apparent improvement in a seven-day toy table is not convincing evidence.

12.3.2 Clustering key trade-offs

Explicit clustering is generally for sufficiently large tables with repetitive selective filters and demonstrably poor natural clustering. Keys can aid pruning, but maintenance consumes compute and storage; high-cardinality timestamps may be more expensive to maintain than a date-level expression. Avoid clustering every small table, and do not apply a key to our tiny teaching fixture.

Decision before DDL.
Only propose `ALTER TABLE ... CLUSTER BY` after baseline profile evidence, an account-level cost estimate, representative query frequency and written approval. New clustering maintenance costs must be counted alongside per-query gains.

Check. Why can a timestamp predicate be selective at the row level yet show weak partition elimination?

12.3.3 Measuring pruning and maintenance overhead

A useful pruning ratio is $P =$ partitions scanned/partitions total for the *relevant scan*, with the caveat that query-wide totals can aggregate several table scans. Compare the same fact scan's numerator and denominator before and after a controlled change. A decrease from 900/1000 to 120/1000 would be evidence of improved pruning; it is *illustrative*, not a reading from this book's account.

Snowflake's system functions for clustering information can help examine table clustering characteristics where applicable. Such information must be interpreted in the context of selective query shapes and table size. Better clustering depth is not synonymous with lower total spend: frequent ingestion or updates may increase background maintenance credits. Evaluate net savings over the expected workload window, not just the fastest cherry-picked SQL statement.

Scenario	Likely action	Evidence needed
Small seven-day fixture	No clustering change	Correctness and pedagogical trace
Large, time-ordered fact	Test range filter first	Low pruning ratio already?
Large, scrambled, repeated scans	Consider cluster key	Measured savings vs. upkeep
Frequent append/corrections	Count maintenance cost	Volume, DML pattern and spend

Listing 6: Read-only partition history fields (approved role).

```
SELECT query_id, partitions_scanned,
       partitions_total, bytes_scanned
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE query_tag = 'book_ch12_energy_benchmark_v1'
  AND partitions_total IS NOT NULL
ORDER BY start_time DESC;
```

Failure case. An analyst adds clustering, reruns exactly one query, and attributes a short runtime to clustering without checking result cache, warehouse start-up or data snapshot. That is a before/after anecdote, not a reliable experiment.

Project reference: P136: clustering and materialised view comparison. *Not independently executed; rights review pending.*

Check. What makes a maintenance-heavy cluster key economically irrational even if it reduces p95 latency?

12.4 Warehouse sizing and concurrency management

12.4.1 Warehouse size versus throughput

Resizing a warehouse changes the compute and memory available for supported workloads, but not every query scales proportionally. Highly parallel scans, large joins and spill-heavy transformations can benefit; compile-bound statements, tiny queries and serial bottlenecks may not. Compare the same source snapshot, query mix and controlled session conditions under distinct warehouse sizes. Record the minimum billing behavior, auto-suspend settings and exact account pricing rather than inserting universal euro amounts.

The *engineering objective* is a service-level target at a defensible cost, not the shortest possible runtime. A twice-as-large warehouse that halves execution time may have similar theoretical execution credits, but real invoices depend on billing windows, idling, concurrency and auxiliary service charges. In addition, warehouse size is not an intrinsic guarantee of query correctness.

12.4.2 Multi-cluster and queueing

When many Power BI interactions arrive together, a warehouse may queue queries even though each individual query is efficient. Multi-cluster configurations, where the feature is available for the customer's edition, can add capacity for parallel demand. This is a concurrency strategy rather than a way to make one query automatically run faster; sizing the base cluster and optimising generated SQL remain relevant.



Figure 12.3. Scaling concurrent admission is different from shortening one scan.

Failure case. A BI dashboard uses seven visuals, each generating SQL. Fifty active users can create a much more demanding query rate than fifty visual pages alone imply. Measure realistic interaction sequences and cache behaviour. **Check.** Why might an extra cluster reduce queue wait without changing the profile of an individual aggregation?

12.4.3 Cost-normalised capacity planning

A good capacity study includes both *latency distribution* and *credits over the observation window*. For a workload with N completed requests, record throughput as N/T and compute spend as C/N credits per successful request. If requests have different complexity, report each family separately; otherwise a changing mixture can falsely suggest a platform regression.

Snowflake's warehouse metering is not identical to per-query cost attribution. Credits billed during idle time, warehouse provisioning, shared concurrency and optional services complicate simple allocation. If you use query-level attributed compute, state exactly which Account Usage view was queried, its data latency and which cost classes are excluded. Do not infer full invoice totals from elapsed seconds alone.

Example (fully hypothetical, not measured)
At the same workload mix, configuration A serves 100 successful requests in ten minutes at 0.20 credits for the measured interval, while B serves 160 at 0.36 credits. A: 0.002 credits/request; B: 0.00225. B offers higher throughput at a higher average credit cost per request. The choice depends on p95 target and capacity need, not a simplistic winner label.

A disciplined load test increases concurrency in stages, randomises query order, keeps a consistent data snapshot and monitors queueing as well as execution. Do not conduct a load test on a shared production warehouse without operational approval. Add auto-suspend and a spending cap in a dedicated experiment plan.

Metric	What it says	What it cannot prove
p95 wall time	User-facing tail	Which phase is slow
Execution time	Snowflake work duration	External report/render time
Queued overload	Capacity contention	That query SQL is inefficient
Credits/request	Normalised spend	Full cost without idle/services

Project reference: P137: warehouse and SQL tuning reference. *Not independently executed; rights review pending.*

Check. What workload characteristic must stay stable when comparing credits per query?

12.5 Caching behavior and materialized results

12.5.1 Result cache and warehouse-local cache

Snowflake can reuse a persisted query result when eligibility conditions are satisfied. The warehouse also has a local data cache that may accelerate reads after earlier scans. These are different mechanisms with different conditions, and neither is equivalent to a precomputed business mart. A query returning an already cached result can appear exceptionally fast while revealing almost nothing about the original scan's cost.

12.5.2 Controlled warm/cold experiments

For a baseline scan study, explicitly record whether USE_CACHED_RESULT is enabled and repeat measurements with the same policy. Even when persisted-result reuse is disabled, warehouse-local cache, data state and concurrency may vary. A "cold run" after resume and a "warm run" should be labelled operational scenarios, not interchangeable treatments. Alternate baseline and candidate query order to limit drift and do not throw away an inconvenient slow run without a documented reason.

Listing 7: An isolated, reversible experimental session.

```
ALTER SESSION SET USE_CACHED_RESULT=FALSE;
SELECT technology, SUM(energy_mwh)
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
GROUP BY technology;
ALTER SESSION UNSET USE_CACHED_RESULT;
```

12.5.3 Materialization versus recomputation

A materialised result is a design choice: precompute when the same expensive transformation is repeatedly requested, and when refresh semantics and maintenance spending are justified. Options include a scheduled Gold table, a materialized view where SQL/function restrictions allow, or a Dynamic Table with supported refresh behavior. Unlike a cached query result, these objects may incur ongoing refresh costs and operational ownership obligations.

> **Do not cache a semantic error.**
> Preaggregating twelve raw revisions into a fast BI mart can make a wrong number arrive sooner. Materialization must occur after the revision-resolution and grain checks established in earlier chapters.

The six-row one-interval fixture is intentionally too small to motivate materialized infrastructure by runtime alone. Its purpose is to teach a correct baseline and illustrate reproducible control variables.

Project reference: P013: pandas on Snowflake teaching reference. *Not independently executed; rights review pending.*

Check. How would you distinguish an improved cached-result hit rate from a more selective scan?

12.6 Query acceleration and search optimization concepts

12.6.1 Query acceleration options

The Query Acceleration Service can provide additional compute for eligible portions of queries under supported configurations. It is not a generic speed switch for every SQL statement: eligibility, estimated benefit and consumption must be investigated. If it is activated, record service-specific credits separately from virtual-warehouse spend. A query with no qualifying work may see little improvement. The account's current feature availability, privileges and pricing must be checked before making claims in a production design.

12.6.2 Search optimization and suitable predicates

Search Optimization Service targets particular selective access patterns, including applicable point lookup and other documented predicates on large datasets. It maintains a search access path, which introduces maintenance and storage-related costs. It is inappropriate to enable solely because a report feels slow; first verify a suitable predicate, cardinality, data size and measured profile. For the small energy teaching fixture, ordinary range filtering is the correct starting point.

12.6.3 Edition, cost, and support checks

Feature support can depend on edition, object type, SQL shape, region and product evolution. A design document must state prerequisites, estimated recurring cost, pilot success metric, rollback step and service ownership. When availability is unknown, label the option a *candidate for evaluation*, not an active control.

Mechanism	Workload clue	Trade-off
Query acceleration	Eligible costly query portions	Additional service credits
Search optimisation	Selective point-like access	Access-path maintenance
Clustering	Repeated range filtering	Background reclustering cost
Larger warehouse	Memory-bound heavy compute	Higher running rate

Approval gate.
Never enable a paid optimisation feature in the companion lab without explicit approval, a bounded pilot window and a method for verifying *both* query results and metered credits. The printed code only describes read-only diagnostic queries.

Project reference: P138: alternate query-performance diagnostic reference. *Not independently executed; rights review pending.*

Check. Why should an optimisation feature's maintenance or service credits enter the final benchmark even if the query latency improves?

12.7 Improving joins, filters, and aggregations

12.7.1 Join cardinality and filter pushdown

The fastest fix is sometimes to remove work that never should have existed. Confirm the relationship and grain of every join. A many-to-many lookup can multiply energy rows and misstate totals; an accidental Cartesian join can create very large intermediates and spill. Push selective constraints toward inputs where doing so preserves semantics, and inspect operator cardinalities in Query Profile rather than assuming the optimiser behaves like handwritten execution order.

12.7.2 Aggregation and column selection

Return only columns a consumer needs; avoid `SELECT *` on wide facts when a report uses four measures. Aggregate at the approved grain rather than repeating a costly row-level join for every visual. However, an early aggregation is not safe if it discards keys needed for correct joins, changes distinctness or merges time zones incorrectly. *Performance and semantics are coupled.*

12.7.3 Rewriting without changing semantics

Listing 8: A safe baseline aggregation for the sample fixture.

```
SELECT technology, SUM(energy_mwh) AS mwh
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
WHERE interval_utc >= '2026-10-01'::TIMESTAMP_NTZ
  AND interval_utc <  '2026-10-08'::TIMESTAMP_NTZ
GROUP BY technology ORDER BY technology;
```

For the seven-day fixture, expect each technology's single-day energy multiplied by seven: Bioenergy 56, Hydro 126, Nuclear 112, Other 28, Solar 42, Wind 98, total **462.000 MWh**. The example is deliberately arithmetic, not a query-speed benchmark. Test key uniqueness after any additional dimension joins.

Rewrite review: change one idea.
Do not simultaneously change warehouse size, clustering, SQL predicates and table layout. First establish exact output equivalence; then compare the targeted physical operator. If the profile does not move, reject the hypothesis and select a new one.

Project reference: P139: Snowflake CTE Analyzer code utility. *Not independently executed; rights review pending.*

Check. Why can removing a duplicate join improve both latency and data quality?

12.8 Benchmarking performance, credits, and latency

12.8.1 Workload definition and dataset snapshots

A reproducible benchmark is a *workload contract*. Specify the dataset build and immutable snapshot, schemas and row counts, query families, parameters, selectivity, arrival distributions and concurrency. Pin the warehouse and session settings, compiler-relevant SQL, benchmark window, permissions and product version where visible. A script that runs a single small SELECT five times is a useful teaching exercise but not a production workload assessment.

Our companion includes three query families: **daily interval selection**, **per-technology total** and **fact-to-dimension lookup**. The fixture starts with the six accepted observations from prior chapters and expands them deterministically to seven dates: **42 rows / 462.000 MWh**. The input data are synthetic and extremely small; Snowflake performance on them will be dominated by overhead, caching and environment effects. Any benchmark report must say so near its headline.

Recorded field	Purpose	Sample value
Input release	Same rows in every variant	synthetic-v1
SQL version	Comparable query semantics	baseline / candidate
Warehouse	Controlled compute	approved test WH
Cache flag	Explain reuse	persisted result off
Concurrency	Reproducible load	single session
Source evidence	Trace actual run	query ID and timestamp

A production study must include representative volumes and customer-specific privacy constraints. Generating millions of synthetic rows may itself consume substantial credits; plan dataset build and cleanup separately from measurements.

Project reference: P140: cross-platform benchmark method reference. *Not independently executed; rights review pending.*

Check. Which omitted benchmark variable would make a warehouse-size comparison non-repeatable?

12.8.2 Median, p95, credits, and repeat counts

Mean runtime can hide tail latency. With a sample of at least several runs per variant, report the *median*, *p95*, minimum/maximum and sample count. State the percentile method because small samples can give unstable p95 estimates. Pair wall-clock latency with query execution and wait times; a slow BI page may not correspond to a slow query. Collect enough runs to reveal noise, alternate treatments and report uncertainty rather than choosing the best run.

A speed ratio is $S = \text{median}(t_{\text{baseline}})/\text{median}(t_{\text{candidate}})$, provided both are evaluated on equivalent inputs and correctness is established. A cost ratio is computed over identical workloads and time windows, *including* idle/overhead and any enabled paid services. An apparent improvement in one query with a maintenance-heavy clustering feature may increase total account spend.

Listing 9: Read-only history of approved tagged runs.

```
SELECT query_id, query_tag,
       total_elapsed_time / 1000.0 AS elapsed_s,
       execution_time / 1000.0 AS exec_s,
       queued_overload_time / 1000.0 AS queue_s
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE query_tag='book_ch12_energy_benchmark_v1'
ORDER BY start_time;
```

Illustrative numbers, not a Snowflake run.
A fabricated six-run series might yield a baseline median of roughly 1.9 s and a candidate near 1.2 s, but those values establish *nothing* about this platform. The companion flags every such CSV row as `synthetic_illustration`. Genuine results require captured query IDs, Account Usage/Snowsight evidence and billing observations.

Per-query attribution and warehouse metering answer different questions. The former may exclude idle warehouse spend; the latter must be distributed carefully under concurrent use. Label each metric's source, units, reporting lag and excluded cost classes.

Project reference: P117: Snowflake Labs data-eng-bench. *Not independently executed; rights review pending.*

Check. How can a faster median coexist with a worse p95, and which number matters for an on-call dashboard SLA?

12.8.3 Limitations and benchmark report

A defensible report separates *measurement*, *interpretation* and *decision*. A measurement states the environment, version, observed numbers and query IDs. An interpretation states the most likely mechanism and alternative explanations. A decision says whether to deploy, revert or collect more data, and names the owner and rollback path.

In the accompanying report template, explicitly distinguish the reproducible **offline correctness oracle** from the **fabricated performance demonstration CSV** and **real Snowflake metrics still to be collected**. No numerical speedup, p95 or credit saving for Snowflake is claimed by this chapter. If the pilot is ever run, add actual query IDs and a timestamp before replacing the placeholders.

Gate	Required evidence	Fail response
Correctness	Same keyed result set	Stop experiment
Repeatability	Run count; stable snapshot	Collect more runs
Diagnosis	Scan/queue/spill change	Reject hypothesis
Economics	Credits and auxiliary charges	Estimate TCO again
Deployability	Owner, permission, rollback	Do not enable feature

Common bias. Selecting only queries that benefit a proposed warehouse upgrade creates an invalid cost argument. Include ordinary report queries, expensive peak-time queries and repeated user interactions. If the dataset is small, state that warehouse provisioning and fixed overhead dominate; extrapolation to hundreds of millions of records would be misleading.

The most useful outcome may be "do not optimise".
If p95 already meets the requirement, the warehouse is inexpensive, and the query is correct, a new maintenance-intensive feature may decrease reliability for negligible benefit. Recording a rational no-change decision is successful performance engineering.

Project reference: P141: Snowflake FinOps optimisation example. *Not independently executed; rights review pending.*

Check. What two facts must be established before claiming a percent reduction in Snowflake credit spending?

12.L Guided lab, chapter review, and exercises

12.L.1 Prerequisites, inputs, environment, permissions, and cost guardrails

Lab mission: compare two semantically equivalent energy-report SQL patterns, produce a correctness certificate and prepare a measurement plan. The companion includes `benchmark_queries.sql`, `energy_sample.csv`, `benchmark_results.csv`, `offline_validate.py` and `tuning_report.md`. The results CSV is **synthetic illustration only**: it was not extracted from any warehouse.

Offline route (no cost): Python 3 standard library; run the offline validator and the Python unit-test command listed in the companion README. **Cloud option (billable):** only with an approved isolated database and dedicated small test warehouse, least-privilege role, auto-suspend, verified budget and explicit cleanup plan. Never run optional DDL on a shared/production account.

12.L.2 Step-by-step procedure and selected code

Step 1. Validate six source energy values, their uniqueness and **66.000 MWh** single-day sum. **Step 2.** Expand the same six values across seven UTC dates for a **42-row / 462.000 MWh** reproducible exercise. **Step 3.** Examine two query shapes: a function-based date predicate and a half-open range; confirm their resulting aggregates agree for every requested day. **Step 4.** If authorised, capture multiple tagged Query History IDs and inspect Query Profile. **Step 5.** Record medians and p95 only from actual measurements, not from the demonstration CSV.

Listing 10: Sample range query from the companion: 4 October only.

```
SELECT technology, SUM(energy_mwh) AS mwh
FROM BOOK_DEV.CH12_LAB.ENERGY_FACT
WHERE interval_utc >= '2026-10-04'::TIMESTAMP_NTZ
  AND interval_utc <  '2026-10-05'::TIMESTAMP_NTZ
GROUP BY technology ORDER BY technology;
```

Expected aggregate for 4 October: **66.000 MWh**. *Performance expectation: none.* The fixture has no meaningful partition-scale power to demonstrate a speed improvement; query timing is environmental.

Output folder
Submit an oracle test log, versioned benchmark SQL, environment/privilege declaration, result rows and Query IDs (if actually measured), a comparison table and an ADR stating why a change is accepted, rejected or postponed.

12.L.3 Expected results, assertions, failure diagnosis, and cleanup

Expected *offline* invariants: six distinct technologies per date; exactly seven dates and 42 accepted fact rows; daily generation 66.000 MWh; total 462.000 MWh; both date-filter formulations have the same keyed output; no negative energy; and the benchmark demonstration file is explicitly marked as synthetic. Failures can reveal mistaken inclusive end boundaries, duplicated joins, unstable fixture order or incorrectly treated null values. Repair the correctness model before comparing speed.

Cleanup. Reset QUERY_TAG and the result-cache session override. With authorised ownership only, remove lab-specific tables and suspend the isolated warehouse after capturing evidence. The companion does not automatically drop objects. Do not remove data or grants belonging to other work.

12.L.4 Foundation exercise, practitioner exercise, and production-design decision

1. **Foundation (20 points).** Explain persisted result cache versus warehouse-local cache and give two sources of apparent timing improvement that are *not* SQL optimisation.
2. **Practitioner (30 points).** Design a seven-day query-equivalence test, collect sufficient tagged runs on an authorised Snowflake lab (or write *not measured*), and record median, p95, partitions scanned and wait/spill indicators.
3. **Architectural decision (30 points).** A Power BI dashboard slows at peak usage. Compare query rewrite, warehouse resizing, multi-cluster, clustering and a serving mart; specify the evidence necessary to reject each option.
4. **Evidence and safety (20 points).** State account permissions, budget guardrails, data provenance, rollback, time window, reviewer and unresolved limitations.

Chapter review. (1) Why must query correctness precede tuning? (2) What distinguishes provisioning from overload wait? (3) When can clustering be cost-negative? (4) Why is one benchmark run insufficient? (5) Which credits are missing from per-query execution attribution?

Chapter synthesis

A credible tuning claim has three legs: *an equal correct answer, a repeatable explanation of the bottleneck, and a measured overall cost.* Remove one leg and the conclusion becomes guesswork. Chapter 13 extends this discipline to governance, reliability, access and cost ownership.

Authoritative technical references (reviewed 22 Oct 2026). Snowflake Documentation: Query History; Query Profile / Snowsight Activity; Clustering Keys; Query Acceleration Service; Search Optimization Service; Warehouse considerations; Account Usage Query Attribution. Query History; Query Profile; Clustering; Acceleration. GitHub repositories cited above are external teaching references, *not* executed or licensed for republication in this chapter.

CHAPTER 13 TRUST / RECOVERY / UNIT ECONOMICS

Security, Governance, Reliability, and Cost

When a data platform works technically, who ensures it is safe to trust?

The dashboard was correct, but the delivery was not.
A Finnish energy analyst opens a carefully reconciled generation dashboard: six accepted observations, 66.000 MWh in the synthetic teaching fixture. An hourly service identity powers the refresh. It also happens to have the power to delete the raw feed, read an unrelated customer directory, and create an unrestricted warehouse. Nobody noticed until a mistaken deployment revoked the report's access and an unattended workload kept running after the analyst went home. Correct numbers alone did not make this a reliable data product.

The architecture built through Chapters 3–12 now has a second contract: protect what the product knows, account for what it spends, and recover from what will go wrong. Security is an engineering property of roles, boundaries and tested denial. Governance identifies who owns meaning and decisions. Reliability translates incidents into rehearsed action. Cost management gives those decisions a financial constraint. None can be outsourced to a single configuration switch.

Learning outcomes
After this chapter, you can (1) draw a threat boundary and least-privilege role graph; (2) distinguish masking and row-access policies from access grants; (3) design identity, tagging, auditing and privacy controls; (4) attribute warehouse and serverless usage without inventing prices; (5) explain the different coverage of monitors and budgets; (6) develop an incident runbook with measurable recovery objectives; and (7) execute negative-access and cost-ledger checks in a safe lab.

Prerequisites. Chapters 3, 5, 10 and 12, and a basic understanding of schemas, roles and query history. Exercises are designed around *synthetic* energy data. Live Snowflake security policies may require Enterprise Edition, administrator approval, and billable resources; do not deploy these examples to shared or production environments without review.



Figure 13.1. Trust requires identity, data, expense and recovery controls together.

13.1 Security architecture and least privilege

13.1.1 Threat boundaries and trust assumptions

Consider five actors around the energy platform: an ingestion service, a transformation runner, a dashboard reader, a platform operator and an external data recipient. Their credentials, networks, data domains and actions differ. Draw boundaries between the identity provider, client device, Snowflake account, storage or external integrations, reporting workspace and downstream consumers. At every boundary ask: *Who can impersonate whom, see which records, trigger what execution, or obtain a secret?*

A threat model is not an assertion that the cloud platform is insecure. It makes assumptions explicit. In our exercise we assume the identity provider controls human SSO; privileged Snowflake administration is separated from routine querying; source files are untrusted until validated; and application operators cannot silently change security policy. Identify adverse events: exposed PAT or private key, broad grant to a role, malicious SQL in an unsafe client, incorrect masking context, unrestricted outgoing integration, and accidental sharing of a raw table.

Threat boundary worksheet
Asset: energy readings and contact metadata. **Actor:** scheduled loader. **Permitted:** insert into a designated landing table and use a small warehouse. **Forbidden:** create users, read contact details, grant roles and export to an unrelated destination. **Control:** scoped role plus protected secret. **Evidence:** denied operations, access events and grant-diff review.

13.1.2 Privilege minimization begins at the task

Start from a job statement and derive precisely the privileges it requires. A Power BI consumer normally needs usage on its warehouse and parent containers plus select on approved serving objects, not ownership of source tables. An ingest worker may need insert or a scoped load operation, not unrestricted access to every schema. Privilege reduction is also temporal: a migration or incident role can be activated for an approved window rather than granted permanently.

Common failure. A service identity has `ACCOUNTADMIN` because setup was easier. This combines credentials, powerful access and continuous machine execution into a large blast radius. Replace it with purpose-built account or database roles, limit secrets to the runtime that needs them, and require an independently reviewed escalation path.

Project reference: P142: Snowflake Horizon governance guide. *Not independently executed; rights review pending.*

Check. Can each permission in the architecture be traced to a business operation and an owner?

13.1.3 Separation of duties and audits

Separation of duties breaks the chain from authoring a change to silently authorizing, deploying and hiding it. A data engineer can propose a new masked field; a governance steward approves the classification; a security operator approves the grant and policy; a release pipeline deploys an auditable artifact. Small teams may combine roles, but should still require a separate reviewer for high-impact changes and retain the decision record. The key distinction is between accountability and technical privilege: a person might own an outcome without holding unrestricted database rights.

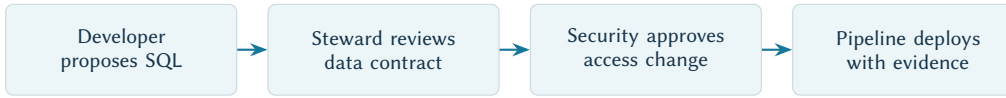


Figure 13.2. A reviewed change creates evidence before it reaches consumers.

A good audit record has the change ID, requester, approver, affected account and objects, before/after grants, linked SQL commit, test evidence, deployment identity and rollback plan. Logs cannot substitute for prevention: detecting a data export after the data has left is too late. Conversely, access restriction without evidence can conceal a broken data product for days. Put both in the control design.

Listing 1: Read-only inspection; use a dedicated approved review role.

```
SHOW GRANTS TO ROLE BOOK13_READER;
SHOW GRANTS ON TABLE BOOK13_SANDBOX.RAW.METER_EVENT;
-- Change documentation is external to these results:
-- approval ID; captured timestamp; release commit.
```

The current role and any active secondary roles affect authorization; policy tests must record session context. A successful SELECT by an administrator does not show that a reader role is safe. A denied SELECT from a restricted role is valuable evidence, provided the test actually ran under that role rather than inheriting broader permissions.

Decision rule. For every control ask both questions: “What will this prevent?” and “How will we detect a regression?” A defensible answer names a scoped object, an owner, an automated check and a human review cadence.

Audit closure: evidence, not screenshots alone

Treat an approval, a negative-access query, and a query-history record as three different kinds of evidence. Approval proves authorization to change; an access test proves an expected behavior under a stated session; a history record helps reconstruct what was run. None independently proves continuing compliance. Repeat policy assertions after role migrations, identity provider changes and upgrades to reporting integrations.

Choose tests that fail when someone broadens access accidentally. For example, a dashboard-reader role must not be able to select `RAW.METER_EVENT`, insert into `GOLD.GENERATION_DAILY` or assume the ingestion role. Verify the corresponding positive behavior as well: that the reader *can* query an approved, masked serving view. A permission system that denies everything is not a functioning product.

> **Regression case: new inherited privilege**
> A reviewer grants `BOOK13_READER` a parent role to simplify deployments. The new hierarchy unexpectedly inherits `SELECT` on raw contact records. A grant-diff check flags the unexpected edge, while an authorized-session probe demonstrates the new access. Roll back the role edge, not just the user assignment, and rerun the negative-access suite.

What to capture: timestamp, role graph snapshot, policy DDL, object inventory, permitted and denied query IDs, incident ticket, evaluator and limitations. An offline model of the role graph is useful for reasoning but does not replace an actual Snowflake permission test. Where audit views have latency or require a licensed feature, state that limitation rather than treating missing rows as evidence of safety.

Trade-off. More tightly scoped roles increase administration overhead; broad roles lower initial friction but enlarge impact and undermine explainability. Prefer reusable access roles combined into functional roles and automate their review. Chapter 10's release checklist should require a security diff just as it requires data-quality and schema compatibility tests. **Check.** Could a teammate reproduce an access-denied assertion from an archived release record, without borrowing an administrator session?

13.2 Role-based access control and role hierarchies

13.2.1 Role hierarchy design

Snowflake uses grants to roles and role inheritance. Model the *minimum* object privileges first, then compose them into functional roles. For the energy case, `BOOK13_RAW_WRITER` writes an isolated raw table, `BOOK13_GOLD_READER` reads serving tables, and `BOOK13_ANALYST` inherits only the latter. Database roles can scope object grants inside a database; account roles hold warehouse privileges and are active in sessions. Account roles may inherit database roles, but database roles cannot inherit account roles.

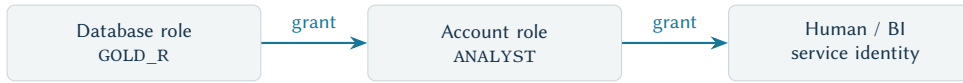


Figure 13.3. Grant direction matters: parent roles inherit child privileges.

13.2.2 Object ownership and grants

`OWNERSHIP` is powerful: it controls an object and influences who can grant access. In a managed access schema, grants on contained objects are centrally managed by the schema owner or a role with `MANAGE GRANTS`; object owners do not grant access themselves. Choose managed access for governed serving zones when centralized approval matters, but document the resulting administration requirements. Avoid automatic future grants until the whole object class has a consistent sensitivity profile.

Listing 2: Illustrative administrator-approved setup, isolated sandbox.

```
CREATE ROLE IF NOT EXISTS BOOK13_ANALYST;
GRANT USAGE ON DATABASE BOOK13_SANDBOX
  TO ROLE BOOK13_ANALYST;
GRANT USAGE ON SCHEMA BOOK13_SANDBOX.GOLD
  TO ROLE BOOK13_ANALYST;
GRANT SELECT ON TABLE
  BOOK13_SANDBOX.GOLD.GENERATION_DAILY
  TO ROLE BOOK13_ANALYST;
-- Warehouse USAGE is granted separately.
```

Project reference: P024: Terraform Snowflake RBAC example. *Not independently executed; rights review pending.*

13.2.3 Permission regression tests

Your access model is a graph, not a spreadsheet of names. Suppose BOOK13_ANALYST is later granted BOOK13_LOADER; its new effective privileges may exceed the design even if no direct raw-table grant was added. Review role-to-role edges, role-to-user assignments, database roles and active secondary roles. Testing only direct grants misses inherited privilege.

Acceptance matrix for the sandbox

Operation	Analyst	Loader
Read GOLD.GENERATION_DAILY	Allow	Deny
Read RAW.METER_EVENT	Deny	As scoped
Insert into raw event table	Deny	Allow
Create account user or grant roles	Deny	Deny

Use actual dedicated test identities after an administrator approves the test. USE ROLE alone is insufficient if secondary roles remain active, the identity can assume additional roles, or a direct user grant is in effect. Set secondary-role behavior explicitly where permitted and record the active context. Never present a failed syntax statement as an access-control success: the negative test must fail for the expected authorization reason.

Listing 3: Reader probe: run only with the approved reader identity.

```
SELECT CURRENT_ROLE(), CURRENT_USER();
SELECT COUNT(*)
FROM BOOK13_SANDBOX.GOLD.GENERATION_DAILY;
-- Expected permission denial (separate statement):
SELECT COUNT(*)
FROM BOOK13_SANDBOX.RAW.METER_EVENT;
```

Tests also need boundaries: an authorized administrator can read or change many objects; a test runner that assumes that role invalidates the evidence. An automated role-lint script can detect unwanted GRANT edges offline and should complement, never replace, real negative-access tests.

Project reference: P143: Terraform role module. *Not independently executed; rights review pending.*

Check. Which privilege, inherited by which path, could explain a newly successful query against the raw table?

13.3 Authentication, secrets, and network controls

13.3.1 SSO and key-based authentication options

Human access should normally be tied to an organization-managed identity, with appropriate single sign-on and multifactor controls. Programmatic clients may use a supported non-interactive authentication method such as key-pair authentication or an approved token-based integration. Authentication proves an identity; authorization still determines its roles and object privileges. Neither a private network nor SSO makes an overprivileged service account harmless.

13.3.2 Secret rotation and vaulting

Do not put passwords, connection strings or private keys in a Git repository, LaTeX example, dashboard parameter export or CI log. Store secrets in an approved vault or workload identity flow; restrict who can retrieve them and rotate on a planned schedule or immediately after suspected exposure. Record key owners, expiry alerts, rotation steps and safe decommissioning. A second valid credential permits a controlled transition without simultaneously breaking every refresh job.

13.3.3 Network and endpoint controls

Snowflake network rules and policies can restrict allowed or blocked network origins; authentication policies constrain allowed authentication methods. Roll out network changes in stages, retain an approved break-glass path and test from the actual connector location. A rule suitable for an office IP may block a cloud-hosted Power BI refresh service. Restrict outbound destinations through separate integration controls rather than assuming inbound network policy covers egress.

Operational check
Before changing a network or authentication policy, identify supported client versions, remote worker locations, automation endpoints and fallback administrators. Require a successful human login and scheduled-job smoke test from authorized locations, with an agreed rollback window.

Project reference: P146: Securing a Snowflake account. *Not independently executed; rights review pending.*

Check. What would remain exposed if a service key were stolen but an IP allowlist were perfectly enforced?

13.4 Masking policies, row access, and classification

13.4.1 Dynamic data masking

Column masking changes what a permitted query returns based on policy logic and session context; it does not by itself grant table access. A masked contact identifier may remain hidden for analysts and visible to a designated steward. In Snowflake, dynamic data masking requires a supported edition (Enterprise Edition or higher as documented at this editorial checkpoint). Create and apply policies only in a permitted sandbox.

Listing 4: Illustrative masking policy; requires approved security role.

```
CREATE MASKING POLICY BOOK13_SANDBOX.GOV.EMAIL_MASK
  AS (val STRING) RETURNS STRING ->
  CASE WHEN IS_ROLE_IN_SESSION('BOOK13_STEWARD')
       THEN val ELSE '[MASKED]' END;
-- Attach in the approved security deployment:
ALTER TABLE BOOK13_SANDBOX.GOLD.CONTACT_CONTEXT
  MODIFY COLUMN CONTACT_EMAIL SET MASKING POLICY
  BOOK13_SANDBOX.GOV.EMAIL_MASK;
```

13.4.2 Row access policies

Row access policies decide which records a role can see, for example only an authorized plant region. They are evaluated at query time and do not automatically stop unauthorized inserts. Use ordinary grants to stop writes and row policies to constrain qualifying reads. Avoid a policy that relies on caller-controlled session variables for sensitive authorization without a protected mapping.

13.4.3 Classification and authorized test users

Classify by sensitivity and business need: public aggregate, internal operational, confidential contact metadata and regulated personal data. Validate masking for a real approved reader and a real steward. Verify both positive and negative cases and protect the evidence itself: a screenshot of unmasked sensitive values can be a new exposure.

Project reference: P147: Snowflake security demonstration. *Not independently executed; rights review pending.*

Check. Why can a policy return masked values to a reader even after SELECT has been granted?

13.5 Tags, lineage, auditing, and data sharing

13.5.1 Tags and lineage provenance

A governance tag supplies machine-readable metadata such as DATA_CLASS = INTERNAL or COST_CENTER = ENERGY. It is useful for discovery and reporting, but a label does not enforce access unless linked to a specific policy and verified. Store a data dictionary that connects each column's meaning to its source, refresh logic, owner and retention decision. Separate structural lineage (object A depends on B) from event-level lineage (this accepted row was derived from source file X under transform release Y).

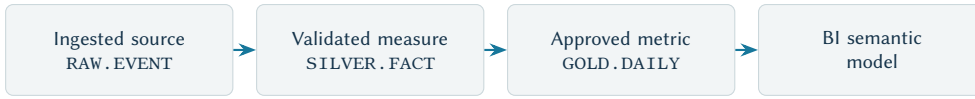


Figure 13.4. Governance metadata must follow the whole measurement path.

Listing 5: Metadata inventory to support a governed review.

```
SHOW TAGS IN SCHEMA BOOK13_SANDBOX.GOV;
SELECT CURRENT_DATABASE(), CURRENT_SCHEMA();
-- Review lineage and catalog with an approved role;
-- exact availability and privileges are account-specific.
```

Trade-off. Attaching many tags without defined owners produces metadata noise. Start with a few required classifications tied to visible decisions, then verify completeness in release checks. Energy totals must retain provenance and units even when no personal information is involved.

Project reference: P133: Intra-organization sharing guide. *Not independently executed; rights review pending.*

Check. Can an analyst trace the value 66.000 MWh through the source version, accepted rows, model version and aggregation grain?

13.5.2 Access audit and usage history

Security operations need to answer three questions: who ran a statement, which object was accessed and which policy or grant change enabled the behavior. Query History, Access History, Login History and Grant views serve related but different purposes; they have separate availability, privilege requirements and ingestion delays. A query record may show submitted text and execution time, while access history may enumerate objects accessed. Do not infer "no access" from an empty view before considering lag, retention, unsupported object types or insufficient viewing privileges.

Listing 6: Read-only audit pattern; account-usage rights required.

```
SELECT query_id, query_type, user_name,
       role_name, start_time
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_HISTORY
WHERE start_time >= DATEADD(day, -1, CURRENT_TIMESTAMP())
  AND query_tag LIKE 'book13_%'
ORDER BY start_time DESC;
```

Protect audit records from broad access; query text can itself contain sensitive literals or identifiers. Keep operations evidence longer than a single incident when justified, but avoid collecting unbounded identity data. Prefer parameters instead of embedding secrets in SQL text. Define a minimal investigation record: time window, identity, role, object, query ID, user impact, hypothesis, evidence and response decision.

Investigation example
A user reports seeing a column unexpectedly. First reproduce using the approved role and masked query. Then inspect relevant masking policy DDL and active role hierarchy; correlate query ID and deployment record. Only then decide whether the incident was a policy bug, a broadened grant, a changed semantic model or a misunderstanding of the reported value.

Check. Which timestamps, log delays and privilege requirements would you document before claiming an access event is absent?

13.5.3 Secure sharing and policy enforcement

Snowflake Secure Data Sharing can expose selected governed objects without creating a separate file-export pipeline. That is useful for partners, but the design question remains what the consumer may learn, not simply whether a share can be created. Prefer approved, stable projections over raw internal tables, document downstream interpretations and confirm row/column policies behave as expected in the intended sharing context. Some security features impose restrictions on shared objects; validate the complete combination before committing to a delivery promise.

A provider's share inventory should record: provider account and region, recipient account, dataset owner, field classifications, allowed purpose, change notice, expiration or revocation plan, applicable contractual review, and monitored usage. Do not assume that revoking future access deletes material a recipient has already copied or derived. Sharing and deletion are two different operational controls.

Data-product contract for a hypothetical regional partner
Dataset: hourly wind generation aggregate. Allowed values: timestamp UTC, production region, MWh total, refresh timestamp. Excluded: personal contacts, source-system credentials, raw device identifiers. Service expectation: revised records within the agreed reporting window. Consumer test: known fixture reproduces the published totals and exposes no restricted columns. Revocation test: removal of the documented grant blocks further approved-account reads.

Keep provenance visible after publication. The consumer needs to know whether a revised total replaces a previous total, whether a zero means observed zero or missing input, and whether daylight-saving conversions were applied. A secure share of semantically ambiguous data is still an unsafe data product.

Incident case. A BI refresh includes `SOURCE_CONTACT_EMAIL` after a schema evolution. The correct response is not merely to rename the column in Power BI. Contain the feed, remove the field from the shared projection, investigate who received the earlier output, test the new schema and document notification decisions with the appropriate owner and counsel.

Project reference: P064: ShareWatch schema-drift monitor. *Not independently executed; rights review pending.*

Check. What can the provider technically revoke, and what cannot be guaranteed after a consumer has exported the result?

13.6 Privacy, retention, and compliance basics

13.6.1 Personal data handling

The running energy fixture is entirely synthetic. A real industrial platform may nonetheless hold addresses, contact names, device identifiers or detailed behavioral records. Determine whether an element is personal data in context: a seemingly harmless site ID may become identifying when combined with a tenant registry. Apply data minimization at ingestion, map purposes and lawful processing grounds with an accountable privacy owner, and restrict highly identifying fields to workflows that demonstrably require them.

13.6.2 Retention and deletion

Define separate retention schedules for raw data, curated facts, backups or recoverable history, logs, exports and BI caches. An DELETE statement on a table does not prove the value is immediately gone from Time Travel, Fail-safe, replication destinations or copies obtained by recipients. Snowflake's retention and recovery features have object- and edition-dependent limits. Document the actual lifecycle, and involve the organization privacy team in decisions about erasure, archival and legal holds.

13.6.3 GDPR and legal review triggers

For operations affecting people in the European Economic Area, the GDPR may require a lawful basis, transparency, contract terms with processors, appropriate safeguards for international transfers, rights-handling processes and sometimes a data protection impact assessment. The applicable obligations depend on roles, purposes and risk. A technical mask is not a blanket compliance certificate; encryption and pseudonymization do not always make data anonymous. Legal review is necessary before making product-specific compliance claims.

Privacy design worksheet
Field: source-operator email. Purpose: operational escalation. Need: incident-only access. Storage: separate restricted reference table. Access: named steward role; masked for all BI readers. Retention: approved duration tied to business purpose. Audit: reviewed access records. Deletion: cataloged copies and recovery windows. Review trigger: external sharing or use in a new model.

Project reference: P149: Limited-access Native Apps example. *Not independently executed; rights review pending.*

Check. Why might deleting an email from the gold model leave an unresolved retention obligation elsewhere?

13.7 Compute, storage, serverless, and AI cost drivers

13.7.1 Virtual warehouse compute cost

A credit is an accounting unit, not a universal euro price. The effective monetary rate depends on commercial agreement, region, edition and contract terms. Warehouse compute is related to warehouse size and active duration, including rules for minimum billing and auto-suspend. Concurrency can change how many clusters run; idle time can create spend without a useful query. Estimate credits separately from converting those credits to money, and never hardcode one supposed globally valid tariff into a book example.

Illustrative scenario, not a live bill
Assume a hypothetical warehouse consumes r credits per hour and is active for t billable hours during a period. A first-order estimate is $C_{\text{compute}} = r\,t$ credits. For $r = 1$ credit/hour and $t = 2.5$ hours, the modeled consumption is 2.5 credits. This is a mathematical teaching assumption, *not* a verified rate or Snowflake invoice. Confirm billing minima and current size rates before using this model for decisions.

A fair optimization comparison holds the business result constant and measures total cost for repeated realistic workloads. A larger warehouse might finish a batch sooner but cost more, less or about the same depending on runtime, efficiency and startup overhead. A lower query execution time does not imply proportionally lower credits. Start with the agreed freshness target, result correctness and workload concurrency; then measure.

Listing 7: Read-only warehouse metering; approved account-usage rights.

```
SELECT warehouse_name,
       SUM(credits_used_compute) AS compute_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.WAREHOUSE_METERING_HISTORY
WHERE start_time >= DATEADD(day, -7, CURRENT_DATE())
GROUP BY warehouse_name
ORDER BY compute_credits DESC;
```

Metering reports have publication latency and may not immediately reflect a running workload. Document the accounting interval and timezone. Do not combine warehouse-level credits with per-query attributed credits as if the latter were an additional bill: those are different views of potentially overlapping consumption.

Project reference: P031: Snowflake monitoring with dbt. *Not independently executed; rights review pending.*

13.7.2 Storage, serverless, and AI cost categories

Storage is not merely the visible active table size. Historical data retained for Time Travel and Fail-safe, staged files, clones with independent modifications, and other retained objects can contribute. A transient table has a different recovery lifecycle from a permanent table; choosing one to reduce retention cost also changes recoverability. Evaluate both sides of the decision and do not treat destructive cleanup as the first cost optimization.

Serverless ingestion and transformation may consume separately metered credits, depending on feature and workload. AI inference or specialized processing can use its own measurement model, so warehouse credits alone understate total expense. Data transfer, external network traffic or marketplace products may introduce further categories. Before launching a new feature, identify the account's actual metering view, unit, ownership tag, budget applicability and shutdown route.

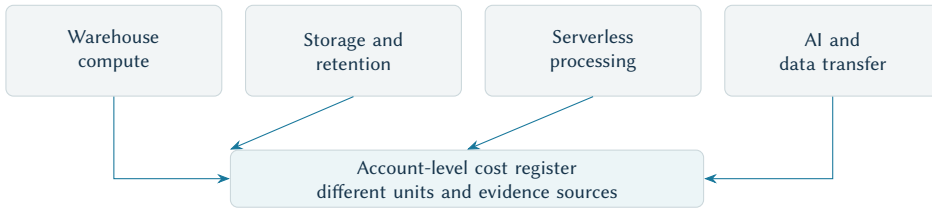


Figure 13.5. A warehouse-only cost report is not a complete bill.

Cost category checklist
For each charge, record a service name, usage period, resource or owner, raw quantity and unit, published or contracted price source, currency, taxes if relevant, data freshness, and whether the number is measured, modeled or illustrative. A missing cost category should be reported as unknown, not zero.

A fixed-cost spreadsheet is suitable for checking scenario arithmetic, but not for proving the availability or price of a cloud feature. Keep version-specific rates in a reviewed external configuration file and capture the date and region of every quoted figure. **Check.** Why might an account have modest virtual-warehouse usage yet a materially larger total cloud bill?

13.7.3 Unit economics and cost attribution

Cost attribution connects a spend category to a decision maker. Shared warehouses complicate this because idle time, queued requests and queries from different teams overlap. Query attribution can estimate execution compute by query, but it does not automatically allocate all warehouse idle time, storage, cloud-service overhead, serverless services or AI usage. One honest accounting method reports warehouse metering as a source of truth, then assigns allocation rules with disclosed assumptions. Never add attributed query credits to the same warehouse's billed credits.

Listing 8: Separate per-query attributed compute, not an extra charge.

```
SELECT COALESCE(NULLIF(query_tag, ''), 'untagged')
         AS workload,
       SUM(credits_attributed_compute) AS query_credits
FROM SNOWFLAKE.ACCOUNT_USAGE.QUERY_ATTRIBUTION_HISTORY
WHERE start_time >= DATEADD(day, -7, CURRENT_DATE())
GROUP BY workload
ORDER BY query_credits DESC;
```

Assign workload-level QUERY_TAGs during ETL, API serving and scheduled reporting; tag shared physical resources with their cost center where appropriate. Record untagged usage and the governance process for correcting it. Avoid putting secret or personal information in query tags. For a complete monthly report reconcile metering and billing on the same reporting boundary, with a documented treatment of delays and adjustments.

Unit economics, with denominators
If an illustrative weekly run consumes 12 credited units for 24 accepted batches, cost is 0.5 credits per batch. If the same input creates 2400 accepted observations, it is 0.005 credits per observation. These values describe the modeled run only. Always state the denominator, excluded categories, data quality, and actual or assumed credit quantity before using a ratio for planning.

Cost per dashboard load, per fresh MWh observation and per validated file measure different products. Choose metrics tied to a business service-level expectation, and reject a cost reduction that simply skips data validation or reduces refreshes below the agreed tolerance.

Project reference: P151: Example Snowflake cost monitoring. *Not independently executed; rights review pending.*

Check. How would you fairly apportion a warehouse's idle expense across teams, and what assumptions would the resulting dashboard expose?

13.8 Budgets, monitoring, and workload guardrails

13.8.1 Resource Monitors and their scope

Snowflake Resource Monitors primarily govern supported warehouse credit consumption: they can notify and apply suspend actions to monitored user-managed warehouses at configured thresholds. They are not universal spending caps, cannot stop every in-flight operation immediately, and do not cover serverless or AI service consumption. Test notification recipients and stop behavior in a restricted environment rather than trusting configuration names alone.

13.8.2 Budgets and alert response

Budgets monitor supported consumption against a defined spending target and forecast; they issue alerts when a threshold is or may be exceeded, depending on configuration. A budget is not a general-purpose hard shutoff. Route alerts to an accountable owner with explicit escalation, and confirm whether each relevant service is included in its scope. Alert noise without a response owner is only another operational failure.

13.8.3 Stop conditions and cost incident handling

Define two thresholds: an early-review threshold that triggers diagnosis and an emergency stop condition authorizing suspension of particular approved workloads. Before stopping, assess safety and data freshness implications. A necessary daily compliance report should not vanish because an unrelated experimental warehouse consumed credits. Pause the experiment first; capture query history, warehouse ownership, expected next run and the recovery plan.

Illustrative two-stage response
At 70% of a modeled monthly allocation, the owner checks anomalous resource usage. At 90%, the owner requests a scoped review and pauses nonessential experiments where authorized. These percentages are an organizational exercise, not Snowflake defaults or claims of fixed enforcement behavior.

Project reference: P152: AI cost governance example. *Not independently executed; rights review pending.*

Check. Which charges can continue even when every monitored virtual warehouse is suspended?

13.9 Operational reliability, recovery, and incident response

13.9.1 Recoverability, backups, and limits

Snowflake Time Travel permits supported historical queries and restoration within the configured retention period; Fail-safe is a platform-assisted recovery mechanism rather than an ordinary customer backup or instant rollback. Edition, object type and retention settings matter. An independent export, replicated environment or protected source replay may be required for a business recovery promise. Test actual recovery from an isolated object, including grants and dependent applications; do not assume that cloned data includes every service, stage or operational integration.

13.9.2 Runbook-driven incident response

Define states: *detect, triage, contain, recover, verify, communicate, learn.* Assign roles for incident commander, technical responder, data steward and stakeholder communications. Preserve necessary logs before cleanup, but restrict incident artifacts because they can contain identities or other sensitive details. For a mistakenly broadened role, containment may mean revoking an inheritance edge and rotating a secret, not destroying the affected table.

13.9.3 RPO/RTO and testing recovery drills

Recovery Point Objective (RPO) bounds acceptable data loss; Recovery Time Objective (RTO) bounds restoration time. These are negotiated business requirements, not automatic guarantees provided by any single Snowflake feature. A dry run should include detection delay, re-establishing credentials, rebuilding a scheduled task, replaying missing events, checking accepted counts and restoring dashboard freshness.

Recovery drill acceptance
Simulate a broken gold table in a disposable sandbox. Record the incident start, suspected loss window, steps taken, actual data restored, check totals and achieved time to recovery. If the result exceeds the target, record the gap and follow-up; do not silently change the target after observing the outcome.

Project reference: P153: Snowflake failover script reference. *Not independently executed; rights review pending.*

Check. If a consumer can tolerate one hour of delayed reporting but no permanently missing confirmed reading, how would the RPO and RTO differ?

13.10 Governance and platform runbooks

13.10.1 Governance ownership assignments

A governance council is not needed for every small data team, but every critical data product needs named decision owners. The platform owner controls Snowflake configuration and reliability; the data owner approves legitimate uses; the steward maintains definitions and quality thresholds; the security owner authorizes controls; finance approves spending thresholds; the product owner resolves delivery trade-offs. One person may fill several roles in a small team, but conflicting approvals should receive independent review.

Example responsibility table

Decision	Accountable	Required evidence
Share a new KPI	Data owner	Contract and recipient scope
Change masking	Security owner	Policy test and review
Change warehouse size	Platform owner	Cost/latency benchmark
Delete a source history	Data/privacy owner	Retention and recovery review

Avoid governance theater. Naming roles in a slide is not control. Add a reliable queue for requests, explicit response time, delegation rules, documented approval evidence and a mechanism for correcting mistakes. Ensure platform teams can answer simple questions promptly: who owns this table, what is the data classification, who may read it, when did it last refresh, and who pays for it?

Listing 9: A neutral context probe for a governed change request.

```
SELECT CURRENT_ACCOUNT(), CURRENT_ROLE(),
       CURRENT_DATABASE(), CURRENT_SCHEMA();
-- Never attach personal identifiers or credentials to
-- the query tag or shared approval ticket.
```

Project reference: P154: SnowAlert policy monitoring reference. *Not independently executed; rights review pending.*

Check. If a sensitive field moves into a newly published view, whose approval is required and what artifact proves it occurred?

13.10.2 Audit evidence and recurring reviews

Operational evidence degrades unless reviewed. A monthly security review compares effective role grants to the approved matrix, identifies dormant service identities and checks high-risk policy changes. A weekly data quality review checks freshness, rejected records, duplicate rate, accepted volume and incident backlog. A cost review reconciles billed categories to owners and follows up untagged workloads. Quarterly, run a recovery drill and reassess retention and sharing inventory.

Record failures as well as passes. A security test skipped because an Enterprise-only feature was unavailable is not equivalent to a successful test. Mark it **not executed** and identify the control that substitutes in the lab, such as a restricted secure view or a separate account-level verification path. Similarly, an offline cost register validates arithmetic but cannot establish billed Snowflake credits or quoted EUR figures.

Evidence bundle for release `13.0-lab`
`security_policies.sql`: proposed policy DDL with privileges and review gates.
`security_tests.sql`: expected permitted and denied SQL, to run under real authorized users.
`cost_register.csv`: synthetic usage assumptions and unit labels.
`incident_runbook.md`: stepwise containment and recovery.
`tests/`: executed offline test output and limitations.
`QUALITY_REPORT.txt`: scope, test status, result summary and required cloud checks.

A review should ask about *control drift*: did a legitimate shortcut create a long-lived privilege, did an analytics-only service gain insert access, or did a new AI workload bypass the ownership budget? Drift can arise even when every single earlier change was approved, because the combined state of the system changed.

Decision point. Prefer the least invasive remediation consistent with safety: remove an accidental role edge before restructuring an entire database, lower experimental schedule frequency before disabling every report, or fix a model contract before creating a new layer of data copies. **Check.** Which required evidence distinguishes a policy that was proposed in Git from a policy actually deployed and verified under the intended role?

13.10.3 Platform operating procedures and checklists

A runbook must work when the engineer who wrote it is asleep. Use imperative steps, with preconditions, approval scope and rollback. Include how to detect the trigger, confirm account and role, preserve evidence, contain safely, verify the data contract, communicate impact, and retire temporary privileges. Keep commands explicit about whether they are read-only, privileged or destructive. If a user cannot tell whether DROP deletes lab-only resources, it does not belong in a copy-and-run recovery procedure.

One-page operational checklist

1. Identify incident ID, severity, time window, affected data product and owner.
2. Verify account, region, role, active secondary roles and required approvals.
3. Preserve query IDs, grant snapshots, scheduled-job status and cost context.
4. Apply the smallest authorized containment action; log its exact effects.
5. Recover from tested source replay, retained history or approved backup.
6. Reconcile row counts, units, watermark, 66.000 MWh fixture invariant, and freshness.
7. Rerun permitted and denied access tests; restore alert routing and schedules.
8. Notify consumers, record follow-ups, remove temporary rights, schedule review.

Example: runaway experimental load. An alert identifies the book lab warehouse as unexpectedly active. Confirm no critical work shares it; capture active statement IDs; notify the owner; suspend the isolated warehouse if authorized; inspect query history and schedule; record total actual metered credits when data becomes available; restart only after fixing the cause. Do not claim the monitor can stop all future serverless or AI activity.

Example: unauthorized raw access. Confirm effective role and identity, revoke the unintended grant edge, rotate any exposed credential, assess data exposure and preserve evidence. Do not hide the incident by truncating tables or deleting audit records. A follow-up change must include positive and negative access assertions from approved test users and an explicit review of inherited permissions.

Project reference: P155: Application control framework. *Not independently executed; rights review pending.*

Check. Could an on-call engineer locate the right owner, determine whether to stop a pipeline, and prove it is safe to resume?

13.L Guided lab, chapter review, and exercises

13.L.1 Prerequisites, inputs, environment, permissions, and cost guardrails

The companion `lab/` directory contains an original author-written security and cost exercise, not copied repository code. Its source fixture is synthetic: six accepted energy observations total 66.000 MWh. The optional cloud steps use disposable `BOOK13_SANDBOX` objects. Obtain explicit approval for role creation, grants and masking policy application; enterprise-only features may not be available. Avoid `ACCOUNTADMIN` for notebook execution and never embed real secrets.

Preflight decisions
Execution profile A (default): local Python tests; no Snowflake credentials, no cloud billing, no personal data.
Profile B (optional): approved disposable Snowflake account; restricted provisioning identity, dedicated small auto-suspending warehouse, documented credit guardrail and verified cleanup plan. Run security negative cases with two separate approved identities; never assume inherited rights away.

13.L.2 Step-by-step procedure with selected code and illustrative results

1. Review `security_policies.sql` and approve object and role names. Run read-only inventory first.
2. In a disposable environment, provision the synthetic `GOLD.GENERATION_DAILY` object and narrow reader access. Apply masking only where supported, approved and independently reviewed.
3. Run `security_tests.sql` under each designated identity and verify the explicit allow/deny matrix. Record query IDs and denial causes.
4. Run local `python -m unittest discover -s tests -v` to validate the offline access graph, cost ledger, and runbook invariant checks.
5. Fill `cost_register.csv` with scenario consumption; keep actual Snowflake metering and monetary rate separate until verified.
6. Rehearse access and cost incidents using the companion runbook. Record detection, containment, verification and handoff times.

Listing 10: Illustrative approved-reader test in an isolated account.

```
SELECT CURRENT_USER(), CURRENT_ROLE();
SELECT COUNT(*) AS rows_visible
FROM BOOK13_SANDBOX.GOLD.GENERATION_DAILY;
-- Separate unauthorized test: expected denied.
SELECT COUNT(*) FROM BOOK13_SANDBOX.RAW.METER_EVENT;
```

Expected result of the offline fixture: six accepted observations, total 66.000 MWh; no asserted cloud result. Account behavior, denial text, edition constraints and metered credits must be recorded after a real supervised run.

13.L.3 Expected result, assertions, failure diagnosis, rollback, and cleanup

The lab passes only if approved access succeeds, forbidden actions genuinely fail with authorization errors, sensitive values are masked for unauthorized readers (if the masking feature is enabled), and the cost register reconciles assumptions without confusing estimated and observed usage. A `SQL compilation error` is not proof of least privilege. The offline suite verifies scenario rules and metadata completeness; its passing status does not establish Snowflake security compliance.

If an access case unexpectedly succeeds, stop and inspect secondary roles, user grants and inherited role paths. If a masking policy is unavailable, log *feature not tested* and never represent a view-based substitution as equivalent. If a bill estimate differs, verify units, minimum billing intervals and excluded services before blaming the data. Cleanup must be owner-approved, scoped to `BOOK13_SANDBOX`, and executed from the safe checklist; destructive SQL is commented out by default.

13.L.4 Foundation, practitioner and production design challenges

Concept review. (1) Why is `SELECT` authorization different from masking? (2) How can role inheritance expand access unexpectedly? (3) What does a warehouse Resource Monitor not cover? (4) Why is Fail-safe not a general backup strategy? (5) How can a query-cost report double-count expenditure?

Foundation exercise. Draw a three-role access graph and write a permitted/denied action matrix. *Acceptance:* no loader privilege reaches the BI reader, and every high-risk action names an owner.

Practitioner exercise. Extend the cost ledger to include a modeled serverless usage line and verify that the warehouse subtotal is not presented as the total account bill. *Acceptance:* units, provenance, pricing assumptions and omissions are explicit.

Architecture decision. A partner requests regional generation data, the dashboard must recover in two hours after failure, and finance demands an auditable monthly cost ceiling. Propose sharing controls, retention policy, RPO/RTO, owner responsibilities and an alert/response route. *Acceptance:* identify which controls alert, which can suspend, and which do not provide hard guarantees.

Scoring rubric (20 points)
Correct access and test identities (5); masking/classification and sharing logic (4); separate metering categories and limits (4); runnable incident and recovery verification (4); citations, provenance and honest untested labels (3). A passing score needs at least 16 points and no critical security or fabricated-evidence claim.

Chapter synthesis. Governance is what keeps a data product understandable when people, systems and threats change. Least privilege is enforceable only when denied behavior is tested; cost is controllable only when its categories are visible; recovery is believable only when rehearsed. Chapter 14 adds AI workloads to this governed platform, increasing the need for all three disciplines. **Authoritative references (verify before print).** [1] Access control; [2] Dynamic masking; [3] Row access; [4] Resource monitors; [5] Cost attribution; [6] Time Travel; [7] EU data protection.

Documentation checked 22 October 2026; all third-party GitHub references are leads, not independently executed or cleared for republication.

CHAPTER 14 EVALUATION / EVIDENCE / COST

Snowflake Cortex AI and Machine Learning

Does a model improve the decision, or only make the system more complicated?

An appealing forecast is not yet a useful forecast.
The Finland Energy Analytics dashboard has a carefully validated, synthetic history and a clear metric contract. Someone proposes adding an AI assistant that predicts tomorrow's generation and answers operational questions. A colourful curve appears quickly; the actual task is harder. Could yesterday's value or last week's value predict as well? Was future information leaked into training? Will the assistant invent a data source? Who pays for repeated prompts, and where are those prompts processed?

Chapter 13 established access controls, provenance, runbooks and workload budgets. Here those controls become prerequisites for model development. Machine learning (ML) estimates patterns from observations; generative AI produces content from learned distributions. Neither automatically makes data accurate or a forecast actionable. The engineer must define a decision, preserve a chronological holdout, measure error, inspect failures and record the cost of each alternative.

Learning outcomes
By the end, you should be able to (1) decide whether SQL, statistics, ML or a language model fits a task; (2) evaluate Cortex availability and inference residency; (3) design and test AI SQL prompts; (4) explain grounded retrieval and natural-language analytics; (5) build leakage-safe energy features; (6) compare naive and seasonal forecasting baselines; (7) write a model card; and (8) estimate and monitor AI-related credit usage without assuming a universal price.

Prerequisites. Chapters 9, 11–13, elementary Python, basic notions of forecast error and Snowflake roles. All printed sample observations and scores are explicitly synthetic. The offline lab needs only the Python standard library; optional AI calls require permission, supported models/regions and billing approval.



Figure 14.1. Prove a baseline before introducing a billable AI extension.

14.1 AI/ML use cases in a modern data platform

14.1.1 Choose a workload, not a fashionable model

An energy engineer can use ordinary SQL to total generation, flag negative readings or join a device dimension. Deterministic rules should remain deterministic: an AI-generated answer to a calculation already defined by a validated SQL expression adds uncertainty with little value. ML is more appropriate when the sought relationship is uncertain: estimating load one day ahead, ranking likely faults or detecting unexpected behaviour in an evolving time series. A language model can help interpret unstructured incident notes, but it cannot replace the measured fact table.

Question	First credible method	Why it fits
How much energy?	SQL aggregate	Exact, reproducible arithmetic
Is reading valid?	Typed rule and contract	Explicit threshold and failure reason
What will happen?	Naive / seasonal forecast	Testable prediction benchmark
Why was equipment stopped?	Search plus cited notes	Unstructured evidence and provenance
Which action to take?	Human-governed decision	Requires operational accountability

14.1.2 Value, failure impact and permitted data

Express value as a decision with a measurable consequence. A grid scheduling assistant might reduce manual inspection time, but a false "normal" designation might delay maintenance. Prioritise use cases by expected value, error severity, required freshness, observability, sensitivity, and whether a human can reject a suggestion. A promising proof of concept may still be unsuitable for unattended operation.

A data contract precedes a model contract. Confirm ownership, licensing, retention and lawful processing for each input: raw telemetry, operator notes and external weather predictions can have different constraints. A prompt is a data transfer, not a harmless string. Minimise sensitive data, retain structured identifiers needed for provenance, and evaluate downstream sharing rules for generated summaries.

Decision checkpoint. If errors can be described and fixed with an approved SQL condition, choose SQL. If success requires estimating an unknown target from past observations, compare statistical and ML baselines. If the input is narrative text, test retrieval or AI-assisted extraction with an audit trail rather than accepting an unsupported explanation.

Project reference: P156: getting started with machine learning. *Not independently executed; rights review pending.*

14.1.3 Build an evaluation baseline before an AI prototype

A model comparison is meaningful only if every candidate receives the same input rights, prediction time, validation windows and scoring procedure. For daily energy forecasting, choose a forecast horizon (tomorrow), a unit (MWh per day), a decision consumer (planning analyst), and a loss function (for example, mean absolute error, MAE). Specify the acceptable error *before* seeing model scores. Record the timestamp at which each predictor would actually have been known.

Our running energy case offers six accepted generation observations totalling 66.000 MWh in the earlier chapters. Six observations cannot demonstrate a trustworthy multiweek forecast. Therefore Chapter 14 creates an *additional, explicitly synthetic* 35-day teaching series with a weekly cycle and one artificial spike. These rows are neither sampled Finnish energy measurements nor copied observations; they exist to test chronological splits, naive predictions and anomaly alert logic under controlled conditions.

A minimum experiment has three comparisons: a last-value forecast ($\hat{y}_t = y_{t-1}$), a seasonal last-week forecast ($\hat{y}_t = y_{t-7}$), and an optional learned model evaluated on the same held-out days. A language-model-generated narrative is assessed separately for factual consistency and evidence support; its fluency is not a numeric forecasting score.

Pre-registered experiment card
Input: synthetic daily generation; one observation per UTC date. **Split:** days 0–20 train, 21–27 validation, 28–34 final test. **Prediction horizon:** one day. **Baselines:** lag 1 and lag 7. **Metrics:** test MAE plus anomaly false/true positives. **Guardrail:** future target values may not appear as current features. **Decision:** add a complex model only if it beats the seasonal baseline and meets safety/cost requirements.

Common failure. Randomly shuffle a chronological series before splitting and train a model on tomorrow's meter reading to predict tomorrow's meter reading. The resulting high accuracy measures leakage, not predictive ability. In production, holdouts should include shifts in season, holidays, missing sensors and regime changes; our tiny lab does not establish reliability in those conditions.

Check. Write a one-sentence claim about improvement that could be *falsified* by a future test, not one that merely describes how sophisticated the model is.

14.2 Cortex AI capabilities and availability caveats

14.2.1 Separate product families and operational jobs

The Snowflake AI and ML portfolio spans managed AI SQL functions for text and multimodal inputs; search services supporting retrieval-augmented generation (RAG); semantic natural-language analytics; and ML capabilities for forecasting, anomaly detection and model development. The functions answer different questions. For example, `AI_CLASSIFY` assigns supplied categories; `AI_COMPLETE` generates responses; a search service retrieves relevant indexed text; a forecasting model estimates a numeric trajectory. Treat "Cortex" as a product family, not one uniform endpoint or billing meter.[1]

14.2.2 Availability is account-specific

Feature status, model allow-lists, access privileges, region, cloud provider, and preview restrictions change. The same SQL identifier can be valid in one account and unavailable in another. Before a paid trial, have an administrator review the documented function privileges, model lifecycle, available models and the account's relevant feature/edition constraints. In 2026, Cortex model RBAC and allow-lists have become important enforcement points; never infer permission from the ability to query a conventional table.

```
-- Read-only account check; admin may need to perform it.
SHOW PARAMETERS LIKE 'CORTEX_ENABLED_CROSS_REGION'
  IN ACCOUNT;
-- Additional model/feature listings vary by privilege.
```

14.2.3 Storage residency is not inference residency

Data stored in an account region can still be sent temporarily to a permitted processing region if cross-region inference is enabled. The request payload and generated answer may cross a regional boundary. A strict data-residency policy therefore requires a specific review of inference routing, not only the location of source tables. Do not change the account-level `CORTEX_ENABLED_CROSS_REGION` parameter from a teaching notebook; it is an administrator-controlled governance decision.

Deployment gate. Record home cloud/region; supported processing region; model ID and lifecycle; approved inference route; sensitive input classification; relevant privilege; anticipated usage; and owner approval. "Available somewhere" is not evidence of local availability or compliance.

Project reference: P157: Cortex AI functions guide. *Not independently executed; rights review pending.*

[1] Snowflake documentation: https://docs.snowflake.com/en/user-guide/snowflake-cortex/llm-functions and https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-search/cortex-search-overview.

14.3 AI SQL functions and evaluation principles

14.3.1 Understand AI SQL semantics before writing queries

A conventional SQL aggregate is deterministic given unchanged data and query semantics. An AI function evaluates an input using a managed model and may be probabilistic, evolve over time or return an error/NULL for a subset of rows. Do not equate the presence of a result with correctness. Snowflake documents `AI_COMPLETE`, `AI_CLASSIFY`, `AI_SUMMARIZE` and other functions; exact function signatures, available models, error behaviour and privileges require a documentation check before deployment.

```
-- OPTIONAL BILLABLE EXAMPLE. Test in an approved sandbox.
-- The model identifier must be checked for your account.
SELECT AI_CLASSIFY(
  'Turbine B was offline after an inverter alarm',
  ['maintenance', 'operations', 'commercial']
) AS suggested_category;
```

The classification should be compared with a human-labeled answer. Even a syntactically valid label might be unsupported by the note. For a multi-label task, decide whether several classes are acceptable; for an exclusive taxonomy, define what to do when the model gives more than one or no recognized category.

14.3.2 Prompts are interfaces with contracts

A reliable prompt identifies the task, permitted context, output type, evidence rules, fallback and forbidden behaviour. Consider an incident-summary assistant: it must distinguish a confirmed diagnosis from a technician's hypothesis, preserve the original device identifier, not invent measured outage duration, and answer "insufficient evidence" when the note is ambiguous. Put required output fields in a validated schema and enforce application-side type checks after inference.

Proposed output contract
`incident_id`: identifier from source; `summary`: short statement grounded in the note; `category`: one of the approved labels; `evidence_spans`: quoted source spans/offsets; `needs_review`: boolean. Reject unknown keys or references to nonexistent source spans. This contract is original example logic, not a claim that every model will follow it unaided.

Failure path. If the model returns malformed JSON or a confident diagnosis absent from the note, quarantine the response and retain the input hash, model ID, version/time and evaluation status. Never silently coerce unrecognized categories into "normal".

14.3.3 Build labelled cases and measure more than fluency

An evaluation set should include typical, rare, adversarial and intentionally unanswerable inputs. For the energy incident assistant, include a clear inverter alarm, a contradictory maintenance note, a prompt-injection sentence embedded in a source note, an incident with no device ID, a multilingual note and an explicit "no known cause" example. Source text is evidence to be interpreted, not instructions to override the assistant's task.

Dimension	Metric	A practical failure test
Classification	Precision, recall, F1	Confusing cause with symptom
Extraction	Field accuracy	Invented device identifier
Grounding	Citation support rate	Citation does not contain claim
Abstention	Correct refusal rate	Answer despite absent evidence
Safety	Rule-violation count	Source note attempts prompt override
Operation	Latency and credits	Large prompt repeated across many rows

In a classification experiment, report a confusion matrix and the number of examples, not a single vague "accuracy" statement. Human raters need a consistent rubric; disagreements should be adjudicated, and labels must be stored independently of model outputs. Evaluate at least once on a frozen holdout set not used to refine prompts.

The workflow matters as much as the score. Fix prompt template and model before a batch evaluation. Capture failures as evidence, revise in a separate development cycle, and rerun the unchanged final holdout. A retrieval component can be evaluated independently (did it find the correct note?) before the generation component (did it use only retrieved evidence?). This helps distinguish missing information from invented content.

Metric integrity. Suppose 18 of 20 incident summaries cite the right document, but only 12 cite a span supporting their central claim. "90% contain a citation" is not evidence that 90% are grounded. Prefer a claim-level evidence check and include the denominator and confidence limitations.

Project reference: P161: prompt engineering and LLM evaluation. *Not independently executed; rights review pending.*

Check. Which metric should trigger an immediate human-review requirement, even if a model's overall score improves?

14.4 Search, retrieval, and natural-language analytics

14.4.1 Retrieve evidence before composing a response

Retrieval-augmented generation (RAG) brings permitted supporting records to the model at question time. A search service indexes selected content and returns candidate passages. A generation step then answers with explicit citations to those passages. Cortex Search uses a mixture of lexical and semantic matching with reranking; quality still depends on source coverage, freshness, permission design, chunking and relevance. Unlike a static glossary, an index can incur refresh, embedding and serving charges.



Figure 14.2. Retrieval narrows evidence; it does not prove that generated claims are true.

14.4.2 Attribution and the hidden access boundary

A useful answer records source document ID, excerpt, version and retrieval timestamp, with claim-level citation checks. If an incident report is reclassified as restricted, refresh paths and cached/indexed copies also require review. Search-service access can operate with owner-rights semantics; a user permitted to query a service may see indexed rows unless the service is designed with appropriate per-user boundaries. Do not assume a row-access policy on a source table is automatically applied during every search response.[2]

14.4.3 Natural-language SQL needs semantic constraints

Cortex Analyst turns business questions into SQL-oriented responses using a defined semantic interpretation of tables and metrics. "Energy generated" must carry the right unit, grain, filters and sign conventions; otherwise a plausible query can be numerically wrong. Apply an allow-list of approved serving views, inspect generated SQL where applicable, restrict execution privileges and verify result totals against certified KPI queries. The system is not a substitute for a trusted metric contract.

Project reference: P162: document RAG with Cortex Search. *Not independently executed; rights review pending.*

[2] See the official Cortex Search overview and its access-control discussion: https://docs.snowflake.com/en/user-guide/snowflake-cortex/cortex-search/cortex-search-overview.

14.5 Data preparation and feature engineering with Snowpark

14.5.1 A time series is not merely a sorted CSV

Time-based predictions depend on a canonical business grain. The energy fact from previous chapters is keyed by source, metric and event time, while today's forecasting lab uses one aggregate generation value for each UTC date. First determine whether missing days mean zero generation, missing telemetry or a source outage. Never fill a missing day with zero without a domain rule: this creates artificial collapses that may look like anomalies.

In Snowpark, use warehouse-side filters and aggregations rather than collecting unrestricted raw data to a laptop. A preparation query might collapse approved hourly readings into UTC days, check valid units, remove superseded revisions and retain an `as_of` timestamp for reproducibility. Only after that contract is satisfied should Python create forecasting features.

```
-- Illustration: use an approved, unique daily-serving view.
SELECT DATE_TRUNC('DAY', event_time_utc)::DATE AS d,
       SUM(generation_mwh) AS total_mwh
FROM BOOK_CH14.GOLD.V_ENERGY_ACCEPTED
WHERE event_time_utc < :as_of_utc
GROUP BY 1 ORDER BY 1;
-- View and bind variable are illustrative prerequisites.
```

14.5.2 Construct past-only lag and rolling features

The lag-7 feature for day t is y_{t-7}. A seven-day rolling average predicting day t should use $y_{t-1}, \ldots, y_{t-7}$, *not* y_t. The difference is one misplaced window boundary. For a one-day-ahead forecast made at the end of day $t-1$, weather forecasts issued before that time may be eligible predictors; realized temperatures for day t are not.

```
-- Conceptual Snowflake window; verify grain and order.
SELECT d, total_mwh,
       LAG(total_mwh, 7) OVER (ORDER BY d) AS lag_7,
       AVG(total_mwh) OVER (
         ORDER BY d ROWS BETWEEN 7 PRECEDING
                            AND 1 PRECEDING) AS avg_7_past
FROM BOOK_CH14.GOLD.V_DAILY_ENERGY;
```

A row-number window assumes one row per calendar day; otherwise a lag of seven rows is not seven days. Validate continuity before applying it. Keep the feature transformation and any imputation policy in version-controlled code so evaluation and serving use identical logic.

14.5.3 Split by time and freeze the training view

For the synthetic 35-day lab, day indices 0–20 supply training history, 21–27 are a validation interval and 28–34 form a sealed test interval. Set hyperparameters on training/validation data, and compute the final score only once for the chosen design. The seasonal baseline does not "fit" coefficients, but its anomaly threshold is calibrated from validation residuals, not from the test spike.

A deployment often requires a *rolling-origin* evaluation rather than one lucky holdout: train to an early cutoff, predict the next horizon, advance the cutoff and repeat. When a source has multiple devices, group identifiers and time boundaries carefully; mixing measurements from the same physical incident across train and test can leak its signature. Compare forecasts at the exact lead time required by the operational decision.



Figure 14.3. Chronological evaluation prevents test outcomes from choosing the model.

Leakage checklist
(1) Is every training record timestamped before the cutoff? (2) Were late revisions frozen as they were known at inference time? (3) Are window features shifted correctly? (4) Is the scaling or imputation fitted only on past values? (5) Were thresholds fixed without looking at test errors? (6) Can the exact feature table be reconstructed from its inputs and `as_of` metadata?

Trade-off. Snowpark and an in-platform feature store can centralise governance, but introduce additional objects and operational costs; a modest, fixed SQL view may be sufficient for a small forecasting application. Select infrastructure only after identifying reuse, update frequency and access boundaries.

Project reference: P165: ML feature engineering in Python. *Not independently executed; rights review pending.*

14.6 Forecasting and anomaly detection use cases

14.6.1 Naive baselines make sophisticated claims falsifiable

The last-value prediction assumes tomorrow resembles today: $\hat{y}_t = y_{t-1}$. The seasonal naive baseline repeats the same weekday one week ago: $\hat{y}_t = y_{t-7}$. Both are cheap, interpretable and defensible first benchmarks. If an ML model cannot consistently improve on these at the required lead time, its maintenance and inference bill require special justification.

14.6.2 Forecasts, alarms and operational thresholds are different

A forecast answers "how much?" An anomaly detector answers "is this observation unexpectedly different from a justified reference?" A maintenance decision asks "what should someone do?" Those are three distinct stages. Our offline lab uses *seasonal residuals* $e_t = |y_t - y_{t-7}|$ and sets a threshold from the earlier validation period, then injects an artificial spike on the final test day. This is a teaching heuristic, *not* an industry-calibrated prediction interval or production fault detector.

```
# Excerpt from accompanying ml_baseline.py
seasonal = values[t - 7]  # known at prediction time
error = abs(values[t] - seasonal)  # scoring afterwards
alert = error > threshold  # threshold fixed from validation
```

Snowflake ML also documents managed forecasting and anomaly-detection objects in the `SNOWFLAKE.ML` namespace. Those are optional comparative systems, not hidden prerequisites for the offline script. In a live experiment, first verify input-shape constraints and training history requirements; preserve result history and the model's own evaluation metadata.

14.6.3 Score both error and confidence honestly

Mean absolute error is MAE $= n^{-1} \sum_t |y_t - \hat{y}_t|$ in MWh, directly interpretable at the original unit. Percentage errors become unstable near zero generation; use care with MAPE and report the count of zero targets. An interval's usefulness requires empirical coverage on unseen data, not merely a visually wide shaded region. For alarms, track missed events, false alerts, time-to-detection and whether humans actually changed an action.

Verified offline fixture (not a Snowflake benchmark). On seven held-out synthetic days the seasonal baseline scores **1.0643 MWh MAE**; last-value scores **3.2500 MWh MAE**. A validation-derived threshold flags the one injected test spike (index 34). With one deliberately placed event, this does not estimate a real-world detection rate.

Project reference: P010: predictive grid maintenance case study. *Not independently executed; rights review pending.*

14.7 Model evaluation, security, and responsible AI

14.7.1 Model permissions and data permissions form one boundary

A secured warehouse view is not sufficient if an application forwards its raw fields into a broader model invocation. Authorisation must cover the service principal, eligible input columns, region processing route, model family, produced artefacts, logs, and users allowed to read those outputs. Deny sensitive incident notes by default; introduce a reviewed, minimised view for a specific approved use case.

For Cortex AI Functions, access involves the appropriate Snowflake database role and, depending on the model, model-specific access control and account allow-lists. In contrast, an externally hosted application may add network policies, secret rotation and vendor contracts. A model registry does not automatically authorise serving. Use a narrow execution role for training and a separate role for inference and publication.

14.7.2 Reproducibility includes human judgment

Retain the dataset fingerprint, split dates, code commit, feature version, model name, settings, scoring script, error examples and reviewer decision. Avoid placing raw sensitive prompts in indiscriminate diagnostics. For a human-in-the-loop workflow, show the source evidence next to the model suggestion and allow reviewers to reject it, flag an unsupported claim, or correct an incident category. Their overrides become auditable labels rather than invisible manual fixes.

Promotion evidence packet
Purpose: energy planning assistance. **Data:** approved synthetic prototype, then governed operational slice. **Validation:** chronological, named holdouts. **Baseline:** seasonal naive. **Model:** exact identifier, lifecycle and settings. **Safety:** access denial and evidence attribution cases. **Operator:** named owner, rollback procedure. **Cost:** measured credits or declared assumptions. **Approval:** signed decision with unresolved limitations.

If a model is retrained or silently upgraded, rerun the evaluation. A monthly drift graph cannot substitute for observing severe failure modes: predictions that break physical plausibility, fabricated sources, and outcomes whose performance disproportionately deteriorates for underrepresented asset types.

Project reference: P171: ML observability in Snowflake. *Not independently executed; rights review pending.*

14.7.3 Model cards explain when not to trust a result

A model card makes scope and failure boundaries visible. It includes the intended consumer, prediction horizon, target and units, eligible data, exclusion criteria, benchmark, observed test results, sensitivity to distribution changes, and a plan for ongoing monitoring and withdrawal. "Production-ready" cannot be inferred from a working demonstration. The six accepted observations from earlier chapters remain a reconciliation fixture, not training evidence for the expanded synthetic forecasting task.



Figure 14.4. A model is released only after independent quality and control gates.

Wrong-task risk. A text generation model cannot establish the underlying physical cause of a turbine trip without calibrated evidence; an apparent causal explanation may only paraphrase a technician's initial suspicion. Keep diagnostic statements, recommendations and factual events separate in the application schema. Distinguish "operational support" from autonomous control of physical equipment.

Misleading benchmark risk. A clean 35-row synthetic experiment says nothing about cold starts, high-wind curtailment, seasonality shifts, meter downtime or the thousands of correlated hourly observations in a real system. Confidence claims must match observed scope. Any live pilot should add backtesting over multiple months or seasons, a change-detection plan and escalation when data is incomplete.

Review protocol. For every failure case, record the expected behaviour, actual behaviour, severity, responsible reviewer and decision. If the model repeatedly cites unsupported facts, disable narrative explanations while keeping verified numerical views online. If live forecasting beats the baseline only on one unusually easy week, decline promotion until independent slices confirm the gain.

Responsible AI is also a reliability discipline. Every AI component needs the same operational obligations as the rest of the data platform: an owner, a minimum test suite, monitoring, versioned outputs, security boundaries, a fallback and an explicit withdrawal criterion.

Project reference: P170: Cortex agent evaluations. *Not independently executed; rights review pending.*

14.8 Token usage, compute charges, and AI cost controls

14.8.1 Tokens are not queries, and neither is a fixed bill

AI inference services frequently charge according to model-dependent input/output token consumption, page counts or other usage units. The same prompt repeated against many data rows can multiply cost, and a model producing a long response can be materially dearer than one returning a short validated label. A retrieval system has additional indexing and ongoing serving considerations; model training and Snowpark transformations may consume warehouse or serverless compute.

Treat cost as several meters, not a single undifferentiated credit figure. For AI SQL, log request count, model, input units, output units, timestamp, status and charge source. For search services, include index refresh, embeddings, serving footprint and source-query compute. For ML functions, include training and inference compute with edition-specific behaviour. Avoid adding a guessed "EUR per prompt" without the applicable consumption table and contractual credit rate.

14.8.2 A cost formula that remains valid when rates change

A useful illustrative token model is

$$C_{\text{tokens}} = \sum_{m} \left(\frac{I_m}{10^6} r_m^{\text{in}} + \frac{O_m}{10^6} r_m^{\text{out}} \right),$$

where I_m and O_m are observed input/output token totals for model m, and r_m are the applicable *date- and region-specific* credit rates per million tokens. If the meter is pages, compute pages at the published page rate instead; do not force everything into tokens. Credits and the organisation's currency conversion must be recorded separately.

Hypothetical sizing example (not a quote).
An approved dry run considers 1,000 calls, an estimated 400 input tokens and 60 output tokens per call: 0.40 million input and 0.06 million output tokens. The illustrative formula is $0.40 r^{\text{in}} + 0.06 r^{\text{out}}$ credits *before* other compute. No monetary or actual credit total is claimed until rates and measurements are obtained.

Project reference: P173: Cortex cost governance reference. *Not independently executed; rights review pending.*

14.8.3 Enforce exposure limits before the first batch

Snowflake publishes account-usage views such as CORTEX_AI_FUNCTIONS_USAGE_HISTORY for AI function monitoring. Data availability and latency may vary; consult the current schema before writing a production report. Budget alerts and usage dashboards are useful, but they are not by themselves guaranteed hard caps on every AI request. Resource monitors largely address warehouse usage rather than every serverless AI meter. The practical control is a combination of scoped privileges, approved workload size, query tagging, request throttling, alerting and a tested stop path.

```
-- Read-only example: inspect available usage metadata.
SELECT *
FROM SNOWFLAKE.ACCOUNT_USAGE.CORTEX_AI_FUNCTIONS_USAGE_HISTORY
LIMIT 10;
-- Requires rights; schema/latency can change over time.
```

Guardrail design. (1) Begin with 10 synthetic test cases. (2) Estimate worst-case prompt growth and generated length. (3) Approve a finite number of rows. (4) Use an explicit timeout and retry limit. (5) Monitor measured consumption. (6) Define who can revoke the AI role or stop scheduled jobs. (7) Require reapproval for new models, regions or larger batches.

Failure example. A scheduled report generates a new summary for every record at every refresh, even when the underlying note has not changed. The query is functionally correct, but compute and token usage scale with reprocessing, not new information. Cache only permitted, versioned results when freshness requirements allow, and keep an auditable invalidation policy when source content changes.

> **Cost and security link.** A permissive AI execution role is a spending risk. If any analyst can run unrestricted model invocations against a large table, dashboards cannot guarantee that the credit bill remains within an informal target. Restrict the capability by role and explicitly review batch size before enabling recurring schedules.

Check. List which meters your proposed workload actually exercises, which ones a given monitor can affect and what human or automated intervention ends a runaway experiment.

Project reference: P174: AI observability guided example. *Not independently executed; rights review pending.*

14.L Guided lab, chapter review, and exercises

14.L.1 Goal, prerequisites, permissions and safeguards

Goal. Compare an interpretable one-day energy forecast with a second baseline and identify an injected anomaly; construct an evaluation specification for an optional, paid AI explanation without actually calling any model. The offline path is entirely synthetic, has no credential requirement, and incurs no Snowflake bill. Optional Snowflake exercises require a restricted sandbox role, administrator approval of region and model, time limits and measured usage.

Inputs. The companion package contains `ml_baseline.py`, a generated daily CSV, a model card, labelled AI cases, optional SQL, unit tests and a README. The generator builds 35 UTC days with a weekly pattern, deliberately small variation and a +6 MWh spike on day index 34. This extended series is a *new synthetic test fixture*, separate from the six 66.000 MWh observations in Chapters 8–13.

14.L.2 Reproduce the baseline and examine the outputs

```
# At the lab directory; Python 3.10+; no packages needed:
python ml_baseline.py
python -m unittest discover -s tests -v
```

Procedure. (1) Inspect the CSV and confirm the daily UTC key and declared synthetic provenance. (2) Verify that training, validation and test days are strictly chronological. (3) Calculate last-value and lag-7 seasonal forecasts. (4) Derive the residual alert threshold using validation days alone. (5) Score the final seven days exactly once. (6) Inspect the flagged spike and explain why a single synthetic outlier is not an operational accuracy claim. (7) Complete the model card with deployment objections.

Observed result in the supplied offline test environment
Train / validate / test: 21 / 7 / 7 daily points. Seasonal test MAE: **1.0643 MWh**; last-value test MAE: **3.2500 MWh**. Residual threshold: **0.8500 MWh**. The test spike at day index 34 is flagged; an otherwise identical series with no injected spike does not flag that final day. All 16 Python `unittest` cases pass. These figures are deterministic synthetic results, *not* Cortex or Finnish grid measurements.

14.L.3 Assertions, diagnosis, rollback and cleanup

Core acceptance checks: 35 unique contiguous UTC dates; only synthetic input; nonnegative finite values; past-only lag features; no test-derived threshold; expected held-out MAE; a flagged injected spike; and rejection of duplicate dates, negative values and unexpected provenance. The accompanying tests verify these invariants. The default SQL file is read-only or commented where billable examples would otherwise run.

If a test fails: a duplicate date means the business grain is broken; an unexpected model score may indicate future leakage or fixture drift; a spurious anomaly may reflect an improperly calibrated threshold; an access error in a proposed Cortex call requires a privileges/region check, not an administrator-privilege shortcut. Roll back by returning to the validated fixture and versioned baseline, disabling optional calls, and retaining evidence. Cleanup needs only deletion of disposable local output files; destructive Snowflake cleanup statements are deliberately omitted.

14.L.4 Review questions, exercises and design rubric

Concept checks. (1) Why is SQL a better fit for an exact MWh total than text generation? (2) How can a forecast feature leak future information? (3) What distinguishes retrieved evidence from a grounded claim? (4) Why is storing data in the EU insufficient to prove inference stays there? (5) Which AI expenses may remain outside ordinary warehouse resource monitors?

Foundation exercise. Reproduce both holdout MAEs, show the individual test residuals and explain the intentional anomaly. *Acceptance:* matching values, correct units and a clear synthetic-data disclaimer.

Practitioner exercise. Create an additional missing-day case and adapt the series validator to reject it without interpreting it as zero production. Add two labelled AI evaluation cases: one note with insufficient evidence and one note attempting a prompt override. *Acceptance:* explicit rejection conditions and unchanged baseline splits.

Architectural decision. A team asks for automated explanations of future energy shortage events. Recommend deterministic rules, forecasting, RAG and/or Cortex functions. Document access, residency, human review, true evaluation criteria and a financial stop policy. *Acceptance:* an inspectable alternative comparison, a versioned test plan and no claims of untested model performance.

Rubric (20 points).
Task-to-method reasoning (4); chronological validation and reproducibility (5); grounded evaluation and responsible deployment (5); region/role/cost controls (4); explicit limitations and provenance (2). Pass at 16 or above with no invented measured credit charge or fabricated evidence.

Chapter synthesis. AI can enrich a governed data platform, but cannot excuse a weak baseline, missing provenance or uncontrolled expense. In Chapter 15 the same discipline will turn the evolving Finland Energy Analytics example into a capstone whose outputs, operational decisions and limitations are auditable. **Technical references (checked 22 October 2026).**

[1] AI COMPLETE; [2] Cross-region inference; [3] Cortex Search; [4] Forecast ML; [5] AI cost management. *Referenced GitHub projects are not executed or cleared for republication.*

CHAPTER 15 INTEGRATE / VERIFY / OPERATE

Capstone: A Production-Ready Energy Data Platform

Can another engineer rebuild, verify and operate the whole system?

The dashboard is not the deliverable. Imagine a Finnish energy analyst opening a clear Power BI report at 08:00. A wind-generation number is wrong. Did the source publish a correction? Did yesterday's batch run twice? Was an invalid row quietly accepted? Or did the report refresh before the silver table? Without a traceable answer, the pleasant dashboard is an operational liability.

This final chapter turns the concepts developed across Chapters 1–14 into one governed engineering product: the *Finland Energy Analytics* capstone. The aim is a system that can be inspected, rerun, tested and handed over. The printed walkthrough provides architecture, acceptance criteria, selected implementation patterns and evidence. The companion project contains longer scripts, fixtures, validation, decision records and incident procedures.

What you will deliver

A documented ingestion and transformation chain from immutable synthetic source events to bronze, silver and gold; versioned contracts and reproducible checks; role and cost policies; a BI semantic contract; a monitoring and recovery runbook; and a release review that distinguishes proved offline behaviour from Snowflake features still requiring live integration tests.

Prerequisites. Chapters 1–14 and elementary SQL/Python. No Snowflake credits are needed for the *offline acceptance fixture*; a real deployment requires a test account, controlled role grants, an approved spend limit and a designated owner. All sample energy readings in this chapter are synthetic; no production Finnish electricity data has been downloaded into the book package.



Figure 15.1. Every arrow needs a contract, observable evidence and a recovery path.

Release principle. "Production-ready" describes the engineered safeguards and documented gates, not a claim that this teaching package has passed a live Snowflake security review or been deployed to the Finnish grid.

15.1 Business scenario, assumptions, and acceptance criteria

15.1.1 Business scenario and stakeholder decisions

The business sponsor needs one trusted view of renewable and other generation readings at an agreed time grain. The operations analyst wants to know whether an observed change is a real generation movement or a corrected source delivery. A data steward wants evidence of origin and licence. A platform operator needs to diagnose missing data without guessing which job to restart. These are different decisions, so a single dashboard acceptance test cannot represent all of them.

Stakeholder	Decision	Required evidence
Energy analyst	Compare generation categories	UTC grain, unit and version
Data steward	Permit publication	Licence and source lineage
Operator	Recover a failed ingest	File checksum and run ledger
Platform owner	Approve release	Tests, grants and spend review

In the educational scenario, a source provider may publish a revised wind value for an existing UTC interval. The platform must retain the earlier evidence, select the valid newer revision, and publish a single current observation. A deliberate duplicate delivery must not double the energy KPI. A logical deletion is not equivalent to an energy reading of zero: deletion changes which key is visible; zero is a legitimate measurement where the source contract permits it.

15.1.2 Dataset assumptions and limitations

The companion fixture has twelve *synthetic* delivered rows, including a duplicate event, one superseded version, a logical deletion and two invalid records. Its final accepted view contains six readings totalling **66.000 MWh**. The source label `DEMO_GRID` is invented. The timestamp is in UTC; generation categories are invented teaching values, not estimates of Finnish production. These six rows cannot support a credible national trend, a machine-learning forecast or an operational energy decision.

> **Assumption register.** Define the natural key as *provider + technology + interval_utc*; define `REVISION` as a positive, monotonically increasing integer for each key. Accept only `UPSERT` or `DELETE`; require nonnegative finite MWh; require DELETE to carry zero; and reject conflicting payloads at an equal key and revision. Each rule is intentionally testable.

Failure to avoid. A business sponsor signs off on a attractive chart without noticing that a wind revision is counted twice. The correct first acceptance meeting examines a six-row gold export, a rejected-row log and a small set of named invariants.

15.1.3 Explicit acceptance and review criteria

Define acceptance before implementation. A successful pipeline is not merely one that returns rows; it demonstrates functional correctness, privacy of access, recoverability, repeatability and an economically bounded operating path. A reproducible trial records the input hash, code revision, run commands, timestamp, failed checks and operator identity. The acceptance document should name who is authorised to waive a check; the default for a broken invariant is **do not publish**.

ID	Release criterion	Evidence artifact
A01	Exactly six current rows	Gold CSV and uniqueness test
A02	Total equals 66.000 MWh	Independent Python oracle
A03	Two rows quarantined	Row-level reasons and count
A04	Replay leaves result unchanged	Double-run comparison
A05	Access policy reviewed	Negative permission test plan
A06	Recovery and cost defined	Runbook, budget and rollback

The first four checks can be demonstrated offline with the shipped fixture. A05 and the Snowflake part of A06 cannot be declared successful by a local Python test. The release record therefore has two dimensions: *offline demonstration passed* and *cloud deployment not yet approved*. This distinction prevents a common portfolio mistake: reporting an architectural design as if production controls were already operating. A useful acceptance matrix has three columns for every criterion: the observable result, the evidence path and the responsible reviewer. For example, an idempotent replay check must compare *business results and audit state*, not merely show that SQL completed without an exception. An apparently identical final sum could hide two duplicated wind rows and two missing solar rows; include key-level comparisons and row counts.

Decision record 15-01: proof before deployment. The capstone uses the offline fixture as a deterministic correctness oracle and treats Snowflake integration, Power BI report operation, hosted event notifications and external AI as separately gated extensions. **Reason:** local reproducibility must not depend on account permissions or variable credit charges. **Cost:** the offline run cannot reveal warehouse performance or actual account privileges.

Project reference: P006: Northstar data engineering; P176: telecom operations analytics. *Not independently executed; rights review pending.*

Learning check. Which two accepted results would you inspect to prove that the same input file produced the same publishable facts after replay? Include a business-key comparison, not only a checksum of the source.

15.2 Source selection, architecture, and data contracts

15.2.1 Fingrid and Statistics Finland source options

The starting point for an actual Finnish energy product is a *source catalogue*, not a web scraper. Fingrid's open-data portal publishes datasets relevant to Finland's electricity system; Statistics Finland offers statistical datasets and machine-readable services. They serve different grains and meanings. A Fingrid series might describe power or energy at operational intervals; a Statistics Finland table might describe regional, monthly or annual statistical aggregates. Do not join these blindly on a date and call the result "hourly generation." For each candidate, record dataset title and identifier, publisher, definition, measurement unit, spatial coverage, update policy, timestamp semantics, historical revisions, rate limits, reuse terms and the exact time of collection. A source may express power in MW, while an accepted observation fact uses energy in MWh. Conversion requires an interval duration and an explicit integration rule; multiplying a power reading by "one hour" without verifying its sampling semantics is an invented measurement.

Question	Fingrid-type feed	Statistics Finland-type table
Likely grain	Operational series; varies by dataset	Published statistical periods
Primary check	Timestamp/unit revision	Definitions, classifications, vintages
Collection	API or published export	PX-Web/structured download
Caution	MW is not automatically MWh	National totals are not event rows

The teaching package does not include a live fetcher or redistribute an external dataset. That avoids an implicit claim that any one Fingrid endpoint, key policy or licence will stay unchanged. The next real-data milestone would select one precise dataset, snapshot its metadata and run a limited authorised retrieval; it should be reviewed as a separate deliverable.

Source portals: Fingrid Open Data; Statistics Finland open data.

Source checkpoint. A credible lineage row is *publisher + dataset ID + source URL + access time + version or ETag/hash + licence snapshot.* A convenient public URL is not proof of continuous access or commercial redistribution rights for every asset on the portal.

15.2.2 Dataset licensing, snapshots, and provenance

A dataset is useful only if a reviewer can say which version produced an answer. Store the original bytes, compute SHA-256 and keep the publisher metadata separately. An API response might change without a filename changing; an immutable snapshot plus request metadata makes investigations possible even after the upstream resource has been revised. If the provider forbids redistribution, keep the raw data in authorised storage and ship only independently authored synthetic fixtures with the book. Statistics Finland describes its open statistical data as available under Creative Commons Attribution 4.0 with source attribution; the project still must check each dataset and any exceptions at acquisition time. For Fingrid, inspect the terms attached to the exact dataset and API rather than transferring a generic licence assumption from another portal. This publication is independent of both institutions and is not an endorsed Finnish energy service. The manifest should record `sha256`, `original_name`, `retrieved_at_utc`, `publisher`, `dataset_id`, `licence_url`, `schema_version` and `row_count`. Use checksums to detect altered input bytes, not to prove truthfulness of a measurement. Source accuracy requires separate domain validation and, where justified, cross-provider reconciliation.

```
# Pure local provenance check; no cloud call.
from hashlib import sha256
from pathlib import Path
p = Path('project/data/energy_revisions.csv')
digest = sha256(p.read_bytes()).hexdigest()
assert len(digest) == 64
print('fixture sha256:', digest)
```

A timestamp is also part of provenance. Record both *event time* (when the measurement applies) and *processing time* (when the pipeline received or validated it). When a correction arrives late, event-time aggregation remains stable while ingest-time audit reflects the revision. That separation is essential for reconstructing what a user could have known at a past reporting cutoff.

Rights and source ledger. The companion `references/source_register.md` identifies external portals as optional research options, not as executed inputs. It labels every delivered reading as synthetic, so a redistribution audit can distinguish source citations from actual data assets.

15.2.3 Architecture and versioned data contracts

The reference design separates immutable evidence from mutable interpretation. Ingestion writes the bronze event ledger. Silver enforces schema and business rules, resolves versions and quarantines failures. Gold publishes an intentionally narrow metric contract. A separate access boundary exposes reporting views to Power BI rather than handing the BI role permission to query raw event payloads.

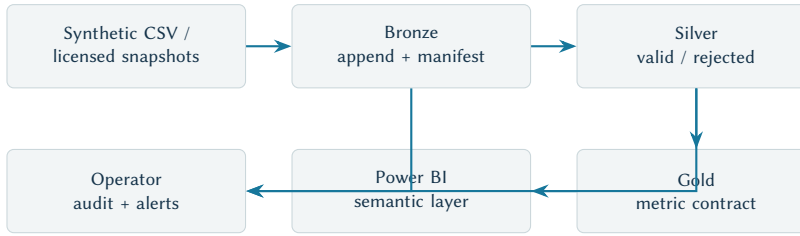


Figure 15.2. Audit and consumption are separate access paths.

The `energy-event/v1` contract has seven required fields: event ID, provider, technology, UTC timestamp, positive revision, decimal MWh and UPSERT/DELETE operation. A later version may add an optional quality code; it must not silently reinterpret historical MWh as kWh. Changing units or natural-key meaning is a breaking change and deserves a new schema version with an explicit migration plan.

Contract review questions. What is the business key? Can two events disagree at the same revision? Is a missing hour an error, or a source that did not promise hourly completeness? What identifies a deleted observation? Which columns are sensitive, and what role may see them? Each answer belongs in the contract, not in a dashboard footnote.

Project reference: P011: green-energy IoT example; P169: predictive maintenance. *Not independently executed; rights review pending.*

Failure mode. Treating the CSV's delivery order as the truth causes an older Hydro event to replace a newer one after replay. Sort by business revision, and fail the release on an ambiguous same-revision conflict.

15.3 Reproducible environment and repository structure

15.3.1 Environment automation and roles

A reproducible environment has more than a diagram of objects. It specifies a version-controlled deployment order, who can create objects and which tests run without cloud access. The least-privileged application role should not create account-wide grants, policies or integrations. One-time administrator-approved bootstrap is distinct from repeatable engineering deployment. Production and development objects must not share an unqualified schema name or a write-enabled service identity. Create a dedicated database or sandbox namespace only after the account owner approves the plan. Use a small, auto-suspending warehouse for test SQL, record its name in environment configuration and prohibit accidental escalation to a large size. Resource monitors can govern certain warehouse consumption, but do not promise they cap every serverless or AI charge. Avoid provisioning a continuously refreshing object merely to demonstrate one reading update.

```
-- Cloud example: run ONLY in an approved sandbox.
USE ROLE <APPROVED_DEPLOY_ROLE>;
USE WAREHOUSE <APPROVED_SMALL_WAREHOUSE>;
CREATE SCHEMA IF NOT EXISTS ENERGY_LAB.BRONZE;
CREATE SCHEMA IF NOT EXISTS ENERGY_LAB.SILVER;
CREATE SCHEMA IF NOT EXISTS ENERGY_LAB.GOLD;
```

Important: placeholder role and warehouse names are not executable Snowflake identifiers. Replace them from an approved environment file, verify privileges and record the grants separately. Database creation and role management are *not* auto-run by the companion project.

15.3.2 Folder structure and immutable dependencies

The repository root contains `project/`, `operations/`, an ADR folder and a test-evidence folder. The project holds a fixture, a Python acceptance oracle, SQL examples, contracts, BI definitions and CI notes. A future deployed release should include a lockfile or immutable container digest; a vague request to install the latest packages makes yesterday's run irreproducible. Avoid embedding passwords in config files or example commands.

Two execution tiers. Tier 1 runs offline on Python's standard library and enforces data invariants. Tier 2 runs only after cloud approval: create scoped objects, stage bytes, load, compare the gold view and perform negative grants tests. Success in Tier 1 is necessary, but not sufficient, for a production deployment.

15.3.3 Test fixtures and cleanup procedures

A good fixture represents the failures the system promises to tolerate. Our twelve deliveries contain an identical repeated event, a revised Wind observation, an old Hydro version, a tombstone for a TEST category and invalid source values. Rather than treating the fixture as disposable random data, record its SHA-256 hash in the acceptance output. Re-running the same code against the same file must give the same six visible business keys and total. A negative test should be reversible. For example, corrupt a copy of the file to change a field name, run the schema-contract assertion and confirm a hard failure. Another test can insert two different payloads for the same natural key and revision; this must block publication rather than elect an arbitrary winner. No test should need to alter the original fixture.

Fixture variant	Expected behaviour
Repeated identical event	Deduplicate; no extra energy
Valid higher revision	Select newer reading at same key
Revision zero	Quarantine under v1 contract
Negative MWh	Quarantine with reason
Equal-revision conflict	Stop release and investigate
Zero-valued DELETE	Remove key from current view

Cleanup is also part of reproducibility. Do not place an unconditional DROP DATABASE inside a reader lab. Instead, use a clearly named sandbox deployment, enumerate the exact objects the exercise created, retain evidence outside that sandbox, and require an authorised person to uncomment destructive SQL. Confirm that no production references or shared stages depend on the objects before removal.

Rebuild proof. A reviewer should clone the archive, run `python -m unittest discover -s project/tests -v`, execute the offline oracle, and compare its structured output with the acceptance contract. If an assertion fails, the review stops; the engineer diagnoses the change and supplies a new fixture version and release note rather than weakening the test.

Project reference: P120: Snowflake DevOps data projects. *Not independently executed; rights review pending.*

Check. Why is testing the cleanup path not equivalent to adding DROP SCHEMA CASCADE to an automated pipeline? Write a rollback plan that protects evidence and does not exceed the role's authority.

15.4 Batch and continuous ingestion design

15.4.1 Reliable batch ingestion

For the baseline, batch ingestion wins on simplicity. Freeze the input, sign a manifest, load the raw rows, retain filename and row number, then validate each typed field in silver. A batch boundary provides a natural replay unit and makes a postmortem easier: an operator can identify exactly which archived input produced a broken run. A unique event ID is useful metadata, but the *business* identity still consists of provider, technology and interval.

```
-- Simplified source audit: inspect after COPY, not before.
SELECT source_file, COUNT(*) AS delivered_rows
FROM ENERGY_LAB.BRONZE.EVENT_LEDGER
GROUP BY source_file ORDER BY source_file;
```

Store the SHA-256 digest in an independent manifest; source metadata alone does not prove file integrity. Snowflake's `COPY INTO` maintains load history, but file-level deduplication and load metadata are not substitutes for correction semantics at the business-key level. A `FORCE` reload, accidental file rename or a different ingestion route can duplicate deliveries. The silver contract must survive those conditions.

15.4.2 Optional continuous ingestion

An event-driven variant may use Snowpipe for staged-file notifications or a streaming connector for row-level delivery. Select it only when measured freshness needs justify a harder operational model. Decide how offsets, acknowledgements, schema changes and dead-letter queues work before advertising "exactly once." Source acknowledgements cannot prevent all application-level replays; deduplicate downstream with stable identities and revision rules.

15.4.3 Audit metadata and replay checks

A load ledger records run ID, source snapshot hash, ingest start and end, observed row count, quarantine count, publish outcome and operator. On retry, reconcile both source and target counts, and compare gold business-key sets. A successful `COPY` does not certify valid business values or dashboard consistency.

Correctness gate. Replay the twelve-row synthetic fixture twice. The second pass may produce another processing log, but it must not produce a seventh gold observation or change 66.000 MWh. Treat increased processing cost as a separate operational concern.

Project reference: P073: Kafka streaming reference; P177: IoT pipeline. *Not independently executed; rights review pending.*

15.5 Bronze-to-silver data quality and transformations

15.5.1 Raw bronze lineage

Bronze should preserve evidence, not attempt to repair the past. Retain the original delivery order, literal values, operation and revision even when a value is malformed. Add only audit columns such as ingestion timestamp, source file, source row number and a stable run identifier. A negative value must remain visible to the quarantine investigation rather than disappear inside an "ignore bad records" option.

```
CREATE TABLE IF NOT EXISTS ENERGY_LAB.BRONZE.EVENT_LEDGER (
  event_id VARCHAR, provider VARCHAR,
  technology VARCHAR, interval_utc VARCHAR,
  revision_raw VARCHAR, energy_raw VARCHAR,
  operation VARCHAR, source_file VARCHAR,
  source_row NUMBER, loaded_at TIMESTAMP_LTZ
);
```

The sample SQL uses raw strings for uncertain fields so the platform can retain an invalid revision or quantity without aborting the entire file. In a live account, configure the file format and stage independently, test row parsing with a tiny file, and inspect whether the chosen error mode matches the capture obligation. Keep the complete audit trail only as long as the approved retention policy permits. A commonly tempting shortcut is to trust an automatically inferred schema and cast every field during COPY. That may be acceptable for a controlled internal feed, but it can destroy the particular malformed value an analyst needs to diagnose. Bronze and silver have different jobs: one captures what arrived; the other declares what the business accepts.

Lineage invariant. For every accepted silver observation, a reviewer must be able to locate its source event and input snapshot. Conversely, every delivered source row has a documented disposition: selected, superseded, exact duplicate, logically deleted or rejected. No row should vanish without classification.

Learning check. A source sends wind 12.000 MWh, then wind 14.000 MWh for the same interval. Which bronze rows remain? Which silver value is current? What evidence is necessary to show that the first row was deliberately superseded rather than lost?

15.5.2 Silver standardisation and DST handling

The acceptance contract requires an ISO-8601 UTC timestamp with a trailing Z. This deliberately avoids ambiguous local times during Finland's daylight-saving transitions. A time like `03:30` on a local clock may be nonexistent during spring transition or occur twice during autumn transition. Never infer the offset from a naive timestamp using a fixed "Finland is UTC+2" rule. Instead, preserve the source timezone identifier and offset, normalise to UTC, and retain a separate display dimension for `Europe/Helsinki`. For the synthetic fixture, `2026-03-01T10:00:00Z` is already UTC. The test suite verifies a canonical structure, positive revision, declared operation and decimal quantity. The source contract defines what happens if timestamps include milliseconds or offsets: either version the contract and parse them explicitly, or quarantine them. A permissive parser that silently changes interpretation creates a false sense of completeness.

```
-- Inspect validation results before publishing.
SELECT event_id, technology, energy_raw,
       TRY_TO_DECIMAL(energy_raw,18,3) AS energy_mwh,
       TRY_TO_NUMBER(revision_raw) AS revision
FROM ENERGY_LAB.BRONZE.EVENT_LEDGER;
```

Casting with TRY_TO_DECIMAL allows the query to expose unparseable values as NULL. However, it must be paired with an explicit *error classification*: NULL, nonfinite/invalid input and legitimate zero are not interchangeable. A valid deletion carries zero MWh but changes current-row visibility. A valid zero production reading remains an UPSERT.

> **A dangerous shortcut.** Grouping by TO_DATE(`interval_utc`) before establishing the intended timezone can shift observations across local reporting dates. Publish an explicit UTC reporting grain; local calendar presentation belongs in a named, tested conversion layer.

The final model uses consistent decimal arithmetic, not floating-point sums, for deterministic demonstration totals. Real source precision may differ by dataset and must be recorded with units and acceptable rounding rules.

15.5.3 Completeness, uniqueness, and range tests

Data quality is a contract with failure categories. First verify schema shape and parseable typed values. Next apply domain conditions: revision greater than zero, operation in the approved list, MWh nonnegative and deletion quantity equal to zero. Finally apply stateful consistency rules: unique current business key and no conflicting payloads at the same revision. Detect exact duplicate deliveries without treating them as evidence of additional electricity. The table below separates *fixture* assertions from *real-world* expectations. Because the fixture contains six categories at one interval, its final row count is six. A real feed cannot be assumed to provide exactly six categories or a new row every hour unless the provider's contract makes that guarantee.

Control	Synthetic fixture expectation	Production interpretation
Uniqueness	Six distinct current keys	Unique at agreed fact grain
Validity	Two rejected deliveries	Quarantine rate threshold
Energy range	No negative MWh	Domain and unit policy
Timeliness	Fixed UTC interval	Source SLA and observed lag
Revisions	Wind rev. 2 wins	Deterministic version ordering

Test both positive and negative behaviour. The companion unit suite injects header drift and a same-revision conflict and expects hard failures. An acceptance suite that only checks successful records is dangerously incomplete: it cannot prove that a malformed event would be stopped before reaching a report.

Quality disposition ledger. The offline oracle records twelve delivered rows; one exact repeated delivery; two rejected rows; one tombstoned business key; six visible rows; and 66.000 MWh. These categories describe different stages of a stateful projection and are not additive in one simple row-count equation. Use the event-level disposition ledger to reconcile each record exactly once.

Project reference: P178: medallion case; P179: bronze-silver-gold pipeline. *Not independently executed; rights review pending.*

Check. Explain why a successful type cast does not guarantee a correct business key, and why a null count alone cannot validate the impact of a logical deletion.

15.6 Gold marts, KPIs, and incremental refreshes

15.6.1 Gold star schema and published KPIs

Gold is a product contract rather than a dumping ground for every source column. Define `fct_energy_hourly` at one row per provider, technology and UTC interval, with an MWh measure and a selected revision. A technology dimension contains controlled labels; a calendar dimension makes UTC and agreed local-display attributes explicit. If there is only one source provider in the demonstration, do not pretend that the fact supports nationwide reconciliation.

```
-- Demonstration acceptance query after the silver view.
SELECT COUNT(*) AS visible_rows,
       SUM(energy_mwh) AS total_mwh
FROM ENERGY_LAB.GOLD.FCT_ENERGY_HOURLY;
-- Expected for the shipped fixture: 6 and 66.000.
```

15.6.2 Incremental refresh correctness

An incremental pipeline must preserve a key's *latest approved state*, including a tombstone, not merely append each newly received UPSERT. Reprocessing a batch can change history when corrections arrive, so the incremental predicate must include a revision or processing-watermark strategy and a repair window for late events. Prefer a deterministic MERGE source with exactly one row per target business key. When multiple source rows match a target, many MERGE designs become nondeterministic or error-prone. For a Dynamic Table alternative, select and test a supported refresh mode and monitor actual lag; TARGET_LAG is a freshness goal, not a guaranteed refresh interval. A dbt incremental model is another option, but its unique key and delete strategy must be explicit. The reference architecture documents one chosen method per deployment rather than mixing three orchestration mechanisms for the same gold table.

15.6.3 Stakeholder acceptance checks

The BI owner's sign-off should compare exact key-level results, not only totals. Require the expected Wind revision 2 value of 14.000 MWh, no visible TEST tombstone key, six distinct rows and the accepted total. Also show a metric-definition sheet: whether *net position* includes consumption, whether an MWh sum covers a full reporting period, and what a missing observation means. In this capstone, consumption is not in the base six-row source; any consumption KPI is a separate, explicitly synthetic extension.

Gold promise. The same filtered fact table and metric definition must serve the SQL check, the Python oracle and the BI measure. A dashboard must not have a separate, undocumented rounding or duplicate-resolution rule.

Project reference: P039: Snowflake dbt medallion example. *Not independently executed; rights review pending.*

15.7 Python components, tests, and CI/CD

15.7.1 Reusable Python components

Separate parsing, validation, revision resolution, publication and the release gate. The supplied `project/python/capstone.py` exposes these steps as importable functions and uses Python's standard library to minimise external dependencies. It is an *offline acceptance oracle*, not a substitute Snowpark runtime or a connector to real Fingrid measurements. This small boundary permits a reviewer to establish the intended behaviour before validating equivalent SQL against a Snowflake test schema.

```
from pathlib import Path
from capstone import evaluate, release_gate
rows, rejected, audit = evaluate(
    Path('project/data/energy_revisions.csv'))
assert release_gate(rows, rejected, audit)['status'] == 'ACCEPT'
assert audit['visible_rows'] == 6
assert audit['total_mwh'] == '66.000'
```

15.7.2 Unit and integration test matrix

Unit tests prove the local code rejects schema drift, invalid decimals, negative values, duplicate deliveries, forbidden operations and conflicting revisions. Integration tests must separately prove the cloud SQL yields the same expected rows, that scoped roles cannot read raw protected data, and that the BI dataset can refresh after a verified publish. Query correctness can be tested offline; network policy, billing effects and grants cannot.

Test tier	Evidence	Release state
Python offline	Actual unittest result	Executed in package
Snowflake SQL	Query ID and assertion output	Pending approved account
Power BI	Refresh log, screenshots	Pending Desktop/service
Security/cost	Negative access and meter	Pending administrator review

15.7.3 CI/CD safety and release tags

The CI example runs offline tests on an isolated runner without credentials. It must not deploy the cloud schema from a public pull request. A separately reviewed deployment job should use short-lived credentials from a managed secret store, verify the target account and environment, require an approval step and emit immutable release metadata. Name a tag such as `ch15-capstone-review-v1`; the archive does not falsely claim that a public tag already exists.

Project reference: P108: Airflow and Snowflake integration example. *Not independently executed; rights review pending.*

Check. Why can a successful GitHub Actions job still be insufficient evidence that a role cannot select from a sensitive Snowflake table?

15.8 Power BI dashboards and optional AI extension

15.8.1 BI semantic model and dashboards

The report contract should expose the approved gold view with a clearly declared UTC time grain and a sum of MWh. Use a star schema with single-direction dimension-to-fact relationships where practicable. Measures should compute at the model's filter context; they must not silently aggregate a second, incompatible grain. A summary page can show total generation, generation by technology, time-series trend and a source-freshness indicator. An audit page should display the dataset version and last completed refresh, not a fabricated "real-time" badge.

```
-- DAX measure for a table of additive MWh facts.
Generation MWh :=
    SUM ( 'fct_energy_hourly'[energy_mwh] )
```

Power BI Import can simplify interactive reads but adds a dataset-refresh boundary; DirectQuery asks Snowflake for data during interactions and can amplify warehouse work. Choose based on volume, concurrency, required freshness and measured response time rather than assuming one is universally faster. Review the current connector, identity, privacy level and gateway or cloud-authentication requirements before publishing a service dataset.

15.8.2 Optional AI baseline and extended model

If stakeholders want forecasting, do not convert the tiny six-row fixture into a predictive success story. Chapter 14's separate 35-day synthetic series can illustrate chronological holdouts and seasonal baselines, but it does not establish a reliable Finnish grid model. Define forecast horizon, training cutoff, MAE and backtesting policy before requesting more expensive models. Evaluate new AI functionality only after the ordinary SQL report is correct and approved.

15.8.3 Privacy, performance, and interpretability

The BI role receives read-only gold exposure, not a direct connection to raw source notes or personal identifiers. Document row filters, identity mapping and export controls. If a generative assistant cites operational incidents, retain source references, redact sensitive fields and test unsupported answers. Observe actual Snowflake query history while exercising dashboard filters; simple report pages can still generate unexpected numbers of expensive queries.

> **Deliverable boundary.** This package ships a semantic-model specification and example DAX/M, not a compiled `.pbix`. No Power BI Desktop or hosted refresh was executed. Report acceptance therefore remains an explicit manual/cloud gate.

Connector documentation: Microsoft Power Query Snowflake connector.

Project reference: P121: Snowflake and Power BI walkthrough; P180: external DE case. *Not independently executed; rights review pending.*

15.9 Access controls, performance, and cost budget

15.9.1 Least-privilege access review

Security is an executable contract with negative tests. Define distinct deployment, ingestion, transformation, report-reader and auditor roles. A BI user should see the gold reporting objects needed for its semantic model but not the raw input stage, rejected source rows or administrative audit records. A transformation service can read bronze and write silver/gold without inheriting an account-administrator role. Human administrators approve role creation and exceptional grants separately from the application release.

Role	Required access	Denied by default
INGEST	Stage and bronze writes	Gold publication grants
TRANSFORM	Bronze reads; silver/gold changes	Security-policy administration
BI_READER	Selected gold reporting views	Bronze raw and quarantine
AUDITOR	Scoped evidence and logs	Data pipeline writes

A test plan must attempt the forbidden action *under the actual restricted role.* The claim "least privilege" is not proved by inspecting a grant diagram; inherited roles, future grants and ownership grants can expand effective access. Never run negative tests as ACCOUNTADMIN and interpret success as a restriction on the BI identity.

```
-- Illustrative NEGATIVE test, approved account only:
USE ROLE <APPROVED_BI_READER>;
SELECT COUNT(*)
FROM ENERGY_LAB.BRONZE.EVENT_LEDGER;
-- Expected: access denied, not a successful count.
```

Role selection placeholders must be resolved through the approved cloud testing procedure. Store only the outcome and non-sensitive metadata in evidence; avoid pasting full account identifiers or passwords into a public repository. Access expectations are incomplete if the source includes personal data, row masking or jurisdiction-specific residency constraints. Add policies at the proper object boundary and validate both permitted and denied queries.

Separate trust boundary. The offline Python oracle can prove deterministic aggregation. It cannot prove Snowflake role inheritance, external authentication policy, Power BI identity propagation or runtime access to staged files. Mark these tests *PENDING* until there is a recorded query ID and an authorised reviewer.

15.9.2 Query tuning and credit accounting

First measure, then tune. Capture a stable workload: a gold aggregate, a time-range filter and a representative BI query. Record query ID, warehouse size, cold/warm cache context, execution time, bytes scanned, partitions scanned and queue time where available. Compare result equality before comparing latency. A fast query returning fewer rows because a filter was rewritten incorrectly is a regression, not an optimisation.

```
-- Timed queries must be run in a controlled test account.
SELECT technology, SUM(energy_mwh) AS mwh
FROM ENERGY_LAB.GOLD.FCT_ENERGY_HOURLY
WHERE interval_utc >= '2026-03-01T00:00:00Z'
GROUP BY technology ORDER BY technology;
```

Micro-partition pruning, an appropriately sized warehouse and minimising unnecessary joins can improve workloads, but none has a universal speedup percentage. One small fixture cannot benchmark pruning behaviour; it fits in negligible storage and is intended for correctness. To evaluate performance responsibly, construct a separate volume-scaled synthetic dataset, label its generator and compare multiple controlled runs. Isolate BI from ELT compute when user-facing concurrency requires it and the account permits the extra cost. Warehouse credit estimates depend on warehouse size, suspension/idle periods and actual metering. Serverless features and Cortex AI may have different billing categories. Do not multiply all platform usage by a single virtual-warehouse rate or claim that an X-Small warehouse resource monitor stops independent serverless charges.

Benchmark field	Reporting requirement
Query/warehouse identifiers	Real IDs, redacted in public copy
Result hash or key comparison	Proves semantic equality
Latency distribution	Median, p95, sample count
Credit components	Warehouse versus other service classes
Run conditions	Cache, concurrency, interval and release

Review question. A new SQL query completes in half the time but uses twice the billed credits. Is that a win? The answer depends on the declared user latency requirement, frequency of execution, total cost envelope and whether results are unchanged.

15.9.3 Budget and cost-failure checks

A platform budget is a forecast and accountability mechanism, not evidence that consumption cannot exceed a threshold. Record a monthly planned credit allowance, separate alerts for abnormal growth and a response policy for nonessential processes. Resource monitors can be useful for supported warehouse consumption; Budgets provide spending visibility and alerts for broader supported scopes. Neither is a universal, instantaneous hard kill-switch for every billable Snowflake service.

Cost drill: synthetic accounting only.
Suppose the exercise ledger records 2.500 hypothetical warehouse credits, 0.750 hypothetical serverless credits and 0.400 hypothetical AI credits. The illustrated total is **3.650 credits**, not a measured Snowflake invoice. The categories matter because a warehouse-only alert would not by itself govern all components. For a live deployment, read metering views with their documented latency and reconcile categories before making a financial claim.

A practical guardrail stack combines a deliberately small test warehouse, auto-suspend, limited integration scope, capped optional workloads, no unapproved AI functions, budget alerts, named approval for escalation and a rehearsed process to suspend the right objects. Some features cannot be turned off by suspending a warehouse, and a stop command may arrive after accrued usage. A credible incident procedure acknowledges this residual spend.



Figure 15.3. Alerts start an accountable response; they are not billing guarantees.

Official reference: Snowflake resource monitors; cost controls.

Project reference: P152: AI cost governance reference. *Not independently executed; rights review pending.*

Failure injection. Imagine a scheduled refresh begins scanning much more data than expected while a test script calls an AI endpoint repeatedly. Which owners are notified, which workloads may be suspended, and how is it confirmed that metering returns to baseline? Write an escalation path without promising zero residual costs.

15.10 Monitoring, failure drills, and operational handover

15.10.1 Freshness monitors and alerting

A production-minded monitor watches a business promise: how long after its source interval does an approved gold row become visible? Monitor source watermark, last delivered file, last successful publish, rejected-row rate and report-refresh completion separately. A single green "pipeline job succeeded" light can conceal a stale data feed or a failed dashboard refresh. Do not describe a Dynamic Table's target lag as a guaranteed delivery time; monitor actual refresh outcomes and late intervals.

Signal	Response
Missing expected source window	Check provider availability and contract
Sudden invalid-value increase	Quarantine; stop affected publication
Duplicate/revision anomaly	Inspect event keys and manifest
Stale gold/public BI result	Verify publish and refresh boundaries
Unexpected spend or queue	Reduce approved workload and escalate

15.10.2 Simulated incident and recovery

Run a tabletop exercise: the provider republishes an earlier interval with a correction while the batch loader retries a previously delivered file. First freeze further publication, identify the affected snapshots, preserve the original event ledger and run the independent oracle. Next recompute current silver keys, compare them with the accepted fixture and publish only after review. A partial retry should be safe: never use a blind append into the gold table when a deterministic merge or rebuild is required.

15.10.3 Handover, RPO/RTO, and operating runbooks

Recovery point objective (RPO) specifies acceptable data loss relative to an incident; recovery time objective (RTO) specifies a target time to restore service. These are *design targets*, not guarantees achieved by writing numbers in a runbook. Set them with business owners, test a real restore and record actual elapsed time. State who may pause ingestion, rotate credentials, restore a prior snapshot and authorise a republish.

Incident evidence. The handover bundle records detection, impact, frozen source hash, diagnostic query IDs, decision owner, recovery steps, validation results and a follow-up action. A postmortem without a reproducible affected dataset cannot reliably distinguish data repair from luck.

Project reference: P055: Snowflake alert-hub reference. *Not independently executed; rights review pending.*

15.11 Documentation, architecture decision records, and demo

15.11.1 Architecture decision records

An architecture decision record (ADR) captures the decision, its context, considered alternatives, consequences, owner, date and revisit condition. The capstone includes an ADR choosing batch-first ingestion with an optional streaming extension. Another ADR records why the local oracle is independent of Snowflake: it gives deterministic acceptance evidence but cannot test access-control enforcement or production throughput. An ADR is a decision history, not a marketing description of a perfect design. A valuable record names its cost. Batch-first reduces infrastructure and incident surface at the expense of ingestion freshness. Import-mode Power BI often reduces interactive warehouse queries but introduces scheduled refresh lag. Separating roles improves security, while requiring deliberate grants and more governance work. These trade-offs help the next maintainer understand why the architecture is shaped as it is.

15.11.2 README and usage instructions

The project README starts with boundaries: synthetic input, supported Python version, no network dependency for offline tests, exact commands and expected output. It then points to the SQL examples, environment prerequisites, known limitations, BI definitions and operations runbook. A reviewer should not have to search an unstructured repository to learn which script makes destructive schema changes. Place the safest, cheapest verification path first.

```
# From the companion bundle root:
python -m unittest discover -s project/tests -v
python project/python/capstone.py
# Inspect project/output/acceptance.json.
# No Snowflake connection is made by these commands.
```

15.11.3 Reproducible five-minute demonstration

An honest demonstration has five beats. Minute one: show the problem and provenance, including the synthetic-data label. Minute two: inspect the source contract and deliberately flawed deliveries. Minute three: run the offline oracle and 21 tests. Minute four: show the gold export with six rows, 66.000 MWh and the quarantined rows. Minute five: explain the architecture, role boundary, cloud release gates and incident response. This is a *reproducible local demonstration*, not evidence of a deployed warehouse.

Demo acceptance. A colleague can reproduce the same numeric output without credentials, locate one rejected event by ID and describe why an older revision cannot replace Wind 14.000 MWh. They can also name which cloud and Power BI tests remain unexecuted.

Project reference: P028: Snowflake DevOps guide. *Not independently executed; rights review pending.*

15.12 Project review and future improvements

15.12.1 Peer review and limitations

A capstone review should challenge the most important claims first. Another engineer independently checks the business key, revision conflict behaviour, UTC interpretation, negative-value quarantine and what happens on replay. The BI reviewer validates that visual labels are consistent with the gold metric contract. Security reviewers inspect actual effective role grants, not only a diagram. Finance or platform owners inspect cost classes and the plan for suspending unintended usage. The shipped capstone meets its *offline* acceptance contract: 21 unit tests execute locally and the deterministic output has six visible observations totalling 66.000 MWh. It does *not* prove live Snowflake SQL compatibility, public-source access, a working Power BI `.pbix`, measured cloud cost, production freshness, a real restore or any AI model's operational quality. These are deliberate limitations printed next to the evidence, not hidden in an appendix.

15.12.2 Product improvements and future extensions

An approved next iteration could add a selected public source with pinned schema and licence snapshot, an automated Snowflake integration test account, a fault-injection test against staged-file loading and a production-safe deployment mechanism. Other extensions include a controlled streaming source, a true incremental backfill strategy, BI report performance evidence and forecast evaluation against a suitable time-series history. None should be admitted into the release solely because the feature exists. Prioritise improvements by reduction of operational risk, demonstrable stakeholder value, incremental cost and reversibility. The first new dependency should solve a measured problem; avoid adding an orchestrator, queue, vector index and AI service simultaneously to a pipeline that processes one daily CSV.

15.12.3 Retrospective evidence and final acceptance

The final gate asks: can a reviewer rebuild from checked-in assets, prove the accepted result, account for all rejected inputs, explain access boundaries and recover from a planned failure? If cloud evidence is absent, document an honest status such as *local capstone accepted; production deployment pending*. If a new source alters the data grain or unit, update the contract, tests, ADR and version tag together.

Closing principle. Production engineering is not the accumulation of platform features. It is a chain of justified decisions with observable outcomes, bounded failure modes, accountable ownership and reproducible evidence. The best final demonstration is the one an independent maintainer can explain and safely repeat.

Project reference: P181: root-cause and quality analysis reference. *Not independently executed; rights review pending.*

15.L Guided lab, chapter review, and exercises

15.L.1 Prerequisites, inputs, environment, permissions, and cost guardrails

Goal. Deliver a checkable final Finland Energy Analytics package with architecture decisions, code, audit artifacts and operational instructions. The default exercise is entirely offline and uses only Python's standard library. A separate Snowflake integration path is documented, but cloud deployment requires a designated test account, least-privilege role, reviewed scripts and a pre-agreed cost ceiling.

Asset	Purpose
`project/data/`	Synthetic input and exact fixture bytes
`project/python/`	Acceptance oracle, no cloud credentials
`project/tests/`	Deterministic positive/negative tests
`project/sql/`	Scoped Snowflake examples, not executed
`project/contracts/`	Versioned schema and invariants
`operations/`	Deployment and incident procedure
`architecture_decisions/`	Trade-offs and revisit criteria

Input assumptions. Twelve delivered synthetic CSV rows; one exact repeated event; a Wind correction to revision 2; a deleted `TEST` key; two invalid records. The provider label `DEMO_GRID` is invented. No real Fingrid or Statistics Finland data is bundled, and no live credentials are requested to complete the lab. **Permission and cost gate.** Offline: local disk write permission only, expected network traffic zero and Snowflake credit consumption zero. Optional cloud: obtain a sandbox role with restricted object rights and a small auto-suspending warehouse. Do not run administrative grants or cleanup without reviewer approval. For Power BI, a semantic model specification is provided, but Desktop and service validation remain manual gates.

Before starting. Confirm `python --version`, inspect the contract, compute the fixture hash, then copy the bundle to a clean folder. If a reviewer cannot repeat the verification without editing secrets or a global Python installation, report an environment defect before continuing.

15.L.2 Step-by-step procedure with selected code and expected results

Step 1 — inspect input. Open `project/data/energy_revisions.csv`. Verify the seven declared headers and note repeated `f-007`, corrected Wind, invalid revision zero and negative Solar. Do not delete the bad rows: they are part of the acceptance proof. **Step 2 — run the independent oracle.** From the companion bundle root:

```
python project/python/capstone.py
python -m unittest discover -s project/tests -v
```

The oracle writes `gold_energy.csv`, `quarantine.json` and `acceptance.json` under `project/output/`. The unit suite checks 21 conditions, including schema drift, equal-revision conflict, replay, approved operations, date format, no negative energy, the expected Wind revision and file export. The shipped execution recorded all 21 tests passing in the current isolated offline environment. **Step 3 — inspect state.** Confirm six visible gold rows and a **66.000 MWh** total. Wind must be 14.000 MWh; the deleted TEST key must be absent. Confirm exactly two source rows in quarantine and one exact repeated delivery in the audit. Record the fixture SHA-256 shown in the output to ensure the reviewer is evaluating the same input bytes.

Assertion	Actual offline output
Delivered / duplicates	12 / 1
Rejected / tombstoned keys	2 / 1
Visible gold rows	6
Gold sum and unit	66.000 MWh
Release gate	ACCEPT (offline fixture only)

Step 4 — cross-check the cloud design. Read SQL files in order. If an approved Snowflake account is available, deploy only to a sandbox, perform the source import and compare the resulting key set with the oracle output. Record actual SQL query IDs and query results. Otherwise mark this step *NOT EXECUTED*; do not improvise a cloud success report.

Result label. "Actual" above means the Python program really ran against authored *synthetic* data; it does not mean the measurements originate from Finland or the SQL was run on Snowflake.

15.L.3 Assertions, diagnosis, rollback, and cleanup

A failed assertion must prevent release. If total MWh increases after replay, inspect duplicate event IDs and the current-key ranking. If Wind shows 12.000 MWh, verify revision sorting. If a zero-valued DELETE remains visible, inspect operation filtering. If the input header changed, reject the file and request a versioned contract update. If an equal-revision collision occurs, quarantine the affected key and ask the source owner to resolve it; never choose an arbitrary last writer. For a failed offline run, preserve the input and failing traceback, remove only generated `project/output/` files if necessary, then rerun from the immutable source. For the cloud extension, pause publication, take an approved snapshot, resolve the root cause and perform a staged repair using the deployment runbook. Destructive SQL remains commented out; no automatic DROP is included in the offline workflow.

15.L.4 Exercises and assessment rubric

Foundation (20 points). Explain the business key, time grain, MWh unit and why the source includes revisions. List two false conclusions a dashboard could reach if corrections or duplicates are ignored. **Practitioner (35 points).** Add a valid seventh technology in a *copy* of the fixture. Update the expected acceptance contract to include its MWh value, demonstrate a passing revised test and show that old release results remain available under their previous fixture hash. Explain why changing the hard-coded original invariant without release documentation is not acceptable. **Production design (30 points).** Design a controlled release from development to production with restricted roles, data-source licence review, an incremental correction path, actual lag monitoring and a cost response plan. Mark which checks must run in the cloud. Describe one realistic failure injection and its rollback. **Communication (15 points).** Present a five-minute demo with data provenance, evidence paths, at least one negative test and the exact statement of limitations. A reviewer must be able to repeat the offline results.

> **Scoring guidance.** Correctness/invariants 35; reproducibility and tests 25; security, cost and recovery 25; clarity, attribution and honest limitations 15. A fabricated benchmark, invented cloud query ID, concealed source licence or unapproved destructive action is an automatic release blocker regardless of total score.

Chapter conclusion. The system is ready for a documented *engineering handover and cloud verification phase*, not certified production operation. The next pages of the book are reference appendices; the capstone's lasting product is the traceable relationship between a stakeholder question, a data contract, tested code, a published metric and an accountable operator.

SNOWFLAKE DATA ENGINEERING

From Fundamentals to Production-Ready Data Pipelines

From Snowflake fundamentals to reliable, production-oriented data pipelines.

Modern data engineering is not just about loading tables. It means making deliberate decisions about architecture, data quality, performance, security, reliability, and cost. This practical guide shows how those decisions connect in the Snowflake data platform.

Progress from Snowflake architecture and SQL to ingestion, transformation, orchestration, analytics, and AI. Each chapter connects concepts with engineering trade-offs, guided exercises, common failure modes, and companion GitHub resources.

WHAT YOU WILL LEARN

- Design and implement robust ETL and ELT data pipelines.
- Master practical SQL and Python for data processing.
- Understand data warehouses, data lakes, and lakehouse architectures.
- Explore batch processing, streaming, and orchestration (Snowpipe, Streams, Tasks, Dynamic Tables).
- Apply data quality, testing, observability, and governance practices.
- Build cloud-based data solutions and production-oriented workflows.

FROM THEORY TO PRACTICE

Explore practical engineering exercises, real-world case studies, architectural examples, and companion GitHub resources designed to connect technical concepts with hands-on implementation.

WHO SHOULD READ THIS BOOK?

Designed for aspiring and practicing Data Engineers, Analytics Engineers, Software Engineers, Data Analysts, and university students seeking a structured understanding of modern data engineering with Snowflake.



ABOUT THE AUTHOR

Arman Golbidi is a Data Engineer with professional experience in Azure, Python, SQL, Power BI, and data analytics. He holds an MSc in Data-Centric Engineering from LUT University, Finland. His background combines practical data platform development, analytics engineering, and applied machine learning.

COMPANION RESOURCES

Source code • Practical exercises • GitHub projects

Access the companion repository for code, datasets, and additional learning materials.

SCAN FOR PROJECT REPOSITORY

ISBN 978-1-736849-00-0

9 781736 849000